# Python 101: A Function-by-Function Code Walkthrough

Data preparation, demand estimation, and dynamic pricing

November 1, 2026

# Contents

**Python 101: learning to read this research code** **7**
1. A program is a sequence of instructions . . . 7
2. Values, types, and containers . . . 7
3. Functions: inputs, work, and outputs . . . 8
4. Branches, loops, and scope . . . 9
5. Arrays: many numbers with one operation . . . 9
6. Masks and comparisons . . . 10
7. pandas tables and Polars query expressions . . . 10
8. Strings, regular expressions, and paths . . . 10
9. Numerical optimization and symbolic algebra are different . . . 11
10. Modules, entry points, and tests . . . 11
11. A tiny trace you can do on paper . . . 12

**Worked reading lessons: unpacking the intimidating lines** **13**
Lesson 1: a table pipeline is a sequence of ordinary calls . . . 13
Lesson 2: masks express a sentence one cell at a time . . . 14
Lesson 3: from named coefficients to a utility matrix . . . 14
Lesson 4: softmax, log probabilities, and zero counts . . . 14
Lesson 5: one household must keep its latent type across weeks . . . 15
Lesson 6: understanding the gradient's einsum without memorizing it . . . 15
Lesson 7: first-stage least squares does not mean matrix inversion . . . 16
Lesson 8: symbolic expressions are not hidden numerical functions . . . 16
Lesson 9: closures and the expanding price bracket . . . 16
Lesson 10: practice before editing the real scripts . . . 17

**data_code: source walkthrough** **18**
data_code/cleaning_merging/master_dat.py . . . 18
Module setup and direct-execution blocks . . . 18
data_code/cleaning_merging/panel_concat.py . . . 26
Module setup and direct-execution blocks . . . 26
data_code/cleaning_merging/panel_convert.py . . . 28
Module setup and direct-execution blocks . . . 28
data_code/cleaning_merging/panel_dat.py . . . 33
Module setup and direct-execution blocks . . . 33
data_code/cleaning_merging/panel_merge.py . . . 33
Module setup and direct-execution blocks . . . 33
data_code/cleaning_merging/panel_week.py . . . 38
Module setup and direct-execution blocks . . . 38
data_code/cleaning_merging/retail_concat.py . . . 39
Module setup and direct-execution blocks . . . 39
data_code/cleaning_merging/retail_convert.py . . . 42
Module setup and direct-execution blocks . . . 42
data_code/cleaning_merging/retail_dat.py . . . 48

Module setup and direct-execution blocks . . . 48
data_code/cleaning_merging/retail_merge.py . . . 48
Module setup and direct-execution blocks . . . 49
data_code/recover_flavors.py . . . 54
Module setup and direct-execution blocks . . . 54
main . . . 55
data_code/summ_stats.py . . . 56
Module setup and direct-execution blocks . . . 56
normalize upcs . . . 57
normalized upc expr . . . 58
valid flavor labels . . . 59
scan purchases . . . 60
repeated yogurt households . . . 63
household summary . . . 65
as lazy . . . 67
partition purchases . . . 68
household batches . . . 70
save table . . . 71
demographic tables . . . 72
scan reference . . . 75
recover flavor reference . . . 76
flavor reference . . . 79
yogurt weeks . . . 83
flavor summaries . . . 85
make hhi figures . . . 91
make figures . . . 93
run . . . 96
main . . . 103

**est_code: source walkthrough** **107**
est_code/bootstrap_se.py . . . 107
Module setup and direct-execution blocks . . . 107
resample households . . . 107
bootstrap standard errors . . . 108
est_code/est.py . . . 112
Module setup and direct-execution blocks . . . 112
normalize upcs . . . 113
normalized upc expr . . . 115
read flavors . . . 116
week dates . . . 122
prepare data . . . 123
add stocks . . . 134
make arrays . . . 136
make draws . . . 138
parameter names . . . 139
household loglike . . . 141
demand objective gradient . . . 143
optimize demand . . . 146
optimize demand.objective . . . 150
optimize demand.progress . . . 151
plot gamma distribution . . . 152
fit model . . . 155
fit model.estimate . . . 159
main . . . 160
est_code/est_gamma.py . . . 164
Module setup and direct-execution blocks . . . 164

est_code/est_het_gamma.py . . . 165
Module setup and direct-execution blocks . . . 165
est_code/est_het_naive.py . . . 165
Module setup and direct-execution blocks . . . 166
est_code/est_naive.py . . . 166
Module setup and direct-execution blocks . . . 166
est_code/price_control.py . . . 167
Module setup and direct-execution blocks . . . 167
check dma . . . 167
other dma instruments . . . 168
absorb effects . . . 171
add price controls . . . 172

**supply_code: source walkthrough** **177**
supply_code/comparative_statics/___init___.py . . . 177
Module setup and direct-execution blocks . . . 177
supply_code/comparative_statics/alpha_p1.py . . . 177
Module setup and direct-execution blocks . . . 177
supply_code/comparative_statics/alpha_p2.py . . . 177
Module setup and direct-execution blocks . . . 178
supply_code/comparative_statics/beta_p1.py . . . 178
Module setup and direct-execution blocks . . . 178
supply_code/comparative_statics/beta_p1gamma.py . . . 178
Module setup and direct-execution blocks . . . 179
supply_code/comparative_statics/beta_p1p2.py . . . 179
Module setup and direct-execution blocks . . . 179
supply_code/comparative_statics/beta_p2.py . . . 179
Module setup and direct-execution blocks . . . 179
supply_code/comparative_statics/beta_profit.py . . . 180
Module setup and direct-execution blocks . . . 180
supply_code/comparative_statics/canonical.py . . . 180
Module setup and direct-execution blocks . . . 180
price2 derivative . . . 182
value2 derivative . . . 183
price1 derivative . . . 183
total derivative . . . 184
terminal total derivative . . . 185
profit derivative . . . 186
supply_code/comparative_statics/delta_p1.py . . . 187
Module setup and direct-execution blocks . . . 187
supply_code/comparative_statics/delta_p2.py . . . 188
Module setup and direct-execution blocks . . . 188
supply_code/comparative_statics/gamma2_p1.py . . . 188
Module setup and direct-execution blocks . . . 188
supply_code/comparative_statics/gamma2_p2.py . . . 188
Module setup and direct-execution blocks . . . 188
supply_code/comparative_statics/gamma_p1.py . . . 189
Module setup and direct-execution blocks . . . 189
supply_code/comparative_statics/gamma_p1beta.py . . . 189
Module setup and direct-execution blocks . . . 189
supply_code/comparative_statics/gamma_p1p2.py . . . 190
Module setup and direct-execution blocks . . . 190
supply_code/comparative_statics/gamma_p2.py . . . 190
Module setup and direct-execution blocks . . . 190
supply_code/comparative_statics/gamma_p2beta.py . . . 190
Module setup and direct-execution blocks . . . 190

supply_code/comparative_statics/gamma_profit.py . . . 191
Module setup and direct-execution blocks . . . 191
supply_code/comparative_statics_symbolic.py . . . 191
Module setup and direct-execution blocks . . . 191
supply_code/export/___init___.py . . . 192
Module setup and direct-execution blocks . . . 192
supply_code/export/latex_export.py . . . 192
Module setup and direct-execution blocks . . . 192
display equation . . . 192
supply_code/export/pretty_print.py . . . 193
Module setup and direct-execution blocks . . . 193
show . . . 193
supply_code/make_comparative_static_figures.py . . . 194
Module setup and direct-execution blocks . . . 194
save . . . 195
main . . . 196
supply_code/make_latex_document.py . . . 200
Module setup and direct-execution blocks . . . 200
main . . . 202
supply_code/model/___init___.py . . . 208
Module setup and direct-execution blocks . . . 209
supply_code/model/assumptions.py . . . 209
Module setup and direct-execution blocks . . . 209
supply_code/model/numerical.py . . . 209
Module setup and direct-execution blocks . . . 209
inside share . . . 210
optimal price . . . 211
evaluate . . . 212
solve . . . 215
supply_code/model/profit.py . . . 216
Module setup and direct-execution blocks . . . 216
supply_code/model/shares.py . . . 216
Module setup and direct-execution blocks . . . 216
supply_code/model/symbols.py . . . 217
Module setup and direct-execution blocks . . . 217
supply_code/model/utility.py . . . 218
Module setup and direct-execution blocks . . . 218
signed state . . . 219
raw utilities . . . 220
supply_code/pricing/___init___.py . . . 220
Module setup and direct-execution blocks . . . 220
supply_code/pricing/period1_foc.py . . . 220
Module setup and direct-execution blocks . . . 220
supply_code/pricing/period1_price.py . . . 221
Module setup and direct-execution blocks . . . 221
supply_code/pricing/period2_foc.py . . . 221
Module setup and direct-execution blocks . . . 222
supply_code/pricing/period2_price.py . . . 222
Module setup and direct-execution blocks . . . 222
supply_code/run_all.py . . . 222
Module setup and direct-execution blocks . . . 223
supply_code/structural_switching_reference.py . . . 224
Module setup and direct-execution blocks . . . 224
expit safe . . . 225
inside share . . . 225
p2 opt . . . 226

evaluate . . . 227
verify . . . 230
plot series . . . 231
main . . . 233

**Tests and verification: worked calling examples** **237**
data_code/test_summ_stats_memory.py . . . 237
Module setup and direct-execution blocks . . . 237
MemorySummaryChecks.test batches preserve global statistics . . . 238
MemorySummaryChecks.test demographics match previous fieldwise sort . . . 242
MemorySummaryChecks.test recovery preserves existing labels and uses earliest year . . . 244
est_code/test_est.py . . . 245
Module setup and direct-execution blocks . . . 245
EstimationChecks.frame . . . 246
EstimationChecks.test history excludes current counts and accumulates all occasions . . . 247
EstimationChecks.test xi sign and unlimited history . . . 248
EstimationChecks.test parquet pipeline and fit . . . 249
EstimationChecks.test parquet pipeline and fit.append . . . 253
EstimationChecks.test unavailable flavor is excluded from logit denominator . . . 254
EstimationChecks.test persistent household integration . . . 256
EstimationChecks.test analytic gradient . . . 258
EstimationChecks.test four model specifications . . . 260
EstimationChecks.test mixed naive integrates only alpha . . . 261
EstimationChecks.test zero heterogeneity nests homogeneous model . . . 263
est_code/test_price_control.py . . . 264
Module setup and direct-execution blocks . . . 264
synthetic panel . . . 265
PriceControlChecks.test excludes consumer dma even when exposure store elsewhere . . . 267
PriceControlChecks.test absorption matches explicit dummy ols . . . 268
PriceControlChecks.test first stage residuals orthogonal and in utility . . . 270
PriceControlChecks.test first stage uses only covered flavor prices . . . 271
PriceControlChecks.test resampling preserves histories and relabels duplicates . . . 274
PriceControlChecks.test bootstrap refits first stage and reports empirical sd . . . 275
PriceControlChecks.test bootstrap refits first stage and reports empirical sd.estimate . . . 277
PriceControlChecks.test controlled demand optimizer runs . . . 277
est_code/test_summary_stats.py . . . 279
Module setup and direct-execution blocks . . . 279
SummaryChecks.test demographics and mixed baskets . . . 280
supply_code/checks/___init___.py . . . 283
Module setup and direct-execution blocks . . . 283
supply_code/checks/check_alpha.py . . . 283
Module setup and direct-execution blocks . . . 283
supply_code/checks/check_beta.py . . . 284
Module setup and direct-execution blocks . . . 284
supply_code/checks/check_delta.py . . . 284
Module setup and direct-execution blocks . . . 284
supply_code/checks/check_focs.py . . . 285
Module setup and direct-execution blocks . . . 285
supply_code/checks/check_gamma.py . . . 285
Module setup and direct-execution blocks . . . 285
supply_code/checks/check_numerical.py . . . 286
Module setup and direct-execution blocks . . . 286
supply_code/checks/check_pricing_derivatives.py . . . 286
Module setup and direct-execution blocks . . . 286
supply_code/checks/test_symmetric.py . . . 287
Module setup and direct-execution blocks . . . 287

SymbolicChecks.equal . . . 288
SymbolicChecks.test raw normalization and both states . . . 289
SymbolicChecks.test focs from actual profit . . . 290
SymbolicChecks.test envelope and implicit gamma derivatives . . . 291
SymbolicChecks.test all parameter implicit derivatives . . . 292
NumericalChecks.test reference on entire benchmark grid . . . 293
NumericalChecks.test independent reoptimization and derivatives . . . 294
NumericalChecks.test independent reoptimization and derivatives.negative profit . . . 296
NumericalChecks.test all parameter and mixed derivatives numerically . . . 297
NumericalChecks.test all parameter and mixed derivatives numerically.arguments at . . . 299
NumericalChecks.test zero switching static case and symmetry . . . 300

**Coverage, rebuilding, and next steps** **302**

# Python 101: learning to read this research code

This is a separate companion to the economic and pipeline guides. Those guides explain the models. This book teaches you to read their implementation, beginning with Python punctuation and ending with each function and script in `data_code`, `est_code`, and `supply_code`. You do not need to understand all the economics before learning what a line of Python does.

The book contains the actual source, source line references, function-specific explanations, statement-by-statement reading notes, alternatives, and small examples. Nested helper functions and test methods are included. Scripts with no functions receive the same treatment at module level. Empty package files are identified explicitly. Source excerpts are a snapshot of the working tree at build time; they are not edited versions of the executable research code.

Read the foundations first, then the small `make_draws` and `signed_state` functions. After that, try `add_stocks`, `resample_households`, and `optimal_price`. Return to the long preparation functions after you can recognize loops, columns, masks, and returned values. The table of contents and PDF bookmarks let you find any function by file and name. Appendix tests are useful worked examples of how to call the code without the private datasets.

## 1. A program is a sequence of instructions

Python executes statements in order. An assignment evaluates its right-hand side, then binds its result to the name on its left:

```
packages = 3
price = 1.50
spending = packages * price
```

`spending` is now 4.5. The equals sign is an instruction to store a result; it is not a mathematical claim to solve. `spending == 4.5` instead asks a question and produces the Boolean value `True`.

Names such as `weekly` and `frame` are ordinary variable names chosen by the author. They have no special meaning to Python. A name can later refer to a different object. `frame = frame.copy()` first copies the old table and then rebinds `frame` to the new one. Assigning `other = frame` does not copy the table; it gives the same object another name.

`#` begins a comment outside a string. Python ignores the rest of that line. Triple-quoted text at the start of a module, class, or function is a docstring: it is documentation retrievable with `help`, not a hidden calculation.

## 2. Values, types, and containers

| Example | Type | How this code uses it |
|---|---|---|
| `3` | Integer, `int` | Counts, loop limits, seeds |
| `3.0`, `.7`, `1e-6` | Floating-point number, `float` | Prices, coefficients, tolerances |
| `"berry"` | String, `str` | Column labels, model names, paths |

| Example | Type | How this code uses it |
|---|---|---|
| `True`, `False` | Boolean, `bool` | Settings and validation |
| `None` | The special absence value | Optional input not supplied |
| `["berry", "plain"]` | List | Ordered, changeable sequence |
| `("j", "0")` | Tuple | Ordered collection, often used for fixed choices |
| `{"price": 1.5}` | Dictionary, `dict` | Named values looked up by key |
| `{"upc", "flavor"}` | Set | Unordered distinct labels and membership checks |

Square brackets have several jobs. In `[1, 2]` they create a list. In `items[0]` they select an item. In `frame["price"]` they select a named table column. Read the object to the left before deciding what the brackets mean.

Python indexes start at zero. For `names = ["other", "berry", "plain"]`, `names[0]` is other and `names[2]` is plain. `names[:2]` selects positions zero and one; slice stops are excluded. `names[1:]` starts at berry and goes to the end. A NumPy slice such as `prices[:, j]` means every row of column j.

`dict(zip(names, theta))` pairs labels with values. If the labels are `["beta", "gamma"]` and values `[2.0, -.5]`, the result is `{"beta": 2.0, "gamma": -.5}`. Ordinary `zip` stops at the shorter input; the surrounding code is responsible for consistent lengths. `p["gamma"]` requires the key; `p.get("gamma", 0)` supplies zero if it is absent.

`a, b = (10, 20)` unpacks two returned values. `_` is a conventional name for an ignored value; Python does not magically discard it. In `fit, _, _ = ...`, the author keeps the fit and does not use the other returned objects.

## 3. Functions: inputs, work, and outputs

```python
def package_cost(quantity, price=1.0):
    total = quantity * price
    return total

answer = package_cost(3, price=1.5)
```

`def` defines a function. The name is `package_cost`; parentheses contain its parameters. The colon introduces the body. Indentation determines which statements belong to that body. Defining the function does not yet execute its body. Calling it with parentheses does.

In the call, 3 is a positional argument and `price=1.5` is a keyword argument. The `=` inside a call names an argument; it is not assignment to a variable outside the function. `price=1.0` in the definition supplies a default when the caller omits that argument. `return` ends this call and gives its value back to the caller. Printing 4.5 displays text; returning 4.5 lets another calculation use the number. A function that falls off its end returns `None`.

`def solve(*, initial_state="0", **parameters)` contains two more pieces of syntax: bare `*` makes following arguments keyword-only; `**parameters` collects additional named arguments into a dictionary. In a call, `evaluate(gamma, state, **parameters)` expands such a dictionary back into named arguments. `*command` in a list expands a sequence into its elements.

Functions defined inside another function are nested functions. They can read names from the surrounding call. In `optimize_demand`, the nested `objective` remembers `data`, `draws`, and `args`; SciPy only has to pass the changing parameter vector. This is a closure. It avoids global variables, whose contents could accidentally affect another fit.

`lambda p: p * p` is a compact function returning the square of p. It is handy for a one-expression root equation. A named `def` is clearer if the calculation needs several steps, comments, or reuse. A function object can itself be passed as an argument: `brentq(f, lo, hi)` asks a solver to call f repeatedly.

## 4. Branches, loops, and scope

```
if quantity <= 0:
    raise ValueError("Quantity must be positive")
else:
    print("Usable quantity")
```

`if` evaluates a condition, `elif` tries another condition when the previous one was false, and `else` handles the remaining case. `raise` stops normal execution by creating an exception. Rejecting unusable counts is preferable to quietly turning them into valid-looking zeros.

`for j, name in enumerate(FLAVORS):` repeats a block once for each flavor, providing both its numeric position and its label. It is equivalent to looping over positions and looking up labels, but makes the correspondence explicit. `range(3)` produces 0, 1, 2. A `while` loop instead keeps going while its condition is true; the price solver uses it because it does not know in advance how many bracket expansions are needed.

`["price_" + f for f in FLAVORS]` is a list comprehension. A slower-to-write but equally valid spelling is:

```
columns = []
for flavor in FLAVORS:
    columns.append("price_" + flavor)
```

An optional `if` at the end filters elements. `{key: value for ...}` builds a dictionary. `(value for ...)` is a generator expression: it yields values as needed rather than first making a complete list. `next(generator, None)` takes the first yielded value, or returns `None` if no match exists.

`try` executes code that might fail. A matching `except` handles named errors. The bootstrap records identification failures rather than accepting them as estimates. Catching every possible exception would also hide programming bugs, so it catches specific expected failure types.

## 5. Arrays: many numbers with one operation

NumPy arrays are rectangular numerical containers. Their `shape` is a tuple of axis lengths. A three-week, three-flavor table has shape `(3, 3)`. Axis zero indexes rows; axis one indexes columns. Consequently, `.sum(axis=1)` adds across flavors for each week, while `.sum(axis=0)` adds across weeks for each flavor.

```
import numpy as np
counts = np.array([[2., 0., 1.], [0., 4., 0.]])
totals = counts.sum(axis=1)             # [3., 4.]
shares = counts / totals[:, None]
```

`totals[:, None]` changes a shape `(2,)` vector into a `(2, 1)` column. NumPy then broadcasts each row total across its three columns. Without that extra axis, `(2,3) / (2,)` cannot broadcast: the trailing dimensions 3 and 2 conflict. Broadcasting is controlled shape matching, not a guess at intended economics.

Similarly, adding a three-element beta vector to a W-by-3 utility matrix adds the flavor intercepts to every row. Array multiplication `*` is elementwise. Matrix multiplication `@` sums products over the matching inner dimension. For one-dimensional vectors it is the dot product, e.g. `[1,2] @ [3,4] = 11`. `**2` squares a value; `^` is not Python's exponentiation operator.

`np.zeros((W,3))` allocates an array of zeros. `np.zeros_like(prices)` matches another array's shape and dtype. Preallocating avoids repeatedly growing an array inside a long loop. `np.column_stack` joins columns; `np.vstack` stacks rows. `np.bincount(hh, weights=row_ll)` sums contributions by nonnegative integer household label, avoiding a slower Python loop over each household.

`np.nan` represents a missing floating-point value; `np.inf` represents positive infinity. NaN is not equal to itself, so use `np.isnan` or table missing-value methods rather than `x == np.nan`. `np.isfinite` rejects both NaN and infinities.

## 6. Masks and comparisons

```
covered = np.isfinite(prices) & (prices > 0)
clean_prices = np.where(covered, prices, 0.)
```

A mask is an array of True/False values. `&` combines array conditions with elementwise AND; `|` is elementwise OR; `~` reverses Booleans. Put each comparison in parentheses because Python operator precedence otherwise changes the grouping. `and`, `or`, and `not` instead combine scalar truth values and short-circuit; they generally cannot combine an entire pandas Series unambiguously.

`.any()` asks whether at least one entry is true; `.all()` asks whether every entry is true. `available.any(axis=1)` asks separately for each row. A useful translation is: `((counts == 0) | available).all(axis=1)` means "every flavor was either unpurchased or available." It rejects a purchased flavor without a price.

`np.where` selects values element by element. It does not lazily evaluate its two alternatives: both argument expressions are evaluated before the call. Use safe denominators or explicit indexing when the unused branch would raise or perform an invalid calculation. A zero price placeholder is not a free good when the corresponding alternative is subsequently masked out of utility.

## 7. pandas tables and Polars query expressions

A pandas `DataFrame` is a table with named columns and row labels; a `Series` is a single labeled column. `frame["price"]` selects one Series; `frame[["price", "quantity"]]` selects a two-column DataFrame. `frame.loc[mask, "price"]` selects specified rows and one column. `frame.iloc[0]` selects by numeric row position. `.copy()` makes an independent table before modifications, avoiding accidental changes to the original.

`.groupby(keys)["quantity"].sum()` splits rows by common keys and sums within each group. `.transform("nunique")` broadcasts a group's distinct count back to every original row. In contrast, an ordinary group aggregation returns one row per group. `.unstack` and `.pivot` turn flavor labels into columns; `.reindex(columns=FLAVORS)` makes their ordering explicit for numerical arrays.

A merge is a lookup using matching key columns. `how="left"` keeps left rows, `inner` keeps matches, `semi` keeps only matching left rows without adding right columns, and `anti` keeps unmatched left rows. A duplicate right key can multiply rows. `validate="many_to_one"` checks the right side is unique; it does not force an invalid table to become unique. Deduplication and validation are different decisions.

Polars uses `pl.col("price")` to build a column expression. Think of it as a recipe, not yet a column of numbers. `pl.lit(0)` is a literal-valued expression; `.alias("price")` names an expression's output. In `pl.when(condition).then(a).otherwise(b)`, each row chooses an outcome. `.with_columns` adds or replaces columns, while `.select` projects only the requested columns/expressions.

`pl.scan_parquet(path)` builds a lazy plan. `.filter`, `.select`, and `.join` extend that plan. `.collect()` executes it and materializes a table in memory; `.sink_parquet(path)` executes it while writing output. This allows early projection/filtering and streaming execution. It does not guarantee tiny memory: joins, sorts, and exact distinct counts still need state. `.to_pandas()` converts to an eager pandas table, appropriate for the later NumPy computations.

Method chaining means "take the result so far, then call the next method":

```
cleaned = frame.dropna().sort_values("week").reset_index(drop=True)
```

The code often places the chain inside parentheses so it can span physical lines. Leading dots on later lines continue the same expression; they are not new statements. Nested function calls evaluate their arguments before the outer call. Starting with the innermost parentheses is often the easiest reading strategy.

## 8. Strings, regular expressions, and paths

`"price_" + flavor` concatenates text. `f"Start {s + 1}"` inserts an expression into a formatted string. `:.4f` means four digits after the decimal point, `:,` adds thousands separators, and `:.1%` displays a fraction as a percentage.

`flush=True` asks printed progress to reach the log promptly instead of waiting in a buffer.

Regexes describe string patterns. In `r"[0-9]+(?:\.0+)?"`, `[0-9]+` means one or more digits, `(?:...)` groups without capturing, `\.` is a literal dot, and `?` makes the decimal-zero suffix optional. The raw-string prefix r keeps Python from processing backslashes first. `^` and `$` anchor start and end. The UPC normalization removes only a numeric `.0` suffix, not every decimal point or arbitrary letters. String handling avoids rounding a large UPC through a floating-point number.

`Path("/scratch") / "Data"` joins filesystem components. The slash is overloaded by `Path`; here it does not divide numbers. `.exists()` checks presence, `.mkdir(parents=True, exist_ok=True)` creates needed directories, and `.resolve()` normalizes a path for comparisons. A `with TemporaryDirectory() as tmp:` block cleans up the temporary directory on exit, including ordinary exception exits. `with path.open("w") as file:` similarly ensures the file is closed.

## 9. Numerical optimization and symbolic algebra are different

`minimize(objective, initial, ...)` repeatedly calls a numerical function, changing an array of parameter values to lower its returned loss. `jac=True` means this objective supplies both the number and its derivative vector. `brentq(f, lo, hi)` finds a zero of a continuous scalar function with a sign-changing bracket; it does not minimize the function. A root of a profit derivative is a candidate optimum, whose optimality needs the argument in the economic guide.

`sp.symbols("alpha beta")` creates symbolic mathematical placeholders. It does not create estimated numerical values. `sp.diff(expr, alpha)` differentiates algebra; `.subs({alpha: -1})` substitutes; `sp.simplify` tries to reduce an expression. `sp.lambdify(arguments, expression, "numpy")` turns that symbolic formula into a callable numerical function with an explicit argument order.

A crucial trap: SymPy treats distinct symbols as independent unless dependence is encoded. If `s` is a separate symbol, differentiating `s*p` with respect to p gives s, not `s + p*ds/dp`. The supply code's total-derivative helper manually adds that chain-rule contribution for equilibrium shares and prices.

For likelihood calculations, `logsumexp` computes the logarithm of a sum of exponentials stably. `np.log(np.exp(x).sum())` can overflow even when its final mathematical answer is finite. The optimizer would then receive unusable values. The analytic gradient is exact for the fixed simulated draws, while the simulation integral itself still approximates the population integral.

## 10. Modules, entry points, and tests

`import numpy as np` imports a library and gives it a short local name. `from price_control import check_dma` imports one object from a neighboring module. Imports can execute a module's top-level code. That is why the legacy cleaning scripts should not be imported merely for inspection: they perform data work at module level.

```
if __name__ == "__main__":
    main()
```

When Python runs a file directly, its `__name__` is `"__main__"`, so this block calls its entry point. When another file imports it, the guard is false and the guarded work is skipped. ArgumentParser translates command-line text into attributes such as `args.draws`; `type=int` converts text to integers and `action="store_true"` creates a Boolean flag.

A class groups behavior. `class Checks(unittest.TestCase):` inherits the testing tools. A method is a function attached to a class; `self` refers to the current instance and is supplied automatically in calls such as `self.frame()`. `self.assertEqual(actual, expected)` fails the test if the values differ. Floating-point results need tolerances: `.1 + .2` is generally not represented exactly as .3. `assert_allclose` compares arrays element by element with stated absolute/relative tolerances. A passing test confirms its constructed case, not every possible input or the truth of an economic assumption.

## 11. A tiny trace you can do on paper

```
stock = np.zeros(3)
counts = np.array([0., 2., 0.])
share = counts / counts.sum()
before = stock.copy()
stock = stock + share
```

Initially stock is `[0,0,0]`. Counts sum to 2, so share is `[0,1,0]`. `before` preserves zero history; stock becomes `[0,1,0]`. The order explains why the current purchase does not enter its own pre-choice state. Reversing the last two statements would change the statistical model, even though both versions are valid Python.

Try these questions before looking back: what does `np.arange(4)` return? Why is `frame[["price"]]` different from `frame["price"]`? When does a nested objective run: when it is defined or when SciPy calls it? Does `new = old` copy a table? Does `.all(axis=1)` summarize rows or columns? The answers are 0–3, table versus Series, when called, no, and one result per row after combining columns. These are the building blocks used repeatedly below.

# Worked reading lessons: unpacking the intimidating lines

The file-by-file chapters below are a reference. These short lessons demonstrate how to slow down and reason through a dense expression. The expanded versions are teaching equivalents, not suggested edits to the production code.

## Lesson 1: a table pipeline is a sequence of ordinary calls

The estimator contains this expression:

```
weekly = (lines.groupby(keys + ["flavor"])["quantity"].sum()
          .unstack("flavor", fill_value=0)
          .reindex(columns=FLAVORS, fill_value=0).reset_index())
```

Read left to right after identifying the innermost list expression:

1. `keys + ["flavor"]` concatenates the household/week key list with one more column label. The plus sign joins lists; it is not adding column values.
2. `lines.groupby(...)` splits purchase rows into household/week/flavor groups.
3. `["quantity"]` selects the quantity Series from that grouped object.
4. `.sum()` adds package counts within each group. Two separate berry purchases in the same household-week become one berry total.
5. `.unstack("flavor", fill_value=0)` moves the flavor part of the grouped index into columns. An unobserved flavor within an existing week gets zero count.
6. `.reindex(columns=FLAVORS, fill_value=0)` fixes the columns to other, berry, plain, in that order, adding any completely absent flavor with zeros.
7. `.reset_index()` turns household/week index levels into ordinary columns.
8. `weekly = ...` names the resulting table only after the whole chain completes.

You could write intermediate objects instead:

```
group_columns = keys + ["flavor"]
grouped = lines.groupby(group_columns)["quantity"]
totals = grouped.sum()
wide = totals.unstack("flavor", fill_value=0)
ordered = wide.reindex(columns=FLAVORS, fill_value=0)
weekly = ordered.reset_index()
```

Both forms express the same intended pandas operations. Intermediates are useful while learning or debugging because you can inspect each type and shape. Chaining keeps a completed transformation compact once its stages are familiar. It is not intrinsically more statistically correct, and it is not automatically lazy just because it is written across several lines.

For a household with berry purchases 2 and 1 and plain purchase 4 in one week, the final count columns are `(other=0, berry=3, plain=4)`. Index order becomes important only when those columns are converted to position-based arrays.

## Lesson 2: masks express a sentence one cell at a time

```
complete = available.any(axis=1) & ((counts == 0) | available).all(axis=1)
```

Suppose one week has counts `[0,2,1]` and availability `[True,True,False]`. The zero-count comparison gives `[True,False,False]`. OR with availability gives `[True,True,False]`: the plain purchase has no price. `.all(axis=1)` therefore gives False for that row. Although `.any(axis=1)` confirms a nonempty menu, AND combines it with False, so the week is rejected.

If plain count were zero, the OR vector would be all True, so the missing plain price would be acceptable: plain can be excluded from the choice set. This distinguishes "unpriced and unpurchased" from "unpriced but purchased."

The expression could be expanded into a row loop and an inner flavor loop:

```
complete_rows = []
for row_counts, row_available in zip(counts, available):
    nonempty_menu = any(row_available)
    every_purchase_covered = all(
        q == 0 or offered
        for q, offered in zip(row_counts, row_available)
    )
    complete_rows.append(nonempty_menu and every_purchase_covered)
```

The loop uses scalar `or` and `and` because q and offered are individual values. The production expression uses `|` and `&` because its comparisons concern whole arrays. Vectorization moves the element loops into NumPy, but the logical rule remains the same. Parentheses are necessary: do not rely on English-like operator precedence in an array expression.

## Lesson 3: from named coefficients to a utility matrix

```
p = dict(zip(parameter_names(model, data["outside"], data["control"]), theta))
beta = np.array([0.0, p["beta_berry"], p["beta_plain"]])
```

The first call obtains parameter labels for this particular model. `zip` pairs each with the coefficient in the same position; `dict` makes the lookup mapping. For a naive three-parameter model, `[.4, -.2, 0.]` corresponds to berry intercept .4, plain intercept -.2, and mean log-minus-alpha zero. It does not mean alpha is zero: the later transform is `-exp(0) = -1`.

With no gamma or price-control effect and one week's prices `[1,1.2,1.5]`, the utility calculation is

```
beta = np.array([0., .4, -.2])
alpha = -1.
prices = np.array([[1., 1.2, 1.5]])
utility = beta + alpha * prices
# utility is [[-1., -.8, -1.7]]
```

Beta has shape `(3,)`, prices `(1,3)`, and the output `(1,3)`. Each flavor's intercept is added to that flavor's price term. Alpha's negative sign means higher prices lower utility. The other-flavor intercept is fixed at zero as a normalization; that does not imply other flavor has zero total utility.

Why not use positions such as `theta[0]` everywhere? That is possible, but a different model inserts/removes coefficients and can change positions. A single ordered label helper plus dictionary lookup makes the interpretation visible. The numeric values remain in an array for efficient optimization.

## Lesson 4: softmax, log probabilities, and zero counts

```
logprob = utility - logsumexp(utility, axis=1, keepdims=True)
row_ll = (data["counts"] * np.where(data["counts"] > 0, logprob, 0.)).sum(axis=1)
```

For logit, the logarithm of a probability is utility minus the log of the sum of exponentiated utilities. `axis=1` performs that denominator calculation across alternatives within each week. `keepdims=True` preserves a W-by-1 column so it can be subtracted from the W-by-J utility matrix.

An unavailable alternative has utility minus infinity and probability zero. Its log probability is minus infinity. A zero purchase count should contribute zero to the log likelihood, but floating point computes `0 * -inf` as NaN. The `where` replaces that log probability with zero **where count is zero** before multiplying, protecting the valid zero contribution.

For counts `[0,2,1]`, the row contribution is two times berry log probability plus one times plain log probability. This is why changing an actual package count changes the likelihood's weight; it is not just a single flavor label. Subtracting raw `np.exp(utility).sum()` rather than its log would be the wrong formula, and computing exp then log naively can overflow. `logsumexp` handles the same mathematics in a stable numerical form.

## Lesson 5: one household must keep its latent type across weeks

Assume two possible types, equally weighted. Type A gives the household's chosen flavor probability .9 in each of two weeks; type B gives .1 in each. Persistent types imply the history probability

```
likelihood = (.9 * .9 + .1 * .1) / 2
# .41
```

Averaging probabilities independently each week instead gives

```
wrong_history = ((.9 + .1) / 2) * ((.9 + .1) / 2)
# .25
```

The difference is not rounding: it is whether the household can redraw its type between weeks. The estimator first adds weekly log likelihoods within household for each draw, then integrates those full-history likelihoods. `np.bincount` performs the within-household sum. A list accumulates one household vector per draw. The final `logsumexp(..., axis=0) - log(R)` integrates the draw dimension while preserving the household dimension.

This also explains why the gradient uses posterior draw weights. A type assigning higher probability to the entire observed history receives greater weight in that household's score. Those weights are calculation intermediates, not new estimated household coefficients written by the program.

## Lesson 6: understanding the gradient's einsum without memorizing it

```
gradient = -np.einsum("rh,rhp->p", weights, np.asarray(scores)) / data["n_households"]
```

The quoted letters label axes; they are not Python variable names. `r` is draw, `h` household, and `p` parameter. Weights has shape `(R,H)`, scores `(R,H,P)`. Multiply matching draw/household entries, sum over r and h because they do not appear after the arrow, and retain p. The resulting vector has length P.

Here is the same sum written in a teaching loop:

```
gradient = np.zeros(P)
for parameter in range(P):
    for draw in range(R):
        for household in range(H):
            gradient[parameter] += (
                weights[draw, household] * scores[draw, household, parameter]
            )
gradient = -gradient / H
```

The negative sign converts a log-likelihood score to a negative-log-likelihood gradient. Division by H matches the mean loss returned to the optimizer. The compact NumPy form avoids Python executing three nested numerical loops, while the teaching loop makes the axis meaning explicit. In this example P/R/H are dimension labels you would obtain from the actual arrays, not existing global constants in the source.

## Lesson 7: first-stage least squares does not mean matrix inversion

```
z_partial = z - controls @ np.linalg.lstsq(controls, z, rcond=None)[0]
```

Starting inside, `lstsq(controls,z,...)` fits z on the control columns. Its return contains coefficients plus diagnostics, so `[0]` extracts the coefficient vector. Matrix multiplication produces fitted z, then subtraction leaves the part not linearly explained by those controls. `rcond=None` requests NumPy's default singular-value cutoff convention for that calculation.

For a three-row example, let controls be a column of ones and z `[1,2,6]`. Least squares estimates the mean 3, fitted values `[3,3,3]`, and residual `[-2,-1,3]`. The production controls are Xi after fixed-effect absorption, but the projection arithmetic has the same structure.

The alternative formula involving an inverse of `X.T @ X` requires nonsingular cross-products and can be numerically less stable. `lstsq` handles rank deficiency using its numerical decomposition. It cannot create excluded-instrument variation where none exists; the following sum-of-squares check rejects that case.

## Lesson 8: symbolic expressions are not hidden numerical functions

```
alpha, beta, gamma = sp.symbols("alpha beta gamma", real=True)
price = sp.Symbol("price", real=True)
index = beta + alpha * price + gamma
direct = sp.diff(index, gamma)
```

`direct` is the symbolic number 1. No household data, optimizer, or probability is involved. If a separate symbol s is introduced, SymPy does not know that s depends on price unless the dependence is encoded. This is why the supply code implements explicit total-derivative rules.

`sp.lambdify((alpha,beta,gamma,price), index, "numpy")` builds a callable expecting four numbers in that order. A call with `(-1,2,.5,1)` gives 1.5. Passing beta where alpha belongs gives a different value without necessarily raising an error. Named dictionaries at output do not remove the need to keep this positional bridge correct. Symbolic algebra improves formula reuse, while numerical tests at asymmetric states catch argument-order mistakes.

## Lesson 9: closures and the expanding price bracket

```
f = lambda p: alpha*(p-effective_cost) + 1 + np.exp(
    np.clip(intercept + alpha*p, -745, 700)
)
lo = effective_cost
width = max(2., 2/abs(alpha))
hi = lo + width
while f(hi) > 0:
    width *= 2
    hi = lo + width
```

Only p is written as a lambda parameter; alpha, effective cost, and intercept come from the enclosing call. Evaluating `f(hi)` substitutes the current hi without changing those fixed inputs. Under the maintained negative-alpha domain, f starts positive at lo and eventually becomes negative as p increases. Doubling width searches for a reliable sign-changing bracket rather than guessing a universal maximum price.

Once bracketed, Brent's root method solves f(p)=0. It is solving the transformed FOC, not trying to make the price itself zero. A loop that forgot to update hi could never change its condition; a loop with `width += 2` would expand more slowly for far-away roots. The mathematical guide establishes why this unique FOC root is the optimum for the current unconstrained logit problem.

## Lesson 10: practice before editing the real scripts

For each exercise, predict the result before checking the answer:

1. For model `naive`, outside False, control True, what parameter names are returned? Answer: beta_berry, beta_plain, mu_log_minus_alpha, price_control.
2. If a household is selected twice by bootstrap, why change its ID twice? Answer: the two sampled histories must remain separate likelihood contributions.
3. What does `.copy()` protect when changing bootstrap IDs? Answer: the original household subtable that may be selected again in another draw.
4. Is a lazy query just a synonym for a small table? Answer: no; it is a plan, and executing it can still need large intermediate state.
5. In `return fitted, results, bounds`, why do the commas matter? Answer: they construct a tuple the caller can unpack; there are three outputs.
6. Why is saving `.tex` not the same as saving `.pdf`? Answer: the former is source text requiring a TeX engine to typeset it.
7. Can a zero outside count justify adding missing purchase weeks? Answer: no; the implemented sample contains only observed purchase weeks.
8. Does a first-stage F statistic establish exclusion? Answer: no; it describes conditional instrument relevance, while exclusion is an economic assumption.

You can learn most of the syntax by running tiny independent arrays and tables. Read the legacy scripts as text before executing them: several contain top-level filesystem writes and replacements. The following chapters reproduce that source without executing it. The numbered listings are fragments for reading; a fragment beginning inside a function is not necessarily runnable on its own.

# data_code: source walkthrough

## data_code/cleaning_merging/master_dat.py

This matches household purchases to retail quotes. The four-element key list is reused for projection, deduplication, and joins, reducing accidental differences between stages. The filtered retail yogurt table is collected once into memory; panel purchases are scanned and joined year by year to bound panel-side work. An inner join means a price match is a sample-selection condition.

Yearly results are staged as Parquet and scanned together into the final sample. `os.remove` deletes the generated temporary files and `os.rmdir` removes their directory after consolidation. These are real filesystem mutations. Final diagnostic queries return scalar counts or small tables. `next(..., None)` picks supported household/price names rather than assuming a suffix. Building all years eagerly in one pandas merge would be shorter but much more memory-hungry. Resolve its retail input path against the retail merge's actual output before running a fresh pipeline.

### Module setup and direct-execution blocks

Source lines 1–7:

```
1 import glob
2 import os
3 import polars as pl
4
5 input_dir = "/scratch/dtm63837/Kilts_Panel/Data"
6 final_out = os.path.join(input_dir, "yogurt_estimation_sample.parquet")
7 temp_dir = os.path.join(input_dir, "temp_years")
```

**Read this block:**

- Line 1: import `glob`. An as-name is a convenient local alias, not another library.
- Line 2: import `os`. An as-name is a convenient local alias, not another library.
- Line 3: import `polars as pl`. An as-name is a convenient local alias, not another library.
- Line 5, `input_dir`: Set this name to the literal text `'/scratch/dtm63837/Kilts_Panel/Data'`.
- Line 6, `final_out`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 7, `temp_dir`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.

Source lines 8–16:

```
8 os.makedirs(temp_dir, exist_ok=True)
9
10 composite_keys = ["panel_year", "week_end", "upc", "store_code_uc"]
11
12 # 1. Load RMS with explicit column projection (ONLY load what you need)
```

```
13 print("[STEP 1/3] Materializing unique Yogurt RMS keys...", flush=True)
14
15 # Define exact RMS columns needed (join keys + filter col + price/movement cols)
16 rms_needed_cols = composite_keys + ["product_module_code", "price"]
```

**Read this block:**

- Line 8: call `os.makedirs` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 10, `composite_keys`: Set this name to the list `['panel_year', 'week_end', 'upc', 'store_code_uc']`.
- Line 13: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 16, `rms_needed_cols`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

**Syntax and calls used here:**

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Source lines 18–31:

```
18 rms_yogurt = (
19     pl.scan_parquet(os.path.join(input_dir, "merged_rms.parquet"))
20     # Select ONLY required columns up front (drops ~85% of memory loading)
21     .select([c for c in rms_needed_cols])
22     # Filter immediately
23     .filter(
24         pl.col("product_module_code").is_in([3603, 3612])
25     )  # Update to your yogurt code
26     # Deduplicate on composite keys
27     .unique(subset=composite_keys, keep="first")
28     .collect()
29 )
30
31 print(f"--> Yogurt RMS loaded successfully: {len(rms_yogurt):,} rows.", flush=True)
```

**Read this block:**

- Line 18, `rms_yogurt`: Set this name to the result of calling `.collect(...)`; the arguments and preceding method chain are shown above.
- Line 31: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.is_in`: Build a Polars membership expression, often used for eligible household IDs or allowed labels.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `len`: Count elements/rows rather than summing their numerical values.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 34–39:

```
34 panel_years = list(range(2006, 2016))
35
36 print(
37     f"[STEP 2/3] Processing panel year-by-year across {len(panel_years)} years...",
38     flush=True,
39 )
```

**Read this block:**

- Line 34, `panel_years`: Set this name to the result of calling `list(...)`; the arguments and preceding method chain are shown above.
- Line 36: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `list`: Materialize iterable entries as an ordered Python list.
- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.

Source lines 41–61:

```
41 for idx, yr in enumerate(panel_years, 1):
42     yr_out = os.path.join(temp_dir, f"yogurt_{yr}.parquet")
43     print(
44         f"  --> [{idx}/{len(panel_years)}] Joining & streaming Year {yr}...",
45         flush=True,
46     )
47
48     # Scan HMS
49     hms_yr = pl.scan_parquet(os.path.join(input_dir, "merged_panel.parquet")).filter(
50         pl.col("panel_year") == yr
51     )
52
53     # Filter in-memory RMS to current year
54     rms_yr = rms_yogurt.filter(pl.col("panel_year") == yr)
55
56     # Stream inner join
57     hms_yr.join(
58         rms_yr.lazy(), on=composite_keys, how="inner", suffix="_rms"
59     ).sink_parquet(yr_out)
60
61     print(f"      [DONE] Saved Year {yr} chunk.", flush=True)
```

**Read this block:**

- Line 41: for each entry yielded by `enumerate(panel_years, 1)`, bind/unpack it into `(idx, yr)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 42, `yr_out`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 43: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 49, `hms_yr`: Set this name to the result of calling `.filter(...)`; the arguments and preceding method chain are shown above.
- Line 54, `rms_yr`: Set this name to the result of calling `rms_yogurt.filter(...)`; the arguments and preceding method chain are shown above.
- Line 57: call `.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 61: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `.lazy`: Wrap an eager Polars table as a lazy query so later operations can be planned together.

Source lines 64–68:

```
print(
    "\n[STEP 3/3] Combining yearly files into final estimation sample...",
    flush=True,
)
pl.scan_parquet(os.path.join(temp_dir, "yogurt_*.parquet")).sink_parquet(final_out)
```

**Read this block:**

- Line 64: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 68: call `.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 71–72:

```
for f in glob.glob(os.path.join(temp_dir, "yogurt_*.parquet")):
    os.remove(f)
```

**Read this block:**

- Line 71: for each entry yielded by `glob.glob(os.path.join(temp_dir, 'yogurt_*.parquet'))`, bind/unpack it into `f` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 72: call `os.remove` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.glob`: Find filesystem paths matching a wildcard pattern, which may include stale files unless the folder is controlled.
- `.remove`: Delete the named file; in these scripts it cleans generated temporary year chunks.

Source lines 73–74:

```
if os.path.exists(temp_dir):
    os.rmdir(temp_dir)
```

**Read this block:**

- Line 73: test `os.path.exists(temp_dir)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 74: call `os.rmdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- `.rmdir`: Remove an empty directory after its temporary files have been deleted.

Source lines 76–79:

```
print(
    f"\n[SUCCESS] Saved 10-year Yogurt estimation sample to: {final_out}",
    flush=True,
)
```

**Read this block:**

- Line 76: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 84–91:

```
84 print("\n" + "=" * 70, flush=True)
85 print("RUNNING ESTIMATION SAMPLE AUDIT DIAGNOSTICS", flush=True)
86 print("=" * 70, flush=True)
87
88 # Scan final dataset lazily for memory-efficient auditing
89 sample_scan = pl.scan_parquet(final_out)
90 schema = sample_scan.collect_schema()
91 col_names = schema.names()
```

**Read this block:**

- Line 84: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 85: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 86: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 89, `sample_scan`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 90, `schema`: Set this name to the result of calling `sample_scan.collect_schema(...)`; the arguments and preceding method chain are shown above.
- Line 91, `col_names`: Set this name to the result of calling `schema.names(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.
- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.

Source lines 94–104:

```
94 total_rows = sample_scan.select(pl.len()).collect().item()
95 unique_upcs = sample_scan.select(pl.col("upc").n_unique()).collect().item()
96 unique_stores = sample_scan.select(pl.col("store_code_uc").n_unique()).collect().item()
97
98 # Check household column dynamically depending on HMS schema
99 hh_col = next((c for c in ["household_code", "hh_id"] if c in col_names), None)
100 unique_hhs = (
101     sample_scan.select(pl.col(hh_col).n_unique()).collect().item() if hh_col else "N/A"
102 )
103
104 print("\n[1] RECORD COUNTS & ENTITY COVERAGE:", flush=True)
```

**Read this block:**

- Line 94, `total_rows`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 95, `unique_upcs`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 96, `unique_stores`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 99, `hh_col`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.
- Line 100, `unique_hhs`: Set this name to the true-branch value if `hh_col` holds, otherwise the else-branch value.
- Line 104: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.item`: Extract a scalar from a one-element array/one-cell result rather than passing a table wrapper onward.
- `.n_unique`: Count distinct Polars values within the relevant aggregation scope.
- `next`: Take the first item from an iterator, or the supplied default when no item matches.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 105–116:

```
105 print(f"    - Total Estimation Rows:        {total_rows:,}", flush=True)
106 print(
107     f"    - Unique Households Matched:    {unique_hhs:,}"
108     if isinstance(unique_hhs, int)
109     else f"    - Unique Households Matched:    {unique_hhs}",
110     flush=True,
111 )
112 print(f"    - Unique Yogurt UPCs Matched:   {unique_upcs:,}", flush=True)
113 print(f"    - Unique Retail Stores Matched: {unique_stores:,}", flush=True)
114
115 # --- DIAGNOSTIC 2: Join Key Integrity & Null Check ---
116 print("\n[2] JOIN KEY INTEGRITY (NULL CHECK):", flush=True)
```

**Read this block:**

- Line 105: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 106: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 112: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 113: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 116: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `isinstance`: Check a runtime type to select the appropriate conversion path.

Source lines 117–121:

```
117 null_counts = (
118     sample_scan.select([pl.col(k).null_count().alias(k) for k in composite_keys])
119     .collect()
120     .to_dict(as_series=False)
121 )
```

**Read this block:**

- Line 117, `null_counts`: Set this name to the result of calling `.to_dict(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.to_dict`: Convert table contents to a Python dictionary; output orientation/options determine its nested layout.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.

Source lines 123–125:

```
123 for key, count in null_counts.items():
124     null_pct = (count[0] / total_rows) * 100 if total_rows > 0 else 0
```

```
125     print(f"    - Nulls in '{key}': {count[0]:,} ({null_pct:.2f}%)", flush=True)
```

**Read this block:**

- Line 123: for each entry yielded by `null_counts.items()`, bind/unpack it into `(key, count)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 124, `null_pct`: Set this name to the true-branch value if `total_rows > 0` holds, otherwise the else-branch value.
- Line 125: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.

Source lines 128–134:

```
128 print("\n[3] PANEL YEAR DISTRIBUTION:", flush=True)
129 year_counts = (
130     sample_scan.group_by("panel_year")
131     .agg(pl.len().alias("count"))
132     .sort("panel_year")
133     .collect()
134 )
```

**Read this block:**

- Line 128: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 129, `year_counts`: Set this name to the result of calling `.collect(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.
- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.

Source lines 136–140:

```
136 for row in year_counts.iter_rows(named=True):
137     print(
138         f"    - Year {row['panel_year']}: {row['count']:,} observations",
139         flush=True,
140     )
```

**Read this block:**

- Line 136: for each entry yielded by `year_counts.iter_rows(named=True)`, bind/unpack it into `row` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 137: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 143–143:

```
143 price_col = next((c for c in ["price", "price_rms"] if c in col_names), None)
```

**Read this block:**

- Line 143, `price_col`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.

Source lines 144–178:

```
144 if price_col:
145     price_stats = (
146         sample_scan.select(
147             [
148                 pl.col(price_col).min().alias("min_price"),
149                 pl.col(price_col).mean().alias("mean_price"),
150                 pl.col(price_col).median().alias("median_price"),
151                 pl.col(price_col).max().alias("max_price"),
152                 pl.col(price_col).null_count().alias("null_price_count"),
153                 (pl.col(price_col) <= 0).sum().alias("zero_price_count"),
154             ]
155         )
156         .collect()
157         .to_dict(as_series=False)
158     )
159 
160     print(
161         f"\n[4] PRICE COLUMN DIAGNOSTICS ('{price_col}'):",
162         flush=True,
163     )
164     print(f"    - Min Price:    ${price_stats['min_price'][0]:.2f}", flush=True)
165     print(f"    - Mean Price:   ${price_stats['mean_price'][0]:.2f}", flush=True)
166     print(
167         f"    - Median Price: ${price_stats['median_price'][0]:.2f}",
168         flush=True,
169     )
170     print(f"    - Max Price:    ${price_stats['max_price'][0]:.2f}", flush=True)
171     print(
172         f"    - Null Prices:  {price_stats['null_price_count'][0]:,}",
173         flush=True,
174     )
175     print(
176         f"    - Zero/Negative:{price_stats['zero_price_count'][0]:,}",
177         flush=True,
178     )
```

**Read this block:**

- Line 144: test `price_col`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 145, `price_stats`: Set this name to the result of calling `.to_dict(...)`; the arguments and preceding method chain are shown above.
- Line 160: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 164: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 165: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 166: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 170: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 171: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 175: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.min`: Take the smallest value, or a per-group/axis minimum when used on a table expression.

- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.
- `.median`: Compute the middle-value statistic rather than a mean; global medians cannot generally be averaged across batches.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.

Source lines 181–188:

```
181 file_size_mb = os.path.getsize(final_out) / (1024**2)
182
183 print("\n[5] OUTPUT FILE SUMMARY:", flush=True)
184 print(f"    - Total Schema Columns: {len(col_names)}", flush=True)
185 print(f"    - File Size on Disk:    {file_size_mb:.2f} MB", flush=True)
186 print(f"    - Output Location:      {final_out}", flush=True)
187
188 print("=" * 70, flush=True)
```

**Read this block:**

- Line 181, `file_size_mb`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 183: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 184: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 185: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 186: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 188: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.getsize`: Read filesystem byte size, later divided by powers of 1024 for MiB/GiB display.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.

Source lines 189–189:

```
189 print("AUDIT COMPLETE — Yogurt estimation sample verified.", flush=True)
```

**Read this block:**

- Line 189: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

# data_code/cleaning_merging/panel_concat.py

This stacks annual cleaned panel tables into combined files. `concat_list` is an ordinary list of three table names. In the loop, an f-string builds a wildcard such as `cleaned_trips_*.parquet`. A lazy scan over those files followed by `sink_parquet` writes one combined table. It does not add rows by matching keys: concatenation stacks records, whereas a join combines columns through matches. There is no deduplication or explicit year reconciliation here. A stale matching file is therefore part of the input unless the directory is audited.

## Module setup and direct-execution blocks

Source lines 1–2:

```
1 import os
2 import polars as pl
```

**Read this block:**

- Line 1: import `os`. An as-name is a convenient local alias, not another library.
- Line 2: import `polars as pl`. An as-name is a convenient local alias, not another library.

Source lines 6–8:

```
6 base_out_dir = "/scratch/dtm63837/Kilts_Panel/Data"
7 final_dir = os.path.join(base_out_dir, "combined")
8 os.makedirs(final_dir, exist_ok=True)
```

**Read this block:**

- Line 6, `base_out_dir`: Set this name to the literal text `'/scratch/dtm63837/Kilts_Panel/Data'`.
- Line 7, `final_dir`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 8: call `os.makedirs` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.

Source lines 12–12:

```
12 concat_list = ["panelists", "purchases", "trips"]
```

**Read this block:**

- Line 12, `concat_list`: Set this name to the list `['panelists', 'purchases', 'trips']`.

Source lines 13–20:

```
13 for table in concat_list:
14     file_pattern = os.path.join(base_out_dir, f"cleaned_{table}_*.parquet")
15     concat_out = os.path.join(final_dir, f"{table}.parquet")
16 
17     print(f"Combining {table} files...")
18 
19     (pl.scan_parquet(file_pattern).sink_parquet(concat_out))
20     print(f"Saved: {concat_out}")
```

**Read this block:**

- Line 13: for each entry yielded by `concat_list`, bind/unpack it into `table` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 14, `file_pattern`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 15, `concat_out`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 17: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 19: call `pl.scan_parquet(file_pattern).sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 20: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

# data_code/cleaning_merging/panel_convert.py

This script runs at module level: importing it can perform data work. First the imports make filesystem and Polars tools available; path strings select raw/server locations. Integer switches `annual` and `master` are truth-tested to decide which conversion loops run. `range(2006, 2016)` includes 2006 through 2015, excluding the upper endpoint. Dictionaries translate source table names to output stems. Nested loops construct one TSV/Parquet pair per year/table, and `exists` prevents trying to read missing files.

After conversion, wildcard scans discover Parquet files. `columns_to_keep` is a dictionary of table-specific projection lists, sometimes containing Polars alias expressions. `split("_")[0]` extracts a table prefix before selecting its projection. Scanning and sinking keep large tables out of ordinary Python lists; eagerly reading every annual table would need much more memory. Lowercasing column names standardizes downstream joins but does not reconcile every schema difference or verify raw-codebook meaning. No CLI parser protects the hard-coded paths; inspect them before running.

## Module setup and direct-execution blocks

Source lines 2–4:

```
2 import glob
3 import os
4 import polars as pl
```

**Read this block:**

- Line 2: import `glob`. An as-name is a convenient local alias, not another library.
- Line 3: import `os`. An as-name is a convenient local alias, not another library.
- Line 4: import `polars as pl`. An as-name is a convenient local alias, not another library.

Source lines 8–14:

```
8 annual = 0
9 master = 0
10
11 years = range(2006, 2016)
12 base_raw_dir = "/scratch/dtm63837/Kilts_Panel/nielsen_extracts/nielsen_extracts/nielsen_extracts/HMS"
13 base_out_dir = "/scratch/dtm63837/Kilts_Panel/Data"
14 os.makedirs(base_out_dir, exist_ok=True)
```

**Read this block:**

- Line 8, `annual`: Set this name to the literal value `0`.
- Line 9, `master`: Set this name to the literal value `0`.
- Line 11, `years`: Set this name to the result of calling `range(...)`; the arguments and preceding method chain are shown above.
- Line 12, `base_raw_dir`: Set this name to the literal text `'/scratch/dtm63837/Kilts_Panel/nielsen_extracts/nielsen_extracts/`.
- Line 13, `base_out_dir`: Set this name to the literal text `'/scratch/dtm63837/Kilts_Panel/Data'`.
- Line 14: call `os.makedirs` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.

Source lines 16–21:

```
base_mapping = {
    "panelists": "panelists",
    "purchases": "purchases",
    "trips": "trips",
}
master_mappings = {"products": "products", "retailers": "retailers"}
```

**Read this block:**

- Line 16, `base_mapping`: Set this name to a dictionary with named entries `panelists`, `purchases`, `trips`.
- Line 21, `master_mappings`: Set this name to a dictionary with named entries `products`, `retailers`.

Source lines 24–46:

```
if annual:
    for y in years:
        print(f"\n--- Converting year {y} ---")

        for base_tsv, out_name in base_mapping.items():
            tsv_filename = f"{base_tsv}_{y}.tsv"
            tsv_path = os.path.join(base_raw_dir, str(y), "Annual_Files", tsv_filename)
            parquet_out = os.path.join(base_out_dir, f"{out_name}_{y}.parquet")

            if os.path.exists(tsv_path):
                (
                    pl.scan_csv(
                        tsv_path,
                        separator="\t",
                        quote_char=None,
                        infer_schema_length=10000,
                    )
                    .rename(str.lower)
                    .sink_parquet(parquet_out)
                )
                print(f"Converted: {tsv_filename} -> {out_name}_{y}.parquet")
            else:
                print(f"WARNING: File not found {tsv_path}")
```

**Read this block:**

- Line 24: test `annual`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 25: for each entry yielded by `years`, bind/unpack it into `y` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 26: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 28: for each entry yielded by `base_mapping.items()`, bind/unpack it into `(base_tsv, out_name)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 29, `tsv_filename`: Set this name to formatted text, substituting the brace-enclosed values into the f-string.
- Line 30, `tsv_path`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 31, `parquet_out`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 33: test `os.path.exists(tsv_path)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 34: call `.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 44: call `print` with the displayed arguments. No assignment stores its return here; its documented side

effect or assertion is what matters.
- Line 46: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- `str`: Convert to text for paths, messages, or serialization.
- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `.rename`: Change column labels (or apply a renaming function); this does not change their underlying values.
- `.scan_csv`: Build a lazy delimited-text read plan; parser options determine types and missing values.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 47–64:

```
47 if master:
48     for base_tsv, out_name in master_mappings.items():
49         tsv_filename = f"{base_tsv}.tsv"
50         tsv_path = os.path.join(base_raw_dir, "Master_Files/Latest", tsv_filename)
51         parquet_out = os.path.join(base_out_dir, f"{out_name}.parquet")
52 
53         if os.path.exists(tsv_path):
54             (
55                 pl.scan_csv(
56                     tsv_path,
57                     separator="\t",
58                     quote_char=None,
59                     infer_schema_length=10000,
60                     encoding="utf8-lossy",
61                 )
62                 .rename(str.lower)
63                 .sink_parquet(parquet_out)
64             )
```

**Read this block:**

- Line 47: test `master`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 48: for each entry yielded by `master_mappings.items()`, bind/unpack it into `(base_tsv, out_name)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 49, `tsv_filename`: Set this name to formatted text, substituting the brace-enclosed values into the f-string.
- Line 50, `tsv_path`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 51, `parquet_out`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 53: test `os.path.exists(tsv_path)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 54: call `.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 68–68:

```
68 parquet_files = glob.glob(os.path.join(base_out_dir, "*.parquet"))
```

**Read this block:**

- Line 68, `parquet_files`: Set this name to the result of calling `glob.glob(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.glob`: Find filesystem paths matching a wildcard pattern, which may include stale files unless the folder is controlled.

Source lines 70–74:

```
70 for filepath in sorted(parquet_files):
71     filename = os.path.basename(filepath)
72     cols = pl.scan_parquet(filepath).collect_schema().names()
73     print(f"\n--- {filename} ({len(cols)} columns) ---")
74     print(cols)
```

**Read this block:**

- Line 70: for each entry yielded by `sorted(parquet_files)`, bind/unpack it into `filepath` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 71, `filename`: Set this name to the result of calling `os.path.basename(...)`; the arguments and preceding method chain are shown above.
- Line 72, `cols`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.
- Line 73: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 74: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.
- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.
- `len`: Count elements/rows rather than summing their numerical values.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.

Source lines 76–116:

```
76 columns_to_keep = {
77     "panelists": [
78         pl.col("household_cd").alias("household_code"),
79         "panel_year",
80         "household_income",
81         "household_size",
82         "male_head_age",
83         "female_head_age",
84         "male_head_employment",
85         "female_head_employment",
86         "race",
87         "hispanic_origin",
88         "dma_cd",
89         "dma_name",
90     ],
91     "purchases": [
92         "trip_code_uc",
93         "upc",
94         "quantity",
95         "total_price_paid",
96         "deal_flag_uc",
97     ],
```

```
    "trips": [
        "trip_code_uc",
        "household_code",
        "purchase_date",
        "store_code_uc",
        "panel_year",
    ],
    "products": [
        "upc",
        "product_module_code",
        "product_module_descr",
        "product_group_code",
        "product_group_descr",
        "brand_code_uc",
        "brand_descr",
        "size1_amount",
        "size1_units",
    ],
}
```

**Read this block:**

- Line 76, `columns_to_keep`: Set this name to a dictionary with named entries `panelists`, `purchases`, `trips`, `products`.

**Syntax and calls used here:**

- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.

Source lines 118–132:

```
for filepath in glob.glob(os.path.join(base_out_dir, "*.parquet")):
    filename = os.path.basename(filepath)

    # Remove .parquet extension first
    clean_name = os.path.splitext(filename)[0]  # e.g., "retailers" or "trips_2011"

    # Split by underscore if present to get the table name
    table_type = clean_name.split("_")[0]  # e.g., "retailers", "trips", "panelists"

    if table_type in columns_to_keep:
        target_cols = columns_to_keep[table_type]
        cleaned_path = os.path.join(base_out_dir, f"cleaned_{filename}")

        (pl.scan_parquet(filepath).select(target_cols).sink_parquet(cleaned_path))
        print(f"Cleaned {filename} -> saved to {cleaned_path}")
```

**Read this block:**

- Line 118: for each entry yielded by `glob.glob(os.path.join(base_out_dir, '*.parquet'))`, bind/unpack it into `filepath` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 119, `filename`: Set this name to the result of calling `os.path.basename(...)`; the arguments and preceding method chain are shown above.
- Line 122, `clean_name`: Set this name to the entry/slice selected by `os.path.splitext(filename)[0]`; brackets select from the object to their left.
- Line 125, `table_type`: Set this name to the entry/slice selected by `clean_name.split('_')[0]`; brackets select from the object to their left.
- Line 127: test `table_type in columns_to_keep`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 128, `target_cols`: Set this name to the entry/slice selected by `columns_to_keep[table_type]`; brackets select from the object to their left.

- Line 129, `cleaned_path`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 131: call `.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 132: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.

# data__code/cleaning__merging/panel__dat.py

This file is empty in the source snapshot: it contains no function or executable data-processing block to explain.

## Module setup and direct-execution blocks

There are no statements in this file.

# data__code/cleaning__merging/panel__merge.py

Purchase lines anchor the merge. Join-key casts align integer dtypes before an inner trip join, then left joins attach annual household and UPC product metadata. Method chaining expresses a sequence of relational operations whose result is sunk to `merged_panel.parquet`. An inner join drops unmatched purchase lines; a left join retains them but may add missing metadata. Duplicate right keys can multiply rows in either case.

The second half scans the output and prints diagnostics. `collect().item()` extracts a scalar from a one-cell query result; `to_dict(as_series=False)` makes a dictionary of Python lists for null-count reporting. `count[0]` picks its only row. File byte size divided by `1024**3` expresses GiB. The empty-output check raises, but a nonempty result is not proof of correct joins. Explicit key validation would be a stronger alternative to relying only on total row ratios.

## Module setup and direct-execution blocks

Source lines 1–9:

```
1 import os
2 import polars as pl
3
4 base_dir = "/scratch/dtm63837/Kilts_Panel/Data"
5 final_out = os.path.join(base_dir, "merged_panel.parquet")
6
7 ## === Load Data === ##
8 print("Loading lazy frames...")
9 panelists = pl.scan_parquet(os.path.join(base_dir, "combined/panelists.parquet"))
```

**Read this block:**

- Line 1: import `os`. An as-name is a convenient local alias, not another library.
- Line 2: import `polars as pl`. An as-name is a convenient local alias, not another library.
- Line 4, `base_dir`: Set this name to the literal text `'/scratch/dtm63837/Kilts_Panel/Data'`.
- Line 5, `final_out`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 8: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 9, `panelists`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.

Source lines 10–12:

```
10 purchases = pl.scan_parquet(os.path.join(base_dir, "combined/purchases.parquet"))
11 trips = pl.scan_parquet(os.path.join(base_dir, "combined/trips.parquet"))
12 products = pl.scan_parquet(os.path.join(base_dir, "cleaned_products.parquet"))
```

**Read this block:**

- Line 10, `purchases`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 11, `trips`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 12, `products`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.

Source lines 16–30:

```
16 purchases = purchases.with_columns(
17     [pl.col("trip_code_uc").cast(pl.Int64), pl.col("upc").cast(pl.Int64)]
18 )
19
20 trips = trips.with_columns(
21     [
22         pl.col("trip_code_uc").cast(pl.Int64),
23         pl.col("household_code").cast(pl.Int64),
24         pl.col("panel_year").cast(pl.Int32),
25     ]
26 )
27
28 panelists = panelists.with_columns(
29     [pl.col("household_code").cast(pl.Int64), pl.col("panel_year").cast(pl.Int32)]
30 )
```

**Read this block:**

- Line 16, `purchases`: Set this name to the result of calling `purchases.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 20, `trips`: Set this name to the result of calling `trips.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 28, `panelists`: Set this name to the result of calling `panelists.with_columns(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.

Source lines 32–44:

```
32 products = products.with_columns([pl.col("upc").cast(pl.Int64)])
33
34 ## === Merge === ##
35 merged = (
36     purchases.join(trips, on="trip_code_uc", how="inner")
37     .join(panelists, on=["household_code", "panel_year"], how="left")
38     .join(products, on="upc", how="left")
```

```
)

print("Merging purchases, trips, panelists, and products...")
print("Streaming to disk (this will take a few minutes)...")
merged.sink_parquet(final_out)
print(f"\n[SUCCESS] Successfully saved merged file to: {final_out}")
```

**Read this block:**

- Line 32, `products`: Set this name to the result of calling `products.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 35, `merged`: Set this name to the result of calling `.join(...)`; the arguments and preceding method chain are shown above.
- Line 41: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 42: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 43: call `merged.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 44: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 47–56:

```
print("\n" + "=" * 50)
print("RUNNING DIAGNOSTICS ON MERGED DATASET")
print("=" * 50)

# 1. Scan output dataset lazily
merged_scan = pl.scan_parquet(final_out)

# 2. Total Row Count Check
total_rows = merged_scan.select(pl.len()).collect().item()
print(f"\n[1] Total Row Count: {total_rows:,}")
```

**Read this block:**

- Line 47: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 48: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 49: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 52, `merged_scan`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 55, `total_rows`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 56: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.item`: Extract a scalar from a one-element array/one-cell result rather than passing a table wrapper onward.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.

- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.

Source lines 58–59:

```
58 if total_rows == 0:
59     raise ValueError("CRITICAL ERROR: Merged dataset contains 0 rows! Check join keys.")
```

**Read this block:**

- Line 58: test `total_rows == 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 59: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

**Syntax and calls used here:**

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 62–69:

```
62 purchases_rows = purchases.select(pl.len()).collect().item()
63 pct_retained = (total_rows / purchases_rows) * 100
64 print(f"    - Anchor purchases rows: {purchases_rows:,}")
65 print(f"    - Retained post-join: {pct_retained:.2f}%")
66
67 # 4. Null / NA Check across key columns
68 print("\n[2] Null Value Diagnostics (Key Columns):")
69 key_cols = ["trip_code_uc", "household_code", "upc", "panel_year"]
```

**Read this block:**

- Line 62, `purchases_rows`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 63, `pct_retained`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 64: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 65: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 68: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 69, `key_cols`: Set this name to the list `['trip_code_uc', 'household_code', 'upc', 'panel_year']`.

Source lines 70–76:

```
70 existing_keys = [col for col in key_cols if col in merged_scan.collect_schema().names()]
71
72 null_counts = (
73     merged_scan.select([pl.col(c).null_count().alias(c) for c in existing_keys])
74     .collect()
75     .to_dict(as_series=False)
76 )
```

**Read this block:**

- Line 70, `existing_keys`: Set this name to a list computed by the displayed for-expression, retaining only entries passing its if filter.
- Line 72, `null_counts`: Set this name to the result of calling `.to_dict(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.

- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `.to_dict`: Convert table contents to a Python dictionary; output orientation/options determine its nested layout.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.

Source lines 78–80:

```
78 for col, count in null_counts.items():
79     null_pct = (count[0] / total_rows) * 100
80     print(f"    - {col}: {count[0]:,} nulls ({null_pct:.2f}%)")
```

**Read this block:**

- Line 78: for each entry yielded by `null_counts.items()`, bind/unpack it into `(col, count)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 79, `null_pct`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 80: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.

Source lines 83–89:

```
83 file_size_gb = os.path.getsize(final_out) / (1024**3)
84 schema_cols = merged_scan.collect_schema().names()
85
86 print("\n[3] Output File Summary:")
87 print(f"    - Total Columns: {len(schema_cols)}")
88 print(f"    - Disk Size: {file_size_gb:.2f} GB")
89 print(f"    - Columns: {schema_cols}")
```

**Read this block:**

- Line 83, `file_size_gb`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 84, `schema_cols`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.
- Line 86: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 87: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 88: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 89: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.getsize`: Read filesystem byte size, later divided by powers of 1024 for MiB/GiB display.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 90–91:

```
90 print("=" * 50)
91 print("DIAGNOSTICS COMPLETE - File is verified and ready.")
```

**Read this block:**

- Line 90: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 91: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

# data_code/cleaning_merging/panel_week.py

This adds a Saturday-ending integer key to the merged panel. The nested expression parses purchase-date strings, extracts weekdays, computes a duration, adds it, formats `%Y%m%d`, and casts to Int64. Read inside out. With Polars' Monday=1 through Sunday=7 convention, `6-(weekday % 7)` gives Monday five days, Saturday zero, and Sunday six days. It agrees with the downstream modulo-seven formula over those weekday values.

The query writes a separate temporary file before `os.replace` replaces the original. Writing directly to the source path while scanning it could corrupt the read; staging then replacement avoids that pattern. It still overwrites the original file after success, so this is not an inspection-only script.

## Module setup and direct-execution blocks

Source lines 1–8:

```
1 import os
2 import polars as pl
3
4 input_dir = "/scratch/dtm63837/Kilts_Panel/Data"
5 panel_path = os.path.join(input_dir, "merged_panel.parquet")
6 panel_temp = os.path.join(input_dir, "merged_panel_updated.parquet")
7
8 print("Pre-computing week_end on merged_panel.parquet...")
```

**Read this block:**

- Line 1: import `os`. An as-name is a convenient local alias, not another library.
- Line 2: import `polars as pl`. An as-name is a convenient local alias, not another library.
- Line 4, `input_dir`: Set this name to the literal text `'/scratch/dtm63837/Kilts_Panel/Data'`.
- Line 5, `panel_path`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 6, `panel_temp`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 8: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Source lines 10–24:

```
10 (
11     pl.scan_parquet(panel_path)
12     .with_columns(
13         # Convert "YYYY-MM-DD" to Date, compute Saturday week-ending, format to YYYYMMDD Int64
14         week_end=(
15             pl.col("purchase_date").str.to_date()
16             + pl.duration(
17                 days=(6 - (pl.col("purchase_date").str.to_date().dt.weekday() % 7))
18             )
19         )
20         .dt.strftime("%Y%m%d")
21         .cast(pl.Int64)
```

```
    )
    .sink_parquet(panel_temp)
)
```

**Read this block:**

- Line 10: call `.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- `.strftime`: Format a parsed date according to directives, e.g. %Y%m%d for an eight-digit week-ending key.
- `.to_date`: Parse text into calendar-date values before weekday extraction and date arithmetic.
- `.duration`: Build a date/time duration from the named unit, allowing a different day offset per row.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.weekday`: Extract a Polars weekday number, Monday=1 through Sunday=7, for the Saturday offset calculation.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.

Source lines 27–28:

```
os.replace(panel_temp, panel_path)
print("Updated merged_panel.parquet successfully with Int64 'week_end'!")
```

**Read this block:**

- Line 27: call `os.replace` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 28: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.

# data__code/cleaning__merging/retail__concat.py

This scans annual retail outputs, harmonizes selected dtypes, and vertically concatenates tables. `glob.glob` returns matching filenames; `sorted` supplies deterministic order. A missing match list triggers `continue`, skipping just that target rather than the whole script. The list comprehension creates casts only for columns actually present in a particular schema.

`how="vertical_relaxed"` allows compatible dtype coercion when stacking frames; it is not permission to ignore arbitrary incompatible schemas. Explicit casts for flavor/size codes and join IDs reduce annual type conflicts. Keeping codes as strings can preserve nonnumeric categories that an integer cast would reject.

## Module setup and direct-execution blocks

Source lines 1–7:

```
import glob
import os
import polars as pl
```

```
4
5 base_out_dir = "/scratch/dtm63837/Kilts_Panel/Data/RMS"
6 final_dir = os.path.join(base_out_dir, "combined")
7 os.makedirs(final_dir, exist_ok=True)
```

**Read this block:**

- Line 1: import `glob`. An as-name is a convenient local alias, not another library.
- Line 2: import `os`. An as-name is a convenient local alias, not another library.
- Line 3: import `polars as pl`. An as-name is a convenient local alias, not another library.
- Line 5, `base_out_dir`: Set this name to the literal text `'/scratch/dtm63837/Kilts_Panel/Data/RMS'`.
- Line 6, `final_dir`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 7: call `os.makedirs` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.

Source lines 9–15:

```
9 concat_targets = {
10     "stores": "cleaned_stores_*.parquet",
11     "products_extra": "cleaned_products_extra_*.parquet",
12     "3612": "cleaned_3612_*.parquet",
13     "3603": "cleaned_3603_*.parquet",
14     "rms": "rms_*.parquet",
15 }
```

**Read this block:**

- Line 9, `concat_targets`: Set this name to a dictionary with named entries `stores`, `products_extra`, `3612`, `3603`, `rms`.

Source lines 18–24:

```
18 CAST_SCHEMA = {
19     "flavor_code": pl.String,
20     "size1_code_uc": pl.String,
21     "upc": pl.Int64,
22     "store_code_uc": pl.Int64,
23     "panel_year": pl.Int32,
24 }
```

**Read this block:**

- Line 18, `CAST_SCHEMA`: Set this name to a dictionary with named entries `flavor_code`, `size1_code_uc`, `upc`, `store_code_uc`, `panel_year`.

Source lines 26–57:

```
26 for table, pattern in concat_targets.items():
27     file_pattern = os.path.join(base_out_dir, pattern)
28     matching_files = sorted(glob.glob(file_pattern))
29
30     if not matching_files:
31         print(f"WARNING: No files found matching {file_pattern}. Skipping {table}...")
32         continue
33
34     concat_out = os.path.join(final_dir, f"{table}.parquet")
35     print(f"Combining {len(matching_files)} files for '{table}'...")
36
37     # Load files individually and align column types to avoid SchemaError
```

```
38      lazy_frames = []
39      for f in matching_files:
40          lf = pl.scan_parquet(f)
41
42          # Cast any problematic columns present in this table schema
43          schema_names = lf.collect_schema().names()
44          cast_exprs = [
45              pl.col(col).cast(target_type)
46              for col, target_type in CAST_SCHEMA.items()
47              if col in schema_names
48          ]
49
50          if cast_exprs:
51              lf = lf.with_columns(cast_exprs)
52
53          lazy_frames.append(lf)
54
55      # Concatenate using relaxed type resolution
56      (pl.concat(lazy_frames, how="vertical_relaxed").sink_parquet(concat_out))
57      print(f"Saved: {concat_out}")
```

**Read this block:**

- Line 26: for each entry yielded by `concat_targets.items()`, bind/unpack it into `(table, pattern)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 27, `file_pattern`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 28, `matching_files`: Set this name to the result of calling `sorted(...)`; the arguments and preceding method chain are shown above.
- Line 30: test `not matching_files`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 31: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 32: skip the rest of this iteration and continue the nearest enclosing loop.
- Line 34, `concat_out`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 35: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 38, `lazy_frames`: Set this name to the list `[]`.
- Line 39: for each entry yielded by `matching_files`, bind/unpack it into `f` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 40, `lf`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 43, `schema_names`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.
- Line 44, `cast_exprs`: Set this name to a list computed by the displayed for-expression, retaining only entries passing its if filter.
- Line 50: test `cast_exprs`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 51, `lf`: Set this name to the result of calling `lf.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 53: call `lazy_frames.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 56: call `.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 57: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `.glob`: Find filesystem paths matching a wildcard pattern, which may include stale files unless the folder is controlled.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `len`: Count elements/rows rather than summing their numerical values.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

# data_code/cleaning_merging/retail_convert.py

This is the retail analogue of panel conversion. Multiple mapping dictionaries separate annual metadata, movement modules, and master files. All are ordinary Python loops around lazy TSV reads and Parquet writes. `quote_char=None` disables CSV-style quote interpretation for TSV inputs. Schema inference inspects a prefix, so later inconsistent values may still need attention.

Cleaning keys are sorted by descending length, and a boundary-aware prefix test distinguishes `products_extra` from `products`. This is a concrete reason for the `sorted(..., key=len, reverse=True)` choice: testing the shorter prefix first could select the wrong projection. `continue` skips already cleaned files; `break` stops the key-search loop after a match. A `try/except` around schema inspection reports unreadable files without aborting all inspection.

## Module setup and direct-execution blocks

Source lines 1–7:

```
1 import glob
2 import os
3 import polars as pl
4
5 input_dir = "/scratch/dtm63837/Kilts_Panel/nielsen_extracts/nielsen_extracts/RMS"
6 output_dir = "/scratch/dtm63837/Kilts_Panel/Data/RMS"
7 os.makedirs(output_dir, exist_ok=True)
```

**Read this block:**

- Line 1: import `glob`. An as-name is a convenient local alias, not another library.
- Line 2: import `os`. An as-name is a convenient local alias, not another library.
- Line 3: import `polars as pl`. An as-name is a convenient local alias, not another library.
- Line 5, `input_dir`: Set this name to the literal text `'/scratch/dtm63837/Kilts_Panel/nielsen_extracts/nielsen_extracts/`.
- Line 6, `output_dir`: Set this name to the literal text `'/scratch/dtm63837/Kilts_Panel/Data/RMS'`.
- Line 7: call `os.makedirs` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 9–23:

```
annual = 1
master = 1

years = range(2006, 2016)

annual_mapping = {
    "products_extra": "products_extra",
    "rms_versions": "rms",
    "stores": "stores",
}
movement_mapping = {
    "3603": "3603",
    "3612": "3612",
}
master_mapping = {"brand_variations": "brand_variations", "products": "products"}
```

**Read this block:**

- Line 9, `annual`: Set this name to the literal value `1`.
- Line 10, `master`: Set this name to the literal value `1`.
- Line 12, `years`: Set this name to the result of calling `range(...)`; the arguments and preceding method chain are shown above.
- Line 14, `annual_mapping`: Set this name to a dictionary with named entries `products_extra`, `rms_versions`, `stores`.
- Line 19, `movement_mapping`: Set this name to a dictionary with named entries `3603`, `3612`.
- Line 23, `master_mapping`: Set this name to a dictionary with named entries `brand_variations`, `products`.

**Syntax and calls used here:**

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.

Source lines 25–66:

```
if annual:
    for y in years:
        for base_mapping, out_name in annual_mapping.items():
            tsv_filename = f"{base_mapping}_{y}.tsv"
            tsv_path = os.path.join(input_dir, str(y), "Annual_Files", tsv_filename)
            parquet_out = os.path.join(output_dir, f"{out_name}_{y}.parquet")

            if os.path.exists(tsv_path):
                (
                    pl.scan_csv(
                        tsv_path,
                        separator="\t",
                        quote_char=None,
                        infer_schema_length=10000,
                    )
                    .rename(str.lower)
                    .sink_parquet(parquet_out)
                )
                print(f"Converted: {tsv_filename} -> {out_name}_{y}.parquet")
            else:
                print(f"FAILED: File not found {tsv_filename}")
        for base_mapping, out_name in movement_mapping.items():
            tsv_filename = f"{base_mapping}_{y}.tsv"
            tsv_path = os.path.join(
                input_dir, str(y), f"Movement_Files/2510_{y}", tsv_filename
            )
            parquet_out = os.path.join(output_dir, f"{out_name}_{y}.parquet")

            if os.path.exists(tsv_path):
```

```
54                (
55                    pl.scan_csv(
56                        tsv_path,
57                        separator="\t",
58                        quote_char=None,
59                        infer_schema_length=10000,
60                    )
61                    .rename(str.lower)
62                    .sink_parquet(parquet_out)
63                )
64                print(f"Converted: {tsv_filename} -> {out_name}_{y}.parquet")
65            else:
66                print(f"FAILED: File not found {tsv_filename}")
```

**Read this block:**

- Line 25: test `annual`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 26: for each entry yielded by `years`, bind/unpack it into `y` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 27: for each entry yielded by `annual_mapping.items()`, bind/unpack it into `(base_mapping, out_name)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 28, `tsv_filename`: Set this name to formatted text, substituting the brace-enclosed values into the f-string.
- Line 29, `tsv_path`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 30, `parquet_out`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 32: test `os.path.exists(tsv_path)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 33: call `.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 43: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 45: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 46: for each entry yielded by `movement_mapping.items()`, bind/unpack it into `(base_mapping, out_name)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 47, `tsv_filename`: Set this name to formatted text, substituting the brace-enclosed values into the f-string.
- Line 48, `tsv_path`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 51, `parquet_out`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 53: test `os.path.exists(tsv_path)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 54: call `.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 64: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 66: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.

- `str`: Convert to text for paths, messages, or serialization.
- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.rename`: Change column labels (or apply a renaming function); this does not change their underlying values.
- `.scan_csv`: Build a lazy delimited-text read plan; parser options determine types and missing values.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 68–90:

```
68 if master:
69     for base_mapping, out_name in master_mapping.items():
70         tsv_filename = f"{base_mapping}.tsv"
71         tsv_path = os.path.join(
72             input_dir, "Master_Files_2006-2020/Latest", tsv_filename
73         )
74         parquet_out = os.path.join(output_dir, f"{out_name}.parquet")
75 
76         if os.path.exists(tsv_path):
77             (
78                 pl.scan_csv(
79                     tsv_path,
80                     separator="\t",
81                     quote_char=None,
82                     infer_schema_length=10000,
83                     encoding="utf8-lossy",
84                 )
85                 .rename(str.lower)
86                 .sink_parquet(parquet_out)
87             )
88             print(f"Converted: {tsv_filename} -> {out_name}.parquet")
89         else:
90             print(f"FAILED: File not found {tsv_filename}")
```

**Read this block:**

- Line 68: test `master`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 69: for each entry yielded by `master_mapping.items()`, bind/unpack it into `(base_mapping, out_name)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 70, `tsv_filename`: Set this name to formatted text, substituting the brace-enclosed values into the f-string.
- Line 71, `tsv_path`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 74, `parquet_out`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 76: test `os.path.exists(tsv_path)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 77: call `.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 88: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 90: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 93–93:

```
93 parquet_files = glob.glob(os.path.join(output_dir, "*.parquet"))
```

**Read this block:**

- Line 93, `parquet_files`: Set this name to the result of calling `glob.glob(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.glob`: Find filesystem paths matching a wildcard pattern, which may include stale files unless the folder is controlled.

Source lines 95–108:

```
95 for filepath in sorted(parquet_files):
96     filename = os.path.basename(filepath)
97 
98     # Check if file is empty before reading
99     if os.path.getsize(filepath) == 0:
100         print(f"\n--- {filename} is EMPTY (0 bytes), skipping ---")
101         continue
102 
103     try:
104         cols = pl.scan_parquet(filepath).collect_schema().names()
105         print(f"\n--- {filename} ({len(cols)} columns) ---")
106         print(cols)
107     except Exception as e:
108         print(f"\n--- Could not read {filename} (corrupted or partial file): {e} ---")
```

**Read this block:**

- Line 95: for each entry yielded by `sorted(parquet_files)`, bind/unpack it into `filepath` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 96, `filename`: Set this name to the result of calling `os.path.basename(...)`; the arguments and preceding method chain are shown above.
- Line 99: test `os.path.getsize(filepath) == 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 100: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 101: skip the rest of this iteration and continue the nearest enclosing loop.
- Line 103: try the indented operation; matching except blocks handle specified exceptions. Unhandled error types still propagate instead of being silently treated as success.
- Line 104, `cols`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.
- Line 105: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 106: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 107: catch `Exception` from the preceding try; the optional as-name holds the error object for logging.
- Line 108: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.
- `.getsize`: Read filesystem byte size, later divided by powers of 1024 for MiB/GiB display.
- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.
- `len`: Count elements/rows rather than summing their numerical values.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.

Source lines 110–138:

```
110 columns_to_keep = {
111     "3603": ["store_code_uc", "upc", "week_end", "price"],
112     "3612": ["store_code_uc", "upc", "week_end", "price"],
113     "products": [
114         "upc",
115         "upc_ver_uc",
116         "product_module_code",
117         "brand_code_uc",
118         "brand_descr",
119         "size1_code_uc",
120         "size1_amount",
121         "size1_units",
122     ],
123     "products_extra": [
124         "upc",
125         "upc_ver_uc",
126         "panel_year",
127         "flavor_code",
128         "flavor_descr",
129     ],
130     "stores": [
131         "store_code_uc",
132         "year",  # Keep as string 'year' during initial extraction
133         "retailer_code",
134         "channel_code",
135         "dma_code",
136         "dma_descr",
137     ],
138 }
```

**Read this block:**

- Line 110, `columns_to_keep`: Set this name to a dictionary with named entries `3603`, `3612`, `products`, `products_extra`, `stores`.

Source lines 141–141:

```
141 table_keys_by_length = sorted(columns_to_keep.keys(), key=len, reverse=True)
```

**Read this block:**

- Line 141, `table_keys_by_length`: Set this name to the result of calling `sorted(...)`; the arguments and preceding method chain are shown above.

Source lines 143–167:

```
143 for filepath in glob.glob(os.path.join(output_dir, "*.parquet")):
144     filename = os.path.basename(filepath)
145 
146     # Skip files that have already been cleaned
147     if filename.startswith("cleaned_"):
148         continue
149 
150     # Remove extension -> "products_extra_2011"
151     clean_name = os.path.splitext(filename)[0]
152 
153     # Match against dictionary keys taking exact boundaries into account
154     matched_table = None
155     for key in table_keys_by_length:
156         if clean_name == key or clean_name.startswith(f"{key}_"):
157             matched_table = key
158             break
159 
```

```
160     if matched_table:
161         target_cols = columns_to_keep[matched_table]
162         cleaned_path = os.path.join(output_dir, f"cleaned_{filename}")
163 
164         (pl.scan_parquet(filepath).select(target_cols).sink_parquet(cleaned_path))
165         print(
166             f"Cleaned {filename} (Matched: {matched_table}) -> saved to cleaned_{filename}"
167         )
```

**Read this block:**

- Line 143: for each entry yielded by `glob.glob(os.path.join(output_dir, '*.parquet'))`, bind/unpack it into `filepath` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 144, `filename`: Set this name to the result of calling `os.path.basename(...)`; the arguments and preceding method chain are shown above.
- Line 147: test `filename.startswith('cleaned_')`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 148: skip the rest of this iteration and continue the nearest enclosing loop.
- Line 151, `clean_name`: Set this name to the entry/slice selected by `os.path.splitext(filename)[0]`; brackets select from the object to their left.
- Line 154, `matched_table`: Set this name to the literal value `None`.
- Line 155: for each entry yielded by `table_keys_by_length`, bind/unpack it into `key` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 156: test `clean_name == key or clean_name.startswith(f'{key}_')`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 157, `matched_table`: Set this name to the object currently named `key`; plain assignment does not copy it.
- Line 158: exit the nearest enclosing loop immediately, rather than merely skipping one iteration.
- Line 160: test `matched_table`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 161, `target_cols`: Set this name to the entry/slice selected by `columns_to_keep[matched_table]`; brackets select from the object to their left.
- Line 162, `cleaned_path`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 164: call `.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 165: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.

# data__code/cleaning__merging/retail__dat.py

This file is empty in the source snapshot: it contains no function or executable data-processing block to explain.

## Module setup and direct-execution blocks

There are no statements in this file.

# data__code/cleaning__merging/retail__merge.py

This concatenates movement modules and uses them as the retail anchor. Integer floor division by 10000 extracts a calendar year from a YYYYMMDD integer. Several metadata tables are deduplicated before left joins to avoid

multiplying movement rows. `suffix` names colliding non-key columns, while composite `on=[...]` lists specify exact matching keys.

First-row deduplication is a policy, not an inference of the correct UPC version. Extra-product sorting prioritizes nonnull flavor descriptions before retaining a UPC/year row. The diagnostics compare anchor and merged row counts and inspect nonnull metadata. A conditional expression chooses which brand field is available for a match diagnostic, so that diagnostic may inspect a source key rather than a successful dictionary match. Output is under `Data/RMS/`, unlike the current master script's `Data/merged_rms.parquet` input path.

## Module setup and direct-execution blocks

Source lines 1–7:

```
1 import os
2 import polars as pl
3
4 base_dir = "/scratch/dtm63837/Kilts_Panel/Data/RMS"
5 final_out = os.path.join(base_dir, "merged_rms.parquet")
6
7 print("Loading lazy frames...")
```

**Read this block:**

- Line 1: import `os`. An as-name is a convenient local alias, not another library.
- Line 2: import `polars as pl`. An as-name is a convenient local alias, not another library.
- Line 4, `base_dir`: Set this name to the literal text `'/scratch/dtm63837/Kilts_Panel/Data/RMS'`.
- Line 5, `final_out`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 7: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Source lines 10–22:

```
10 movements = pl.concat(
11     [
12         pl.scan_parquet(os.path.join(base_dir, "combined/3612.parquet")),
13         pl.scan_parquet(os.path.join(base_dir, "combined/3603.parquet")),
14     ]
15 ).with_columns(
16     [
17         pl.col("upc").cast(pl.Int64),
18         pl.col("store_code_uc").cast(pl.Int64),
19         # Integer division converts YYYYMMDD (e.g., 20120107) directly to YYYY (2012)
20         (pl.col("week_end") // 10000).cast(pl.Int32).alias("panel_year"),
21     ]
22 )
```

**Read this block:**

- Line 10, `movements`: Set this name to the result of calling `.with_columns(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.

- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- The // operator is floor division, used here for integer counts or calendar-year extraction.

Source lines 25–36:

```
25 rms = (
26     pl.scan_parquet(os.path.join(base_dir, "combined/rms.parquet"))
27     .with_columns([pl.col("upc").cast(pl.Int64), pl.col("upc_ver_uc").cast(pl.Int32)])
28     .unique(subset=["upc"], keep="first")
29 )
30 
31 # 3. Products table (deduplicated on exact UPC + Version Code composite key)
32 products = (
33     pl.scan_parquet(os.path.join(base_dir, "cleaned_products.parquet"))
34     .with_columns([pl.col("upc").cast(pl.Int64), pl.col("upc_ver_uc").cast(pl.Int32)])
35     .unique(subset=["upc", "upc_ver_uc"], keep="first")
36 )
```

**Read this block:**

- Line 25, `rms`: Set this name to the result of calling `.unique(...)`; the arguments and preceding method chain are shown above.
- Line 32, `products`: Set this name to the result of calling `.unique(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.

Source lines 39–49:

```
39 stores = (
40     pl.scan_parquet(os.path.join(base_dir, "combined/stores.parquet"))
41     .rename({"year": "panel_year"})  # Safely renames 'year' to 'panel_year'
42     .with_columns(
43         [
44             pl.col("store_code_uc").cast(pl.Int64),
45             pl.col("panel_year").cast(pl.Int32),
46         ]
47     )
48     .unique(subset=["store_code_uc", "panel_year"], keep="first")
49 )
```

**Read this block:**

- Line 39, `stores`: Set this name to the result of calling `.unique(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.rename`: Change column labels (or apply a renaming function); this does not change their underlying values.

Source lines 52–64:

```
52 products_extra = (
53     pl.scan_parquet(os.path.join(base_dir, "combined/products_extra.parquet"))
54     .with_columns([pl.col("upc").cast(pl.Int64), pl.col("panel_year").cast(pl.Int32)])
55     .sort("flavor_descr", descending=False, nulls_last=True)
56     .unique(subset=["upc", "panel_year"], keep="first")
57 )
58 
59 # 6. Brand Variations (deduplicated on brand code)
60 brand_var = (
61     pl.scan_parquet(os.path.join(base_dir, "brand_variations.parquet"))
```

```
    .with_columns(pl.col("brand_code_uc").cast(pl.Int64))
    .unique(subset=["brand_code_uc"], keep="first")
)
```

**Read this block:**

- Line 52, `products_extra`: Set this name to the result of calling `.unique(...)`; the arguments and preceding method chain are shown above.
- Line 60, `brand_var`: Set this name to the result of calling `.unique(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.

Source lines 67–81:

```
merged = (
    movements
    # Step A: Attach upc_ver_uc from RMS
    .join(rms, on="upc", how="left", suffix="_rms")
    # Step B: Match exact version in products
    .join(products, on=["upc", "upc_ver_uc"], how="left", suffix="_prod")
    # Step C: Match store metadata by store
    .join(stores, on=["store_code_uc", "panel_year"], how="left", suffix="_str")
    # Step D: Match extra annual metadata
    .join(products_extra, on=["upc", "panel_year"], how="left", suffix="_xtra")
    # Step E: Match master brand dictionary
    .join(brand_var, on="brand_code_uc", how="left", suffix="_brand")
)

print("Streaming merged RMS dataset to disk...")
```

**Read this block:**

- Line 67, `merged`: Set this name to the result of calling `.join(...)`; the arguments and preceding method chain are shown above.
- Line 81: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 82–90:

```
merged.sink_parquet(final_out)
print(f"Successfully saved: {final_out}")

## === Run Post-Merge Audit === ##
print("\n" + "=" * 60)
print("RUNNING POST-MERGE AUDIT: RMS DATASET")
print("=" * 60)

merged_scan = pl.scan_parquet(final_out)
```

**Read this block:**

- Line 82: call `merged.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 83: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 86: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 87: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 88: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 90, `merged_scan`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 93–99:

```
93 total_merged_rows = merged_scan.select(pl.len()).collect().item()
94 anchor_rows = movements.select(pl.len()).collect().item()
95 pct_retained = (total_merged_rows / anchor_rows) * 100
96
97 print(f"\n[1] Row Count & Integrity Audit:")
98 print(f"    - Movements Anchor Rows: {anchor_rows:,}")
99 print(f"    - Merged File Rows:     {total_merged_rows:,}")
```

**Read this block:**

- Line 93, `total_merged_rows`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 94, `anchor_rows`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 95, `pct_retained`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 97: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 98: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 99: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.item`: Extract a scalar from a one-element array/one-cell result rather than passing a table wrapper onward.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.

Source lines 100–100:

```
100 print(f"    - Row Ratio:           {pct_retained:.2f}%")
```

**Read this block:**

- Line 100: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 102–105:

```
102 if total_merged_rows != anchor_rows:
103     print("    ⚠  WARNING: Row count discrepancy detected! Check join keys.")
104 else:
105     print("    ✓ PERFECT MATCH: Exact 1:1 row retention (0 rows added or dropped).")
```

**Read this block:**

- Line 102: test `total_merged_rows != anchor_rows`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 103: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 105: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 108–116:

```
print("\n[2] Dimension Table Match Rates:")
match_targets = {
    "products (brand_descr)": "brand_descr",
    "stores (retailer_code)": "retailer_code",
    "products_extra (flavor_descr)": "flavor_descr",
    "brand_var": "brand_code_uc_brand"
    if "brand_code_uc_brand" in merged_scan.collect_schema().names()
    else "brand_code_uc",
}
```

**Read this block:**

- Line 108: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 109, `match_targets`: Set this name to a dictionary with named entries `products (brand_descr)`, `stores (retailer_code)`, `products_extra (flavor_descr)`, `brand_var`.

**Syntax and calls used here:**

- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 118–127:

```
for dim_name, col_name in match_targets.items():
    if col_name in merged_scan.collect_schema().names():
        matched_count = (
            merged_scan.filter(pl.col(col_name).is_not_null())
            .select(pl.len())
            .collect()
            .item()
        )
        match_pct = (matched_count / total_merged_rows) * 100
        print(f"    - {dim_name:<30}: {matched_count:,} matched ({match_pct:.2f}%)")
```

**Read this block:**

- Line 118: for each entry yielded by `match_targets.items()`, bind/unpack it into `(dim_name, col_name)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 119: test `col_name in merged_scan.collect_schema().names()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 120, `matched_count`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 126, `match_pct`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 127: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.

- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.is_not_null`: Create a Polars nonnull mask; additional range checks can still be needed.

Source lines 130–136:

```
130 file_size_gb = os.path.getsize(final_out) / (1024**3)
131 schema_cols = merged_scan.collect_schema().names()
132
133 print("\n[3] Output File Summary:")
134 print(f"    - Total Columns: {len(schema_cols)}")
135 print(f"    - File Size:     {file_size_gb:.2f} GB")
136 print(f"    - Output Path:   {final_out}")
```

**Read this block:**

- Line 130, `file_size_gb`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 131, `schema_cols`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.
- Line 133: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 134: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 135: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 136: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.getsize`: Read filesystem byte size, later divided by powers of 1024 for MiB/GiB display.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 137–138:

```
137 print("=" * 60)
138 print("AUDIT COMPLETE.")
```

**Read this block:**

- Line 137: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 138: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

# data_code/recover_flavors.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

## Module setup and direct-execution blocks

Source lines 1–6:

```
1 """Supplement the flavor CSV from annual RMS product metadata on the server."""
2 import argparse
3 from pathlib import Path
4
```

```
5
6 from summ_stats import DATA_DIR, FLAVOR_FILE, recover_flavor_reference as recover
```

**Read this block:**

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `argparse`. An as-name is a convenient local alias, not another library.
- Line 3: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 6: from `summ_stats`, import `DATA_DIR, FLAVOR_FILE, recover_flavor_reference as recover` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 20–21:

```
20 if __name__ == "__main__":
21     main()
```

**Read this block:**

- Line 20: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 21: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

## main

Original function: `main`. Source begins at line 11.

This small command-line wrapper imports the existing recovery function under the shorter local name `recover`. `as recover` changes only the imported name, not the implementation. Three Path arguments specify the original CSV, annual products, and output file; `recover(...)` does the actual work. Reusing that function avoids a second recovery algorithm drifting away from the summary pipeline. The guarded entry point lets tests import the module without running the recovery job.

**Definition and stated intent:**

Source lines 11–11:

```
11 def main():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 12–17:

```
12 parser = argparse.ArgumentParser(description=__doc__)
13 parser.add_argument("--flavors", type=Path, default=FLAVOR_FILE)
14 parser.add_argument("--products", type=Path, default=DATA_DIR / "RMS/combined/products_extra.parquet")
15 parser.add_argument("--output", type=Path, default=DATA_DIR / "recovered_flavor_reference.csv")
16 args = parser.parse_args()
17 recover(args.flavors, args.products, args.output)
```

**Read this block:**

- Line 12, `parser`: Set this name to the result of calling `argparse.ArgumentParser(...)`; the arguments and preceding method chain are shown above.

- Line 13: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 14: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 15: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 16, `args`: Set this name to the result of calling `parser.parse_args(...)`; the arguments and preceding method chain are shown above.
- Line 17: call `recover` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.ArgumentParser`: Create the command-line parser object that will register and validate options.
- `.add_argument`: Register a command-line option and its type/default/choices/flag behavior before parsing.
- `.parse_args`: Read command-line arguments into a Namespace of named attributes.
- `recover`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

**Output reminder:** reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

# data_code/summ_stats.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

## Module setup and direct-execution blocks

Source lines 1–12:

```
1  """Readable, unweighted summaries for the full panel and matched yogurt sample.
2
3  Run on the server: python3 summ_stats.py --help
4  No data are loaded when this file is imported.
5  """
6
7  from pathlib import Path
8  import argparse
9  import tempfile
10 import time
11
12 import numpy as np
```

**Read this block:**

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 7: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 8: import `argparse`. An as-name is a convenient local alias, not another library.
- Line 9: import `tempfile`. An as-name is a convenient local alias, not another library.
- Line 10: import `time`. An as-name is a convenient local alias, not another library.
- Line 12: import `numpy as np`. An as-name is a convenient local alias, not another library.

Source lines 13–22:

```
13 import pandas as pd
14 import polars as pl
```

```
15
16 DATA_DIR = Path("/scratch/dtm63837/Kilts_Panel/Data")
17 # Set this to the flavor-reference CSV's full path on the Slurm server.
18 FLAVOR_FILE = (
19     DATA_DIR
20     / "../nielsen_extracts/nielsen_extracts/RMS/Reference_Documentation/2006-
       ↪ 2020_Documentation/Latest_Flavor_2010.csv"
21 )
22 FLAVORS = ["other", "berry", "plain"]
```

**Read this block:**

- Line 13: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 14: import `polars as pl`. An as-name is a convenient local alias, not another library.
- Line 16, `DATA_DIR`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 18, `FLAVOR_FILE`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 22, `FLAVORS`: Set this name to the list `['other', 'berry', 'plain']`.

**Syntax and calls used here:**

- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 27–30:

```
27 RACE_LABELS = {}
28 INCOME_LABELS = {}
29 HISPANIC_LABELS = {}
30 AGE_LABELS = {}
```

**Read this block:**

- Line 27, `RACE_LABELS`: Set this name to a dictionary from the displayed keys and values.
- Line 28, `INCOME_LABELS`: Set this name to a dictionary from the displayed keys and values.
- Line 29, `HISPANIC_LABELS`: Set this name to a dictionary from the displayed keys and values.
- Line 30, `AGE_LABELS`: Set this name to a dictionary from the displayed keys and values.

Source lines 651–652:

```
651 if __name__ == "__main__":
652     main()
```

**Read this block:**

- Line 651: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 652: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

## normalize upcs

Original function: `normalize_upcs`. Source begins at line 33.

This is the pandas normalization helper also used in estimation: strip whitespace, recognize numeric-looking identifiers, remove `.0` and leading zeros, and retain nonnumeric strings. Its argument is a Series and its return is a Series. The `where` condition retains original text on nonnumeric rows. The reason for string processing rather

than floating-point conversion is identifier precision. See the estimation function's worked examples; identical naming here does not mean Python automatically shares their definitions across modules.

**Definition and stated intent:**

Source lines 33–34:

```
33 def normalize_upcs(values):
34     """Canonical numeric UPC keys without floating-point conversion."""
```

`def` creates this callable; the parentheses name its inputs (`values`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 35–37:

```
35 text = values.astype("string").str.strip()
36 numeric = text.str.fullmatch(r"[0-9]+(?:\.0+)?", na=False)
37 canonical = text.str.replace(r"\.0+$", "", regex=True).str.lstrip("0")
```

**Read this block:**

- Line 35, `text`: Set this name to the result of calling `.strip(...)`; the arguments and preceding method chain are shown above.
- Line 36, `numeric`: Set this name to the result of calling `text.str.fullmatch(...)`; the arguments and preceding method chain are shown above.
- Line 37, `canonical`: Set this name to the result of calling `.lstrip(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.strip`: Remove surrounding string whitespace; it does not remove internal spaces.
- `.astype`: Convert the pandas/NumPy object's dtype; this changes representation, not merely its displayed label.
- `.fullmatch`: Require the entire string to match a regex; contains would allow a partial match inside unrelated text.
- `.lstrip`: Remove any leading occurrences of the specified character set, such as zeros in numeric UPC text.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.

Source lines 38–38:

```
38 return text.where(~numeric, canonical.replace("", "0"))
```

**Read this block:**

- Line 38: return the result of calling `text.where(...)`; the arguments and preceding method chain are shown above. This ends the call.

**Syntax and calls used here:**

- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

## normalized upc expr

Original function: `normalized_upc_expr`. Source begins at line 41.

This constructs the Polars-expression version of UPC normalization for a lazy pipeline. It returns a recipe named `upc`, not a table. Nested conditions replace an all-zero numeric identifier with `"0"`, preserving nonnumeric text. The function avoids repeating a long expression at every join boundary. Using matching canonical keys on both sides prevents `"001"` and `"1"` from mistakenly appearing unmatched.

**Definition and stated intent:**

Source lines 41–41:

```
41 def normalized_upc_expr():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 42–44:

```
42 text = pl.col("upc").cast(pl.String).str.strip_chars()
43 numeric = text.str.contains(r"^[0-9]+(?:\.0+)?$")
44 canonical = text.str.replace(r"\.0+$", "").str.strip_chars_start("0")
```

**Read this block:**

- Line 42, `text`: Set this name to the result of calling `.strip_chars(...)`; the arguments and preceding method chain are shown above.
- Line 43, `numeric`: Set this name to the result of calling `text.str.contains(...)`; the arguments and preceding method chain are shown above.
- Line 44, `canonical`: Set this name to the result of calling `.strip_chars_start(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.strip_chars`: Trim surrounding characters/whitespace in a Polars string expression.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.contains`: Test whether text contains the supplied regex pattern; anchors can make this an entire-string test.
- `.strip_chars_start`: Trim characters from the start of each Polars string; it does not remove matching characters inside it.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.

Source lines 45–47:

```
45 return pl.when(numeric).then(
46     pl.when(canonical == "").then(pl.lit("0")).otherwise(canonical)
47 ).otherwise(text).alias("upc")
```

**Read this block:**

- Line 45: return the result of calling `.alias(...)`; the arguments and preceding method chain are shown above. This ends the call.

**Syntax and calls used here:**

- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.otherwise`: Supply the fallback expression for rows not selected by the preceding condition.
- `.then`: Supply the value/expression selected where the preceding Polars condition is true.
- `.when`: Start a Polars row-wise conditional expression, completed by then and otherwise.
- `.lit`: Build a literal-valued Polars expression, distinguishing constant text/numbers from column names.

## valid flavor labels

Original function: `valid_flavor_labels`. Source begins at line 50.

Input is a pandas table with `flavor_descr`; output is a Boolean Series. Missing text becomes an empty string, is cast to string, stripped, and lowercased. `.isin` checks a list of unusable sentinel strings, then `~` negates the result. Thus `" N/A "` is invalid while `"Strawberry"` is valid. Normalizing once before comparison is clearer than writing separate conditions for every case and whitespace variant. This helper exists alongside directly written lazy filters; it is not automatically applied to every flavor-processing path.

**Definition and stated intent:**

Source lines 50–50:

```
50 def valid_flavor_labels(frame):
```

`def` creates this callable; the parentheses name its inputs (`frame`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 51–51:

```
51 labels = frame["flavor_descr"].fillna("").astype(str).str.strip().str.lower()
```

**Read this block:**

- Line 51, `labels`: Set this name to the result of calling `.lower(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.lower`: Convert pandas/Python text to lowercase so comparisons need not list every capitalization.
- `.strip`: Remove surrounding string whitespace; it does not remove internal spaces.
- `.astype`: Convert the pandas/NumPy object's dtype; this changes representation, not merely its displayed label.
- `.fillna`: Replace pandas missing entries with the provided fallback; it can change downstream denominators or validity checks.

Source lines 52–52:

```
52 return ~labels.isin(["", "na", "n/a", "nan", "null", "none"])
```

**Read this block:**

- Line 52: return the negated or sign-adjusted expression `~labels.isin(['', 'na', 'n/a', 'nan', 'null', 'none'])`. This ends the call.

**Syntax and calls used here:**

- `.isin`: Test each pandas entry's membership in an allowed sequence, producing a Boolean Series.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

## scan purchases

Original function: `scan_purchases`. Source begins at line 55.

This builds and returns a lazy, projected purchase query. It validates required column labels and maps optional source labels to a common schema. `aliases.items()` supplies output name plus candidate source names. `next(..., None)` picks the first available candidate; otherwise `pl.lit(None, dtype=pl.String)` makes an explicitly typed missing column. A consistent schema simplifies later summaries.

The comprehension builds expressions without reading rows. Dates, UPCs, quantities, and prices get consistent representations. Head-age zero codes become null, then `pl.coalesce` picks the first nonmissing head-age category, preferring female then male. This is a priority rule, not a mean. Eagerly loading all columns first would use more memory; dropping rows because an optional demographic is absent would change the summary population unnecessarily.

**Definition and stated intent:**

Source lines 55–56:

```
55 def scan_purchases(path, first_year, last_year):
56     """Project needed columns before loading; accept the names in current code."""
```

`def` creates this callable; the parentheses name its inputs (`path`, `first_year`, `last_year`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 57–67:

```
57 scan = pl.scan_parquet(path)
58 names = scan.collect_schema().names()
59 required = [
60     "household_code",
61     "panel_year",
62     "purchase_date",
63     "trip_code_uc",
64     "upc",
65     "quantity",
66 ]
67 missing = set(required) - set(names)
```

**Read this block:**

- Line 57, `scan`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 58, `names`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.
- Line 59, `required`: Set this name to a list containing 6 displayed entries.
- Line 67, `missing`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.

**Syntax and calls used here:**

- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.
- `set`: Build a collection of distinct values, used for column membership and missing-field checks.

Source lines 68–69:

```
68 if missing:
69     raise ValueError(f"{path}: missing columns {sorted(missing)}")
```

**Read this block:**

- Line 68: test `missing`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 69: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

**Syntax and calls used here:**

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 70–79:

```
70 aliases = {
71     "race": ["race"],
72     "income": ["household_income", "income"],
73     "hispanic_origin": ["hispanic_origin", "hispanic"],
74     "household_size": ["household_size"],
75     "female_head_age": ["female_head_age"],
76     "male_head_age": ["male_head_age"],
77     "dma_cd": ["dma_cd"],
78 }
79 expressions = [pl.col(c) for c in required]
```

**Read this block:**

- Line 70, `aliases`: Set this name to a dictionary with named entries `race`, `income`, `hispanic_origin`, `household_size`, `female_head_age`, `male_head_age`, `dma_cd`.
- Line 79, `expressions`: Set this name to a list computed by the displayed for-expression.

**Syntax and calls used here:**

- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 80–85:

```
80 for target, choices in aliases.items():
81     source = next((c for c in choices if c in names), None)
82     expression = (
83         pl.col(source).cast(pl.String) if source else pl.lit(None, dtype=pl.String)
84     )
85     expressions.append(expression.alias(target))
```

**Read this block:**

- Line 80: for each entry yielded by `aliases.items()`, bind/unpack it into `(target, choices)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 81, `source`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.
- Line 82, `expression`: Set this name to the true-branch value if `source` holds, otherwise the else-branch value.
- Line 85: call `expressions.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `next`: Take the first item from an iterator, or the supplied default when no item matches.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- `.lit`: Build a literal-valued Polars expression, distinguishing constant text/numbers from column names.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 87–101:

```
87 price_column = next((c for c in ["price", "price_rms"] if c in names), None)
88 expressions.append(
89     pl.col(price_column).cast(pl.Float64).alias("price")
90     if price_column
91     else pl.lit(None, dtype=pl.Float64).alias("price")
92 )
93 purchases = (
94     scan.select(expressions)
95     .filter(pl.col("panel_year").is_between(first_year, last_year))
96     .with_columns(
97         pl.col("purchase_date").cast(pl.String).str.slice(0, 10).str.to_date(),
98         normalized_upc_expr(),
99         pl.col("quantity").cast(pl.Float64),
100     )
101 )
```

**Read this block:**

- Line 87, `price_column`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.

- Line 88: call `expressions.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 93, `purchases`: Set this name to the result of calling `.with_columns(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.to_date`: Parse text into calendar-date values before weekday extraction and date arithmetic.
- `normalized_upc_expr`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.is_between`: Build an inclusive interval check by default, such as selecting the requested panel-year range.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.slice`: Take a substring starting at a position for a given length, here often the first ten date characters.

Source lines 104–110:

```
104 age_columns = ["female_head_age", "male_head_age"]
105 purchases = purchases.with_columns(
106     [
107         pl.when(pl.col(c) == "0").then(None).otherwise(pl.col(c)).alias(c)
108         for c in age_columns
109     ]
110 )
```

**Read this block:**

- Line 104, `age_columns`: Set this name to the list `['female_head_age', 'male_head_age']`.
- Line 105, `purchases`: Set this name to the result of calling `purchases.with_columns(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.otherwise`: Supply the fallback expression for rows not selected by the preceding condition.
- `.then`: Supply the value/expression selected where the preceding Polars condition is true.
- `.when`: Start a Polars row-wise conditional expression, completed by then and otherwise.

Source lines 111–111:

```
111 return purchases.with_columns(pl.coalesce(age_columns).alias("head_age"))
```

**Read this block:**

- Line 111: return the result of calling `purchases.with_columns(...)`; the arguments and preceding method chain are shown above. This ends the call.

## repeated yogurt households

Original function: `repeated_yogurt_households`. Source begins at line 114.

This keeps household histories with at least two distinct purchase weeks after the caller's year/price filters. `week` is a Polars expression using modulo arithmetic to reach Saturday. The query obtains each household's minimum and maximum week: unequal endpoints prove at least two distinct weeks without retaining a full distinct-week set.

Only eligible household IDs are collected. A semi join filters the original lazy purchase table while preserving its columns and all eligible purchase rows. Joining an expanded eligibility table could multiply rows; the semi join expresses membership only. Multiple trips in the same week do not satisfy this rule. Estimation later repeats a similar restriction after additional exclusions.

**Definition and stated intent:**

Source lines 114–119:

```
114 def repeated_yogurt_households(scan):
115     """Keep households with two distinct Saturday-ending purchase weeks.
116
117     Apply after year selection; multiple lines/trips in one week count once.
118     Summaries precede the estimation-only menu and IV exclusions.
119     """
```

`def` creates this callable; the parentheses name its inputs (`scan`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 120–130:

```
120 week = pl.col("purchase_date") + pl.duration(
121     days=(6 - pl.col("purchase_date").dt.weekday()) % 7
122 )
123 ids = (
124     scan.group_by("household_code")
125     .agg(week.min().alias("first_week"), week.max().alias("last_week"))
126     .filter(pl.col("first_week") != pl.col("last_week"))
127     .select("household_code")
128 )
129 # Materialize only household IDs so the repeated scan cannot cache all purchases.
130 ids = ids.collect(engine="streaming")
```

**Read this block:**

- Line 120, `week`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 123, `ids`: Set this name to the result of calling `.select(...)`; the arguments and preceding method chain are shown above.
- Line 130, `ids`: Set this name to the result of calling `ids.collect(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.duration`: Build a date/time duration from the named unit, allowing a different day offset per row.
- `.weekday`: Extract a Polars weekday number, Monday=1 through Sunday=7, for the Saturday offset calculation.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.
- `.min`: Take the smallest value, or a per-group/axis minimum when used on a table expression.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.

Source lines 131–131:

```
131 return scan.join(ids.lazy(), on="household_code", how="semi")
```

**Read this block:**

- Line 131: return the result of calling `scan.join(...)`; the arguments and preceding method chain are shown above. This ends the call.

**Syntax and calls used here:**

- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `.lazy`: Wrap an eager Polars table as a lazy query so later operations can be planned together.

## household summary

Original function: `household_summary`. Source begins at line 134.

Input is a lazy purchase table; output is a lazy one-row-per-household query. The `expressions` list defines exact purchase totals, distinct counts, and observed years. Demographic history follows a separate path: select a narrow table, deduplicate, stably sort, and take each field's first nonmissing value. Aggregating the full purchase table separately preserves exact trip/UPC/line counts while avoiding repeated wide demographic sorts.

`group_by(...).agg([...])` evaluates each listed expression per group. A validated one-to-one join reunites totals and demographic values. Later `with_columns` steps preserve original race, apply the Hispanic override, and calculate approximate weeks/trip rate. The second step can refer to the new `approx_observed_weeks` created in the first. Counting only observed years avoids filling panel gaps but still assumes full-year exposure within those years.

**Definition and stated intent:**

Source lines 134–135:

```
134 def household_summary(scan):
135     """One row per household, so frequent shoppers do not dominate demographics."""
```

`def` creates this callable; the parentheses name its inputs (`scan`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 136–145:

```
136 demographics = [
137     "race",
138     "income",
139     "hispanic_origin",
140     "household_size",
141     "head_age",
142     "female_head_age",
143     "male_head_age",
144     "dma_cd",
145 ]
```

**Read this block:**

- Line 136, `demographics`: Set this name to a list containing 8 displayed entries.

Source lines 147–152:

```
147 scan = scan.with_columns(
148     [
149         pl.when(pl.col(c) == "0").then(None).otherwise(pl.col(c)).alias(c)
150         for c in ["female_head_age", "male_head_age"]
151     ]
152 )
```

**Read this block:**

- Line 147, `scan`: Set this name to the result of calling `scan.with_columns(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.otherwise`: Supply the fallback expression for rows not selected by the preceding condition.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.then`: Supply the value/expression selected where the preceding Polars condition is true.
- `.when`: Start a Polars row-wise conditional expression, completed by then and otherwise.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 153–167:

```
153 expressions = [
154     pl.col("panel_year").n_unique().alias("observed_years"),
155     pl.col("panel_year").min().alias("first_year"),
156     pl.col("panel_year").max().alias("last_year"),
157     pl.col("trip_code_uc").n_unique().alias("trips"),
158     pl.col("purchase_date").n_unique().alias("shopping_days"),
159     pl.col("upc").n_unique().alias("distinct_upcs"),
160     pl.len().alias("purchase_lines"),
161     pl.col("quantity").sum().alias("packages"),
162 ]
163 # Keep exact purchase counts separate from the much narrower demographic
164 # history. Sort that history once for all fields, preserving input order
165 # for conflicting values on the same date. Removing identical rows cannot
166 # change the first nonmissing value.
167 totals = scan.group_by("household_code").agg(expressions)
```

**Read this block:**

- Line 153, `expressions`: Set this name to a list containing 8 displayed entries.
- Line 167, `totals`: Set this name to the result of calling `.agg(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.n_unique`: Count distinct Polars values within the relevant aggregation scope.
- `.min`: Take the smallest value, or a per-group/axis minimum when used on a table expression.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.

Source lines 168–174:

```
168 history = (scan.select(["household_code", "panel_year", "purchase_date"] + demographics)
169            .unique(maintain_order=True)
170            .sort(["household_code", "panel_year", "purchase_date"], maintain_order=True))
171 values = history.group_by("household_code").agg([
172     pl.col(c).drop_nulls().first().alias(c) for c in demographics
173 ])
174 households = totals.join(values, on="household_code", how="left", validate="1:1")
```

**Read this block:**

- Line 168, `history`: Set this name to the result of calling `.sort(...)`; the arguments and preceding method chain are shown above.
- Line 171, `values`: Set this name to the result of calling `.agg(...)`; the arguments and preceding method chain are shown above.
- Line 174, `households`: Set this name to the result of calling `totals.join(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.
- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.first`: Select the first value in the current expression/group; preceding ordering and null filtering determine which one.
- `.drop_nulls`: Remove null values from the Polars expression before selecting a first observed value.
- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.

Source lines 175–182:

```
175 return households.with_columns(
176     pl.col("race").alias("race_original"),
177     pl.when(pl.col("hispanic_origin") == "1").then(pl.lit("Hispanic"))
178     .otherwise(pl.col("race")).alias("race"),
179     (pl.col("observed_years") * 365.25 / 7).alias("approx_observed_weeks"),
180 ).with_columns(
181     (pl.col("trips") / pl.col("approx_observed_weeks")).alias("trips_per_observed_week")
182 )
```

**Read this block:**

- Line 175: return the result of calling `.with_columns(...)`; the arguments and preceding method chain are shown above. This ends the call.

**Syntax and calls used here:**

- `.lit`: Build a literal-valued Polars expression, distinguishing constant text/numbers from column names.

## as lazy

Original function: `as_lazy`. Source begins at line 185.

This small adapter always returns a Polars LazyFrame. `isinstance` checks the runtime type. An existing LazyFrame is returned unchanged, a pandas table is converted through `pl.from_pandas(...).lazy()`, and the remaining expected Polars DataFrame receives `.lazy()`. This lets callers supply convenient small eager tables in tests while production uses lazy scans. It is not a universal converter: an arbitrary unsupported object without `.lazy()` will still fail. Avoiding a copy/conversion when already lazy preserves the existing query plan.

**Definition and stated intent:**

Source lines 185–185:

```
185 def as_lazy(frame):
```

`def` creates this callable; the parentheses name its inputs (`frame`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 186–187:

```
186 if isinstance(frame, pl.LazyFrame):
187     return frame
```

**Read this block:**

- Line 186: test `isinstance(frame, pl.LazyFrame)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 187: return the object currently named `frame`; plain assignment does not copy it. This ends the call.

**Syntax and calls used here:**

- `isinstance`: Check a runtime type to select the appropriate conversion path.

Source lines 188–189:

```
188 if isinstance(frame, pd.DataFrame):
189     return pl.from_pandas(frame).lazy()
```

**Read this block:**

- Line 188: test `isinstance(frame, pd.DataFrame)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 189: return the result of calling `.lazy(...)`; the arguments and preceding method chain are shown above. This ends the call.

**Syntax and calls used here:**

- `.lazy`: Wrap an eager Polars table as a lazy query so later operations can be planned together.
- `.from_pandas`: Convert a pandas object into a Polars table; calling lazy afterward makes a query plan from it.

Source lines 190–190:

```
190 return frame.lazy()
```

**Read this block:**

- Line 190: return the result of calling `frame.lazy(...)`; the arguments and preceding method chain are shown above. This ends the call.

## partition purchases

Original function: `partition_purchases`. Source begins at line 193.

This executes a lazy scan once to write physical household-hash partitions, then returns a list of lazy scans over those files. `hasattr(pl, "PartitionBy")` checks API availability before starting. `(hash(seed=0) % batches)` maps every row with the same household ID to the same bucket. This is deterministic routing, not random sample selection.

`sink_parquet(pl.PartitionBy(...))` writes by bucket. The list comprehension includes only existing bucket directories, important because small examples can leave buckets empty. `sorted(directory.glob(...))` makes input order predictable. `hive_partitioning=False` prevents folder names from automatically adding a partition column that was intentionally excluded. The alternative of filtering/recollecting the original source for every bucket repeats large scans; physical partitions trade temporary disk space for reusable reads.

**Definition and stated intent:**

Source lines 193–194:

```
193 def partition_purchases(scan, work, prefix, batches):
194     """Read the projected source once, writing household hash partitions."""
```

`def` creates this callable; the parentheses name its inputs (`scan`, `work`, `prefix`, `batches`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 195–196:

```
195 if not hasattr(pl, "PartitionBy"):
196     raise RuntimeError("Summary partitioning requires Polars >= 2.0; install est_code/requirements.txt.")
```

**Read this block:**

- Line 195: test `not hasattr(pl, 'PartitionBy')`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 196: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

**Syntax and calls used here:**

- `hasattr`: Check that an object exposes a named attribute before using an optional/version-dependent API.
- `RuntimeError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 197–203:

```
197 directory = work / f"{prefix}_purchases"
198 began = time.monotonic()
199 print(f"{prefix}: partitioning purchases once into {batches} batches", flush=True)
200 (scan.with_columns((pl.col("household_code").hash(seed=0) % batches).alias("_batch"))
201  .sink_parquet(pl.PartitionBy(directory, key="_batch", include_key=False), engine="streaming"))
202 parts = [pl.scan_parquet(sorted(directory.glob(f"_batch={batch}/*.parquet")), hive_partitioning=False)
203          for batch in range(batches) if (directory / f"_batch={batch}").exists()]
```

**Read this block:**

- Line 197, `directory`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 198, `began`: Set this name to the result of calling `time.monotonic(...)`; the arguments and preceding method chain are shown above.
- Line 199: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 200: call `.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 202, `parts`: Set this name to a list computed by the displayed for-expression, retaining only entries passing its if filter.

**Syntax and calls used here:**

- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.
- `.monotonic`: Read a monotonic clock for elapsed durations rather than a wall-clock date/time.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.
- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- `.glob`: Find filesystem paths matching a wildcard pattern, which may include stale files unless the folder is controlled.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 204–205:

```
204 if not parts:
205     raise ValueError(f"Empty {prefix} sample in selected years.")
```

**Read this block:**

- Line 204: test `not parts`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 205: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

**Syntax and calls used here:**

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 206–206:

```
206 print(f"{prefix}: partitioning completed in {time.monotonic() - began:.1f}s", flush=True)
```

**Read this block:**

- Line 206: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 207–207:

```
207 return parts
```

**Read this block:**

- Line 207: return the object currently named `parts`; plain assignment does not copy it. This ends the call.

## household batches

Original function: `household_batches`. Source begins at line 210.

This accepts an optional list of already written partitions. The conditional expression reuses them if supplied; otherwise it creates them once. `enumerate` gives each partition an output filename suffix, `household_summary` runs on it, and `sink_parquet` writes its reduced result. The final wildcard scan returns one logical table of all household outputs without first concatenating them eagerly in Python. Households never cross partition boundaries, so exact household summaries can be computed independently and then stacked.

**Definition and stated intent:**

Source lines 210–211:

```
210 def household_batches(scan, work, prefix, batches, partitions=None):
211     # Each physical partition is read on its own; no full-source rescan per batch.
```

`def` creates this callable; the parentheses name its inputs (`scan`, `work`, `prefix`, `batches`, `partitions`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 212–212:

```
212 partitions = partitions if partitions is not None else partition_purchases(scan, work, prefix, batches)
```

**Read this block:**

- Line 212, `partitions`: Set this name to the true-branch value if `partitions is not None` holds, otherwise the else-branch value.

**Syntax and calls used here:**

- `partition_purchases`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 213–217:

```
213 for batch, subset in enumerate(partitions):
214     began = time.monotonic()
215     print(f"{prefix}: household batch {batch + 1}/{len(partitions)}", flush=True)
216     household_summary(subset).sink_parquet(work / f"{prefix}_{batch}.parquet", engine="streaming")
217     print(f"{prefix}: household batch {batch + 1} completed in {time.monotonic() - began:.1f}s", flush=True)
```

**Read this block:**

- Line 213: for each entry yielded by `enumerate(partitions)`, bind/unpack it into `(batch, subset)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 214, `began`: Set this name to the result of calling `time.monotonic(...)`; the arguments and preceding method chain are shown above.
- Line 215: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 216: call `household_summary(subset).sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 217: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.monotonic`: Read a monotonic clock for elapsed durations rather than a wall-clock date/time.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `household_summary`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `len`: Count elements/rows rather than summing their numerical values.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 218–218:

```
218 return pl.scan_parquet(work / f"{prefix}_*.parquet")
```

**Read this block:**

- Line 218: return the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above. This ends the call.

**Syntax and calls used here:**

- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.

## save table

Original function: `save_table`. Source begins at line 221.

This writes both a machine-readable CSV and an escaped LaTeX table and prints a readable version. Polars inputs are collected and converted to pandas because the chosen table formatting/export methods belong to pandas. A pandas input already has those methods, so conversion is skipped.

`lambda value: f"{value:.2f}"` formats display numbers without changing the stored numerical data. `index=False` suppresses an unwanted row-number column. `escape=True` protects LaTeX syntax characters in labels. CSV retains reusable data whereas LaTeX supplies paper-ready formatting; either output alone would serve only one of those purposes. This function's useful result is its side effects, so no explicit return is required.

**Definition and stated intent:**

Source lines 221–222:

```
221 def save_table(frame, output, name, caption):
222     """Keep machine-readable CSV beside the paper-ready LaTeX table."""
```

`def` creates this callable; the parentheses name its inputs (`frame`, `output`, `name`, `caption`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 223–224:

```
223 if isinstance(frame, (pl.LazyFrame, pl.DataFrame)):
224     frame = as_lazy(frame).collect(engine="streaming").to_pandas()
```

**Read this block:**

- Line 223: test `isinstance(frame, (pl.LazyFrame, pl.DataFrame))`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 224, `frame`: Set this name to the result of calling `.to_pandas(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `isinstance`: Check a runtime type to select the appropriate conversion path.
- `.to_pandas`: Convert a Polars table to pandas, crossing from query/table preparation to eager labeled analysis.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `as_lazy`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 225–230:

```
225 print(f"\n{caption} [{name}]", flush=True)
226 print(frame.to_string(index=False, float_format=lambda value: f"{value:.2f}"), flush=True)
227 frame.to_csv(output / f"{name}.csv", index=False)
228 (output / f"{name}.tex").write_text(
229     frame.to_latex(index=False, escape=True, float_format="%.2f", caption=caption)
230 )
```

**Read this block:**

- Line 225: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 226: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 227: call `frame.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 228: call `(output / f'{name}.tex').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `.to_latex`: Serialize a pandas table as LaTeX source, which needs a separate TeX compilation to become PDF.

**Output reminder:** reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

## demographic tables

Original function: `demographic_tables`. Source begins at line 233.

Input is a one-row-per-household table, an output directory, and filename prefix. The initial count is its denominator. The loop builds one categorical table per demographic using null-to-`Missing` replacement, grouped counts,

optional codebook mappings, and percentages of all households. `.map(labels).fillna(code)` keeps source codes visible when a verified label is absent.

The `specs` list pairs human-readable metric names with Polars aggregation expressions. Enumerated string aliases let all metrics be computed in a single query and then zipped back to names. `rows.extend(generator)` appends race percentages to the overview. Equal household weighting is achieved by summarizing households first; directly counting purchase rows would overweight frequent shoppers. Numeric means of income/head-age codes remain means of categories, not economic levels.

**Definition and stated intent:**

Source lines 233–233:

```
233 def demographic_tables(households, output, prefix):
```

`def` creates this callable; the parentheses name its inputs (`households`, `output`, `prefix`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 234–236:

```
234 households = as_lazy(households)
235 count = households.select(pl.len()).collect(engine="streaming").item()
236 race_table = None
```

**Read this block:**

- Line 234, `households`: Set this name to the result of calling `as_lazy(...)`; the arguments and preceding method chain are shown above.
- Line 235, `count`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 236, `race_table`: Set this name to the literal value `None`.

**Syntax and calls used here:**

- `as_lazy`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.item`: Extract a scalar from a one-element array/one-cell result rather than passing a table wrapper onward.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.

Source lines 237–249:

```
237 for column, labels in [("race", RACE_LABELS), ("hispanic_origin", HISPANIC_LABELS),
238                        ("income", INCOME_LABELS), ("household_size", {}),
239                        ("head_age", AGE_LABELS), ("female_head_age", AGE_LABELS),
240                        ("male_head_age", AGE_LABELS)]:
241     table = (households.select(pl.col(column).cast(pl.String).fill_null("Missing").alias("code"))
242              .group_by("code").len(name="households").sort("code")
243              .collect(engine="streaming").to_pandas())
244     table["category"] = table["code"].map(labels).fillna(table["code"])
245     table["percent_all_households"] = 100 * table["households"] / count
246     save_table(table, output, f"{prefix}_{column}",
247                f"{prefix.replace('_', ' ').title()}: {column.replace('_', ' ')} (unweighted)")
248     if column == "race":
249         race_table = table
```

**Read this block:**

- Line 237: for each entry yielded by `[('race', RACE_LABELS), ('hispanic_origin', HISPANIC_LABELS), ...`, bind/unpack it into `(column, labels)` and run the indented body. Changes inside an iteration can feed subsequent iterations.

- Line 241, `table`: Set this name to the result of calling `.to_pandas(...)`; the arguments and preceding method chain are shown above.
- Line 244, `table['category']`: Store the result of calling `.fillna(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 245, `table['percent_all_households']`: Store division of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 246: call `save_table` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 248: test `column == 'race'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 249, `race_table`: Set this name to the object currently named `table`; plain assignment does not copy it.

**Syntax and calls used here:**

- `.to_pandas`: Convert a Polars table to pandas, crossing from query/table preparation to eager labeled analysis.
- `.fillna`: Replace pandas missing entries with the provided fallback; it can change downstream denominators or validity checks.
- `save_table`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.
- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.
- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.fill_null`: Replace Polars nulls with a chosen value/expression; null and floating-point NaN are distinct representations.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 250–264:

```
250 specs = [
251     ("Households", pl.len()),
252     ("Mean observed panel years", pl.col("observed_years").mean()),
253     ("Mean trips per household", pl.col("trips").mean()),
254     ("Mean trips per approximate observed week", pl.col("trips_per_observed_week").mean()),
255     ("Median trips per approximate observed week", pl.col("trips_per_observed_week").median()),
256     ("Mean distinct UPCs per household", pl.col("distinct_upcs").mean()),
257     ("Mean purchase lines per household", pl.col("purchase_lines").mean()),
258     ("Mean packages per household", pl.col("packages").mean()),
259     ("Mean head age (source category code)", pl.col("head_age").cast(pl.Float64, strict=False).mean()),
260     ("Households with numeric head age", pl.col("head_age").cast(pl.Float64, strict=False).count()),
261     ("Mean household income (source category code)", pl.col("income").cast(pl.Float64, strict=False).mean()),
262     ("Households with numeric income", pl.col("income").cast(pl.Float64, strict=False).count()),
263 ]
264 values = households.select([expr.alias(str(i)) for i, (_, expr) in
    ↪  enumerate(specs)]).collect(engine="streaming").row(0)
```

**Read this block:**

- Line 250, `specs`: Set this name to a list containing 12 displayed entries.
- Line 264, `values`: Set this name to the result of calling `.row(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.

- `.median`: Compute the middle-value statistic rather than a mean; global medians cannot generally be averaged across batches.
- `str`: Convert to text for paths, messages, or serialization.
- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 265–269:

```
265 rows = [(name, value) for (name, _), value in zip(specs, values)]
266 rows.extend((f"Race: {row.category} (% of all households)", row.percent_all_households)
267             for row in race_table.itertuples())
268 save_table(pd.DataFrame(rows, columns=["metric", "value"]), output, f"{prefix}_overview",
269             f"{prefix.replace('_', ' ').title()}: Household purchase summary (unweighted)")
```

**Read this block:**

- Line 265, `rows`: Set this name to a list computed by the displayed for-expression.
- Line 266: call `rows.extend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 268: call `save_table` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.extend`: Append every element from an iterable, unlike append which would add that iterable as one item.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

**Output reminder:** reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

## scan reference

Original function: `scan_reference`. Source begins at line 272.

This wraps `pl.scan_csv` with consistent parsing choices. `infer_schema=False` keeps reference fields as strings instead of guessing that UPCs are numbers. `null_values=[...]` explicitly recognizes several missing-value spellings. The return is lazy, so calling the function sets up a CSV read without loading the entire table. Centralizing these options prevents different recovery and classification paths from interpreting the same file differently.

**Definition and stated intent:**

Source lines 272–272:

```
272 def scan_reference(path):
```

`def` creates this callable; the parentheses name its inputs (`path`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 273–274:

```
273 return pl.scan_csv(path, infer_schema=False,
274                    null_values=["NA", "N/A", "NaN", "nan", "NULL", "null", "None", "<NA>"])
```

**Read this block:**

- Line 273: return the result of calling `pl.scan_csv(...)`; the arguments and preceding method chain are shown above. This ends the call.

**Syntax and calls used here:**

- `.scan_csv`: Build a lazy delimited-text read plan; parser options determine types and missing values.

### recover flavor reference

Original function: `recover_flavor_reference`. Source begins at line 277.

This supplements, rather than overwrites, the descriptor reference. Both input tables are lazy; an anti join excludes product UPCs already present in the CSV. The filter removes missing/placeholder candidate descriptors. If annual metadata exists, each UPC's candidate columns take earliest-year values; otherwise first row deduplication is used. The output is a new combined reference plus a separate CSV of just recovered rows.

`.with_name(output.stem + ...)` constructs a sibling output filename. Comparing resolved paths rejects accidental overwrite of the original reference. After writing additions, `scan_reference` rereads them under the same schema conventions before diagonal relaxed concatenation allows unequal optional columns. The return is the output Path, enabling `run` to use it for subsequent classification. It does not repair blank descriptors on UPCs already in the original reference.

**Definition and stated intent:**

Source lines 277–277:

```
277 def recover_flavor_reference(reference_path, products_path, output):
```

`def` creates this callable; the parentheses name its inputs (`reference_path`, `products_path`, `output`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 278–278:

```
278 reference = scan_reference(reference_path).with_columns(normalized_upc_expr())
```

**Read this block:**

- Line 278, `reference`: Set this name to the result of calling `.with_columns(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `normalized_upc_expr`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `scan_reference`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 279–280:

```
279 if not {"upc", "flavor_descr"}.issubset(reference.collect_schema().names()):
280     raise ValueError("Reference needs upc and flavor_descr columns.")
```

**Read this block:**

- Line 279: test `not {'upc', 'flavor_descr'}.issubset(reference.collect_schema().names())`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 280: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

**Syntax and calls used here:**

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.

Source lines 281–282:

```
281 products = pl.scan_parquet(products_path)
282 names = products.collect_schema().names()
```

**Read this block:**

- Line 281, `products`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 282, `names`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.

Source lines 283–284:

```
283 if not {"upc", "flavor_descr"}.issubset(names):
284     raise ValueError(f"{products_path}: needs upc and flavor_descr columns.")
```

**Read this block:**

- Line 283: test `not {'upc', 'flavor_descr'}.issubset(names)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 284: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

**Syntax and calls used here:**

- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 285–292:

```
285 columns = [c for c in ["upc", "flavor_descr", "panel_year", "product_descr"] if c in names]
286 products = (products.select(columns).with_columns(normalized_upc_expr())
287             .join(reference.select("upc").unique(), on="upc", how="anti"))
288 products = products.filter(
289     pl.col("upc").is_not_null() & pl.col("flavor_descr").is_not_null()
290     & ~pl.col("flavor_descr").cast(pl.String).str.strip_chars().str.to_lowercase()
291     .is_in(["", "na", "n/a", "nan", "null", "none"])
292 )
```

**Read this block:**

- Line 285, `columns`: Set this name to a list computed by the displayed for-expression, retaining only entries passing its if filter.
- Line 286, `products`: Set this name to the result of calling `.join(...)`; the arguments and preceding method chain are shown above.
- Line 288, `products`: Set this name to the result of calling `products.filter(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.is_not_null`: Create a Polars nonnull mask; additional range checks can still be needed.
- `.is_in`: Build a Polars membership expression, often used for eligible household IDs or allowed labels.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.to_lowercase`: Build a lowercase Polars string expression for reproducible label comparisons.
- `.strip_chars`: Trim surrounding characters/whitespace in a Polars string expression.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.

- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

Source lines 293–299:

```
293 if "panel_year" in names:
294     recovered = products.group_by("upc").agg([
295         pl.col(c).sort_by("panel_year", nulls_last=True).first().alias(c)
296         for c in columns if c != "upc"
297     ])
298 else:
299     recovered = products.unique(subset="upc", keep="first", maintain_order=True)
```

**Read this block:**

- Line 293: test `'panel_year' in names`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 294, `recovered`: Set this name to the result of calling `.agg(...)`; the arguments and preceding method chain are shown above.
- Line 299, `recovered`: Set this name to the result of calling `products.unique(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.first`: Select the first value in the current expression/group; preceding ordering and null filtering determine which one.
- `.sort_by`: Order values within an expression/group by another column before selecting a first value.

Source lines 300–301:

```
300 output = Path(output)
301 output.parent.mkdir(parents=True, exist_ok=True)
```

**Read this block:**

- Line 300, `output`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 301: call `output.parent.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.

Source lines 302–303:

```
302 if output.resolve() == Path(reference_path).resolve():
303     raise ValueError("Choose a separate output file to preserve the original reference.")
```

**Read this block:**

- Line 302: test `output.resolve() == Path(reference_path).resolve()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 303: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

**Syntax and calls used here:**

- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.

Source lines 304–309:

```
304 recovered_path = output.with_name(output.stem + "_recovered_rows.csv")
305 recovered.sink_csv(recovered_path, engine="streaming")
306 additions = scan_reference(recovered_path)
307 pl.concat([reference, additions], how="diagonal_relaxed").sink_csv(output, engine="streaming")
308 count = additions.select(pl.len()).collect(engine="streaming").item()
309 print(f"Recovered labels for {count:,} additional UPCs. Existing entries retained. Saved {output}", flush=True)
```

**Read this block:**

- Line 304, `recovered_path`: Set this name to the result of calling `output.with_name(...)`; the arguments and preceding method chain are shown above.
- Line 305: call `recovered.sink_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 306, `additions`: Set this name to the result of calling `scan_reference(...)`; the arguments and preceding method chain are shown above.
- Line 307: call `.sink_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 308, `count`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 309: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.with_name`: Construct a sibling Path with a replacement filename, preserving its parent directory.
- `.sink_csv`: Execute/write the query result as CSV; unlike select/filter, this is a filesystem side effect.
- `.item`: Extract a scalar from a one-element array/one-cell result rather than passing a table wrapper onward.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Source lines 310–310:

```
310 return output
```

**Read this block:**

- Line 310: return the object currently named `output`; plain assignment does not copy it. This ends the call.

## flavor reference

Original function: `flavor_reference`. Source begins at line 313.

This resolves one label per normalized UPC and returns an eager Polars mapping. `detailed` selects two different expression recipes. Detailed labels use trimmed lowercase descriptors with whitespace collapsed; grouped labels use ordered plain/berry text matching and an other fallback. `.replace` maps specific whole values; regex `.str.replace_all` instead matches patterns within text.

Before deduplicating, the function counts UPCs assigned multiple distinct labels and can save their rows for review. `unique(..., keep="first", maintain_order=True)` implements first-reference-row precedence reproducibly. This is a transparent rule but not a semantic resolution of conflicting descriptions. Returning only UPC and final flavor gives downstream joins a small unique lookup table.

**Definition and stated intent:**

Source lines 313–314:

```
313 def flavor_reference(path, detailed=False, output=None):
314     """Use first reference row per normalized UPC; keep missing descriptors."""
```

`def` creates this callable; the parentheses name its inputs (`path`, `detailed`, `output`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 315–317:

```
315 frame = scan_reference(path)
316 names = frame.collect_schema().names()
317 missing = {"upc", "flavor_descr"} - set(names)
```

**Read this block:**

- Line 315, `frame`: Set this name to the result of calling `scan_reference(...)`; the arguments and preceding method chain are shown above.
- Line 316, `names`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.
- Line 317, `missing`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.

**Syntax and calls used here:**

- `scan_reference`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.
- `set`: Build a collection of distinct values, used for column membership and missing-field checks.

Source lines 318–319:

```
318 if missing:
319     raise ValueError(f"{path}: missing columns {sorted(missing)}")
```

**Read this block:**

- Line 318: test `missing`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 319: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

**Syntax and calls used here:**

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 320–320:

```
320 frame = frame.with_columns(normalized_upc_expr())
```

**Read this block:**

- Line 320, `frame`: Set this name to the result of calling `frame.with_columns(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `normalized_upc_expr`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 321–322:

```
321 if frame.select((pl.col("upc").is_null() | (pl.col("upc") == "")).any()).collect(engine="streaming").item():
322     raise ValueError(f"{path}: flavor reference contains missing UPCs.")
```

**Read this block:**

- Line 321: test `frame.select((pl.col('upc').is_null() | (pl.col('upc') == '')).any()).collect...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 322: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

**Syntax and calls used here:**

- `.item`: Extract a scalar from a one-element array/one-cell result rather than passing a table wrapper onward.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `.is_null`: Create a Polars null-value mask, useful when validating joins and conversions.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- The | operator accepts either elementwise condition; Python's scalar or is generally unsuitable for an entire table column.

Source lines 323–323:

```
323 text = pl.col("flavor_descr").fill_null("")
```

**Read this block:**

- Line 323, `text`: Set this name to the result of calling `.fill_null(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.fill_null`: Replace Polars nulls with a chosen value/expression; null and floating-point NaN are distinct representations.

Source lines 324–325:

```
324 if "product_descr" in names:
325     text = text + pl.lit(" ") + pl.col("product_descr").fill_null("")
```

**Read this block:**

- Line 324: test `'product_descr' in names`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 325, `text`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

**Syntax and calls used here:**

- `.lit`: Build a literal-valued Polars expression, distinguishing constant text/numbers from column names.

Source lines 326–326:

```
326 text = text.str.to_lowercase()
```

**Read this block:**

- Line 326, `text`: Set this name to the result of calling `text.str.to_lowercase(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.to_lowercase`: Build a lowercase Polars string expression for reproducible label comparisons.

Source lines 327–333:

```
327 if detailed:
328     flavor = (pl.col("flavor_descr").fill_null("").str.to_lowercase().str.strip_chars()
329               .str.replace_all(r"\s+", " ").replace({"": "unknown", "unflavored": "plain"}))
```

```
330 else:
331     flavor = (pl.when(text.str.contains("plain|unflavored")).then(pl.lit("plain"))
332               .when(text.str.contains("berry|straw|blue|rasp|black|cran|cherry|wildberry"))
333               .then(pl.lit("berry")).otherwise(pl.lit("other")))
```

**Read this block:**

- Line 327: test `detailed`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 328, `flavor`: Set this name to the result of calling `.replace(...)`; the arguments and preceding method chain are shown above.
- Line 331, `flavor`: Set this name to the result of calling `.otherwise(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.
- `.otherwise`: Supply the fallback expression for rows not selected by the preceding condition.
- `.replace_all`: Replace every regex match within each string, such as collapsing repeated whitespace.
- `.then`: Supply the value/expression selected where the preceding Polars condition is true.
- `.when`: Start a Polars row-wise conditional expression, completed by then and otherwise.
- `.strip_chars`: Trim surrounding characters/whitespace in a Polars string expression.
- `.contains`: Test whether text contains the supplied regex pattern; anchors can make this an entire-string test.

Source lines 334–336:

```
334 frame = frame.with_columns(flavor.alias("flavor"))
335 conflicting = frame.group_by("upc").agg(pl.col("flavor").n_unique().alias("n")).filter(pl.col("n") > 1)
336 count = conflicting.select(pl.len()).collect(engine="streaming").item()
```

**Read this block:**

- Line 334, `frame`: Set this name to the result of calling `frame.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 335, `conflicting`: Set this name to the result of calling `.filter(...)`; the arguments and preceding method chain are shown above.
- Line 336, `count`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.
- `.n_unique`: Count distinct Polars values within the relevant aggregation scope.

Source lines 337–341:

```
337 if count:
338     print(f"{count:,} UPCs have conflicting flavor classifications; keeping the first CSV row.", flush=True)
339     if output is not None:
340         name = "detailed_flavor_conflicts.csv" if detailed else "aggregated_flavor_conflicts.csv"
341         frame.join(conflicting.select("upc"), on="upc", how="semi").sink_csv(output / name, engine="streaming")
```

**Read this block:**

- Line 337: test `count`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.

- Line 338: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 339: test `output is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 340, `name`: Set this name to the true-branch value if `detailed` holds, otherwise the else-branch value.
- Line 341: call `.sink_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.sink_csv`: Execute/write the query result as CSV; unlike select/filter, this is a filesystem side effect.
- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 342–343:

```
342 blanks = frame.select((pl.col("flavor_descr").fill_null("").str.strip_chars() ==
    ↪  "").sum()).collect(engine="streaming").item()
343 print(f"Flavor-reference rows with blank descriptors: {blanks:,}", flush=True)
```

**Read this block:**

- Line 342, `blanks`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 343: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.

Source lines 344–344:

```
344 return frame.select("upc", "flavor").unique(subset="upc", keep="first",
    ↪  maintain_order=True).collect(engine="streaming")
```

**Read this block:**

- Line 344: return the result of calling `.collect(...)`; the arguments and preceding method chain are shown above. This ends the call.

**Syntax and calls used here:**

- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.

## yogurt weeks

Original function: `yogurt_weeks`. Source begins at line 347.

This attaches flavor labels, audits matched purchase lines, and returns a lazy weekly quantity query plus an eager small audit table. A left join preserves unmatched purchases long enough to count/report them. Unmatched UPCs are exported and cause a ValueError, preventing an apparently successful summary of a silently reduced sample. Nonpositive/missing quantities also stop the run.

The Saturday expression adds a row-dependent duration to purchase date. Grouping by household/week/flavor and summing quantity keeps only observed combinations. The alternative dense pivot would create a column for every detailed flavor and potentially a great many empty cells. Price problems are audited here; the main matched-sample caller has already applied its positive-price filter.

**Definition and stated intent:**

Source lines 347–348:

```
347 def yogurt_weeks(scan, reference, flavors=FLAVORS, output=None):
348     """Keep sparse household/week/flavor counts rather than a dense pivot."""
```

`def` creates this callable; the parentheses name its inputs (`scan`, `reference`, `flavors`, `output`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 349–355:

```
349 joined = scan.join(reference.lazy(), on="upc", how="left")
350 audit = joined.select(
351     pl.len().alias("purchase_lines"),
352     pl.col("flavor").is_null().sum().alias("unmatched_flavor_lines"),
353     ((~pl.col("price").is_finite()) | (pl.col("price") <=
    ↪  0)).fill_null(True).sum().alias("missing_or_nonpositive_quoted_prices"),
354     ((~pl.col("quantity").is_finite()) | (pl.col("quantity") <=
    ↪  0)).fill_null(True).sum().alias("invalid_quantity_lines"),
355 ).collect(engine="streaming")
```

**Read this block:**

- Line 349, `joined`: Set this name to the result of calling `scan.join(...)`; the arguments and preceding method chain are shown above.
- Line 350, `audit`: Set this name to the result of calling `.collect(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `.lazy`: Wrap an eager Polars table as a lazy query so later operations can be planned together.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.is_null`: Create a Polars null-value mask, useful when validating joins and conversions.
- `.fill_null`: Replace Polars nulls with a chosen value/expression; null and floating-point NaN are distinct representations.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.is_finite`: Build the Polars finite-number test used to reject missing/infinite quotes.
- The | operator accepts either elementwise condition; Python's scalar or is generally unsuitable for an entire table column.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

Source lines 356–361:

```
356 if audit["unmatched_flavor_lines"][0]:
357     unmatched = (joined.filter(pl.col("flavor").is_null()).group_by("upc")
358                  .agg(pl.len().alias("purchase_lines"), pl.col("quantity").sum().alias("packages")))
359     if output is not None:
360         unmatched.sink_csv(output / "unmatched_purchase_upcs.csv", engine="streaming")
361     raise ValueError("Purchased UPCs still lack flavor labels; see unmatched_purchase_upcs.csv.")
```

**Read this block:**

- Line 356: test `audit['unmatched_flavor_lines'][0]`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 357, `unmatched`: Set this name to the result of calling `.agg(...)`; the arguments and preceding method chain are shown above.

- Line 359: test `output is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 360: call `unmatched.sink_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 361: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

**Syntax and calls used here:**

- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.sink_csv`: Execute/write the query result as CSV; unlike select/filter, this is a filesystem side effect.
- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.

Source lines 362–363:

```
362 if audit["invalid_quantity_lines"][0]:
363     raise ValueError("Nonpositive or missing quantities; audit the matched sample first.")
```

**Read this block:**

- Line 362: test `audit['invalid_quantity_lines'][0]`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 363: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 364–366:

```
364 week = pl.col("purchase_date") + pl.duration(days=(6 - pl.col("purchase_date").dt.weekday()) % 7)
365 grouped = (joined.with_columns(week.alias("week"))
366             .group_by(["household_code", "week", "flavor"]).agg(pl.col("quantity").sum()))
```

**Read this block:**

- Line 364, `week`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 365, `grouped`: Set this name to the result of calling `.agg(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.duration`: Build a date/time duration from the named unit, allowing a different day offset per row.
- `.weekday`: Extract a Polars weekday number, Monday=1 through Sunday=7, for the Saturday offset calculation.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.
- `.with_columns`: Add or replace named columns while keeping the other columns in the table.

Source lines 367–367:

```
367 return grouped, audit
```

**Read this block:**

- Line 367: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

## flavor summaries

Original function: `flavor_summaries`. Source begins at line 370.

This is the main descriptive-statistics engine. It returns pandas share and transition tables for plotting, while writing much richer outputs. First it sums quantities by flavor and computes package shares; a rounding filter removes rows whose displayed share is zero but leaves the denominator unchanged. Next it aggregates household/flavor totals to number of flavors, squared quantities, and largest quantity, producing HHI and largest-flavor share.

The basket query groups again by household/week. `pl.when(pl.len() == 1)` gives a single-flavor label only to unmixed baskets. `.shift().over("household_code")` looks at the previous basket within the household, not across household borders. Date differences identify adjacent weeks. A cumulative sum of spell-start flags creates run identifiers; mixed baskets and gaps break the runs.

The expensive basket query is checkpointed once in a temporary directory so transitions, spells, CSV exports, and global medians can reuse it. Transition counts pivot into a small matrix; dividing by row sums produces probabilities. Zero denominators become NaN. Spells are averaged as spells, not as households. Finally a small metric table records basket frequencies and eligible-pair switching. This sequence uses several aggregation units intentionally; replacing them all with one grand groupby would obscure their different denominators.

**Definition and stated intent:**

Source lines 370–370:

```
370 def flavor_summaries(weekly, output, flavors=FLAVORS):
```

`def` creates this callable; the parentheses name its inputs (`weekly`, `output`, `flavors`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 371–372:

```
371 weekly = as_lazy(weekly)
372 totals = weekly.group_by("flavor").agg(pl.col("quantity").sum().alias("packages")).collect(engine="streaming")
```

**Read this block:**

- Line 371, `weekly`: Set this name to the result of calling `as_lazy(...)`; the arguments and preceding method chain are shown above.
- Line 372, `totals`: Set this name to the result of calling `.collect(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `as_lazy`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.

Source lines 373–374:

```
373 if flavors != FLAVORS:
374     flavors = sorted(totals["flavor"].to_list())
```

**Read this block:**

- Line 373: test `flavors != FLAVORS`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 374, `flavors`: Set this name to the result of calling `sorted(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.

Source lines 376–384:

```
shares = (pl.DataFrame({"flavor": flavors}).join(totals, on="flavor", how="left")
          .with_columns(pl.col("packages").fill_null(0))
          .with_columns((100 * pl.col("packages") / pl.col("packages").sum()).alias("percent_packages")))
# Match the table's two-decimal display; keep the original denominator.
shares = shares.filter(pl.col("percent_packages").round(2) > 0).sort(
    ["percent_packages", "flavor"], descending=[True, False]
)
save_table(shares, output, "yogurt_flavor_shares", "Flavor shares of purchased packages")
counts = weekly.group_by(["household_code", "flavor"]).agg(pl.col("quantity").sum())
```

**Read this block:**

- Line 376, `shares`: Set this name to the result of calling `.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 380, `shares`: Set this name to the result of calling `.sort(...)`; the arguments and preceding method chain are shown above.
- Line 383: call `save_table` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 384, `counts`: Set this name to the result of calling `.agg(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.fill_null`: Replace Polars nulls with a chosen value/expression; null and floating-point NaN are distinct representations.
- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.round`: Round numeric values for a check/display rule; rounding quantities is compared to the original before acceptance.
- `save_table`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 385–394:

```
households = (counts.group_by("household_code").agg(
    pl.len().alias("flavors_bought"), pl.col("quantity").sum().alias("packages"),
    pl.col("quantity").pow(2).sum().alias("squared_packages"),
    pl.col("quantity").max().alias("largest_packages"))
    .with_columns((pl.col("squared_packages") / pl.col("packages").pow(2)).alias("flavor_HHI"),
                  (pl.col("largest_packages") / pl.col("packages")).alias("largest_flavor_share"))
    .select("household_code", "flavors_bought", "flavor_HHI", "largest_flavor_share"))
hh_path = output / "yogurt_household_flavors.csv"
households.sink_csv(hh_path, engine="streaming")
hh_metrics = pl.scan_csv(hh_path).select(pl.len(), pl.col("flavors_bought").mean(),
 ↪  pl.col("flavor_HHI").mean()).collect(engine="streaming").row(0)
```

**Read this block:**

- Line 385, `households`: Set this name to the result of calling `.select(...)`; the arguments and preceding method chain are shown above.
- Line 392, `hh_path`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 393: call `households.sink_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 394, `hh_metrics`: Set this name to the result of calling `.row(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.pow`: Raise each expression value to a power, such as squaring quantities for concentration calculations.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.sink_csv`: Execute/write the query result as CSV; unlike select/filter, this is a filesystem side effect.
- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.
- `.scan_csv`: Build a lazy delimited-text read plan; parser options determine types and missing values.

Source lines 395–404:

```
395 baskets = (weekly.group_by(["household_code", "week"]).agg(
396     pl.col("quantity").sum().alias("packages"), pl.len().alias("flavors_in_basket"),
397     pl.when(pl.len() == 1).then(pl.col("flavor").first()).otherwise(None).alias("single_flavor"))
398     .sort(["household_code", "week"])
399     .with_columns(pl.col("single_flavor").shift().over("household_code").alias("previous_flavor"),
400                   (pl.col("week").diff().over("household_code").dt.total_days() / 7).alias("gap_weeks"))
401     .with_columns((pl.col("single_flavor").is_not_null() & pl.col("previous_flavor").is_not_null()
402                    & (pl.col("gap_weeks") == 1)).fill_null(False).alias("eligible"))
403     .with_columns((~(pl.col("eligible") & (pl.col("single_flavor") == pl.col("previous_flavor")))).fill_null(False))
404                   .cast(pl.UInt64).cum_sum().alias("spell")))
```

**Read this block:**

- Line 395, `baskets`: Set this name to the result of calling `.with_columns(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.cum_sum`: Accumulate running totals, used to turn spell-start flags into spell IDs.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- `.over`: Evaluate a Polars expression within the specified group/window, preserving row-level results.
- `.total_days`: Convert a duration expression into its elapsed number of days before comparing weekly gaps.
- `.shift`: Move values by one row by default; over(household) prevents the shift crossing household boundaries.
- `.is_not_null`: Create a Polars nonnull mask; additional range checks can still be needed.
- `.otherwise`: Supply the fallback expression for rows not selected by the preceding condition.
- `.then`: Supply the value/expression selected where the preceding Polars condition is true.
- `.diff`: For SymPy differentiate algebra; for a Polars/pandas date/value expression calculate successive differences. The receiver decides which operation this name means.
- `.first`: Select the first value in the current expression/group; preceding ordering and null filtering determine which one.
- `.when`: Start a Polars row-wise conditional expression, completed by then and otherwise.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

Source lines 406–439:

```
406 with tempfile.TemporaryDirectory(prefix="baskets-", dir=output) as tmp:
407     checkpoint = Path(tmp) / "baskets.parquet"
408     baskets.sink_parquet(checkpoint, engine="streaming")
409     baskets = pl.scan_parquet(checkpoint)
410     baskets.sink_csv(output / "yogurt_weekly_baskets.csv", engine="streaming")
411     weekly.sink_csv(output / "yogurt_weekly_flavor_quantities.csv", engine="streaming")
412     transitions = (baskets.filter(pl.col("eligible")).group_by(["previous_flavor", "single_flavor"])
413                    .len().collect(engine="streaming").to_pandas())
414     if transitions.empty:
415         matrix = pd.DataFrame(0, index=flavors, columns=flavors)
```

```
416     else:
417         matrix = transitions.pivot(index="previous_flavor", columns="single_flavor",
    ↪ values="len").fillna(0).reindex(index=flavors, columns=flavors, fill_value=0)
418     matrix.index.name = "previous_flavor"
419     matrix.columns.name = "single_flavor"
420     matrix.to_csv(output / "yogurt_transition_counts.csv")
421     probabilities = matrix.div(matrix.sum(axis=1).replace(0, np.nan), axis=0)
422     probabilities.to_csv(output / "yogurt_transition_probabilities.csv")
423     print("\nYogurt transition counts", flush=True)
424     print(matrix.to_string(), flush=True)
425     print("\nYogurt transition probabilities", flush=True)
426     print(probabilities.to_string(float_format=lambda v: f"{v:.2f}"), flush=True)
427     spells = (baskets.filter(pl.col("single_flavor").is_not_null()).group_by(["household_code", "spell"])
428               .agg(pl.col("single_flavor").first().alias("flavor"), pl.len().alias("weeks")))
429     spells_path = Path(tmp) / "spells.parquet"
430     spells.sink_parquet(spells_path, engine="streaming")
431     spells = pl.scan_parquet(spells_path)
432     spells.sink_csv(output / "yogurt_single_flavor_spells.csv", engine="streaming")
433     spell_mean = spells.select(pl.col("weeks").mean()).collect(engine="streaming").item()
434     bm = baskets.select(
435         pl.len().alias("weeks"), (100 * (pl.col("flavors_in_basket") > 1).mean()).alias("mixed"),
436         (100 * (pl.col("packages") > 7).mean()).alias("above_seven"), pl.col("eligible").sum().alias("pairs"),
437         (100 * (pl.col("single_flavor") !=
    ↪ pl.col("previous_flavor")).filter(pl.col("eligible")).mean()).alias("switch_rate"),
438         pl.col("gap_weeks").median().alias("gap"),
439     ).collect(engine="streaming").row(0, named=True)
```

**Read this block:**

- Line 406: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 407, `checkpoint`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 408: call `baskets.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 409, `baskets`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 410: call `baskets.sink_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 411: call `weekly.sink_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 412, `transitions`: Set this name to the result of calling `.to_pandas(...)`; the arguments and preceding method chain are shown above.
- Line 414: test `transitions.empty`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 415, `matrix`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 417, `matrix`: Set this name to the result of calling `.reindex(...)`; the arguments and preceding method chain are shown above.
- Line 418, `matrix.index.name`: Set this name to the literal text `'previous_flavor'`.
- Line 419, `matrix.columns.name`: Set this name to the literal text `'single_flavor'`.
- Line 420: call `matrix.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 421, `probabilities`: Set this name to the result of calling `matrix.div(...)`; the arguments and preceding method chain are shown above.
- Line 422: call `probabilities.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 423: call `print` with the displayed arguments. No assignment stores its return here; its documented

side effect or assertion is what matters.
- Line 424: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 425: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 426: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 427, `spells`: Set this name to the result of calling `.agg(...)`; the arguments and preceding method chain are shown above.
- Line 429, `spells_path`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 430: call `spells.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 431, `spells`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 432: call `spells.sink_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 433, `spell_mean`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 434, `bm`: Set this name to the result of calling `.row(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.
- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `.to_pandas`: Convert a Polars table to pandas, crossing from query/table preparation to eager labeled analysis.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `.div`: Divide a pandas object with explicit alignment options; axis=0 aligns the denominator by rows.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.item`: Extract a scalar from a one-element array/one-cell result rather than passing a table wrapper onward.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.reindex`: Align to an explicitly requested row/column order, adding missing labels with a chosen fill value if provided.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.
- `.fillna`: Replace pandas missing entries with the provided fallback; it can change downstream denominators or validity checks.
- `.pivot`: Reshape long labels into matrix columns; index/columns/values specify their roles.
- `.median`: Compute the middle-value statistic rather than a mean; global medians cannot generally be averaged across batches.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 440–447:

```
440 rows = [
441     ("Matched households", hh_metrics[0]), ("Observed yogurt purchase weeks", bm["weeks"]),
442     ("Mean flavors bought per household", hh_metrics[1]), ("Mean household flavor HHI (0 to 1)", hh_metrics[2]),
443     ("Mixed-flavor weeks (%)", bm["mixed"]), ("Weeks above seven packages (%)", bm["above_seven"]),
444     ("Eligible adjacent single-flavor week pairs", bm["pairs"]), ("Switch rate in eligible pairs (%)",
    ↪  bm["switch_rate"]),
```

```
445     ("Mean single-flavor spell length (consecutive calendar weeks)", spell_mean), ("Median gap between purchase
    ↪  weeks", bm["gap"]),
446 ]
447 save_table(pd.DataFrame(rows, columns=["metric", "value"]), output, "yogurt_dynamics", "Matched yogurt sample:
    ↪  purchase dynamics")
```

**Read this block:**

- Line 440, `rows`: Set this name to a list containing 10 displayed entries.
- Line 447: call `save_table` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 448–448:

```
448 return shares.to_pandas(), probabilities
```

**Read this block:**

- Line 448: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

## make hhi figures

Original function: `make_hhi_figures`. Source begins at line 451.

This reads just the saved `flavor_HHI` column and writes histogram and CDF PDFs. `usecols` avoids reading irrelevant household fields. `.dropna().to_numpy()` leaves a numeric vector. If it is empty, a bare return avoids dividing by zero. The histogram weights each household by `100 / n`, so heights are percentages. `linspace(0,1,41)` defines 40 bins.

The CDF uses sorted observations and cumulative percentages. `np.r_` concatenates an initial zero with the vectors to draw the starting segment. `where="post"` creates a right-continuous step plot. A loop over two `kind` strings shares styling/export code without confusing their different calculations. It reuses saved household summaries instead of rerunning a large private-data pipeline.

**Definition and stated intent:**

Source lines 451–452:

```
451 def make_hhi_figures(output):
452     """Equal-household distributions using all flavor labels, without top-20 truncation."""
```

`def` creates this callable; the parentheses name its inputs (`output`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 453–457:

```
453 import matplotlib
454 matplotlib.use("Agg")
455 import matplotlib.pyplot as plt
456
457 values = pd.read_csv(output / "yogurt_household_flavors.csv",
    ↪  usecols=["flavor_HHI"])["flavor_HHI"].dropna().to_numpy()
```

**Read this block:**

- Line 453: import `matplotlib`. An as-name is a convenient local alias, not another library.
- Line 454: call `matplotlib.use` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 455: import `matplotlib.pyplot as plt`. An as-name is a convenient local alias, not another library.
- Line 457, `values`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.use`: Select a Matplotlib backend; Agg renders to files without opening a GUI window.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- `.dropna`: Remove pandas missing observations before the following calculation.
- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.

Source lines 458–459:

```
458 if not len(values):
459     return
```

**Read this block:**

- Line 458: test `not len(values)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 459: bare return ends this call immediately with None. Later statements in the function are skipped.

**Syntax and calls used here:**

- `len`: Count elements/rows rather than summing their numerical values.

Source lines 460–472:

```
460 for kind in ["histogram", "cdf"]:
461     fig, ax = plt.subplots(figsize=(7, 5))
462     if kind == "histogram":
463         ax.hist(values, bins=np.linspace(0, 1, 41), weights=np.full(len(values), 100 / len(values)),
    ↪  color="steelblue")
464         ax.set_ylabel("Households (%)")
465     else:
466         ax.step(np.r_[0., np.sort(values)], np.r_[0., 100 * np.arange(1, len(values) + 1) / len(values)],
    ↪  where="post")
467         ax.set_ylabel("Cumulative households (%)")
468     ax.set(xlim=(0, 1), xlabel="Household flavor HHI (quantity shares)", title="Yogurt flavor concentration")
469     fig.tight_layout()
470     fig.savefig(output / f"flavor_hhi_{kind}.pdf")
471     plt.close(fig)
472     print(f"Saved HHI figure: {output / f'flavor_hhi_{kind}.pdf'}", flush=True)
```

**Read this block:**

- Line 460: for each entry yielded by `['histogram', 'cdf']`, bind/unpack it into `kind` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 461, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 462: test `kind == 'histogram'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 463: call `ax.hist` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 464: call `ax.set_ylabel` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 466: call `ax.step` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 467: call `ax.set_ylabel` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 468: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 469: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 470: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 471: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 472: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.
- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.tight_layout`: Adjust plot spacing to reduce overlap between labels and content.
- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.
- `.close`: Release a Figure or file resource, according to the receiving object.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.hist`: Bin observed values, with optional weights controlling whether heights are counts, percentages, or another unit.
- `.set_ylabel`: Set the vertical-axis label, whose unit should match the plotted values.
- `.step`: Draw an empirical step curve; where='post' extends each value to the right.
- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.
- `.full`: Allocate an array filled with one value, such as equal histogram weights.
- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.
- `.arange`: Create evenly stepped values with an excluded stop, commonly row positions 0 through n-1.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

## make figures

Original function: `make_figures`. Source begins at line 475.

Inputs are already computed flavor-share and transition tables. Sorting then `head(20)` selects a readable subset for plots, and `reindex` aligns matrix rows and columns with the displayed flavor order. It does not recompute transition denominators within that subset, so a plotted row need not sum to one.

`fig, ax = plt.subplots` separates a Figure container from its plotting Axes. `ax.bar` draws shares; `ax.imshow` colors transition cells. Two nested index loops place formatted text inside each cell. Tick rotation improves long-label readability. `tight_layout`, `savefig`, and `close` finish each output. Plotting inside the data aggregation itself would make it harder to skip figures or recreate them from existing summary tables.

**Definition and stated intent:**

Source lines 475–476:

```
475 def make_figures(shares, transitions, output):
476     # Limit figure dimensions; all flavors remain in the saved tables.
```

`def` creates this callable; the parentheses name its inputs (`shares`, `transitions`, `output`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 477–484:

```
477 shares = shares.sort_values("packages", ascending=False).head(20)
478 flavors = shares["flavor"].tolist()
479 transitions = transitions.reindex(index=flavors, columns=flavors)
480 # Import plotting only if requested (use --no-figures for tables alone).
481 import matplotlib
482
483 matplotlib.use("Agg")
484 import matplotlib.pyplot as plt
```

**Read this block:**

- Line 477, `shares`: Set this name to the result of calling `.head(...)`; the arguments and preceding method chain are shown above.
- Line 478, `flavors`: Set this name to the result of calling `shares['flavor'].tolist(...)`; the arguments and preceding method chain are shown above.
- Line 479, `transitions`: Set this name to the result of calling `transitions.reindex(...)`; the arguments and preceding method chain are shown above.
- Line 481: import `matplotlib`. An as-name is a convenient local alias, not another library.
- Line 483: call `matplotlib.use` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 484: import `matplotlib.pyplot as plt`. An as-name is a convenient local alias, not another library.

**Syntax and calls used here:**

- `.sort_values`: Order pandas rows by values; ascending can supply one direction for each listed sort key.
- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.
- `.reindex`: Align to an explicitly requested row/column order, adding missing labels with a chosen fill value if provided.
- `.use`: Select a Matplotlib backend; Agg renders to files without opening a GUI window.

Source lines 486–491:

```
486 fig, ax = plt.subplots(figsize=(max(6, len(flavors) * 0.35), 5))
487 ax.bar(shares["flavor"], shares["percent_packages"], color="steelblue")
488 ax.tick_params(axis="x", labelrotation=90)
489 ax.set(ylabel="Share of purchased packages (%)", title="Yogurt flavor shares (up to 20 most purchased flavors)")
490 fig.tight_layout()
491 fig.savefig(output / "flavor_shares.pdf")
```

**Read this block:**

- Line 486, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 487: call `ax.bar` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 488: call `ax.tick_params` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 489: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 490: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 491: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `len`: Count elements/rows rather than summing their numerical values.
- `.bar`: Draw bar heights from supplied categories and values; it does not compute the underlying shares.
- `.tick_params`: Adjust axis tick appearance, for example rotating long category labels.
- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.tight_layout`: Adjust plot spacing to reduce overlap between labels and content.
- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.

Source lines 492–505:

```
492 plt.close(fig)
493 side = max(6, len(flavors) * 0.4)
494 fig, ax = plt.subplots(figsize=(side, side))
```

```
image = ax.imshow(transitions.to_numpy(), vmin=0, vmax=1, cmap="Blues")
ax.set(
    xticks=range(len(flavors)),
    yticks=range(len(flavors)),
    xticklabels=flavors,
    yticklabels=flavors,
    xlabel="Current flavor",
    ylabel="Previous flavor",
    title="Adjacent single-flavor purchase weeks",
)
ax.tick_params(axis="x", labelrotation=90)
```

**Read this block:**

- Line 492: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 493, `side`: Set this name to the result of calling `max(...)`; the arguments and preceding method chain are shown above.
- Line 494, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 495, `image`: Set this name to the result of calling `ax.imshow(...)`; the arguments and preceding method chain are shown above.
- Line 496: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 505: call `ax.tick_params` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.close`: Release a Figure or file resource, according to the receiving object.
- `.imshow`: Display a numerical matrix as colored cells; row/column label alignment remains the caller's responsibility.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.

Source lines 506–515:

```
for i in range(len(flavors)):
    for j in range(len(flavors)):
        value = transitions.iloc[i, j]
        ax.text(
            j,
            i,
            f"{value:.2f}" if pd.notna(value) else "N/A",
            ha="center",
            va="center",
        )
```

**Read this block:**

- Line 506: for each entry yielded by `range(len(flavors))`, bind/unpack it into `i` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 507: for each entry yielded by `range(len(flavors))`, bind/unpack it into `j` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 508, `value`: Set this name to the entry/slice selected by `transitions.iloc[i, j]`; brackets select from the object to their left.
- Line 509: call `ax.text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.text`: Place a text label on the axes/Figure at the supplied coordinates.
- `.notna`: Create a pandas mask marking nonmissing values; it does not establish numeric positivity or finiteness.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 516–519:

```
516 fig.colorbar(image, ax=ax)
517 fig.tight_layout()
518 fig.savefig(output / "flavor_transitions.pdf")
519 plt.close(fig)
```

**Read this block:**

- Line 516: call `fig.colorbar` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 517: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 518: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 519: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.colorbar`: Add a legend relating matrix colors to numerical values.

**Output reminder:** reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

### run

Original function: `run`. Source begins at line 522.

This orchestrates the summary workflow and writes final artifacts. `sources` and `source_partitions` are dictionaries keyed by sample prefix, keeping full panel and matched yogurt objects separate. It optionally supplements flavor metadata, then prepares each population, writes reusable hash partitions, computes household/demographic summaries, and audits annual coverage.

The second loop runs detailed then grouped flavor taxonomies. Partition-level weekly outputs remain sparse, while small audit tables are summed. The last reference therefore holds the grouped mapping, which is saved for estimation. The final export intentionally reopens the original purchase file, rather than the demographic projection, to preserve store/DMA and other fields. A semi join keeps eligible households and a left join attaches the resolved flavor.

`prepared.drop("flavor")` avoids two competing label columns when reprocessing an already labeled input. The resolved-path check prevents writing over the original source. This orchestration keeps the population restrictions explicit; running the estimator on an arbitrary intermediate table can produce another sample even if its filename looks similar.

**Definition and stated intent:**

Source lines 522–522:

```
522 def run(args, output, work):
```

`def` creates this callable; the parentheses name its inputs (`args`, `output`, `work`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 523–524:

```
523 products_extra = args.products_extra or args.data_dir / "RMS/combined/products_extra.parquet"
524 flavor_path = args.flavors
```

**Read this block:**

- Line 523, `products_extra`: Set this name to the Boolean condition `args.products_extra or args.data_dir / 'RMS/combined/products_...`; it can be a scalar or a row/array mask depending on its inputs.
- Line 524, `flavor_path`: Set this name to the attribute `args.flavors` of the object before the dot.

Source lines 525–532:

```
525 if products_extra.exists():
526     recovered_path = output / "recovered_flavor_reference.csv"
527     if recovered_path.resolve() == flavor_path.resolve():
528         recovered_path = output / "supplemented_flavor_reference.csv"
529     recover_flavor_reference(flavor_path, products_extra, recovered_path)
530     flavor_path = recovered_path
531 else:
532     print(f"Flavor recovery source not found: {products_extra}; using the CSV reference.", flush=True)
```

**Read this block:**

- Line 525: test `products_extra.exists()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 526, `recovered_path`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 527: test `recovered_path.resolve() == flavor_path.resolve()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 528, `recovered_path`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 529: call `recover_flavor_reference` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 530, `flavor_path`: Set this name to the object currently named `recovered_path`; plain assignment does not copy it.
- Line 532: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- `recover_flavor_reference`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 533–534:

```
533 sources = {}
534 source_partitions = {}
```

**Read this block:**

- Line 533, `sources`: Set this name to a dictionary from the displayed keys and values.
- Line 534, `source_partitions`: Set this name to a dictionary from the displayed keys and values.

Source lines 535–563:

```
535 for prefix, filename in [("full_panel", "merged_panel.parquet"), ("matched_yogurt",
    ↪ "yogurt_estimation_sample.parquet")]:
```

```
536     source = scan_purchases(args.data_dir / filename, args.first_year, args.last_year)
537     if prefix == "matched_yogurt":
538         covered = (pl.col("price").is_finite() & (pl.col("price") > 0)).fill_null(False)
539         audit = source.select(pl.len().alias("purchase_lines_before_price_filter"),
540                               (~covered).sum().alias("purchase_lines_without_price_coverage"))
541         save_table(audit, output, "yogurt_price_coverage", "Yogurt matched-price coverage")
542         source = source.filter(covered)
543         source = repeated_yogurt_households(source)
544     parts = partition_purchases(source, work, prefix, args.household_batches)
545     source_partitions[prefix] = parts
546     source = pl.concat(parts)
547     sources[prefix] = source
548     households = household_batches(source, work, prefix, args.household_batches, partitions=parts)
549     if households.select(pl.len()).collect(engine="streaming").item() == 0:
550         raise ValueError(f"Empty {prefix} sample in selected years.")
551     demographic_tables(households, output, prefix)
552     households.sink_csv(output / f"{prefix}_households.csv", engine="streaming")
553     # Exact distinct-trip counts for one year at a time bound aggregation state.
554     years = source.select("panel_year").unique().sort("panel_year").collect(engine="streaming")["panel_year"]
555     coverage_rows = []
556     for year in years:
557         coverage_rows.append(source.filter(pl.col("panel_year") == year).select(
558             pl.lit(year).alias("panel_year"),
559             pl.col("household_code").n_unique().alias("households"),
560             pl.col("trip_code_uc").n_unique().alias("trips"), pl.len().alias("purchase_lines"),
561             pl.col("quantity").sum().alias("packages")).collect(engine="streaming"))
562     coverage = pl.concat(coverage_rows)
563     save_table(coverage, output, f"{prefix}_year_coverage", "Sample coverage by year")
```

**Read this block:**

- Line 535: for each entry yielded by `[('full_panel', 'merged_panel.parquet'), ('matched_yogurt', 'y...`, bind/unpack it into `(prefix, filename)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 536, `source`: Set this name to the result of calling `scan_purchases(...)`; the arguments and preceding method chain are shown above.
- Line 537: test `prefix == 'matched_yogurt'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 538, `covered`: Set this name to the result of calling `.fill_null(...)`; the arguments and preceding method chain are shown above.
- Line 539, `audit`: Set this name to the result of calling `source.select(...)`; the arguments and preceding method chain are shown above.
- Line 541: call `save_table` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 542, `source`: Set this name to the result of calling `source.filter(...)`; the arguments and preceding method chain are shown above.
- Line 543, `source`: Set this name to the result of calling `repeated_yogurt_households(...)`; the arguments and preceding method chain are shown above.
- Line 544, `parts`: Set this name to the result of calling `partition_purchases(...)`; the arguments and preceding method chain are shown above.
- Line 545, `source_partitions[prefix]`: Store the object currently named `parts`; plain assignment does not copy it in the selected entry/column.
- Line 546, `source`: Set this name to the result of calling `pl.concat(...)`; the arguments and preceding method chain are shown above.
- Line 547, `sources[prefix]`: Store the object currently named `source`; plain assignment does not copy it in the selected entry/column.
- Line 548, `households`: Set this name to the result of calling `household_batches(...)`; the arguments and preceding method chain are shown above.
- Line 549: test `households.select(pl.len()).collect(engine='streaming').item() == 0`. Execute the

indented branch only when it is true; an else/elif branch handles its alternative.
- Line 550: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 551: call `demographic_tables` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 552: call `households.sink_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 554, `years`: Set this name to the entry/slice selected by `source.select('panel_year').unique().sort('panel_year').c...`; brackets select from the object to their left.
- Line 555, `coverage_rows`: Set this name to the list `[]`.
- Line 556: for each entry yielded by `years`, bind/unpack it into `year` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 557: call `coverage_rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 562, `coverage`: Set this name to the result of calling `pl.concat(...)`; the arguments and preceding method chain are shown above.
- Line 563: call `save_table` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `scan_purchases`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `partition_purchases`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `household_batches`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `demographic_tables`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.sink_csv`: Execute/write the query result as CSV; unlike select/filter, this is a filesystem side effect.
- `save_table`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.fill_null`: Replace Polars nulls with a chosen value/expression; null and floating-point NaN are distinct representations.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `repeated_yogurt_households`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.item`: Extract a scalar from a one-element array/one-cell result rather than passing a table wrapper onward.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.
- `.is_finite`: Build the Polars finite-number test used to reject missing/infinite quotes.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.lit`: Build a literal-valued Polars expression, distinguishing constant text/numbers from column names.
- `.n_unique`: Count distinct Polars values within the relevant aggregation scope.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

Source lines 564–588:

```
564 for detailed in [True, False]:
565     folder = output if detailed else output / "aggregated_flavors"
566     folder.mkdir(parents=True, exist_ok=True)
567     reference = flavor_reference(flavor_path, detailed=detailed, output=output)
568     flavors = sorted(reference["flavor"].unique().to_list()) if detailed else FLAVORS
569     if detailed:
570         reference.write_csv(output / "detailed_flavor_reference.csv")
571     # Process household batches independently, keeping weekly baskets long.
572     audit_total = None
573     tag = "detailed" if detailed else "grouped"
574     parts = source_partitions["matched_yogurt"]
575     for batch, subset in enumerate(parts):
576         began = time.monotonic()
577         print(f"{tag}: yogurt week batch {batch + 1}/{len(parts)}", flush=True)
578         weekly, audit = yogurt_weeks(subset, reference, flavors, output=folder)
579         weekly.sink_parquet(work / f"{tag}_weeks_{batch}.parquet", engine="streaming")
580         print(f"{tag}: yogurt week batch {batch + 1} completed in {time.monotonic() - began:.1f}s", flush=True)
581         audit_total = audit if audit_total is None else pl.concat([audit_total, audit]).select(pl.all().sum())
582     if detailed:
583         save_table(audit_total, output, "yogurt_sample_audit", "Yogurt sample audit")
584     weekly = pl.scan_parquet(work / f"{tag}_weeks_*.parquet")
585     shares, transitions = flavor_summaries(weekly, folder, flavors)
586     if not args.no_figures:
587         make_hhi_figures(folder)
588         make_figures(shares, transitions, folder)
```

**Read this block:**

- Line 564: for each entry yielded by `[True, False]`, bind/unpack it into `detailed` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 565, `folder`: Set this name to the true-branch value if `detailed` holds, otherwise the else-branch value.
- Line 566: call `folder.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 567, `reference`: Set this name to the result of calling `flavor_reference(...)`; the arguments and preceding method chain are shown above.
- Line 568, `flavors`: Set this name to the true-branch value if `detailed` holds, otherwise the else-branch value.
- Line 569: test `detailed`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 570: call `reference.write_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 572, `audit_total`: Set this name to the literal value `None`.
- Line 573, `tag`: Set this name to the true-branch value if `detailed` holds, otherwise the else-branch value.
- Line 574, `parts`: Set this name to the entry/slice selected by `source_partitions['matched_yogurt']`; brackets select from the object to their left.
- Line 575: for each entry yielded by `enumerate(parts)`, bind/unpack it into `(batch, subset)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 576, `began`: Set this name to the result of calling `time.monotonic(...)`; the arguments and preceding method chain are shown above.
- Line 577: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 578, `(weekly, audit)`: Unpack the result of calling `yogurt_weeks(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 579: call `weekly.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 580: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 581, `audit_total`: Set this name to the true-branch value if `audit_total is None` holds, otherwise the else-branch value.

- Line 582: test `detailed`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 583: call `save_table` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 584, `weekly`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 585, `(shares, transitions)`: Unpack the result of calling `flavor_summaries(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 586: test `not args.no_figures`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 587: call `make_hhi_figures` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 588: call `make_figures` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- `flavor_reference`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `flavor_summaries`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.
- `.write_csv`: Write an eager Polars table to CSV; it does not return a newly analyzed dataset.
- `.monotonic`: Read a monotonic clock for elapsed durations rather than a wall-clock date/time.
- `yogurt_weeks`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `make_hhi_figures`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `make_figures`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `len`: Count elements/rows rather than summing their numerical values.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 591–598:

```
591 reference.write_csv(output / "estimation_flavor_reference.csv")
592 print(f"Saved estimation flavor mapping to {output / 'estimation_flavor_reference.csv'}", flush=True)
593 # Retain every original purchase column needed downstream, including store
594 # and DMA fields that demographic projections do not otherwise need.
595 eligible = sources["matched_yogurt"].select("household_code").unique().collect(engine="streaming")
596 prepared = pl.scan_parquet(args.data_dir / "yogurt_estimation_sample.parquet")
597 original_names = prepared.collect_schema().names()
598 price_column = next((c for c in ["price", "price_rms"] if c in original_names), None)
```

**Read this block:**

- Line 591: call `reference.write_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 592: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 595, `eligible`: Set this name to the result of calling `.collect(...)`; the arguments and preceding method chain are shown above.

- Line 596, `prepared`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 597, `original_names`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.
- Line 598, `price_column`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.
- `next`: Take the first item from an iterator, or the supplied default when no item matches.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Source lines 599–600:

```
599 if price_column is None:
600     raise ValueError("Yogurt purchase table needs price or price_rms for matched-price coverage.")
```

**Read this block:**

- Line 599: test `price_column is None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 600: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 601–602:

```
601 prepared = prepared.filter(pl.col(price_column).cast(pl.Float64).is_finite()
602                            & (pl.col(price_column).cast(pl.Float64) > 0))
```

**Read this block:**

- Line 601, `prepared`: Set this name to the result of calling `prepared.filter(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.

Source lines 603–604:

```
603 if "flavor" in original_names:
604     prepared = prepared.drop("flavor")
```

**Read this block:**

- Line 603: test `'flavor' in original_names`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 604, `prepared`: Set this name to the result of calling `prepared.drop(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.drop`: Remove the specified columns/labels; inspect the receiver because list and table methods have different APIs.

Source lines 605–610:

```
605 prepared = (prepared.filter(pl.col("panel_year").is_between(args.first_year, args.last_year))
606             .with_columns(normalized_upc_expr(),
607                           pl.col("purchase_date").cast(pl.String).str.slice(0, 10).str.to_date())
608             .join(eligible.lazy(), on="household_code", how="semi")
609             .join(reference.lazy(), on="upc", how="left"))
610 sample_path = output / "yogurt_estimation_sample.parquet"
```

**Read this block:**

- Line 605, `prepared`: Set this name to the result of calling `.join(...)`; the arguments and preceding method chain are shown above.
- Line 610, `sample_path`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

**Syntax and calls used here:**

- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `.lazy`: Wrap an eager Polars table as a lazy query so later operations can be planned together.
- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `normalized_upc_expr`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.to_date`: Parse text into calendar-date values before weekday extraction and date arithmetic.
- `.is_between`: Build an inclusive interval check by default, such as selecting the requested panel-year range.
- `.slice`: Take a substring starting at a position for a given length, here often the first ten date characters.

Source lines 611–612:

```
611 if sample_path.resolve() == (args.data_dir / "yogurt_estimation_sample.parquet").resolve():
612     raise ValueError("Choose an output folder separate from the original data directory.")
```

**Read this block:**

- Line 611: test `sample_path.resolve() == (args.data_dir / 'yogurt_estimation_sample.parquet')...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 612: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 613–615:

```
613 prepared.sink_parquet(sample_path, engine="streaming")
614 print(f"Saved full prepared yogurt purchase table to {sample_path}", flush=True)
615 print(f"Saved summaries to {output}", flush=True)
```

**Read this block:**

- Line 613: call `prepared.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 614: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 615: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Output reminder:** reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

## main

Original function: `main`. Source begins at line 618.

This parses summary options and chooses ordinary pipeline work or HHI-only work. `args.output or default_path` uses a fallback when the optional output is None. The HHI-only branch checks its saved input, makes graphs, optionally repeats for grouped summaries, then returns before raw-data setup. It is a true early exit, not merely a plotting flag.

Ordinary runs create the output directory and enter a TemporaryDirectory context there, passing its Path to `run`. Context management ensures intermediate partitions are cleaned on ordinary completion or exceptions. Keeping temporary work under output makes disk requirements predictable. `--no-figures` and `--hhi-only` are rejected together because their requested actions conflict.

**Definition and stated intent:**

Source lines 618–618:

```
618 def main():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 619–624:

```
619 parser = argparse.ArgumentParser(description=__doc__)
620 parser.add_argument("--data-dir", type=Path, default=DATA_DIR)
621 parser.add_argument("--flavors", type=Path, default=FLAVOR_FILE)
622 parser.add_argument("--products-extra", type=Path)
623 parser.add_argument("--output", type=Path)
624 parser.add_argument("--first-year", type=int, default=2006)
```

**Read this block:**

- Line 619, `parser`: Set this name to the result of calling `argparse.ArgumentParser(...)`; the arguments and preceding method chain are shown above.
- Line 620: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 621: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 622: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 623: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 624: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.ArgumentParser`: Create the command-line parser object that will register and validate options.
- `.add_argument`: Register a command-line option and its type/default/choices/flag behavior before parsing.

Source lines 625–631:

```
625 parser.add_argument("--last-year", type=int, default=2015)
626 parser.add_argument("--household-batches", type=int, default=16,
627                     help="Household disk partitions; more batches reduce aggregation memory (default 16).")
628 parser.add_argument("--no-figures", action="store_true")
629 parser.add_argument("--hhi-only", action="store_true",
630                     help="Plot HHI from existing yogurt_household_flavors.csv in --output; skip raw data and all
                     ↪  summaries.")
631 args = parser.parse_args()
```

**Read this block:**

- Line 625: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 626: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 628: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 629: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 631, `args`: Set this name to the result of calling `parser.parse_args(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.parse_args`: Read command-line arguments into a Namespace of named attributes.

Source lines 632–633:

```
632 if args.hhi_only and args.no_figures:
633     parser.error("--hhi-only cannot be combined with --no-figures")
```

**Read this block:**

- Line 632: test `args.hhi_only and args.no_figures`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 633: call `parser.error` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.error`: Report invalid command-line input and exit through argparse; it is not a recoverable data-cleaning step here.

Source lines 634–635:

```
634 if args.household_batches < 1:
635     parser.error("--household-batches must be positive")
```

**Read this block:**

- Line 634: test `args.household_batches < 1`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 635: call `parser.error` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 636–636:

```
636 output = args.output or args.data_dir / "summary_outputs"
```

**Read this block:**

- Line 636, `output`: Set this name to the Boolean condition `args.output or args.data_dir / 'summary_outputs'`; it can be a scalar or a row/array mask depending on its inputs.

Source lines 637–645:

```
637 if args.hhi_only:
638     source = output / "yogurt_household_flavors.csv"
639     if not source.exists():
640         parser.error(f"HHI-only needs the saved detailed household table: {source}")
641     make_hhi_figures(output)
642     grouped = output / "aggregated_flavors"
643     if (grouped / "yogurt_household_flavors.csv").exists():
644         make_hhi_figures(grouped)
645     return
```

**Read this block:**

- Line 637: test `args.hhi_only`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 638, `source`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 639: test `not source.exists()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 640: call `parser.error` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 641: call `make_hhi_figures` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 642, `grouped`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

- Line 643: test `(grouped / 'yogurt_household_flavors.csv').exists()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 644: call `make_hhi_figures` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 645: bare return ends this call immediately with None. Later statements in the function are skipped.

**Syntax and calls used here:**

- `make_hhi_figures`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 646–646:

```
646 output.mkdir(parents=True, exist_ok=True)
```

**Read this block:**

- Line 646: call `output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.

Source lines 647–648:

```
647 with tempfile.TemporaryDirectory(prefix="summary-work-", dir=output) as tmp:
648     run(args, output, Path(tmp))
```

**Read this block:**

- Line 647: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 648: call `run` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.
- `run`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

# est_code: source walkthrough

## est_code/bootstrap_se.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

### Module setup and direct-execution blocks

Source lines 1–5:

```
"""Household bootstrap for the two-stage demand estimator."""
import numpy as np
import pandas as pd

from price_control import add_price_controls
```

**Read this block:**

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 3: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 5: from `price_control`, import `add_price_controls` into this module's namespace. Importing can execute the imported module's top-level setup.

### resample households

Original function: `resample_households`. Source begins at line 8.

Input is the weekly frame and a NumPy random Generator. Output is a concatenated bootstrap sample with the same number of household draws as the original. The comprehension stores whole household subtables. `rng.integers(0, H, size=H)` draws indices with replacement; its upper endpoint H is excluded.

`enumerate(selected)` supplies a fresh ID and selected old-group position. `.copy()` is essential before changing that ID so the original history is not modified. If selection is `[2,2,0]`, two copies of old household 2 get new IDs 0 and 1, keeping their likelihoods separate. Concatenating with `ignore_index=True` gives clean row labels. Resampling individual weeks would break persistent household preferences and within-household dependence.

**Definition and stated intent:**

Source lines 8–13:

```
def resample_households(frame, rng):
    """Keep whole histories; repeated households get distinct bootstrap IDs.

    Stocks/Xi stay attached to the original histories. Rebuilding stocks after
    dropping incomplete price weeks would incorrectly erase observed purchases.
    """
```

`def` creates this callable; the parentheses name its inputs (`frame`, `rng`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 14–16:

```
14 groups = [group for _, group in frame.groupby("household_code", sort=True)]
15 selected = rng.integers(0, len(groups), size=len(groups))
16 samples = []
```

**Read this block:**

- Line 14, `groups`: Set this name to a list computed by the displayed for-expression.
- Line 15, `selected`: Set this name to the result of calling `rng.integers(...)`; the arguments and preceding method chain are shown above.
- Line 16, `samples`: Set this name to the list `[]`.

**Syntax and calls used here:**

- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `.integers`: Draw integer positions with replacement by default; the high endpoint is excluded.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 17–20:

```
17 for new_id, old_id in enumerate(selected):
18     group = groups[old_id].copy()
19     group["household_code"] = new_id
20     samples.append(group)
```

**Read this block:**

- Line 17: for each entry yielded by `enumerate(selected)`, bind/unpack it into `(new_id, old_id)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 18, `group`: Set this name to the result of calling `groups[old_id].copy(...)`; the arguments and preceding method chain are shown above.
- Line 19, `group['household_code']`: Store the object currently named `new_id`; plain assignment does not copy it in the selected entry/column.
- Line 20: call `samples.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.

Source lines 21–21:

```
21 return pd.concat(samples, ignore_index=True)
```

**Read this block:**

- Line 21: return the result of calling `pd.concat(...)`; the arguments and preceding method chain are shown above. This ends the call.

## bootstrap standard errors

Original function: `bootstrap_standard_errors`. Source begins at line 24.

The `estimate` argument is a callable, not an estimate array. Each repetition resamples households, refits enabled price controls, calls `estimate(sample, initial)`, and records its result. `record.update` adds named fields to an existing dictionary. Accepted vectors go to `values`; all attempts go to `records`, so failure does not disappear from the audit trail.

The `try/except` catches expected data/linear-algebra problems. Every attempt checkpoints the records as CSV. After the loop, the threshold requires at least 20 usable fits and at least 80% success. Returning `(None, report)` explicitly signals inference was withheld. Otherwise, `std(ddof=1)` uses sample rather than population SD, `quantile` forms empirical percentile intervals, and `np.cov(..., rowvar=False)` treats columns as parameters. Analytic Hessian SEs would be cheaper but would require handling the estimated first-stage residuals and household dependence; this implementation uses a two-stage cluster bootstrap.

**Definition and stated intent:**

Source lines 24–30:

```
24 def bootstrap_standard_errors(frame, args, estimate, initial, names):
25     """Refit price controls AND demand in every household-cluster replication.
26 
27     estimate(frame, initial) returns a scipy optimizer result. All replications
28     use the same integration draws as the main fit. Retail instruments stay
29     fixed: this is inference conditional on the observed retail price universe.
30     """
```

`def` creates this callable; the parentheses name its inputs (`frame`, `args`, `estimate`, `initial`, `names`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 31–33:

```
31 rng = np.random.default_rng(args.seed + 10000)
32 records = []
33 values = []
```

**Read this block:**

- Line 31, `rng`: Set this name to the result of calling `np.random.default_rng(...)`; the arguments and preceding method chain are shown above.
- Line 32, `records`: Set this name to the list `[]`.
- Line 33, `values`: Set this name to the list `[]`.

**Syntax and calls used here:**

- `.default_rng`: Create a local seeded random Generator, avoiding dependence on a shared global random stream.

Source lines 34–53:

```
34 for replication in range(args.bootstrap_reps):
35     record = {"replication": replication + 1, "converged": False}
36     try:
37         sample = resample_households(frame, rng)
38         if not args.no_price_control:
39             sample, _ = add_price_controls(sample)
40         fit = estimate(sample, initial)
41         record.update(converged=bool(fit.success), objective=float(fit.fun),
42                       message=str(fit.message))
43         record.update(dict(zip(names, fit.x)))
44         if fit.success and np.isfinite(fit.x).all() and np.isfinite(fit.fun):
45             values.append(fit.x)
46     except (ValueError, np.linalg.LinAlgError) as error:
47         # An unidentifiable resample is recorded, never silently accepted.
48         record["message"] = str(error)
49     records.append(record)
50     # Checkpoint after each replication in case a long cluster job ends.
```

```
51      pd.DataFrame(records).to_csv(args.output / "bootstrap_draws.csv", index=False)
52      print(f"Bootstrap {replication+1}/{args.bootstrap_reps}: "
53            f"{len(values)} usable fits", flush=True)
```

**Read this block:**

- Line 34: for each entry yielded by `range(args.bootstrap_reps)`, bind/unpack it into `replication` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 35, `record`: Set this name to a dictionary with named entries `replication`, `converged`.
- Line 36: try the indented operation; matching except blocks handle specified exceptions. Unhandled error types still propagate instead of being silently treated as success.
- Line 37, `sample`: Set this name to the result of calling `resample_households(...)`; the arguments and preceding method chain are shown above.
- Line 38: test `not args.no_price_control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 39, `(sample, _)`: Unpack the result of calling `add_price_controls(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 40, `fit`: Set this name to the result of calling `estimate(...)`; the arguments and preceding method chain are shown above.
- Line 41: call `record.update` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 43: call `record.update` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 44: test `fit.success and np.isfinite(fit.x).all() and np.isfinite(fit.fun)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 45: call `values.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 46: catch `(ValueError, np.linalg.LinAlgError)` from the preceding try; the optional as-name holds the error object for logging.
- Line 48, `record['message']`: Store the result of calling `str(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 49: call `records.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 51: call `pd.DataFrame(records).to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 52: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `resample_households`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `estimate`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.update`: Add/replace dictionary entries in place; unrelated entries remain available.
- `add_price_controls`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- `str`: Convert to text for paths, messages, or serialization.
- `bool`: Convert scalar truthiness to an explicit Boolean flag.

- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `len`: Count elements/rows rather than summing their numerical values.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 54–58:

```
valid = len(values)
report = dict(method=("household bootstrap" if args.no_price_control
                      else "household bootstrap, refitting both stages"),
              requested=args.bootstrap_reps, usable=valid,
              conditional_on="retail instruments and fixed integration draws")
```

**Read this block:**

- Line 54, `valid`: Set this name to the result of calling `len(...)`; the arguments and preceding method chain are shown above.
- Line 55, `report`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 60–62:

```
if valid < max(20, int(np.ceil(.8 * args.bootstrap_reps))):
    report["status"] = "insufficient converged replications; no SEs reported"
    return None, report
```

**Read this block:**

- Line 60: test `valid < max(20, int(np.ceil(0.8 * args.bootstrap_reps)))`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 61, `report['status']`: Store the literal text `'insufficient converged replications; no SEs reported'` in the selected entry/column.
- Line 62: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

**Syntax and calls used here:**

- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `int`: Convert to an integer; it does not by itself validate that a source value was already integral.
- `.ceil`: Round upward, so the minimum usable-bootstrap threshold meets the requested success fraction.

Source lines 63–70:

```
estimates = np.asarray(values)
table = pd.DataFrame({"parameter": names,
                      "std_error": estimates.std(axis=0, ddof=1),
                      "ci_lower": np.quantile(estimates, .025, axis=0),
                      "ci_upper": np.quantile(estimates, .975, axis=0)})
covariance = pd.DataFrame(np.cov(estimates, rowvar=False), index=names, columns=names)
covariance.to_csv(args.output / "covariance.csv")
report["status"] = "computed; percentile 95% intervals"
```

**Read this block:**

- Line 63, `estimates`: Set this name to the result of calling `np.asarray(...)`; the arguments and preceding method chain are shown above.
- Line 64, `table`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.

- Line 68, `covariance`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 69: call `covariance.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 70, `report['status']`: Store the literal text `'computed; percentile 95% intervals'` in the selected entry/column.

**Syntax and calls used here:**

- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `.std`: Compute standard deviation; ddof=1 requests sample SD rather than dividing squared deviations by the full count.
- `.quantile`: Find empirical quantiles, supplying percentile bootstrap endpoints rather than normal-Wald formulas.
- `.cov`: Compute covariance; rowvar=False means columns represent parameters rather than treating rows as variables.

Source lines 71–71:

```
71 return table, report
```

**Read this block:**

- Line 71: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

# est_code/est.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

## Module setup and direct-execution blocks

Source lines 1–12:

```
1 """Weekly yogurt demand: fixed or mixed logit, with or without gamma.
2
3 Read README.md first. Run `python3 est.py --help` for server paths and options.
4 The raw data are not required for importing or testing the likelihood.
5 """
6
7 from pathlib import Path
8 import argparse
9 import json
10 import time
11
12 import numpy as np
```

**Read this block:**

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 7: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 8: import `argparse`. An as-name is a convenient local alias, not another library.
- Line 9: import `json`. An as-name is a convenient local alias, not another library.
- Line 10: import `time`. An as-name is a convenient local alias, not another library.
- Line 12: import `numpy as np`. An as-name is a convenient local alias, not another library.

Source lines 13–19:

```
13 import pandas as pd
14 import polars as pl
15 from scipy.optimize import minimize
16 from scipy.special import logsumexp
17
18 from price_control import check_dma, add_price_controls
19 from bootstrap_se import bootstrap_standard_errors
```

**Read this block:**

- Line 13: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 14: import `polars as pl`. An as-name is a convenient local alias, not another library.
- Line 15: from `scipy.optimize`, import `minimize` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 16: from `scipy.special`, import `logsumexp` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 18: from `price_control`, import `check_dma, add_price_controls` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 19: from `bootstrap_se`, import `bootstrap_standard_errors` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 21–24:

```
21 FLAVORS = ["other", "berry", "plain"]
22 DATA_DIR = Path("/scratch/dtm63837/Kilts_Panel/Data")
23 # Set this to the flavor-reference CSV on the Slurm server.
24 FLAVOR_FILE = DATA_DIR / "summary_outputs/estimation_flavor_reference.csv"
```

**Read this block:**

- Line 21, `FLAVORS`: Set this name to the list `['other', 'berry', 'plain']`.
- Line 22, `DATA_DIR`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 24, `FLAVOR_FILE`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

**Syntax and calls used here:**

- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 686–687:

```
686 if __name__ == "__main__":
687     main()
```

**Read this block:**

- Line 686: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 687: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

## normalize upcs

Original function: `normalize_upcs`. Source begins at line 27.

**Job and contract.** Accept a pandas Series of UPC-like values and return a Series of canonical string identifiers. `values.astype("string")` selects pandas' nullable string dtype; `.str` applies a string operation to every element. `.strip()` removes surrounding whitespace. `numeric` is a Boolean Series, not a single true/false answer.

`fullmatch` requires the entire string to fit the numeric pattern. Replacement removes a trailing decimal-zero suffix, then `lstrip("0")` removes leading zeros. The last `where` keeps the original text where `~numeric` is true and uses the canonical version elsewhere. Thus `"00123.0"` becomes `"123"`, `"000"` becomes `"0"`, and `"A001"` stays `"A001"`. The inner `.replace("", "0")` fixes the empty string produced when all digits were zero.

**Why this choice?** Converting to `float` then `int` looks shorter but can round long identifiers and fails on nonnumeric UPCs. A Python loop would work but duplicates the same string operations per row and loses convenient missing-value handling. `where` is easy to misread: its true branch retains the receiver.

**Definition and stated intent:**

Source lines 27–28:

```
27 def normalize_upcs(values):
28     """Canonical numeric UPC keys without floating-point conversion."""
```

`def` creates this callable; the parentheses name its inputs (`values`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 29–31:

```
29 text = values.astype("string").str.strip()
30 numeric = text.str.fullmatch(r"[0-9]+(?:\.0+)?", na=False)
31 canonical = text.str.replace(r"\.0+$", "", regex=True).str.lstrip("0")
```

**Read this block:**

- Line 29, `text`: Set this name to the result of calling `.strip(...)`; the arguments and preceding method chain are shown above.
- Line 30, `numeric`: Set this name to the result of calling `text.str.fullmatch(...)`; the arguments and preceding method chain are shown above.
- Line 31, `canonical`: Set this name to the result of calling `.lstrip(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.strip`: Remove surrounding string whitespace; it does not remove internal spaces.
- `.astype`: Convert the pandas/NumPy object's dtype; this changes representation, not merely its displayed label.
- `.fullmatch`: Require the entire string to match a regex; contains would allow a partial match inside unrelated text.
- `.lstrip`: Remove any leading occurrences of the specified character set, such as zeros in numeric UPC text.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.

Source lines 32–32:

```
32 return text.where(~numeric, canonical.replace("", "0"))
```

**Read this block:**

- Line 32: return the result of calling `text.where(...)`; the arguments and preceding method chain are shown above. This ends the call.

**Syntax and calls used here:**

- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

## normalized upc expr

Original function: `normalized_upc_expr`. Source begins at line 35.

This returns a **Polars expression**, not a loaded Series. Calling it constructs a reusable recipe for the `upc` column. `pl.col` identifies that column; `cast(pl.String)` standardizes its type. The local names `text`, `numeric`, and `canonical` hold expression objects. Their calculations run later when the surrounding lazy query is executed.

Nested `when/then/otherwise` first decides whether a UPC is numeric, then whether stripping zeros would leave an empty identifier. `pl.lit("0")` means literal text, not a column named zero. `.alias("upc")` ensures `with_columns` replaces the intended column. This version avoids collecting an entire lazy table just to normalize one field. A pandas conversion would forfeit the lazy query's memory and projection advantages.

**Definition and stated intent:**

Source lines 35–35:

```
35 def normalized_upc_expr():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 36–38:

```
36 text = pl.col("upc").cast(pl.String).str.strip_chars()
37 numeric = text.str.contains(r"^[0-9]+(?:\.0+)?$")
38 canonical = text.str.replace(r"\.0+$", "").str.strip_chars_start("0")
```

**Read this block:**

- Line 36, `text`: Set this name to the result of calling `.strip_chars(...)`; the arguments and preceding method chain are shown above.
- Line 37, `numeric`: Set this name to the result of calling `text.str.contains(...)`; the arguments and preceding method chain are shown above.
- Line 38, `canonical`: Set this name to the result of calling `.strip_chars_start(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.strip_chars`: Trim surrounding characters/whitespace in a Polars string expression.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.contains`: Test whether text contains the supplied regex pattern; anchors can make this an entire-string test.
- `.strip_chars_start`: Trim characters from the start of each Polars string; it does not remove matching characters inside it.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.

Source lines 39–44:

```
39 return (
40     pl.when(numeric)
41     .then(pl.when(canonical == "").then(pl.lit("0")).otherwise(canonical))
42     .otherwise(text)
43     .alias("upc")
44 )
```

**Read this block:**

- Line 39: return the result of calling `.alias(...)`; the arguments and preceding method chain are shown above. This ends the call.

**Syntax and calls used here:**

- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.otherwise`: Supply the fallback expression for rows not selected by the preceding condition.
- `.then`: Supply the value/expression selected where the preceding Polars condition is true.
- `.when`: Start a Polars row-wise conditional expression, completed by then and otherwise.
- `.lit`: Build a literal-valued Polars expression, distinguishing constant text/numbers from column names.

## read flavors

Original function: `read_flavors`. Source begins at line 47.

Input `path` is a CSV path; optional `products_extra` is a Parquet path. Output is a two-column pandas table with one row per UPC. This function has an early return: if the CSV already contains resolved `upc,flavor`, validation succeeds and the rest of the recovery/classification code is never executed.

`{"upc", "flavor"}.issubset(frame.columns)` asks whether both required labels exist. `missing = required - set(frame.columns)` is set subtraction. Validation uses `.any()` for "at least one invalid row" and `.all()` for "every label is allowed." `if products_extra is not None` distinguishes an omitted optional argument from a supplied path that happens not to exist.

The metadata path scans only useful columns and anti-joins existing reference UPCs. It appends absent UPCs, rather than overwriting existing classifications. `[... for c in columns if c != "upc"]` builds one aggregation expression per non-key field. `np.select` picks the first matching condition, so plain wins when both plain and berry text occur. A generic substring loop could be easier to write at first, but would make precedence and vectorized missing-value handling less explicit. `drop_duplicates(..., keep="first")` is a reconciliation rule; it does not prove the retained label is correct.

**Definition and stated intent:**

Source lines 47–48:

```
47 def read_flavors(path, products_extra=None):
48     """Use the same plain-first classification as data_code/summ_stats.py."""
```

`def` creates this callable; the parentheses name its inputs (`path`, `products_extra`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 49–49:

```
49 frame = pd.read_csv(path, dtype={"upc": str})
```

**Read this block:**

- Line 49, `frame`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.

Source lines 50–64:

```
50 if {"upc", "flavor"}.issubset(frame.columns):
51     # Consume the exact mapping saved by summ_stats; no recovery or relabeling.
52     frame["upc"] = normalize_upcs(frame["upc"])
53     if (
54         frame["upc"].isna().any()
55         or frame["upc"].eq("").any()
56         or frame["upc"].duplicated().any()
57     ):
58         raise ValueError(f"{path}: saved mapping needs unique, nonmissing UPCs.")
59     if not frame["flavor"].isin(FLAVORS).all():
60         raise ValueError(f"{path}: saved mapping must use other, berry, plain.")
61     print(
62         f"Loaded {len(frame):,} resolved UPC flavor labels from {path}.", flush=True
```

```
63     )
64     return frame[["upc", "flavor"]]
```

**Read this block:**

- Line 50: test `{'upc', 'flavor'}.issubset(frame.columns)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 52, `frame['upc']`: Store the result of calling `normalize_upcs(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 53: test `frame['upc'].isna().any() or frame['upc'].eq('').any() or frame['upc'].duplic...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 58: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 59: test `not frame['flavor'].isin(FLAVORS).all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 60: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 61: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 64: return the entry/slice selected by `frame[['upc', 'flavor']]`; brackets select from the object to their left. This ends the call.

**Syntax and calls used here:**

- `normalize_upcs`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.isna`: Create a pandas mask marking missing values; compare masks rather than using equality to NaN.
- `.eq`: Compare elementwise for equality, the method equivalent of == for these pandas values.
- `.duplicated`: Flag repeated keys/rows so validation can detect ambiguity instead of resolving it silently.
- `.isin`: Test each pandas entry's membership in an allowed sequence, producing a Boolean Series.
- `len`: Count elements/rows rather than summing their numerical values.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 65–65:

```
65 missing = {"upc", "flavor_descr"} - set(frame.columns)
```

**Read this block:**

- Line 65, `missing`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.

**Syntax and calls used here:**

- `set`: Build a collection of distinct values, used for column membership and missing-field checks.

Source lines 66–67:

```
66 if missing:
67     raise ValueError(f"{path}: missing columns {sorted(missing)}")
```

**Read this block:**

- Line 66: test `missing`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 67: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

**Syntax and calls used here:**

- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.

Source lines 68–68:

```
68 frame["upc"] = normalize_upcs(frame["upc"])
```

**Read this block:**

- Line 68, `frame['upc']`: Store the result of calling `normalize_upcs(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

Source lines 69–70:

```
69 if frame["upc"].isna().any() or frame["upc"].eq("").any():
70     raise ValueError(f"{path}: flavor reference contains missing UPCs.")
```

**Read this block:**

- Line 69: test `frame['upc'].isna().any() or frame['upc'].eq('').any()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 70: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 71–112:

```
71 if products_extra is not None:
72     products_extra = Path(products_extra)
73     if products_extra.exists():
74         products = pl.scan_parquet(products_extra)
75         names = products.collect_schema().names()
76         if not {"upc", "flavor_descr"}.issubset(names):
77             raise ValueError(
78                 f"{products_extra}: needs upc and flavor_descr columns."
79             )
80         columns = [
81             c
82             for c in ["upc", "flavor_descr", "product_descr", "panel_year"]
83             if c in names
84         ]
85         products = (
86             products.select(columns)
87             .with_columns(normalized_upc_expr())
88             .join(
89                 pl.DataFrame(
90                     {"upc": frame["upc"].dropna().unique().tolist()}
91                 ).lazy(),
92                 on="upc",
93                 how="anti",
94             )
95         )
96         products = products.filter(
97             pl.col("upc").is_not_null()
98             & pl.col("flavor_descr").is_not_null()
99             & ~pl.col("flavor_descr")
100             .cast(pl.String)
101             .str.strip_chars()
102             .str.to_lowercase()
```

```
            .is_in(["", "na", "n/a", "nan", "null", "none"])
        )
        if "panel_year" in names:
            products = products.group_by("upc").agg(
                [
                    pl.col(c)
                    .sort_by("panel_year", nulls_last=True)
                    .first()
                    .alias(c)
                    for c in columns
```

Source lines 113–130:

```
                if c != "upc"
            ]
        )
    else:
        products = products.unique(
            subset="upc", keep="first", maintain_order=True
        )
    recovered = products.collect(engine="streaming").to_pandas()
    frame = pd.concat([frame, recovered], ignore_index=True)
    print(
        f"Recovered flavor labels for {len(recovered):,} additional UPCs from {products_extra}.",
        flush=True,
    )
else:
    print(
        f"Flavor recovery source not found: {products_extra}; using CSV reference only.",
        flush=True,
    )
```

**Read this block:**

- Line 71: test `products_extra is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 72, `products_extra`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 73: test `products_extra.exists()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 74, `products`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 75, `names`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.
- Line 76: test `not {'upc', 'flavor_descr'}.issubset(names)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 77: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 80, `columns`: Set this name to a list computed by the displayed for-expression, retaining only entries passing its if filter.
- Line 85, `products`: Set this name to the result of calling `.join(...)`; the arguments and preceding method chain are shown above.
- Line 96, `products`: Set this name to the result of calling `products.filter(...)`; the arguments and preceding method chain are shown above.
- Line 105: test `'panel_year' in names`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 106, `products`: Set this name to the result of calling `.agg(...)`; the arguments and preceding method chain are shown above.
- Line 117, `products`: Set this name to the result of calling `products.unique(...)`; the arguments and preceding

method chain are shown above.
- Line 120, `recovered`: Set this name to the result of calling `.to_pandas(...)`; the arguments and preceding method chain are shown above.
- Line 121, `frame`: Set this name to the result of calling `pd.concat(...)`; the arguments and preceding method chain are shown above.
- Line 122: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 127: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.to_pandas`: Convert a Polars table to pandas, crossing from query/table preparation to eager labeled analysis.
- `.lazy`: Wrap an eager Polars table as a lazy query so later operations can be planned together.
- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.
- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `normalized_upc_expr`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.is_not_null`: Create a Polars nonnull mask; additional range checks can still be needed.
- `.is_in`: Build a Polars membership expression, often used for eligible household IDs or allowed labels.
- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.to_lowercase`: Build a lowercase Polars string expression for reproducible label comparisons.
- `.first`: Select the first value in the current expression/group; preceding ordering and null filtering determine which one.
- `.sort_by`: Order values within an expression/group by another column before selecting a first value.
- `.strip_chars`: Trim surrounding characters/whitespace in a Polars string expression.
- `.dropna`: Remove pandas missing observations before the following calculation.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

Source lines 131–131:

```
131 text = frame["flavor_descr"].fillna("")
```

**Read this block:**

- Line 131, `text`: Set this name to the result of calling `frame['flavor_descr'].fillna(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.fillna`: Replace pandas missing entries with the provided fallback; it can change downstream denominators or validity checks.

Source lines 132–133:

```
132 if "product_descr" in frame.columns:
133     text = text + " " + frame["product_descr"].fillna("")
```

**Read this block:**

- Line 132: test `'product_descr' in frame.columns`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 133, `text`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Source lines 134–144:

```
134 text = text.str.lower()
135 frame["flavor"] = np.select(
136     [
137         text.str.contains("plain|unflavored"),
138         text.str.contains("berry|straw|blue|rasp|black|cran|cherry|wildberry"),
139     ],
140     ["plain", "berry"],
141     default="other",
142 )
143 # Match summ_stats: retain missing labels and use the first CSV row per UPC.
144 conflicts = (frame.groupby("upc")["flavor"].nunique() > 1).sum()
```

**Read this block:**

- Line 134, `text`: Set this name to the result of calling `text.str.lower(...)`; the arguments and preceding method chain are shown above.
- Line 135, `frame['flavor']`: Store the result of calling `np.select(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 144, `conflicts`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.lower`: Convert pandas/Python text to lowercase so comparisons need not list every capitalization.
- `.contains`: Test whether text contains the supplied regex pattern; anchors can make this an entire-string test.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.nunique`: Count distinct pandas values, rather than counting repeated purchase rows as separate identities.
- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.

Source lines 145–150:

```
145 if conflicts:
146     print(
147         f"{conflicts:,} UPCs have conflicting flavor classifications; "
148         "keeping the first flavor in CSV row order.",
149         flush=True,
150     )
```

**Read this block:**

- Line 145: test `conflicts`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.

- Line 146: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 151–151:

```
151 return frame[["upc", "flavor"]].drop_duplicates(subset="upc", keep="first")
```

**Read this block:**

- Line 151: return the result of calling `.drop_duplicates(...)`; the arguments and preceding method chain are shown above. This ends the call.

**Syntax and calls used here:**

- `.drop_duplicates`: Retain one row per selected key; keep='first' makes earlier sorting/order economically consequential.

## week dates

Original function: `week_dates`. Source begins at line 154.

`pd.to_datetime` interprets strings with an explicit `%Y%m%d` format: four-digit year, two-digit month, two-digit day. The regex removes a trailing `.0` from integer dates that were stored as float-like text. `errors="raise"` rejects a malformed date instead of silently creating a missing date.

For example, `20100109` should mean January 9, 2010. Passing that integer directly to a general timestamp constructor can instead mean nanoseconds from an epoch. Explicit parsing prevents that mistake. This helper exists in the module, but the current main preparation path parses purchase-date strings with Polars directly; a function's presence does not mean the entry point uses it.

**Definition and stated intent:**

Source lines 154–155:

```
154 def week_dates(values):
155     """Parse YYYYMMDD safely (integer dates are NOT nanoseconds)."""
```

`def` creates this callable; the parentheses name its inputs (`values`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 156–160:

```
156 return pd.to_datetime(
157     values.astype(str).str.replace(r"\.0$", "", regex=True),
158     format="%Y%m%d",
159     errors="raise",
160 )
```

**Read this block:**

- Line 156: return the result of calling `pd.to_datetime(...)`; the arguments and preceding method chain are shown above. This ends the call.

**Syntax and calls used here:**

- `.to_datetime`: Parse/convert dates; an explicit format avoids interpreting numeric YYYYMMDD values as timestamp offsets.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.
- `.astype`: Convert the pandas/NumPy object's dtype; this changes representation, not merely its displayed label.

## prepare data

Original function: `prepare_data`. Source begins at line 163.

This is the data assembly function. `args` is an argparse Namespace: a small object whose attributes hold command-line settings. It returns a pandas table with one row per usable household-week. Its work crosses a deliberate boundary: large input filtering and quote aggregation use lazy Polars, while the smaller household choice table becomes pandas for subsequent array calculations.

Read it in phases. First it scans and filters years, resolves labels, chooses the price field, and projects columns. `getattr(args, name, default)` provides compatibility when a manually constructed Namespace lacks an optional attribute. `next((c for c in candidates if c in names), None)` chooses the first supported price field. It is a small loop written as a generator, not a special price API.

Next it collects selected households, checks quantities, sums counts, and chooses an exposure store. Sorting by descending quantity followed by ascending store ID then keeping the first household-week row implements the tie-break exactly. `groupby(...).sum().unstack(...)` turns long flavor counts into three columns; `reindex` fixes column order even if one flavor is absent from an intermediate table. Merge `validate` arguments catch unintended duplicate keys.

The quote section continues to use full `base`, not the household-limited `selected`. It deduplicates quotes and checks conflicts before constructing menus. The DMA section computes sums and counts and subtracts own-DMA totals. This algebra avoids looping over every possible outside DMA for every household. The same pooled IV is copied into all three flavor-named IV columns.

Finally it constructs histories **before** menu/IV likelihood exclusions, builds Boolean availability masks, filters unusable rows, reapplies the two-week rule, and adds outside counts. `complete &= iv_complete` combines masks in place; it is not a comparison. `transform("nunique")` produces a row-aligned count so the household restriction can be applied without a second merge. Moving `add_stocks` below these exclusions is a plausible-looking alternative that changes the information used to build history.

**Definition and stated intent:**

Source lines 163–168:

```
163 def prepare_data(args):
164     """Use only the merged purchase table, including its existing prices.
165 
166     The IV pools all flavors: the same-week average observed price outside the
167     consumer's DMA. Repeated purchases do not duplicate store/UPC price quotes.
168     """
```

`def` creates this callable; the parentheses name its inputs (`args`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 169–171:

```
169 base = pl.scan_parquet(args.sample)
170 names = base.collect_schema().names()
171 base = base.filter(pl.col("panel_year").is_between(args.first_year, args.last_year))
```

**Read this block:**

- Line 169, `base`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 170, `names`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.
- Line 171, `base`: Set this name to the result of calling `base.filter(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.

- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.is_between`: Build an inclusive interval check by default, such as selecting the requested panel-year range.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.

Source lines 172–177:

```
172 if "flavor" not in names:
173     products_extra = getattr(args, "products_extra", None) or DATA_DIR / "RMS/combined/products_extra.parquet"
174     flavor = read_flavors(args.flavors, products_extra)
175     base = base.with_columns(normalized_upc_expr()).join(pl.from_pandas(flavor).lazy(), on="upc", how="left")
176 else:
177     base = base.with_columns(normalized_upc_expr())
```

**Read this block:**

- Line 172: test `'flavor' not in names`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 173, `products_extra`: Set this name to the Boolean condition `getattr(args, 'products_extra', None) or DATA_DIR / 'RMS/combi...`; it can be a scalar or a row/array mask depending on its inputs.
- Line 174, `flavor`: Set this name to the result of calling `read_flavors(...)`; the arguments and preceding method chain are shown above.
- Line 175, `base`: Set this name to the result of calling `.join(...)`; the arguments and preceding method chain are shown above.
- Line 177, `base`: Set this name to the result of calling `base.with_columns(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `read_flavors`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `getattr`: Retrieve an attribute by its string name, optionally supplying a default if it is absent.
- `.lazy`: Wrap an eager Polars table as a lazy query so later operations can be planned together.
- `normalized_upc_expr`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.from_pandas`: Convert a pandas object into a Polars table; calling lazy afterward makes a query plan from it.

Source lines 178–178:

```
178 price_column = next((c for c in ["price", "price_rms"] if c in names), None)
```

**Read this block:**

- Line 178, `price_column`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `next`: Take the first item from an iterator, or the supplied default when no item matches.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Source lines 179–180:

```
179 if price_column is None:
180     raise ValueError("Merged purchase table needs a price or price_rms column.")
```

**Read this block:**

- Line 179: test `price_column is None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 180: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

**Syntax and calls used here:**

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 181–183:

```
181 consumer_dma = getattr(args, "household_dma_column", "dma_cd")
182 columns = ["household_code", "panel_year", "purchase_date", "trip_code_uc", "upc",
183            "store_code_uc", "quantity", "flavor", pl.col(price_column).cast(pl.Float64).alias("price")]
```

**Read this block:**

- Line 181, `consumer_dma`: Set this name to the result of calling `getattr(...)`; the arguments and preceding method chain are shown above.
- Line 182, `columns`: Set this name to a list containing 9 displayed entries.

**Syntax and calls used here:**

- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.

Source lines 184–185:

```
184 if not args.no_price_control:
185     columns.append(pl.col(consumer_dma).alias("dma_cd"))
```

**Read this block:**

- Line 184: test `not args.no_price_control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 185: call `columns.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.

Source lines 186–192:

```
186 base = base.select(columns).with_columns(
187     pl.col("purchase_date").cast(pl.String).str.slice(0, 10).str.to_date())
188 base = base.filter(pl.col("price").is_finite() & (pl.col("price") > 0))
189 base = base.with_columns((pl.col("purchase_date") + pl.duration(
190     days=(6 - pl.col("purchase_date").dt.weekday()) % 7)).alias("week"))
191 # Keep the full merged table for prices/IVs, even in a household-limited run.
192 selected = base
```

**Read this block:**

- Line 186, `base`: Set this name to the result of calling `.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 188, `base`: Set this name to the result of calling `base.filter(...)`; the arguments and preceding method chain are shown above.
- Line 189, `base`: Set this name to the result of calling `base.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 192, `selected`: Set this name to the object currently named `base`; plain assignment does not copy it.

**Syntax and calls used here:**

- `.to_date`: Parse text into calendar-date values before weekday extraction and date arithmetic.

- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.slice`: Take a substring starting at a position for a given length, here often the first ten date characters.
- `.is_finite`: Build the Polars finite-number test used to reject missing/infinite quotes.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.
- `.duration`: Build a date/time duration from the named unit, allowing a different day offset per row.
- `.weekday`: Extract a Polars weekday number, Monday=1 through Sunday=7, for the Saturday offset calculation.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.

Source lines 193–196:

```
193 if args.max_households:
194     ids = (base.select("household_code").unique().sort("household_code")
195            .limit(args.max_households).collect(engine="streaming")["household_code"])
196     selected = base.filter(pl.col("household_code").is_in(ids.implode()))
```

**Read this block:**

- Line 193: test `args.max_households`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 194, `ids`: Set this name to the entry/slice selected by `base.select('household_code').unique().sort('household_co...`; brackets select from the object to their left.
- Line 196, `selected`: Set this name to the result of calling `base.filter(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `.is_in`: Build a Polars membership expression, often used for eligible household IDs or allowed labels.
- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.
- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.

Source lines 197–198:

```
197 print(f"Loading merged purchase table: {args.sample}", flush=True)
198 lines = selected.collect(engine="streaming").to_pandas()
```

**Read this block:**

- Line 197: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 198, `lines`: Set this name to the result of calling `.to_pandas(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.
- `.to_pandas`: Convert a Polars table to pandas, crossing from query/table preparation to eager labeled analysis.

Source lines 199–200:

```
199 if lines.empty:
200     raise ValueError("No purchase lines in selected sample.")
```

**Read this block:**

- Line 199: test `lines.empty`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 200: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 201–202:

```
201 if lines["flavor"].isna().any() or not lines["flavor"].isin(FLAVORS).all():
202     raise ValueError("Merged purchase table has missing or invalid flavor labels.")
```

**Read this block:**

- Line 201: test `lines['flavor'].isna().any() or not lines['flavor'].isin(FLAVORS).all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 202: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

**Syntax and calls used here:**

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.isna`: Create a pandas mask marking missing values; compare masks rather than using equality to NaN.
- `.isin`: Test each pandas entry's membership in an allowed sequence, producing a Boolean Series.

Source lines 203–203:

```
203 q = lines["quantity"].to_numpy(float)
```

**Read this block:**

- Line 203, `q`: Set this name to the result of calling `lines['quantity'].to_numpy(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.

Source lines 204–205:

```
204 if not np.isfinite(q).all() or (q <= 0).any() or not np.allclose(q, np.round(q)):
205     raise ValueError("Quantity must be positive integer package counts.")
```

**Read this block:**

- Line 204: test `not np.isfinite(q).all() or (q <= 0).any() or (not np.allclose(q, np.round(q)))`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 205: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

**Syntax and calls used here:**

- `.allclose`: Check approximate numerical equality with tolerances, appropriate for floating-point arrays.
- `.round`: Round numeric values for a check/display rule; rounding quantities is compared to the original before acceptance.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.

Source lines 206–212:

```
206 lines["week"] = pd.to_datetime(lines["week"])
207 keys = ["household_code", "week"]
208 weekly = (lines.groupby(keys + ["flavor"])["quantity"].sum().unstack("flavor", fill_value=0)
209           .reindex(columns=FLAVORS, fill_value=0).reset_index())
210 stores = lines.groupby(keys + ["store_code_uc"], as_index=False)["quantity"].sum()
211 stores = stores.sort_values(keys + ["quantity", "store_code_uc"], ascending=[True, True, False, True])
```

```
212 stores = stores.drop_duplicates(keys)[keys + ["store_code_uc"]]
```

**Read this block:**

- Line 206, `lines['week']`: Store the result of calling `pd.to_datetime(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 207, `keys`: Set this name to the list `['household_code', 'week']`.
- Line 208, `weekly`: Set this name to the result of calling `.reset_index(...)`; the arguments and preceding method chain are shown above.
- Line 210, `stores`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.
- Line 211, `stores`: Set this name to the result of calling `stores.sort_values(...)`; the arguments and preceding method chain are shown above.
- Line 212, `stores`: Set this name to the entry/slice selected by `stores.drop_duplicates(keys)[keys + ['store_code_uc']]`; brackets select from the object to their left.

**Syntax and calls used here:**

- `.to_datetime`: Parse/convert dates; an explicit format avoids interpreting numeric YYYYMMDD values as timestamp offsets.
- `.reset_index`: Turn the index back into ordinary row numbering; drop=True discards the old index rather than saving it as a column.
- `.reindex`: Align to an explicitly requested row/column order, adding missing labels with a chosen fill value if provided.
- `.unstack`: Move an index level into columns, such as turning long flavor counts into wide weekly outcomes.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `.sort_values`: Order pandas rows by values; ascending can supply one direction for each listed sort key.
- `.drop_duplicates`: Retain one row per selected key; keep='first' makes earlier sorting/order economically consequential.

Source lines 213–213:

```
213 weekly = weekly.merge(stores, on=keys, validate="one_to_one")
```

**Read this block:**

- Line 213, `weekly`: Set this name to the result of calling `weekly.merge(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.merge`: Join pandas tables through matching keys; how controls retention and validate checks intended key cardinality.

Source lines 214–218:

```
214 if not args.no_price_control:
215     lines["dma_cd"] = check_dma(lines["dma_cd"], "consumer")
216     if (lines.groupby(keys)["dma_cd"].nunique() > 1).any():
217         raise ValueError("Multiple consumer DMAs within a household-week.")
218     weekly = weekly.merge(lines[keys + ["dma_cd"]].drop_duplicates(), on=keys, validate="one_to_one")
```

**Read this block:**

- Line 214: test `not args.no_price_control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 215, `lines['dma_cd']`: Store the result of calling `check_dma(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

- Line 216: test `(lines.groupby(keys)['dma_cd'].nunique() > 1).any()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 217: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 218, `weekly`: Set this name to the result of calling `weekly.merge(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `check_dma`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.nunique`: Count distinct pandas values, rather than counting repeated purchase rows as separate identities.

Source lines 220–221:

```
220 quote_keys = ["store_code_uc", "week", "upc"]
221 price_columns = quote_keys + ["flavor", "price"]
```

**Read this block:**

- Line 220, `quote_keys`: Set this name to the list `['store_code_uc', 'week', 'upc']`.
- Line 221, `price_columns`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Source lines 222–223:

```
222 if not args.no_price_control:
223     price_columns.append("dma_cd")
```

**Read this block:**

- Line 222: test `not args.no_price_control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 223: call `price_columns.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 224–230:

```
224 quotes = (base.filter(pl.col("price").is_finite() & (pl.col("price") > 0))
225           .select(price_columns).unique())
226 # A store/week/UPC should have one price and one flavor. Consumer DMA may
227 # differ when consumers from different markets visit the same store.
228 local = quotes.select(quote_keys + ["flavor", "price"]).unique()
229 conflicting = (local.group_by(quote_keys).len().filter(pl.col("len") > 1)
230                 .limit(1).collect(engine="streaming"))
```

**Read this block:**

- Line 224, `quotes`: Set this name to the result of calling `.unique(...)`; the arguments and preceding method chain are shown above.
- Line 228, `local`: Set this name to the result of calling `.unique(...)`; the arguments and preceding method chain are shown above.
- Line 229, `conflicting`: Set this name to the result of calling `.collect(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.

Source lines 231–232:

```
231 if len(conflicting):
232     raise ValueError("Conflicting merged prices or flavors for the same store/week/UPC.")
```

**Read this block:**

- Line 231: test `len(conflicting)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 232: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

**Syntax and calls used here:**

- `len`: Count elements/rows rather than summing their numerical values.

Source lines 233–239:

```
menus = (local.group_by(["store_code_uc", "week", "flavor"])
         .agg(pl.col("price").mean()).collect(engine="streaming").to_pandas())
menus["week"] = pd.to_datetime(menus["week"])
menus = (menus.pivot(index=["store_code_uc", "week"], columns="flavor", values="price")
         .reindex(columns=FLAVORS).rename(columns={f: "price_" + f for f in FLAVORS}).reset_index())
weekly = weekly.merge(menus, on=["store_code_uc", "week"], how="left", validate="many_to_one")
print("Using merged prices directly; no separate RMS data are loaded.", flush=True)
```

**Read this block:**

- Line 233, `menus`: Set this name to the result of calling `.to_pandas(...)`; the arguments and preceding method chain are shown above.
- Line 235, `menus['week']`: Store the result of calling `pd.to_datetime(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 236, `menus`: Set this name to the result of calling `.reset_index(...)`; the arguments and preceding method chain are shown above.
- Line 238, `weekly`: Set this name to the result of calling `weekly.merge(...)`; the arguments and preceding method chain are shown above.
- Line 239: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.
- `.rename`: Change column labels (or apply a renaming function); this does not change their underlying values.
- `.pivot`: Reshape long labels into matrix columns; index/columns/values specify their roles.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.

Source lines 240–264:

```
if not args.no_price_control:
    invalid_dma = quotes.select(
        ((pl.col("dma_cd").cast(pl.Float64, strict=False).is_null())
         | (pl.col("dma_cd").cast(pl.Float64, strict=False) <= 0)
         | (pl.col("dma_cd").cast(pl.Float64, strict=False) != pl.col("dma_cd").cast(pl.Float64,
         ↪  strict=False).floor()))
        .fill_null(True).any()).collect(engine="streaming").item()
    if invalid_dma:
        raise ValueError("Merged price records contain invalid dma_cd values for the other-DMA IV.")
    # Pool across ALL flavors. Deduplicate quote repeats within each DMA.
    by_dma = (quotes.with_columns(pl.col("dma_cd").cast(pl.Float64).cast(pl.Int64))
              .group_by(["week", "dma_cd"]).agg(
                  pl.col("price").sum().alias("dma_sum"), pl.len().alias("dma_count"))
              .collect(engine="streaming").to_pandas())
    by_dma["week"] = pd.to_datetime(by_dma["week"])
    totals = (by_dma.groupby("week", as_index=False)[["dma_sum", "dma_count"]].sum()
              .rename(columns={"dma_sum": "total_sum", "dma_count": "total_count"}))
    weekly = weekly.merge(totals, on="week", how="left", validate="many_to_one")
    weekly = weekly.merge(by_dma, on=["week", "dma_cd"], how="left", validate="many_to_one")
    denominator = weekly["total_count"] - weekly["dma_count"].fillna(0)
    weekly["iv_overall"] = ((weekly["total_sum"] - weekly["dma_sum"].fillna(0))
```

```
260                             / denominator.where(denominator > 0))
261     for f in FLAVORS:
262         weekly["iv_" + f] = weekly["iv_overall"]
263     weekly = weekly.drop(columns=["total_sum", "total_count", "dma_sum", "dma_count"])
264     print("IV: overall same-week mean price outside consumer dma_cd, pooled across flavors.", flush=True)
```

**Read this block:**

- Line 240: test `not args.no_price_control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 241, `invalid_dma`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 246: test `invalid_dma`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 247: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 249, `by_dma`: Set this name to the result of calling `.to_pandas(...)`; the arguments and preceding method chain are shown above.
- Line 253, `by_dma['week']`: Store the result of calling `pd.to_datetime(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 254, `totals`: Set this name to the result of calling `.rename(...)`; the arguments and preceding method chain are shown above.
- Line 256, `weekly`: Set this name to the result of calling `weekly.merge(...)`; the arguments and preceding method chain are shown above.
- Line 257, `weekly`: Set this name to the result of calling `weekly.merge(...)`; the arguments and preceding method chain are shown above.
- Line 258, `denominator`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 259, `weekly['iv_overall']`: Store division of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 261: for each entry yielded by `FLAVORS`, bind/unpack it into `f` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 262, `weekly['iv_' + f]`: Store the entry/slice selected by `weekly['iv_overall']`; brackets select from the object to their left in the selected entry/column.
- Line 263, `weekly`: Set this name to the result of calling `weekly.drop(...)`; the arguments and preceding method chain are shown above.
- Line 264: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.item`: Extract a scalar from a one-element array/one-cell result rather than passing a table wrapper onward.
- `.drop`: Remove the specified columns/labels; inspect the receiver because list and table methods have different APIs.
- `.fillna`: Replace pandas missing entries with the provided fallback; it can change downstream denominators or validity checks.
- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- `.fill_null`: Replace Polars nulls with a chosen value/expression; null and floating-point NaN are distinct representations.
- `.is_null`: Create a Polars null-value mask, useful when validating joins and conversions.
- `.floor`: Take the greatest integer no larger than each value; equality with it tests whether a positive code is integral.
- The | operator accepts either elementwise condition; Python's scalar or is generally unsuitable for an entire table column.

Source lines 265–269:

```
265 weekly = add_stocks(weekly.sort_values(keys).reset_index(drop=True), args.habit_weight)
266 price_cols = ["price_" + f for f in FLAVORS]
267 available = np.isfinite(weekly[price_cols].to_numpy(float)) & (weekly[price_cols].to_numpy(float) > 0)
268 counts = weekly[FLAVORS].to_numpy(float)
269 complete = available.any(axis=1) & ((counts == 0) | available).all(axis=1)
```

**Read this block:**

- Line 265, `weekly`: Set this name to the result of calling `add_stocks(...)`; the arguments and preceding method chain are shown above.
- Line 266, `price_cols`: Set this name to a list computed by the displayed for-expression.
- Line 267, `available`: Set this name to elementwise AND of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 268, `counts`: Set this name to the result of calling `weekly[FLAVORS].to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 269, `complete`: Set this name to elementwise AND of the displayed operands; parentheses control which subexpressions are evaluated together.

**Syntax and calls used here:**

- `add_stocks`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 270–271:

```
270 for j, f in enumerate(FLAVORS):
271     weekly["available_" + f] = available[:, j]
```

**Read this block:**

- Line 270: for each entry yielded by `enumerate(FLAVORS)`, bind/unpack it into `(j, f)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 271, `weekly['available_' + f]`: Store the entry/slice selected by `available[:, j]`; brackets select from the object to their left in the selected entry/column.

**Syntax and calls used here:**

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.

Source lines 272–272:

```
272 print(f"Weeks excluded for purchased flavors without a covered price: {(~complete).sum():,}", flush=True)
```

**Read this block:**

- Line 272: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

Source lines 273–277:

```
273 if not args.no_price_control:
274     iv_cols = ["iv_" + f for f in FLAVORS]
275     iv_complete = np.isfinite(weekly[iv_cols]).all(axis=1)
276     print(f"Additional weeks excluded for missing other-DMA IVs: {(complete & ~iv_complete).sum():,}", flush=True)
277     complete &= iv_complete
```

**Read this block:**

- Line 273: test `not args.no_price_control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 274, `iv_cols`: Set this name to a list computed by the displayed for-expression.
- Line 275, `iv_complete`: Set this name to the result of calling `.all(...)`; the arguments and preceding method chain are shown above.
- Line 276: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 277: update `complete` using `&=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 278–282:

```
278 weekly = weekly.loc[complete].copy()
279 repeated = weekly.groupby("household_code")["week"].transform("nunique") >= 2
280 print(f"Households excluded for fewer than two usable weeks: {weekly.loc[~repeated,
    ↪  'household_code'].nunique():,}", flush=True)
281 weekly = weekly.loc[repeated].reset_index(drop=True)
282 weekly["outside"] = np.maximum(args.capacity - weekly[FLAVORS].sum(axis=1), 0)
```

**Read this block:**

- Line 278, `weekly`: Set this name to the result of calling `weekly.loc[complete].copy(...)`; the arguments and preceding method chain are shown above.
- Line 279, `repeated`: Set this name to the Boolean condition `weekly.groupby('household_code')['week'].transform('nunique') ...`; it can be a scalar or a row/array mask depending on its inputs.
- Line 280: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 281, `weekly`: Set this name to the result of calling `weekly.loc[repeated].reset_index(...)`; the arguments and preceding method chain are shown above.
- Line 282, `weekly['outside']`: Store the result of calling `np.maximum(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

**Syntax and calls used here:**

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `.transform`: Compute a pandas group result broadcast back to original rows, unlike a reduced one-row-per-group aggregation.
- `.maximum`: Take an elementwise maximum, e.g. max(capacity minus packages, zero) independently each week.

Source lines 283–284:

```
283 if weekly.empty:
284     raise ValueError("No households with at least two usable purchase weeks.")
```

**Read this block:**

- Line 283: test `weekly.empty`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 284: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 285–285:

```
285 return weekly
```

**Read this block:**

- Line 285: return the object currently named `weekly`; plain assignment does not copy it. This ends the call.

## add stocks

Original function: `add_stocks`. Source begins at line 288.

Input is a weekly pandas table; output is a sorted copy with stock, novelty, and Xi columns. The outer loop resets history for each household. `for _, group` ignores the household label but keeps its subtable. The inner `iterrows()` loop supplies a row index and a Series containing that row's values.

`stocks[index] = stock` stores pre-choice history into one output row. The seven-day condition tests genuine adjacent calendar weeks. Then counts are normalized into shares, stock is updated, and the current date becomes the previous date. A gap resets recent novelty by leaving its preallocated zeros, but does not reset cumulative stock. A loop is reasonable here because state at the next observation depends on the preceding update; vectorization requires careful shifted cumulative sums and calendar-gap checks.

The final `enumerate` loop writes one named column per flavor. `stocks[:, j]` extracts all rows of the jth flavor. With `habit_weight=.7`, current Xi uses .3 times novelty minus .7 times own stock. The separately preallocated `stocks` matrix means historical output rows do not change when `stock` is reassigned. Try a one-household, two-week example before reading the likelihood.

**Definition and stated intent:**

Source lines 288–299:

```
288 def add_stocks(weekly, habit_weight=0.7):
289     """All past observed occasions, delta=1; no rolling window or decay.
290 
291     S_jt = sum_{s<t} share_js
292     N_jt = sum_{k != j} share_k,t-1  (immediately preceding calendar week)
293     Xi_jt = (1-habit_weight)*N_jt - habit_weight*S_jt
294 
295     Positive gamma rewards recent other-flavor consumption and penalizes own
296     fatigue. Negative gamma represents habit. A purchase occasion is an observed
297     household purchase week. Current purchases enter history only AFTER Xi is
298     recorded. The first observed occasion starts with zero history.
299     """
```

`def` creates this callable; the parentheses name its inputs (`weekly`, `habit_weight`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 300–304:

```
300 result = (
301     weekly.sort_values(["household_code", "week"]).reset_index(drop=True).copy()
302 )
303 stocks = np.zeros((len(result), 3))
304 novelty = np.zeros_like(stocks)
```

**Read this block:**

- Line 300, `result`: Set this name to the result of calling `.copy(...)`; the arguments and preceding method chain are shown above.
- Line 303, `stocks`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 304, `novelty`: Set this name to the result of calling `np.zeros_like(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `.reset_index`: Turn the index back into ordinary row numbering; drop=True discards the old index rather than saving it as a column.
- `.sort_values`: Order pandas rows by values; ascending can supply one direction for each listed sort key.

- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `len`: Count elements/rows rather than summing their numerical values.
- `.zeros_like`: Create zeros matching another array's shape/dtype instead of manually repeating dimensions.

Source lines 305–316:

```
305 for _, group in result.groupby("household_code", sort=False):
306     stock = np.zeros(3)
307     previous_share = np.zeros(3)
308     previous_week = None
309     for index, row in group.iterrows():
310         stocks[index] = stock
311         if previous_week is not None and (row["week"] - previous_week).days == 7:
312             novelty[index] = previous_share.sum() - previous_share
313         counts = row[FLAVORS].to_numpy(float)
314         previous_share = counts / counts.sum()
315         stock = stock + previous_share
316         previous_week = row["week"]
```

**Read this block:**

- Line 305: for each entry yielded by `result.groupby('household_code', sort=False)`, bind/unpack it into `(_, group)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 306, `stock`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 307, `previous_share`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 308, `previous_week`: Set this name to the literal value `None`.
- Line 309: for each entry yielded by `group.iterrows()`, bind/unpack it into `(index, row)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 310, `stocks[index]`: Store the object currently named `stock`; plain assignment does not copy it in the selected entry/column.
- Line 311: test `previous_week is not None and (row['week'] - previous_week).days == 7`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 312, `novelty[index]`: Store subtraction of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 313, `counts`: Set this name to the result of calling `row[FLAVORS].to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 314, `previous_share`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 315, `stock`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 316, `previous_week`: Set this name to the entry/slice selected by `row['week']`; brackets select from the object to their left.

**Syntax and calls used here:**

- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.

Source lines 317–322:

```
317 for j, name in enumerate(FLAVORS):
318     result["stock_" + name] = stocks[:, j]
319     result["novelty_" + name] = novelty[:, j]
320     result["xi_" + name] = (1 - habit_weight) * novelty[
321         :, j
322     ] - habit_weight * stocks[:, j]
```

**Read this block:**

- Line 317: for each entry yielded by `enumerate(FLAVORS)`, bind/unpack it into `(j, name)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 318, `result['stock_' + name]`: Store the entry/slice selected by `stocks[:, j]`; brackets select from the object to their left in the selected entry/column.
- Line 319, `result['novelty_' + name]`: Store the entry/slice selected by `novelty[:, j]`; brackets select from the object to their left in the selected entry/column.
- Line 320, `result['xi_' + name]`: Store subtraction of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.

**Syntax and calls used here:**

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.

Source lines 323–323:

```
323 return result
```

**Read this block:**

- Line 323: return the object currently named `result`; plain assignment does not copy it. This ends the call.

## make arrays

Original function: `make_arrays`. Source begins at line 326.

This converts a labeled table into a dictionary of NumPy arrays. `pd.factorize` returns both dense integer household codes and their original labels: household IDs `[10,10,30]` become codes `[0,0,1]`. Dense codes let `bincount` sum efficiently. The expression `FLAVORS + (["outside"] if capacity else [])` builds either three or four outcome columns without changing the inside-price array's three columns.

The function validates counts/Xi, checks menu consistency, and inserts zero placeholders for missing prices. The separate availability mask ensures those zeros are not interpreted as offered free goods. `all(c in frame for c in cf_cols)` asks whether every residual column exists. If controls are absent, `zeros_like` supplies the right-shaped residual array. The returned dictionary deliberately keeps flags such as `outside` and `control` beside arrays so the likelihood can use one interface for several models. Passing raw tables directly into every optimization step would repeatedly perform label lookup and conversion.

**Definition and stated intent:**

Source lines 326–327:

```
326 def make_arrays(frame, capacity):
327     """Use household codes to sum log likelihoods before integrating types."""
```

`def` creates this callable; the parentheses name its inputs (`frame`, `capacity`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 328–332:

```
328 hh, labels = pd.factorize(frame["household_code"], sort=True)
329 counts = frame[FLAVORS + (["outside"] if capacity else [])].to_numpy(float)
330 prices = frame[["price_" + f for f in FLAVORS]].to_numpy(float)
331 xi = frame[["xi_" + f for f in FLAVORS]].to_numpy(float)
332 available = np.isfinite(prices) & (prices > 0)
```

**Read this block:**

- Line 328, `(hh, labels)`: Unpack the result of calling `pd.factorize(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 329, `counts`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.

- Line 330, `prices`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 331, `xi`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 332, `available`: Set this name to elementwise AND of the displayed operands; parentheses control which subexpressions are evaluated together.

**Syntax and calls used here:**

- `.factorize`: Encode labels as dense integer positions and return the original-label lookup alongside those positions.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.

Source lines 333–334:

```
333 if not all(np.isfinite(x).all() for x in [counts, xi]):
334     raise ValueError("Nonfinite estimation inputs.")
```

**Read this block:**

- Line 333: test `not all((np.isfinite(x).all() for x in [counts, xi]))`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 334: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

**Syntax and calls used here:**

- `all`: Require every generated condition to be true; this reduces a sequence to one Boolean.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Source lines 335–336:

```
335 if ((counts[:, :3] > 0) & ~available).any() or not available.any(axis=1).all():
336     raise ValueError("Purchased flavors must have covered prices and a nonempty choice set.")
```

**Read this block:**

- Line 335: test `((counts[:, :3] > 0) & ~available).any() or not available.any(axis=1).all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 336: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

**Syntax and calls used here:**

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

Source lines 338–341:

```
338 prices = np.where(available, prices, 0.)
339 cf_cols = ["cf_" + f for f in FLAVORS]
340 control = all(c in frame for c in cf_cols)
341 cf = frame[cf_cols].to_numpy(float) if control else np.zeros_like(prices)
```

**Read this block:**

- Line 338, `prices`: Set this name to the result of calling `np.where(...)`; the arguments and preceding method chain are shown above.
- Line 339, `cf_cols`: Set this name to a list computed by the displayed for-expression.
- Line 340, `control`: Set this name to the result of calling `all(...)`; the arguments and preceding method chain are shown above.
- Line 341, `cf`: Set this name to the true-branch value if `control` holds, otherwise the else-branch value.

**Syntax and calls used here:**

- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- `.zeros_like`: Create zeros matching another array's shape/dtype instead of manually repeating dimensions.

Source lines 342–343:

```
342 if not np.isfinite(cf).all():
343     raise ValueError("Nonfinite price-control residuals.")
```

**Read this block:**

- Line 342: test `not np.isfinite(cf).all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 343: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 344–355:

```
344 return dict(
345     counts=counts,
346     prices=prices,
347     available=available,
348     xi=xi,
349     cf=cf,
350     control=control,
351     hh=hh,
352     labels=labels,
353     n_households=len(labels),
354     outside=bool(capacity),
355 )
```

**Read this block:**

- Line 344: return the result of calling `dict(...)`; the arguments and preceding method chain are shown above. This ends the call.

**Syntax and calls used here:**

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `len`: Count elements/rows rather than summing their numerical values.
- `bool`: Convert scalar truthiness to an explicit Boolean flag.

## make draws

Original function: `make_draws`. Source begins at line 358.

Inputs are an even draw count and an integer seed; output has shape `(number,2)`. `number % 2` is the remainder after division by two: zero for even numbers, one for odd. `number // 2` is integer division. The generator draws half the rows, then `vstack([z, -z])` appends their negatives. With two draws, an initial row `[.4,-1.2]` gives a second row `[-.4,1.2]`.

The seed makes repeated calls reproducible, and a local Generator avoids changing NumPy's global random state. Reusing these fixed antithetic nodes reduces noise when comparing candidate parameter vectors. Drawing afresh

inside each objective call would make the optimizer chase random changes as well as parameter effects. More draws still cost more computation.

**Definition and stated intent:**

Source lines 358–359:

```
358 def make_draws(number, seed):
359     """Fixed antithetic normal draws, reused throughout optimization."""
```

`def` creates this callable; the parentheses name its inputs (`number`, `seed`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 360–361:

```
360 if number < 2 or number % 2:
361     raise ValueError("Use an even number of draws, at least two.")
```

**Read this block:**

- Line 360: test `number < 2 or number % 2`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 361: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

**Syntax and calls used here:**

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.

Source lines 362–362:

```
362 z = np.random.default_rng(seed).standard_normal((number // 2, 2))
```

**Read this block:**

- Line 362, `z`: Set this name to the result of calling `.standard_normal(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.standard_normal`: Draw independent standard-normal values in the requested shape, before coefficient transformations.
- `.default_rng`: Create a local seeded random Generator, avoiding dependence on a shared global random stream.
- The // operator is floor division, used here for integer counts or calendar-year extraction.

Source lines 363–363:

```
363 return np.vstack([z, -z])
```

**Read this block:**

- Line 363: return the result of calling `np.vstack(...)`; the arguments and preceding method chain are shown above. This ends the call.

**Syntax and calls used here:**

- `.vstack`: Stack input arrays vertically, adding rows while preserving their common column layout.

## parameter names

Original function: `parameter_names`. Source begins at line 366.

This returns the ordered labels defining the parameter vector. `names += [...]` extends a list; it does not numerically add its string entries. Membership tests such as `model in ("mixed_naive", "heterogeneous")` let related specifications share branches. The default `control=False` omits a residual coefficient unless the caller explicitly enables it.

For naive conditional demand without controls, the returned labels are two betas and mean log-minus-alpha. Heterogeneous demand adds mean gamma and both SDs; the optional outside constant is prepended and price control appended. Keeping this logic in one helper is safer than independently remembering vector positions in optimization, utility, and output. The order is part of the API: zipping a reordered list with old coefficients silently changes their meaning.

**Definition and stated intent:**

Source lines 366–366:

```
366 def parameter_names(model, outside, control=False):
```

`def` creates this callable; the parentheses name its inputs (`model`, `outside`, `control`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 367–368:

```
367 names = (["inside_constant"] if outside else []) + ["beta_berry", "beta_plain"]
368 names += ["mu_log_minus_alpha"]
```

**Read this block:**

- Line 367, `names`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 368: update `names` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

**Syntax and calls used here:**

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 369–370:

```
369 if model in ("homogeneous", "heterogeneous"):
370     names += ["mean_gamma"]
```

**Read this block:**

- Line 369: test `model in ('homogeneous', 'heterogeneous')`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 370: update `names` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 371–372:

```
371 if model in ("mixed_naive", "heterogeneous"):
372     names += ["sd_log_minus_alpha"]
```

**Read this block:**

- Line 371: test `model in ('mixed_naive', 'heterogeneous')`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 372: update `names` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 373–374:

```
373 if model == "heterogeneous":
374     names += ["sd_gamma"]
```

**Read this block:**

- Line 373: test `model == 'heterogeneous'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 374: update `names` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 375–376:

```
375 if control:
376     names += ["price_control"]
```

**Read this block:**

- Line 375: test `control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 376: update `names` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 377–377:

```
377 return names
```

**Read this block:**

- Line 377: return the object currently named `names`; plain assignment does not copy it. This ends the call.

## household loglike

Original function: `household_loglike`. Source begins at line 380.

Input `theta` is a one-dimensional parameter array, `data` is the dictionary from `make_arrays`, and `draws` is a draw-by-two matrix. The return is one log likelihood per household, not a scalar. `dict(zip(...))` attaches coefficient names so the rest of the function need not remember their numeric positions.

For each latent-type draw, negative exponential constructs alpha and an affine normal transformation constructs gamma. Broadcasting makes beta and these scalar coefficients operate across every row/flavor. Unavailable utility is set to minus infinity; an optional zero outside column is appended. `logprob = utility - logsumexp(..., axis=1, keepdims=True)` normalizes each row. The retained column dimension makes subtraction broadcast across that row.

The count-weighted row sum uses `where(counts > 0, logprob, 0)` to avoid zero times minus infinity. `bincount` adds all weeks of the same household. Only after this aggregation does the final `logsumexp` integrate draws. This order implements persistent preferences. Averaging weekly probabilities first would be simpler computationally but would estimate a different preference process.

**Definition and stated intent:**

Source lines 380–386:

```
380 def household_loglike(theta, data, draws, model):
381     """L_i = mean_r exp(sum_t sum_j count_itj * log P_itj,r).
382 
383     Gamma is normal and common across flavors; alpha = -exp(mu + sd*z). Beta never varies by draw.
384     Process one draw at a time to avoid a weeks x draws x products array.
385     The multinomial combinatorial constant is omitted (independent of theta).
386     """
```

`def` creates this callable; the parentheses name its inputs (`theta`, `data`, `draws`, `model`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 387–393:

```
387 p = dict(zip(parameter_names(model, data["outside"], data["control"]), theta))
388 beta = np.array([0.0, p["beta_berry"], p["beta_plain"]])
389 beta += p.get("inside_constant", 0.0)
390 loglikes = []
391 active_draws = (
392     draws if model in ("mixed_naive", "heterogeneous") else np.zeros((1, 2))
393 )
```

**Read this block:**

- Line 387, `p`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 388, `beta`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 389: update `beta` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 390, `loglikes`: Set this name to the list `[]`.
- Line 391, `active_draws`: Set this name to the true-branch value if `model in ('mixed_naive', 'heterogeneous')` holds, otherwise the else-branch value.

**Syntax and calls used here:**

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 394–412:

```
394 for z in active_draws:
395     alpha = -np.exp(
396         p["mu_log_minus_alpha"] + p.get("sd_log_minus_alpha", 0.0) * z[0]
397     )
398     gamma = p.get("mean_gamma", 0.) + p.get("sd_gamma", 0.) * z[1]
399     utility = (
400         beta
401         + alpha * data["prices"]
402         + gamma * data["xi"]
403         + p.get("price_control", 0.0) * data["cf"]
404     )
405     utility = np.where(data["available"], utility, -np.inf)
406     if data["outside"]:
407         utility = np.column_stack([utility, np.zeros(len(utility))])
408     logprob = utility - logsumexp(utility, axis=1, keepdims=True)
409     row_ll = (data["counts"] * np.where(data["counts"] > 0, logprob, 0.)).sum(axis=1)
410     loglikes.append(
411         np.bincount(data["hh"], weights=row_ll, minlength=data["n_households"])
412     )
```

**Read this block:**

- Line 394: for each entry yielded by `active_draws`, bind/unpack it into `z` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 395, `alpha`: Set this name to the negated or sign-adjusted expression `-np.exp(p['mu_log_minus_alpha'] + p.get('sd_log_minus_alp...`.

- Line 398, `gamma`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 399, `utility`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 405, `utility`: Set this name to the result of calling `np.where(...)`; the arguments and preceding method chain are shown above.
- Line 406: test `data['outside']`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 407, `utility`: Set this name to the result of calling `np.column_stack(...)`; the arguments and preceding method chain are shown above.
- Line 408, `logprob`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 409, `row_ll`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.
- Line 410: call `loglikes.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.column_stack`: Place one-dimensional vectors beside each other as columns of a matrix.
- `logsumexp`: Compute log(sum(exp(values))) stably; axis selects alternatives, draws, or another reduction dimension.
- `.bincount`: Accumulate frequencies or weighted sums by dense nonnegative integer labels; repeated household codes are added together.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 413–413:

```
413 return logsumexp(loglikes, axis=0) - np.log(len(active_draws))
```

**Read this block:**

- Line 413: return subtraction of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

**Syntax and calls used here:**

- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.

## demand objective gradient

Original function: `demand_objective_gradient`. Source begins at line 416.

This returns a tuple `(mean_negative_log_likelihood, gradient)`, which SciPy requests when `jac=True`. The utility and likelihood steps mirror `household_loglike`; the additional work computes derivatives in the same draw loop. `total[:, None]` expands a weekly count vector to a column so it can multiply the matrix of choice probabilities. `residual` is observed count minus predicted count for the three inside alternatives.

The `columns` dictionary holds one weekly score vector per coefficient name. For example, the berry intercept takes column 1 of residual, while the alpha location score uses the price-weighted residual multiplied by alpha itself because differentiating `-exp(mu + sd*z)` contributes alpha. Each score is aggregated to households, and `column_stack` places parameters on a final axis.

`np.einsum("rh,rhp->p", weights, scores)` means multiply posterior weight at draw r/household h by its parameter-p score, then sum r and h, retaining p. A nested loop over r, h, p would express the same sum more verbosely and slowly. The minus sign and household-count division match minimization of average NLL. This is an analytic derivative of the finite-draw objective, not automatic differentiation or a finite-difference estimate.

**Definition and stated intent:**

Source lines 416–417:

```
416 def demand_objective_gradient(theta, data, draws, model):
417     """Exact simulated likelihood score, weighted by household type posteriors."""
```

`def` creates this callable; the parentheses name its inputs (`theta`, `data`, `draws`, `model`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 418–422:

```
418 names = parameter_names(model, data["outside"], data["control"])
419 p = dict(zip(names, theta))
420 active = draws if model in ("mixed_naive", "heterogeneous") else np.zeros((1, 2))
421 likelihoods, scores = [], []
422 total = data["counts"].sum(axis=1)
```

**Read this block:**

- Line 418, `names`: Set this name to the result of calling `parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 419, `p`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 420, `active`: Set this name to the true-branch value if `model in ('mixed_naive', 'heterogeneous')` holds, otherwise the else-branch value.
- Line 421, `(likelihoods, scores)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 422, `total`: Set this name to the result of calling `data['counts'].sum(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.

Source lines 423–443:

```
423 for z in active:
424     alpha = -np.exp(p["mu_log_minus_alpha"] + p.get("sd_log_minus_alpha", 0.) * z[0])
425     gamma = p.get("mean_gamma", 0.) + p.get("sd_gamma", 0.) * z[1]
426     utility = (np.array([0., p["beta_berry"], p["beta_plain"]]) + p.get("inside_constant", 0.)
427                + alpha * data["prices"] + gamma * data["xi"] + p.get("price_control", 0.) * data["cf"])
428     utility = np.where(data["available"], utility, -np.inf)
429     if data["outside"]:
430         utility = np.column_stack([utility, np.zeros(len(utility))])
431     logprob = utility - logsumexp(utility, axis=1, keepdims=True)
432     row_ll = (data["counts"] * np.where(data["counts"] > 0, logprob, 0.)).sum(axis=1)
433     likelihoods.append(np.bincount(data["hh"], weights=row_ll, minlength=data["n_households"]))
434     residual = data["counts"][:, :3] - total[:, None] * np.exp(logprob[:, :3])
435     price_score = (residual * data["prices"]).sum(axis=1) * alpha
436     columns = {"inside_constant": residual.sum(axis=1), "beta_berry": residual[:, 1],
```

```
437                "beta_plain": residual[:, 2], "mu_log_minus_alpha": price_score,
438                "sd_log_minus_alpha": price_score * z[0],
439                "price_control": (residual * data["cf"]).sum(axis=1)}
440    columns["mean_gamma"] = (residual * data["xi"]).sum(axis=1)
441    columns["sd_gamma"] = columns["mean_gamma"] * z[1]
442    scores.append(np.column_stack([np.bincount(data["hh"], weights=columns[n],
443                                               minlength=data["n_households"]) for n in names]))
```

**Read this block:**

- Line 423: for each entry yielded by `active`, bind/unpack it into `z` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 424, `alpha`: Set this name to the negated or sign-adjusted expression `-np.exp(p['mu_log_minus_alpha'] + p.get('sd_log_minus_alp...`.
- Line 425, `gamma`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 426, `utility`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 428, `utility`: Set this name to the result of calling `np.where(...)`; the arguments and preceding method chain are shown above.
- Line 429: test `data['outside']`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 430, `utility`: Set this name to the result of calling `np.column_stack(...)`; the arguments and preceding method chain are shown above.
- Line 431, `logprob`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 432, `row_ll`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.
- Line 433: call `likelihoods.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 434, `residual`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 435, `price_score`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 436, `columns`: Set this name to a dictionary with named entries `inside_constant`, `beta_berry`, `beta_plain`, `mu_log_minus_alpha`, `sd_log_minus_alpha`, `price_control`.
- Line 440, `columns['mean_gamma']`: Store the result of calling `(residual * data['xi']).sum(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 441, `columns['sd_gamma']`: Store multiplication of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 442: call `scores.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.
- `.column_stack`: Place one-dimensional vectors beside each other as columns of a matrix.
- `logsumexp`: Compute log(sum(exp(values))) stably; axis selects alternatives, draws, or another reduction dimension.
- `.bincount`: Accumulate frequencies or weighted sums by dense nonnegative integer labels; repeated household codes are added together.

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `len`: Count elements/rows rather than summing their numerical values.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 444–447:

```
444 likelihoods = np.asarray(likelihoods)
445 normalizer = logsumexp(likelihoods, axis=0)
446 weights = np.exp(likelihoods - normalizer)
447 gradient = -np.einsum("rh,rhp->p", weights, np.asarray(scores)) / data["n_households"]
```

**Read this block:**

- Line 444, `likelihoods`: Set this name to the result of calling `np.asarray(...)`; the arguments and preceding method chain are shown above.
- Line 445, `normalizer`: Set this name to the result of calling `logsumexp(...)`; the arguments and preceding method chain are shown above.
- Line 446, `weights`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 447, `gradient`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

**Syntax and calls used here:**

- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `.einsum`: Perform an explicitly labeled sum of products; rh,rhp->p sums draws/households and retains the parameter axis.

Source lines 448–448:

```
448 return -(normalizer - np.log(len(active))).mean(), gradient
```

**Read this block:**

- Line 448: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

**Syntax and calls used here:**

- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.
- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.

## optimize demand

Original function: `optimize_demand`. Source begins at line 451.

This sets up and runs multiple bounded local searches and returns the selected fit, every starting solution, and the bounds. Initial SDs are .3 because zero can be stationary under symmetric integration. Bounds are pairs `(lower, upper)` in the same order as parameter names. The optional initial vector is copied before perturbation, preventing caller-owned coefficients from being changed.

Each start uses `minimize(..., method="L-BFGS-B", jac=True)`. The callback reports progress every ten iterations, while completed starts are saved to JSON. `np.clip` keeps perturbed starts feasible. `min(results, key=lambda r: r.fun)` uses each result's loss as its comparison key; it selects the smallest loss, not necessarily a converged fit. An alternative is to select only converged results, but that is not the rule this code implements. Multistart search probes local solutions and does not establish a global optimum.

**Definition and stated intent:**

Source lines 451–451:

```
451 def optimize_demand(data, args, initial=None, number_starts=None, quiet=False):
```

`def` creates this callable; the parentheses name its inputs (`data`, `args`, `initial`, `number_starts`, `quiet`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 452–455:

```
452 draws = make_draws(args.draws, args.seed)
453 names = parameter_names(args.model, data["outside"], data["control"])
454 start = np.zeros(len(names))
455 bounds = []
```

**Read this block:**

- Line 452, `draws`: Set this name to the result of calling `make_draws(...)`; the arguments and preceding method chain are shown above.
- Line 453, `names`: Set this name to the result of calling `parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 454, `start`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 455, `bounds`: Set this name to the list `[]`.

**Syntax and calls used here:**

- `make_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 456–463:

```
456 for k, name in enumerate(names):
457     if name.startswith("sd_"):
458         start[k] = 0.3  # Zero SD is a stationary point: start away from zero.
459         bounds.append((0.0, 3.0))
460     elif name == "mu_log_minus_alpha":
461         bounds.append((-6.0, 3.0))
462     else:
463         bounds.append((-20.0, 20.0))
```

**Read this block:**

- Line 456: for each entry yielded by `enumerate(names)`, bind/unpack it into `(k, name)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 457: test `name.startswith('sd_')`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 458, `start[k]`: Store the literal value `0.3` in the selected entry/column.
- Line 459: call `bounds.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 460: test `name == 'mu_log_minus_alpha'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 461: call `bounds.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 463: call `bounds.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.

Source lines 464–465:

```
464 if initial is not None:
465     start = np.asarray(initial).copy()
```

**Read this block:**

- Line 464: test `initial is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 465, `start`: Set this name to the result of calling `.copy(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.

Source lines 466–467:

```
466 rng = np.random.default_rng(args.seed + 1)
467 results = []
```

**Read this block:**

- Line 466, `rng`: Set this name to the result of calling `np.random.default_rng(...)`; the arguments and preceding method chain are shown above.
- Line 467, `results`: Set this name to the list `[]`.

**Syntax and calls used here:**

- `.default_rng`: Create a local seeded random Generator, avoiding dependence on a shared global random stream.

Source lines 469–470:

```
469 def objective(theta):
470     return demand_objective_gradient(theta, data, draws, args.model)
```

**Read this block:**

- Line 469: define nested callable `objective`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 472–506:

```
472 for s in range(args.starts if number_starts is None else number_starts):
473     initial = start.copy()
474     if s:
475         initial += rng.normal(0, 0.5, len(start))
476         initial = np.clip(initial, np.array(bounds)[:, 0], np.array(bounds)[:, 1])
477     began = time.monotonic()
478     iterations = [0]
479 
480     def progress(theta):
481         iterations[0] += 1
482         if not quiet and iterations[0] % 10 == 0:
483             print(f"Start {s + 1}: iteration {iterations[0]}, elapsed {time.monotonic() - began:.0f}s", flush=True)
484 
485     fit = minimize(
486         objective,
487         initial,
488         method="L-BFGS-B",
489         jac=True,
490         callback=progress,
491         bounds=bounds,
```

```
492         options={"maxiter": args.maxiter, "ftol": 1e-10, "gtol": 1e-6},
493     )
494     results.append(fit)
495     if not quiet:
496         args.output.mkdir(parents=True, exist_ok=True)
497         (args.output / "starts_checkpoint.json").write_text(json.dumps([
498             dict(theta=r.x.tolist(), nll=float(r.fun * data["n_households"]),
499                  converged=bool(r.success), iterations=int(r.nit), evaluations=int(r.nfev))
500             for r in results], indent=2))
501     if not quiet:
502         print(
503             f"Start {s + 1}: NLL={fit.fun * data['n_households']:.4f}; "
504             f"converged={fit.success}",
505             flush=True,
506         )
```

**Read this block:**

- Line 472: for each entry yielded by `range(args.starts if number_starts is None else number_starts)`, bind/unpack it into `s` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 473, `initial`: Set this name to the result of calling `start.copy(...)`; the arguments and preceding method chain are shown above.
- Line 474: test `s`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 475: update `initial` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 476, `initial`: Set this name to the result of calling `np.clip(...)`; the arguments and preceding method chain are shown above.
- Line 477, `began`: Set this name to the result of calling `time.monotonic(...)`; the arguments and preceding method chain are shown above.
- Line 478, `iterations`: Set this name to the list `[0]`.
- Line 480: define nested callable `progress`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.
- Line 485, `fit`: Set this name to the result of calling `minimize(...)`; the arguments and preceding method chain are shown above.
- Line 494: call `results.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 495: test `not quiet`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 496: call `args.output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 497: call `(args.output / 'starts_checkpoint.json').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 501: test `not quiet`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 502: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `.monotonic`: Read a monotonic clock for elapsed durations rather than a wall-clock date/time.
- `minimize`: Run iterative numerical minimization; jac=True expects the objective to return value plus gradient.
- `.normal`: Draw normal random values with the supplied location/scale, used for starts or synthetic data.
- `.clip`: Cap values at lower/upper limits; here it keeps starts feasible or exponentials within floating-point range.
- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.dumps`: Serialize Python dictionaries/lists/scalars into a JSON string, with optional readable indentation.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `bool`: Convert scalar truthiness to an explicit Boolean flag.
- `int`: Convert to an integer; it does not by itself validate that a source value was already integral.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 508–508:

```
508 best = min(results, key=lambda r: r.fun)
```

**Read this block:**

- Line 508, `best`: Set this name to the result of calling `min(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `min`: Select the smallest entry; key= can select by objective value rather than object identity.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.

Source lines 509–509:

```
509 return best, results, bounds
```

**Read this block:**

- Line 509: return a tuple of 3 results. The caller can unpack them; return ends the call, unlike print.

## optimize demand.objective

Original function: `optimize_demand.objective`. Source begins at line 469.

This small nested adapter has one explicit input, `theta`. It forwards theta and the enclosing call's data/draws/model to the actual objective/gradient function. SciPy expects a callable accepting the changing candidate vector; the closure supplies the fixed objects. A global variable for data could work but would make concurrent or repeated fits harder to reason about. Returning the whole pair is intentional because the surrounding minimizer sets `jac=True`.

**Definition and stated intent:**

Source lines 469–469:

```
469 def objective(theta):
```

`def` creates this callable; the parentheses name its inputs (`theta`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 470–470:

```
470 return demand_objective_gradient(theta, data, draws, args.model)
```

**Read this block:**

- Line 470: return the result of calling `demand_objective_gradient(...)`; the arguments and preceding method chain are shown above. This ends the call.

**Syntax and calls used here:**

- `demand_objective_gradient`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

## optimize demand.progress

Original function: `optimize_demand.progress`. Source begins at line 480.

SciPy calls this nested callback after iterations, passing the current vector. The body increments `iterations[0]`, checks a remainder modulo ten, and prints elapsed time unless quiet mode is active. The callback does not return an objective value and does not choose the next parameter vector.

The enclosing variable is a one-element list so the nested function can mutate its element without rebinding the outer name. An ordinary integer counter with assignment would require `nonlocal iterations`. That alternative is often clearer; the existing list technique works because objects can be mutated through a captured reference. `time.monotonic` measures elapsed duration without wall-clock adjustments changing the result.

**Definition and stated intent:**

Source lines 480–480:

```
480 def progress(theta):
```

`def` creates this callable; the parentheses name its inputs (`theta`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 481–481:

```
481 iterations[0] += 1
```

**Read this block:**

- Line 481: update `iterations[0]` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 482–483:

```
482 if not quiet and iterations[0] % 10 == 0:
483     print(f"Start {s + 1}: iteration {iterations[0]}, elapsed {time.monotonic() - began:.0f}s", flush=True)
```

**Read this block:**

- Line 482: test `not quiet and iterations[0] % 10 == 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 483: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.monotonic`: Read a monotonic clock for elapsed durations rather than a wall-clock date/time.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

**Output reminder:** reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

## plot gamma distribution

Original function: `plot_gamma_distribution`. Source begins at line 512.

Input is an estimate table, output directory, and convergence flag. This is a side-effect function: it saves a figure and implicitly returns `None`. `set_index("parameter")["estimate"]` creates a name-indexed Series for coefficient lookup. The early bare `return` skips naive models with no gamma.

For positive SD, `linspace` makes a smooth horizontal grid, `norm.pdf` computes density, and `fill_between(..., where=x < 0)` shades the negative region. For zero SD a continuous density would divide by zero or misrepresent a point mass, so the `else` branch draws a vertical line. `norm.cdf(0, ...)` computes the implied population habit share. `Agg` supports file rendering without a GUI; `plt.close(fig)` releases the figure. This is a distribution of latent types, not an uncertainty interval for the mean.

**Definition and stated intent:**

Source lines 512–513:

```
512 def plot_gamma_distribution(table, output, converged=True):
513     """Plot the fitted population distribution, not uncertainty in its mean."""
```

`def` creates this callable; the parentheses name its inputs (`table`, `output`, `converged`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 514–514:

```
514 estimates = table.set_index("parameter")["estimate"]
```

**Read this block:**

- Line 514, `estimates`: Set this name to the entry/slice selected by `table.set_index('parameter')['estimate']`; brackets select from the object to their left.

**Syntax and calls used here:**

- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.

Source lines 515–516:

```
515 if "mean_gamma" not in estimates:
516     return  # Naive models have no gamma parameter.
```

**Read this block:**

- Line 515: test `'mean_gamma' not in estimates`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 516: bare return ends this call immediately with None. Later statements in the function are skipped.

Source lines 517–523:

```
517 import matplotlib
518 matplotlib.use("Agg")
519 import matplotlib.pyplot as plt
520 from scipy.stats import norm
521
522 mean = float(estimates["mean_gamma"])
523 sd = float(estimates.get("sd_gamma", 0.))
```

**Read this block:**

- Line 517: import `matplotlib`. An as-name is a convenient local alias, not another library.

- Line 518: call `matplotlib.use` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 519: import `matplotlib.pyplot as plt`. An as-name is a convenient local alias, not another library.
- Line 520: from `scipy.stats`, import `norm` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 522, `mean`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 523, `sd`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.use`: Select a Matplotlib backend; Agg renders to files without opening a GUI window.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.

Source lines 524–524:

```
524 fig, ax = plt.subplots(figsize=(7, 5))
```

**Read this block:**

- Line 524, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.

**Syntax and calls used here:**

- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.

Source lines 525–539:

```
525 if sd > 0:
526     x = np.linspace(mean - 4 * sd, mean + 4 * sd, 801)
527     density = norm.pdf(x, loc=mean, scale=sd)
528     ax.plot(x, density, color="steelblue")
529     ax.fill_between(x, density, where=x < 0, color="steelblue", alpha=.25)
530     ax.set_ylabel("Population density")
531     negative = norm.cdf(0, loc=mean, scale=sd)
532     description = f"Mean = {mean:.4g}; SD = {sd:.4g}; P(gamma < 0) = {negative:.1%}"
533 else:
534     ax.axvline(mean, color="steelblue", linewidth=2, label="Point mass (SD = 0)")
535     ax.set_xlim(mean - 1, mean + 1)
536     ax.set_yticks([])
537     ax.set_ylabel("Point mass")
538     ax.legend()
539     description = f"All households: gamma = {mean:.4g} (SD = 0)"
```

**Read this block:**

- Line 525: test `sd > 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 526, `x`: Set this name to the result of calling `np.linspace(...)`; the arguments and preceding method chain are shown above.
- Line 527, `density`: Set this name to the result of calling `norm.pdf(...)`; the arguments and preceding method chain are shown above.
- Line 528: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 529: call `ax.fill_between` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 530: call `ax.set_ylabel` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 531, `negative`: Set this name to the result of calling `norm.cdf(...)`; the arguments and preceding method chain are shown above.
- Line 532, `description`: Set this name to formatted text, substituting the brace-enclosed values into the f-string.
- Line 534: call `ax.axvline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 535: call `ax.set_xlim` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 536: call `ax.set_yticks` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 537: call `ax.set_ylabel` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 538: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 539, `description`: Set this name to formatted text, substituting the brace-enclosed values into the f-string.

**Syntax and calls used here:**

- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.
- `.pdf`: Evaluate a probability density (here a normal distribution), not a PDF document file.
- `.plot`: Draw a line from supplied x/y values on the receiving plotting axes.
- `.fill_between`: Shade the area between curves/zero, optionally only where a Boolean condition is true.
- `.set_ylabel`: Set the vertical-axis label, whose unit should match the plotted values.
- `.cdf`: Evaluate cumulative probability below a threshold, used for the population share with negative gamma.
- `.axvline`: Add a vertical reference line, such as zero gamma or a point-mass type.
- `.set_xlim`: Set the visible x-axis limits; this changes presentation rather than the calculated results.
- `.set_yticks`: Choose which vertical tick marks to show, including an empty list for a point-mass display.
- `.legend`: Render labels previously attached to plotted series, identifying curves.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 540–541:

```
ax.axvline(0, color="gray", linestyle="--", linewidth=1)
title = "Fitted population gamma distribution"
```

**Read this block:**

- Line 540: call `ax.axvline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 541, `title`: Set this name to the literal text `'Fitted population gamma distribution'`.

Source lines 542–543:

```
if not converged or table["at_bound"].any():
    title += " (provisional fit)"
```

**Read this block:**

- Line 542: test `not converged or table['at_bound'].any()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 543: update `title` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

**Syntax and calls used here:**

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.

Source lines 544–548:

```
ax.set(xlabel="Gamma (common across flavors)", title=title)
fig.text(.5, .02, description, ha="center")
fig.tight_layout(rect=(0, .06, 1, 1))
fig.savefig(output / "gamma_distribution.pdf")
plt.close(fig)
```

**Read this block:**

- Line 544: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 545: call `fig.text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 546: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 547: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 548: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.text`: Place a text label on the axes/Figure at the supplied coordinates.
- `.tight_layout`: Adjust plot spacing to reduce overlap between labels and content.
- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.
- `.close`: Release a Figure or file resource, according to the receiving object.

## fit model

Original function: `fit_model`. Source begins at line 551.

This coordinates point fitting, reporting, optional inference, and saved files. It turns `best.x` into a labeled table. The boundary list comprehension pairs each coefficient with its bounds and tests distance below `1e-4`; flags do not automatically change the estimate. Saving point estimates before bootstrap protects the useful main result if a long inference job is interrupted.

The nested `estimate` callback supplies demand fitting to the generic bootstrap. The bootstrap runs only after a converged main fit and with the original weekly frame available. Missing inference columns are explicitly filled with NaN so an absent SE cannot be mistaken for zero. `vars(args).items()` enumerates settings; Path objects become strings so JSON serialization can encode them. The returned `best` is still a SciPy result object, while CSV/JSON provide portable outputs.

**Definition and stated intent:**

Source lines 551–552:

```
def fit_model(data, args, frame=None):
    """Estimate demand, run the two-stage bootstrap, and save readable outputs."""
```

`def` creates this callable; the parentheses name its inputs (`data`, `args`, `frame`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 553–560:

```
best, results, bounds = optimize_demand(data, args)
names = parameter_names(args.model, data["outside"], data["control"])
table = pd.DataFrame({"parameter": names, "estimate": best.x})
table["at_bound"] = [
```

```
557     min(abs(v - lo), abs(v - hi)) < 1e-4 for v, (lo, hi) in zip(best.x, bounds)
558 ]
559 args.output.mkdir(parents=True, exist_ok=True)
560 table.to_csv(args.output / "point_estimates.csv", index=False)
```

**Read this block:**

- Line 553, `(best, results, bounds)`: Unpack the result of calling `optimize_demand(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 554, `names`: Set this name to the result of calling `parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 555, `table`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 556, `table['at_bound']`: Store a list computed by the displayed for-expression in the selected entry/column.
- Line 559: call `args.output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 560: call `table.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `optimize_demand`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `min`: Select the smallest entry; key= can select by objective value rather than object identity.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `abs`: Take magnitude, removing a sign before a distance/tolerance comparison.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.

Source lines 561–562:

```
561 plot_gamma_distribution(table, args.output, converged=best.success)
562 inference = {"method": args.se_method, "status": "disabled"}
```

**Read this block:**

- Line 561: call `plot_gamma_distribution` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 562, `inference`: Set this name to a dictionary with named entries `method`, `status`.

**Syntax and calls used here:**

- `plot_gamma_distribution`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 563–583:

```
563 if args.se_method == "bootstrap":
564     if not best.success:
565         inference["status"] = "main fit not converged; bootstrap not run"
566     elif frame is None:
567         raise ValueError("Bootstrap requires the weekly input frame.")
568     else:
569 
570         def estimate(sample, initial):
571             bootstrap_data = make_arrays(sample, args.capacity)
572             fitted, _, _ = optimize_demand(
```

```
573                 bootstrap_data, args, initial, args.bootstrap_starts, quiet=True
574             )
575             return fitted
576 
577         standard_errors, inference = bootstrap_standard_errors(
578             frame, args, estimate, best.x, names
579         )
580         if standard_errors is not None:
581             table = table.merge(
582                 standard_errors, on="parameter", validate="one_to_one"
583             )
```

**Read this block:**

- Line 563: test `args.se_method == 'bootstrap'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 564: test `not best.success`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 565, `inference['status']`: Store the literal text `'main fit not converged; bootstrap not run'` in the selected entry/column.
- Line 566: test `frame is None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 567: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 570: define nested callable `estimate`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.
- Line 577, `(standard_errors, inference)`: Unpack the result of calling `bootstrap_standard_errors(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 580: test `standard_errors is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 581, `table`: Set this name to the result of calling `table.merge(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `bootstrap_standard_errors`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.merge`: Join pandas tables through matching keys; how controls retention and validate checks intended key cardinality.

Source lines 585–587:

```
585 for column in ["std_error", "ci_lower", "ci_upper"]:
586     if column not in table:
587         table[column] = np.nan
```

**Read this block:**

- Line 585: for each entry yielded by `['std_error', 'ci_lower', 'ci_upper']`, bind/unpack it into `column` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 586: test `column not in table`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 587, `table[column]`: Store the attribute `np.nan` of the object before the dot in the selected entry/column.

Source lines 588–588:

```
588 table.to_csv(args.output / "estimates.csv", index=False)
```

**Read this block:**

- Line 588: call `table.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 589–609:

```
589 summary = dict(
590     model=args.model,
591     households=data["n_households"],
592     weeks=len(data["hh"]),
593     capacity=args.capacity,
594     inference=inference,
595     converged=bool(best.success),
596     message=str(best.message),
597     nll=float(best.fun * data["n_households"]),
598     settings={
599         k: str(v) if isinstance(v, Path) else v for k, v in vars(args).items()
600     },
601     starts=[
602         dict(
603             nll=float(r.fun * data["n_households"]),
604             converged=bool(r.success),
605             theta=r.x.tolist(),
606         )
607         for r in results
608     ],
609 )
```

**Read this block:**

- Line 589, `summary`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `len`: Count elements/rows rather than summing their numerical values.
- `bool`: Convert scalar truthiness to an explicit Boolean flag.
- `str`: Convert to text for paths, messages, or serialization.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `isinstance`: Check a runtime type to select the appropriate conversion path.
- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.
- `vars`: Expose an object's attribute dictionary, here the parsed settings saved with the result.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 610–612:

```
610 (args.output / "fit.json").write_text(json.dumps(summary, indent=2))
611 print("Inference: " + inference["status"], flush=True)
612 print(table.to_string(index=False))
```

**Read this block:**

- Line 610: call `(args.output / 'fit.json').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 611: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 612: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `.dumps`: Serialize Python dictionaries/lists/scalars into a JSON string, with optional readable indentation.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Source lines 613–614:

```
613 if not best.success or table["at_bound"].any():
614     print("Provisional fit: inspect convergence/bounds before interpretation.")
```

**Read this block:**

- Line 613: test `not best.success or table['at_bound'].any()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 614: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.

Source lines 615–615:

```
615 return best
```

**Read this block:**

- Line 615: return the object currently named `best`; plain assignment does not copy it. This ends the call.

## fit model.estimate

Original function: `fit_model.estimate`. Source begins at line 570.

This nested function accepts one resampled weekly table and a starting coefficient vector. It rebuilds numerical arrays, runs demand optimization with bootstrap start count, and returns only the selected fit. `fitted, _, _` discards other optimizer outputs. It captures capacity and other settings from outer `args`. Passing this callback avoids importing `est.py` back into `bootstrap_se.py`, which could create a circular module dependency. Price controls are reestimated by the bootstrap before this function is called, not inside this callback.

**Definition and stated intent:**

Source lines 570–570:

```
570 def estimate(sample, initial):
```

`def` creates this callable; the parentheses name its inputs (`sample`, `initial`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 571–574:

```
571 bootstrap_data = make_arrays(sample, args.capacity)
572 fitted, _, _ = optimize_demand(
573     bootstrap_data, args, initial, args.bootstrap_starts, quiet=True
574 )
```

**Read this block:**

- Line 571, `bootstrap_data`: Set this name to the result of calling `make_arrays(...)`; the arguments and preceding method chain are shown above.

- Line 572, `(fitted, _, _)`: Unpack the result of calling `optimize_demand(...)`; the arguments and preceding method chain are shown above into these targets in order.

**Syntax and calls used here:**

- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `optimize_demand`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 575–575:

```
575 return fitted
```

**Read this block:**

- Line 575: return the object currently named `fitted`; plain assignment does not copy it. This ends the call.

## main

Original function: `main`. Source begins at line 618.

This is the command-line entry point. `default_model` lets small wrapper scripts reuse the entire interface while selecting a different default model. Each `add_argument` describes a setting, conversion, default, or permitted choices. The parser turns a flag such as `--no-price-control` into the attribute `args.no_price_control`; hyphens become underscores in attribute names.

Validation runs before expensive data loading. `parser.error` prints usage and exits for invalid settings. After parsing, execution is preparation, optional first stages, weekly input export, array construction, and fitting. This division lets tests construct a small `SimpleNamespace` and call internal functions without pretending to run a shell command. Repeating this code in all four entry scripts would allow defaults and validations to drift apart.

**Definition and stated intent:**

Source lines 618–618:

```
618 def main(default_model="heterogeneous"):
```

`def` creates this callable; the parentheses name its inputs (`default_model`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 619–631:

```
619 parser = argparse.ArgumentParser(description=__doc__)
620 parser.add_argument(
621     "--sample",
622     type=Path,
623     default=DATA_DIR / "summary_outputs/yogurt_estimation_sample.parquet",
624 )
625 parser.add_argument("--flavors", type=Path, default=FLAVOR_FILE)
626 parser.add_argument(
627     "--products-extra",
628     type=Path,
629     help="Annual flavor metadata; defaults to RMS/combined/products_extra.parquet beside the RMS data directory.",
630 )
631 parser.add_argument("--output", type=Path, default=Path("estimation_output"))
```

**Read this block:**

- Line 619, `parser`: Set this name to the result of calling `argparse.ArgumentParser(...)`; the arguments and preceding method chain are shown above.
- Line 620: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 625: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 626: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 631: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.ArgumentParser`: Create the command-line parser object that will register and validate options.
- `.add_argument`: Register a command-line option and its type/default/choices/flag behavior before parsing.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 632–645:

```
632 parser.add_argument(
633     "--model",
634     choices=["naive", "homogeneous", "mixed_naive", "heterogeneous"],
635     default=default_model,
636 )
637 parser.add_argument("--first-year", type=int, default=2006)
638 parser.add_argument("--last-year", type=int, default=2015)
639 parser.add_argument("--max-households", type=int)
640 parser.add_argument(
641     "--capacity",
642     type=int,
643     default=0,
644     help="0: conditional flavor demand; 7: outside=max(7-Q,0)",
645 )
```

**Read this block:**

- Line 632: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 637: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 638: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 639: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 640: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 646–657:

```
646 parser.add_argument(
647     "--no-price-control",
648     action="store_true",
649     help="Disable the price control using the overall other-DMA mean-price IV",
650 )
651 parser.add_argument("--household-dma-column", default="dma_cd")
652 parser.add_argument(
653     "--se-method", choices=["bootstrap", "none"], default="bootstrap"
654 )
655 parser.add_argument("--bootstrap-reps", type=int, default=49)
656 parser.add_argument("--bootstrap-starts", type=int, default=2)
657 parser.add_argument("--habit-weight", type=float, default=0.7)
```

**Read this block:**

- Line 646: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 651: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 652: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 655: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 656: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 657: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 658–662:

```
parser.add_argument("--draws", type=int, default=128)
parser.add_argument("--starts", type=int, default=3)
parser.add_argument("--maxiter", type=int, default=600)
parser.add_argument("--seed", type=int, default=306)
args = parser.parse_args()
```

**Read this block:**

- Line 658: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 659: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 660: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 661: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 662, `args`: Set this name to the result of calling `parser.parse_args(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.parse_args`: Read command-line arguments into a Namespace of named attributes.

Source lines 663–664:

```
if not 0 <= args.habit_weight <= 1 or args.capacity < 0 or args.starts < 1:
    parser.error("Check habit weight, capacity, and starts.")
```

**Read this block:**

- Line 663: test `not 0 <= args.habit_weight <= 1 or args.capacity < 0 or args.starts < 1`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 664: call `parser.error` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.error`: Report invalid command-line input and exit through argparse; it is not a recoverable data-cleaning step here.

Source lines 665–668:

```
if args.first_year > args.last_year or (
    args.max_households is not None and args.max_households < 1
):
    parser.error("Check years and household limit.")
```

**Read this block:**

- Line 665: test `args.first_year > args.last_year or (args.max_households is not None and args...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 668: call `parser.error` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 669–670:

```
669 if args.draws < 2 or args.draws % 2 or args.maxiter < 1:
670     parser.error("Use an even draw count >= 2 and maxiter >= 1.")
```

**Read this block:**

- Line 669: test `args.draws < 2 or args.draws % 2 or args.maxiter < 1`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 670: call `parser.error` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.

Source lines 671–674:

```
671 if args.se_method == "bootstrap" and (
672     args.bootstrap_reps < 20 or args.bootstrap_starts < 1
673 ):
674     parser.error("Bootstrap requires at least 20 replications and one start.")
```

**Read this block:**

- Line 671: test `args.se_method == 'bootstrap' and (args.bootstrap_reps < 20 or args.bootstrap...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 674: call `parser.error` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 675–675:

```
675 frame = prepare_data(args)
```

**Read this block:**

- Line 675, `frame`: Set this name to the result of calling `prepare_data(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `prepare_data`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 676–680:

```
676 if not args.no_price_control:
677     frame, first_stage = add_price_controls(frame)
678     args.output.mkdir(parents=True, exist_ok=True)
679     first_stage.to_csv(args.output / "first_stage.csv", index=False)
680     print(first_stage.to_string(index=False), flush=True)
```

**Read this block:**

- Line 676: test `not args.no_price_control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 677, `(frame, first_stage)`: Unpack the result of calling `add_price_controls(...)`; the arguments and preceding method chain are shown above into these targets in order.

- Line 678: call `args.output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 679: call `first_stage.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 680: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `add_price_controls`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Source lines 681–683:

```
681 args.output.mkdir(parents=True, exist_ok=True)
682 frame.to_csv(args.output / "weekly_inputs.csv", index=False)
683 fit_model(make_arrays(frame, args.capacity), args, frame)
```

**Read this block:**

- Line 681: call `args.output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 682: call `frame.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 683: call `fit_model` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `fit_model`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

**Output reminder:** reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

# est_code/est_gamma.py

This wrapper selects the `homogeneous` default, meaning common alpha and common unrestricted gamma. It imports and calls the shared `main`; it does not define a function locally. The name of a script is not itself a Python function or an instruction to fit a different formula unless its actual call says so.

## Module setup and direct-execution blocks

Source lines 1–2:

```
1 """Common beta, alpha, and flavor-specific gamma. See README.md."""
2 from est import main
```

**Read this block:**

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: from `est`, import `main` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 4–5:

```
4 if __name__ == "__main__":
5     main("homogeneous")
```

**Read this block:**

- Line 4: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 5: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

# est__code/est__het__gamma.py

This wrapper supplies the `heterogeneous` default to shared `main`. Both alpha and gamma vary by persistent type. Parentheses call the imported function; writing just `main` would retrieve the function object without executing it.

## Module setup and direct-execution blocks

Source lines 1–2:

```
1 """Common beta; heterogeneous gamma AND alpha. See README.md."""
2 from est import main
```

**Read this block:**

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: from `est`, import `main` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 4–5:

```
4 if __name__ == "__main__":
5     main("heterogeneous")
```

**Read this block:**

- Line 4: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 5: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

# est__code/est__het__naive.py

This guarded wrapper supplies the `mixed_naive` default. The common estimator then includes price heterogeneity while excluding gamma. The small file is a convenient command name and shares all data, optimizer, and inference options with the main script.

### Module setup and direct-execution blocks

Source lines 1–2:

```
"""Common beta, heterogeneous alpha, and no gamma. See README.md."""
from est import main
```

**Read this block:**

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: from `est`, import `main` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 4–5:

```
if __name__ == "__main__":
    main("mixed_naive")
```

**Read this block:**

- Line 4: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 5: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

## est__code/est__naive.py

This wrapper imports `main` and calls it with the `naive` default only under the entry-point guard. The wrapper contains no new estimation algorithm. `--model` can still override the supplied default because the common parser processes the user's options. Reusing one main function avoids four copies of data preparation and CLI validation.

### Module setup and direct-execution blocks

Source lines 1–2:

```
"""Common beta/alpha and no history effect. See README.md."""
from est import main
```

**Read this block:**

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: from `est`, import `main` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 4–5:

```
if __name__ == "__main__":
    main("naive")
```

**Read this block:**

- Line 4: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 5: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

# est_code/price_control.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

## Module setup and direct-execution blocks

Source lines 1–7:

```
1 """Other-DMA instruments and the price first stage. See README.md."""
2 import numpy as np
3 import pandas as pd
4
5 from scipy.stats import t as student_t
6
7 FLAVORS = ["other", "berry", "plain"]
```

**Read this block:**

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 3: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 5: from `scipy.stats`, import `t as student_t` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 7, `FLAVORS`: Set this name to the list `['other', 'berry', 'plain']`.

## check dma

Original function: `check_dma`. Source begins at line 10.

This converts a pandas Series to integer DMA IDs, rejecting missing, nonnumeric, nonpositive, and fractional entries. `pd.to_numeric(errors="coerce")` turns bad strings into missing numbers so one Boolean mask can find all invalid rows. `codes == np.floor(codes)` checks integrality without rounding fractional inputs into a valid ID. Only after validation does `astype("int64")` convert the dtype. `description` makes an error identify the consumer or retail source. Quietly replacing an invalid DMA with zero would contaminate own-market exclusion.

**Definition and stated intent:**

Source lines 10–11:

```
10 def check_dma(values, description):
11     """Normalize Nielsen's numeric DMA identifiers across panel and RMS files."""
```

`def` creates this callable; the parentheses name its inputs (`values`, `description`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 12–13:

```
12 codes = pd.to_numeric(values, errors="coerce")
13 valid = codes.notna() & (codes > 0) & (codes == np.floor(codes))
```

**Read this block:**

- Line 12, `codes`: Set this name to the result of calling `pd.to_numeric(...)`; the arguments and preceding method chain are shown above.

- Line 13, `valid`: Set this name to elementwise AND of the displayed operands; parentheses control which subexpressions are evaluated together.

**Syntax and calls used here:**

- `.notna`: Create a pandas mask marking nonmissing values; it does not establish numeric positivity or finiteness.
- `.floor`: Take the greatest integer no larger than each value; equality with it tests whether a positive code is integral.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.

Source lines 14–15:

```
14 if not valid.all():
15     raise ValueError(f"Missing/invalid {description} DMA codes: {(~valid).sum():,}")
```

**Read this block:**

- Line 14: test `not valid.all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 15: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

**Syntax and calls used here:**

- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 16–16:

```
16 return codes.astype("int64")
```

**Read this block:**

- Line 16: return the result of calling `codes.astype(...)`; the arguments and preceding method chain are shown above. This ends the call.

**Syntax and calls used here:**

- `.astype`: Convert the pandas/NumPy object's dtype; this changes representation, not merely its displayed label.

## other dma instruments

Original function: `other_dma_instruments`. Source begins at line 19.

This retained helper accepts three pandas tables and returns weekly rows with UPC-matched flavor IVs. It is **not** the pooled instrument construction used by current `est.py`. It first averages retail-DMA prices outside the consumer's DMA per UPC/week, then averages across the exposure store's UPC basket.

Renaming the own-DMA table's columns makes its keys match the merge. `present` is a Boolean Series; converting it to integers subtracts one from the DMA count only when that own-market quote exists. The grouped `count` ignores missing values whereas `size` counts all rows. Comparing them detects an incompletely instrumented basket and marks the whole flavor IV missing, rather than quietly averaging only a subset. Several joins replace a much slower nested lookup loop.

**Definition and stated intent:**

Source lines 19–25:

```
19 def other_dma_instruments(weekly, exposure_upcs, dma_prices):
20     """Match each exposure-store UPC to its same-week price in OTHER DMAs.
21
22     dma_prices has ONE mean package price per UPC/week/retail DMA. First average
23     these means over all DMAs except the CONSUMER's DMA, then average over the
24     exposure store's UPCs in a flavor. This matches the local price's UPC basket.
25     """
```

`def` creates this callable; the parentheses name its inputs (`weekly`, `exposure_upcs`, `dma_prices`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 26–33:

```
26 dma_prices = dma_prices.copy()
27 dma_prices["dma_code"] = check_dma(dma_prices["dma_code"], "retail")
28 pk = ["upc", "week"]
29 totals = (dma_prices.groupby(pk)["dma_price"].agg(["sum", "count"])
30           .rename(columns={"sum": "sum_dma", "count": "n_dma"}).reset_index())
31 exposure = weekly[["store_code_uc", "week", "dma_cd"]].drop_duplicates()
32 exposure = exposure.merge(exposure_upcs, on=["store_code_uc", "week"],
33                           how="left", validate="many_to_many")
```

**Read this block:**

- Line 26, `dma_prices`: Set this name to the result of calling `dma_prices.copy(...)`; the arguments and preceding method chain are shown above.
- Line 27, `dma_prices['dma_code']`: Store the result of calling `check_dma(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 28, `pk`: Set this name to the list `['upc', 'week']`.
- Line 29, `totals`: Set this name to the result of calling `.reset_index(...)`; the arguments and preceding method chain are shown above.
- Line 31, `exposure`: Set this name to the result of calling `.drop_duplicates(...)`; the arguments and preceding method chain are shown above.
- Line 32, `exposure`: Set this name to the result of calling `exposure.merge(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `check_dma`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.reset_index`: Turn the index back into ordinary row numbering; drop=True discards the old index rather than saving it as a column.
- `.rename`: Change column labels (or apply a renaming function); this does not change their underlying values.
- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `.drop_duplicates`: Retain one row per selected key; keep='first' makes earlier sorting/order economically consequential.
- `.merge`: Join pandas tables through matching keys; how controls retention and validate checks intended key cardinality.

Source lines 34–40:

```
34 exposure = exposure.merge(totals, on=pk, how="left", validate="many_to_one")
35 own = dma_prices.rename(columns={"dma_code": "dma_cd", "dma_price": "own_dma_price"})
36 exposure = exposure.merge(own, on=pk + ["dma_cd"], how="left", validate="many_to_one")
37 present = exposure["own_dma_price"].notna()
38 denominator = exposure["n_dma"] - present.astype(int)
39 exposure["iv"] = ((exposure["sum_dma"] - exposure["own_dma_price"].fillna(0))
```

```
40                  / denominator.where(denominator > 0))
```

**Read this block:**

- Line 34, `exposure`: Set this name to the result of calling `exposure.merge(...)`; the arguments and preceding method chain are shown above.
- Line 35, `own`: Set this name to the result of calling `dma_prices.rename(...)`; the arguments and preceding method chain are shown above.
- Line 36, `exposure`: Set this name to the result of calling `exposure.merge(...)`; the arguments and preceding method chain are shown above.
- Line 37, `present`: Set this name to the result of calling `exposure['own_dma_price'].notna(...)`; the arguments and preceding method chain are shown above.
- Line 38, `denominator`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 39, `exposure['iv']`: Store division of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.

**Syntax and calls used here:**

- `.notna`: Create a pandas mask marking nonmissing values; it does not establish numeric positivity or finiteness.
- `.astype`: Convert the pandas/NumPy object's dtype; this changes representation, not merely its displayed label.
- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- `.fillna`: Replace pandas missing entries with the provided fallback; it can change downstream denominators or validity checks.

Source lines 42–46:

```
42 keys = ["store_code_uc", "week", "dma_cd", "flavor"]
43 grouped = exposure.groupby(keys)["iv"].agg(["mean", "count", "size"])
44 grouped.loc[grouped["count"] != grouped["size"], "mean"] = np.nan
45 result = (grouped["mean"].unstack("flavor").reindex(columns=FLAVORS)
46           .rename(columns={f: "iv_" + f for f in FLAVORS}).reset_index())
```

**Read this block:**

- Line 42, `keys`: Set this name to the list `['store_code_uc', 'week', 'dma_cd', 'flavor']`.
- Line 43, `grouped`: Set this name to the result of calling `.agg(...)`; the arguments and preceding method chain are shown above.
- Line 44, `grouped.loc[grouped['count'] != grouped['size'], 'mean']`: Store the attribute `np.nan` of the object before the dot in the selected entry/column.
- Line 45, `result`: Set this name to the result of calling `.reset_index(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.reindex`: Align to an explicitly requested row/column order, adding missing labels with a chosen fill value if provided.
- `.unstack`: Move an index level into columns, such as turning long flavor counts into wide weekly outcomes.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.

Source lines 47–48:

```
47 return weekly.merge(result, on=["store_code_uc", "week", "dma_cd"], how="left",
48                    validate="many_to_one")
```

**Read this block:**

- Line 47: return the result of calling `weekly.merge(...)`; the arguments and preceding method chain are shown above. This ends the call.

## absorb effects

Original function: `absorb_effects`. Source begins at line 51.

Input `values` is a numerical matrix; `groups` lists row-aligned group labels such as week and DMA. Output is a copy with group means removed. Factorization converts arbitrary labels to dense integers. `np.add.at(means, code, residual)` accumulates rows into group sums even when indices repeat; ordinary fancy-index `means[code] += residual` may not accumulate repeats as intended.

`counts[:, None]` makes group counts a column so every variable is divided by its group size. The function cycles week and DMA demeaning until the biggest absolute change falls below tolerance, then returns. A maximum iteration limit raises on failure. A huge explicit dummy matrix is a valid alternative used in tests, but its memory grows with the number of fixed-effect groups. One pass is insufficient when group cells are unbalanced.

**Definition and stated intent:**

Source lines 51–56:

```
51 def absorb_effects(values, groups, tolerance=1e-10, maxiter=2000):
52     """Remove week and DMA means without a huge matrix of dummy variables.
53 
54     Repeated demeaning is needed because DMA/week cells are unbalanced.
55     Each household-week exposure has equal weight in the first stage.
56     """
```

`def` creates this callable; the parentheses name its inputs (`values`, `groups`, `tolerance`, `maxiter`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 57–58:

```
57 residual = np.asarray(values, dtype=float).copy()
58 codes = [pd.factorize(group, sort=True)[0] for group in groups]
```

**Read this block:**

- Line 57, `residual`: Set this name to the result of calling `.copy(...)`; the arguments and preceding method chain are shown above.
- Line 58, `codes`: Set this name to a list computed by the displayed for-expression.

**Syntax and calls used here:**

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `.factorize`: Encode labels as dense integer positions and return the original-label lookup alongside those positions.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 59–68:

```
59 for _ in range(maxiter):
60     previous = residual.copy()
61     for code in codes:
62         counts = np.bincount(code)
63         means = np.zeros((len(counts), residual.shape[1]))
64         np.add.at(means, code, residual)
65         means /= counts[:, None]
66         residual -= means[code]
67     if np.max(np.abs(residual - previous)) < tolerance:
68         return residual
```

**Read this block:**

- Line 59: for each entry yielded by `range(maxiter)`, bind/unpack it into `_` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 60, `previous`: Set this name to the result of calling `residual.copy(...)`; the arguments and preceding method chain are shown above.
- Line 61: for each entry yielded by `codes`, bind/unpack it into `code` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 62, `counts`: Set this name to the result of calling `np.bincount(...)`; the arguments and preceding method chain are shown above.
- Line 63, `means`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 64: call `np.add.at` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 65: update `means` using `/=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 66: update `residual` using `-=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 67: test `np.max(np.abs(residual - previous)) < tolerance`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 68: return the object currently named `residual`; plain assignment does not copy it. This ends the call.

**Syntax and calls used here:**

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `.bincount`: Accumulate frequencies or weighted sums by dense nonnegative integer labels; repeated household codes are added together.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `.add.at`: Accumulate in place at possibly repeated indices; this avoids the lost-repeat behavior of buffered fancy-index updates.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.abs`: Take absolute magnitude, as needed for tolerance tests and bound distances.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 69–69:

```
69 raise ValueError("First-stage fixed-effect demeaning did not converge.")
```

**Read this block:**

- Line 69: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

**Syntax and calls used here:**

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

## add price controls

Original function: `add_price_controls`. Source begins at line 72.

This returns `(augmented_frame, diagnostic_table)`. For each flavor it selects covered prices, absorbs week/DMA effects jointly from price, instrument, and Xi, then partials Xi out of y and z. `within[:, 0]`, `[:, 1]`, `[:, 2:]` split those roles by known column order. `lstsq(...)[0]` keeps only the coefficient vector from the least-squares result tuple.

Dot products compute the IV slope after partialling. A small residual instrument sum of squares raises instead of dividing by an almost zero denominator. `np.linalg.lstsq` tolerates redundant controls more gracefully than

explicitly inverting their cross-product matrix. The residual is assigned only to covered rows; unavailable alternatives retain zero placeholders. Cluster score sums, partial R-squared, and residual SD describe first-stage relevance and fit. They do not prove instrument validity. Reusing this function in bootstrap replications keeps the two-stage estimator consistent across samples.

**Definition and stated intent:**

Source lines 72–77:

```
72 def add_price_controls(frame):
73     """OLS separately by flavor: price ~ IV + all three Xi + week FE + DMA FE.
74 
75     Refit this function in every household bootstrap sample. Return residuals
76     in DOLLARS (no arbitrary rescaling) for the choice-model control function.
77     """
```

`def` creates this callable; the parentheses name its inputs (`frame`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 78–79:

```
78 result = frame.copy()
79 reports = []
```

**Read this block:**

- Line 78, `result`: Set this name to the result of calling `frame.copy(...)`; the arguments and preceding method chain are shown above.
- Line 79, `reports`: Set this name to the list `[]`.

**Syntax and calls used here:**

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.

Source lines 80–119:

```
80 for flavor in FLAVORS:
81     columns = ["price_" + flavor, "iv_" + flavor] + ["xi_" + f for f in FLAVORS]
82     covered = np.isfinite(result["price_" + flavor]) & (result["price_" + flavor] > 0)
83     sample = result.loc[covered]
84     if sample.empty:
85         raise ValueError(f"No covered prices for the {flavor} first stage.")
86     hh, households = pd.factorize(sample["household_code"], sort=True)
87     values = sample[columns].to_numpy(float)
88     if not np.isfinite(values).all():
89         raise ValueError(f"Nonfinite first-stage inputs for {flavor}.")
90     within = absorb_effects(values, [sample["week"], sample["dma_cd"]])
91     y, z, controls = within[:, 0], within[:, 1], within[:, 2:]
92     # Partial out Xi before estimating the excluded instrument's coefficient.
93     # This tolerates redundant Xi columns, but NOT a redundant instrument.
94     z_partial = z - controls @ np.linalg.lstsq(controls, z, rcond=None)[0]
95     y_partial = y - controls @ np.linalg.lstsq(controls, y, rcond=None)[0]
96     z_ss = z_partial @ z_partial
97     if z_ss <= 1e-12 * max(1., values[:, 1] @ values[:, 1]):
98         raise ValueError(f"No excluded-instrument variation after controls for {flavor}.")
99     pi = (z_partial @ y_partial) / z_ss
100    control_beta = np.linalg.lstsq(controls, y - pi*z, rcond=None)[0]
101    residual = y - pi*z - controls @ control_beta
102    if np.std(residual) < 1e-10:
103        raise ValueError(f"First-stage residual for {flavor} is essentially zero.")
104    result["cf_" + flavor] = 0.
105    result.loc[covered, "cf_" + flavor] = residual
106    # Household-clustered partial-IV t/F diagnostic (not a weak-IV-robust test).
```

```
    scores = np.bincount(hh, weights=z_partial * residual, minlength=len(households))
    pi_se = np.sqrt((scores @ scores) / z_ss**2)
    if len(households) > 1:
        pi_se *= np.sqrt(len(households) / (len(households) - 1))
    f_stat = float((pi / pi_se)**2) if pi_se > 0 else np.nan
    restricted_ss = y_partial @ y_partial
    partial_r2 = 1 - (residual @ residual) / restricted_ss if restricted_ss > 0 else np.nan
    reports.append(dict(flavor=flavor, iv_coefficient=float(pi),
                        iv_cluster_se=float(pi_se), iv_cluster_F=f_stat,
                        iv_pvalue=float(2*student_t.sf(np.sqrt(f_stat), len(households)-1))
                        if len(households) > 1 else np.nan,
                        partial_R2=float(partial_r2), residual_sd=float(np.std(residual)),
                        observations=len(sample), households=len(households)))
```

**Read this block:**

- Line 80: for each entry yielded by `FLAVORS`, bind/unpack it into `flavor` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 81, `columns`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 82, `covered`: Set this name to elementwise AND of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 83, `sample`: Set this name to the entry/slice selected by `result.loc[covered]`; brackets select from the object to their left.
- Line 84: test `sample.empty`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 85: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 86, `(hh, households)`: Unpack the result of calling `pd.factorize(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 87, `values`: Set this name to the result of calling `sample[columns].to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 88: test `not np.isfinite(values).all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 89: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 90, `within`: Set this name to the result of calling `absorb_effects(...)`; the arguments and preceding method chain are shown above.
- Line 91, `(y, z, controls)`: Unpack a tuple containing 3 displayed entries into these targets in order.
- Line 94, `z_partial`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 95, `y_partial`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 96, `z_ss`: Set this name to matrix multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 97: test `z_ss <= 1e-12 * max(1.0, values[:, 1] @ values[:, 1])`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 98: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 99, `pi`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 100, `control_beta`: Set this name to the entry/slice selected by `np.linalg.lstsq(controls, y - pi * z, rcond=None)[0]`; brackets select from the object to their left.
- Line 101, `residual`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 102: test `np.std(residual) < 1e-10`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.

- Line 103: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 104, `result['cf_' + flavor]`: Store the literal value `0.0` in the selected entry/column.
- Line 105, `result.loc[covered, 'cf_' + flavor]`: Store the object currently named `residual`; plain assignment does not copy it in the selected entry/column.
- Line 107, `scores`: Set this name to the result of calling `np.bincount(...)`; the arguments and preceding method chain are shown above.
- Line 108, `pi_se`: Set this name to the result of calling `np.sqrt(...)`; the arguments and preceding method chain are shown above.
- Line 109: test `len(households) > 1`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 110: update `pi_se` using `*=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 111, `f_stat`: Set this name to the true-branch value if `pi_se > 0` holds, otherwise the else-branch value.
- Line 112, `restricted_ss`: Set this name to matrix multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 113, `partial_r2`: Set this name to the true-branch value if `restricted_ss > 0` holds, otherwise the else-branch value.
- Line 114: call `reports.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.factorize`: Encode labels as dense integer positions and return the original-label lookup alongside those positions.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- `absorb_effects`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.bincount`: Accumulate frequencies or weighted sums by dense nonnegative integer labels; repeated household codes are added together.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.lstsq`: Solve least squares without explicitly inverting a cross-product matrix; [0] keeps its coefficient vector.
- `.std`: Compute standard deviation; ddof=1 requests sample SD rather than dividing squared deviations by the full count.
- `len`: Count elements/rows rather than summing their numerical values.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `.sf`: Evaluate survival probability above a threshold, useful for a two-sided t diagnostic.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 120–120:

```
120 return result, pd.DataFrame(reports)
```

**Read this block:**

- Line 120: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

# supply_code: source walkthrough

## supply_code/comparative_statics/___init___.py

This empty file marks an importable package directory. It defines no functions and performs no data calculations. Python can support some namespace packages without it, but an explicit marker makes the intended package structure clear.

### Module setup and direct-execution blocks

There are no statements in this file.

## supply_code/comparative_statics/alpha_p1.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

### Module setup and direct-execution blocks

Source lines 1–3:

```
1 from model.symbols import alpha
2 from comparative_statics.canonical import price1_derivative
3 dp1_dalpha = price1_derivative(alpha)
```

**Read this block:**

- Line 1: from `model.symbols`, import `alpha` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 2: from `comparative_statics.canonical`, import `price1_derivative` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3, `dp1_dalpha`: Set this name to the result of calling `price1_derivative(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `price1_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

## supply_code/comparative_statics/alpha_p2.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

### Module setup and direct-execution blocks

Source lines 1–3:

```
1 from model.symbols import alpha
2 from comparative_statics.canonical import price2_derivative
3 dp2_dalpha = price2_derivative(alpha)
```

**Read this block:**

- Line 1: from `model.symbols`, import `alpha` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 2: from `comparative_statics.canonical`, import `price2_derivative` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3, `dp2_dalpha`: Set this name to the result of calling `price2_derivative(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `price2_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

## supply_code/comparative_statics/beta_p1.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

### Module setup and direct-execution blocks

Source lines 1–3:

```
1 from model.symbols import beta
2 from comparative_statics.canonical import price1_derivative
3 dp1_dbeta = price1_derivative(beta)
```

**Read this block:**

- Line 1: from `model.symbols`, import `beta` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 2: from `comparative_statics.canonical`, import `price1_derivative` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3, `dp1_dbeta`: Set this name to the result of calling `price1_derivative(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `price1_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

## supply_code/comparative_statics/beta_p1gamma.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

### Module setup and direct-execution blocks

Source lines 1–1:

```
1 from comparative_statics.gamma_p1beta import dp1dbeta_dgamma_clean
```

**Read this block:**

- Line 1: from `comparative_statics.gamma_p1beta`, import `dp1dbeta_dgamma_clean` into this module's namespace. Importing can execute the imported module's top-level setup.

## supply_code/comparative_statics/beta_p1p2.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

### Module setup and direct-execution blocks

Source lines 1–3:

```
1 from model.symbols import *
2 from comparative_statics.canonical import price1_derivative, price2_derivative
3 ratio_derivative = (p2*price1_derivative(beta)-p1*price2_derivative(beta))/p2**2
```

**Read this block:**

- Line 1: from `model.symbols`, import $*$ into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 2: from `comparative_statics.canonical`, import `price1_derivative, price2_derivative` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3, `ratio_derivative`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

**Syntax and calls used here:**

- `price1_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `price2_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.

## supply_code/comparative_statics/beta_p2.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

### Module setup and direct-execution blocks

Source lines 1–3:

```
1 from model.symbols import beta
2 from comparative_statics.canonical import price2_derivative
3 dp2_dbeta = price2_derivative(beta)
```

**Read this block:**

- Line 1: from `model.symbols`, import `beta` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 2: from `comparative_statics.canonical`, import `price2_derivative` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3, `dp2_dbeta`: Set this name to the result of calling `price2_derivative(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `price2_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

# supply_code/comparative_statics/beta_profit.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

## Module setup and direct-execution blocks

Source lines 1–1:

```
1 from comparative_statics.canonical import dPi_dbeta
```

**Read this block:**

- Line 1: from `comparative_statics.canonical`, import `dPi_dbeta` into this module's namespace. Importing can execute the imported module's top-level setup.

# supply_code/comparative_statics/canonical.py

Functions provide generic derivative rules; module-level assignments call them to define named gamma, beta, curvature, and ratio expressions. The quotient-rule ratio is created once with generic terminal symbols, then `.subs({...})` specializes it to matched-inside, matched-outside, and acquisition histories. This keeps three ratio definitions tied to one algebraic rule instead of three independent hand-written implementations.

## Module setup and direct-execution blocks

Source lines 1–3:

```
1 """All derivatives follow one implicit FOC and the terminal envelope."""
2 import sympy as sp
3 from model.symbols import *
```

**Read this block:**

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `sympy as sp`. An as-name is a convenient local alias, not another library.
- Line 3: from `model.symbols`, import `*` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 33–38:

```
33 dp1_dgamma = price1_derivative(gamma)
34 dp2_dgamma = price2_derivative(gamma)
35 dVj_dgamma, dV0_dgamma = value2_derivative(gamma,u,-1,p2j), value2_derivative(gamma,v,1,p20)
36 ds_dgamma = sp.factor(sj1*s01*(q+alpha*dp1_dgamma))
37 du_dgamma, dv_dgamma = -u*(1-u)**2, v*(1-v)**2
```

```
38 d2p1_dgamma2 = total_derivative(dp1_dgamma,gamma)
```

**Read this block:**

- Line 33, `dp1_dgamma`: Set this name to the result of calling `price1_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 34, `dp2_dgamma`: Set this name to the result of calling `price2_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 35, `(dVj_dgamma, dV0_dgamma)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 36, `ds_dgamma`: Set this name to the result of calling `sp.factor(...)`; the arguments and preceding method chain are shown above.
- Line 37, `(du_dgamma, dv_dgamma)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 38, `d2p1_dgamma2`: Set this name to the result of calling `total_derivative(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `price1_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `price2_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `value2_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.factor`: Factor common algebraic pieces to make a symbolic derivative easier to inspect.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.
- `total_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 53–58:

```
53 d2p2_dgamma2 = terminal_total_derivative(dp2_dgamma,gamma)
54 ds2_dgamma = sp.factor(sj2*s02*(q2+alpha*dp2_dgamma))
55 dp2dbeta_dgamma = terminal_total_derivative(price2_derivative(beta),gamma)
56 dPi_dgamma = profit_derivative(gamma)
57 dPi_dbeta = profit_derivative(beta)
58 ratio_derivative = (p2*dp1_dgamma-p1*dp2_dgamma)/p2**2
```

**Read this block:**

- Line 53, `d2p2_dgamma2`: Set this name to the result of calling `terminal_total_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 54, `ds2_dgamma`: Set this name to the result of calling `sp.factor(...)`; the arguments and preceding method chain are shown above.
- Line 55, `dp2dbeta_dgamma`: Set this name to the result of calling `terminal_total_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 56, `dPi_dgamma`: Set this name to the result of calling `profit_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 57, `dPi_dbeta`: Set this name to the result of calling `profit_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 58, `ratio_derivative`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

**Syntax and calls used here:**

- `terminal_total_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `profit_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 59–61:

```
matched_j = ratio_derivative.subs({q:-1,p2:p2j,sj2:u,q2:-1})
matched_0 = ratio_derivative.subs({q:1,p2:p20,sj2:v,q2:1})
induce_harvest = ratio_derivative.subs({q:1,p2:p2j,sj2:u,q2:-1})
```

**Read this block:**

- Line 59, `matched_j`: Set this name to the result of calling `ratio_derivative.subs(...)`; the arguments and preceding method chain are shown above.
- Line 60, `matched_0`: Set this name to the result of calling `ratio_derivative.subs(...)`; the arguments and preceding method chain are shown above.
- Line 61, `induce_harvest`: Set this name to the result of calling `ratio_derivative.subs(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.subs`: Substitute symbols/expressions in symbolic algebra; this does not by itself solve an equilibrium.

## price2 derivative

Original function: `price2_derivative`. Source begins at line 6.

This returns a **symbolic expression** for the terminal optimal-price derivative with respect to symbolic parameter theta. Default arguments `sj2`, `q2`, and `p2` are shared symbols; callers substitute a particular branch by passing u/-1/p2j or v/+1/p20. `sp.diff(beta, theta)` is one when theta is beta and zero for an independent parameter, so one generic expression covers several comparative statics.

`direct` is the fixed-price derivative of the utility index. The returned formula already incorporates implicit share/price feedback; `simplify` reduces algebra. Differentiating the markup while holding its share fixed would omit that feedback. Unlike numerical finite differences, symbolic derivatives do not need a step size, though they still depend on a correct symbolic model.

**Definition and stated intent:**

Source lines 6–6:

```
def price2_derivative(theta, share=sj2, history=q2, price=p2):
```

`def` creates this callable; the parentheses name its inputs (`theta`, `share`, `history`, `price`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 7–7:

```
direct = sp.diff(beta, theta) + price*sp.diff(alpha, theta) + history*sp.diff(gamma, theta)
```

**Read this block:**

- Line 7, `direct`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

**Syntax and calls used here:**

- `.diff`: For SymPy differentiate algebra; for a Polars/pandas date/value expression calculate successive differences. The receiver decides which operation this name means.

Source lines 8–8:

```
return sp.simplify((1-share)*sp.diff(c, theta) + sp.diff(alpha, theta)/alpha**2 - share*direct/alpha)
```

**Read this block:**

- Line 8: return the result of calling `sp.simplify(...)`; the arguments and preceding method chain are shown above. This ends the call.

**Syntax and calls used here:**

- `.simplify`: Try algebraic simplification; it does not numerically optimize a price or establish every economic sign.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.

## value2 derivative

Original function: `value2_derivative`. Source begins at line 11.

This computes the symbolic envelope derivative of a reoptimized continuation value. It uses the same fixed-price index derivative as terminal pricing, then returns `-share*direct/alpha - share*diff(c,theta)`. There is no explicit derivative of optimal price because the terminal FOC cancels that first-order effect. Omitting price responses here is justified by the envelope theorem, not a general Python or SymPy rule. Substituting fully expanded optimal prices would be harder algebra with the same result under the maintained optimum conditions.

**Definition and stated intent:**

Source lines 11–11:

```
11 def value2_derivative(theta, share, history, price):
```

`def` creates this callable; the parentheses name its inputs (`theta`, `share`, `history`, `price`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 12–12:

```
12 direct = sp.diff(beta, theta) + price*sp.diff(alpha, theta) + history*sp.diff(gamma, theta)
```

**Read this block:**

- Line 12, `direct`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

**Syntax and calls used here:**

- `.diff`: For SymPy differentiate algebra; for a Polars/pandas date/value expression calculate successive differences. The receiver decides which operation this name means.

Source lines 13–13:

```
13 return -share*direct/alpha - share*sp.diff(c, theta)
```

**Read this block:**

- Line 13: return subtraction of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

## price1 derivative

Original function: `price1_derivative`. Source begins at line 16.

This forms the symbolic first-period price derivative. It builds its direct utility effect, obtains the two terminal envelope derivatives, subtracts them, and inserts the result into the implicit first-price formula. `sp.diff(delta, theta)` includes a direct discount-factor change when theta is delta; for gamma that term is zero but the continuation values still change.

The two helper calls use the correct opposite signed terminal histories. Maintaining a single generic function avoids separate alpha/beta/delta/gamma formulas disagreeing about those branches. `sp.simplify` does algebraic cleanup; it does not decide whether a term is positive under the economic domain.

**Definition and stated intent:**

Source lines 16–16:

```
16 def price1_derivative(theta):
```

`def` creates this callable; the parentheses name its inputs (`theta`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 17–18:

```
17 direct = sp.diff(beta, theta) + p1*sp.diff(alpha, theta) + q*sp.diff(gamma, theta)
18 dDV = value2_derivative(theta,u,-1,p2j)-value2_derivative(theta,v,1,p20)
```

**Read this block:**

- Line 17, `direct`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 18, `dDV`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.

**Syntax and calls used here:**

- `.diff`: For SymPy differentiate algebra; for a Polars/pandas date/value expression calculate successive differences. The receiver decides which operation this name means.
- `value2_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 19–20:

```
19 return sp.simplify(s01*sp.diff(c,theta)+sp.diff(alpha,theta)/alpha**2
20                  -sj1*direct/alpha-s01*(sp.diff(delta,theta)*(Vj-V0)+delta*dDV))
```

**Read this block:**

- Line 19: return the result of calling `sp.simplify(...)`; the arguments and preceding method chain are shown above. This ends the call.

**Syntax and calls used here:**

- `.simplify`: Try algebraic simplification; it does not numerically optimize a price or establish every economic sign.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.

## total derivative

Original function: `total_derivative`. Source begins at line 23.

Input is a symbolic expression and differentiation parameter. Output is its total derivative along the solved equilibrium. The first assignments compute price derivatives, then logistic chain rules compute derivatives of the three shares. A dictionary maps each endogenous symbol to its derivative.

The return is explicit partial differentiation plus `sum(diff(expr,k)*val for k,val in substitutions.items())`. This is the chain rule over every mapped endogenous object. The dictionary is a derivative map, not values actually substituted with `.subs` in this function. `sp.factor` collects common algebraic factors. Plain `sp.diff(expr,theta)` would treat shares, prices, and values as independent symbols and miss their indirect changes.

**Definition and stated intent:**

Source lines 23–23:

```
23 def total_derivative(expr, theta):
```

`def` creates this callable; the parentheses name its inputs (`expr`, `theta`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 24–30:

```
24 dp = price1_derivative(theta)
25 dpj, dp0 = price2_derivative(theta,u,-1,p2j), price2_derivative(theta,v,1,p20)
26 ds = sj1*s01*(sp.diff(beta,theta)+p1*sp.diff(alpha,theta)+q*sp.diff(gamma,theta)+alpha*dp)
27 du = u*(1-u)*(sp.diff(beta,theta)+p2j*sp.diff(alpha,theta)-sp.diff(gamma,theta)+alpha*dpj)
28 dv = v*(1-v)*(sp.diff(beta,theta)+p20*sp.diff(alpha,theta)+sp.diff(gamma,theta)+alpha*dp0)
29 substitutions = {p1:dp, p2j:dpj, p20:dp0, sj1:ds, u:du, v:dv,
30                  Vj:value2_derivative(theta,u,-1,p2j), V0:value2_derivative(theta,v,1,p20)}
```

**Read this block:**

- Line 24, `dp`: Set this name to the result of calling `price1_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 25, `(dpj, dp0)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 26, `ds`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 27, `du`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 28, `dv`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 29, `substitutions`: Set this name to a dictionary from the displayed keys and values.

**Syntax and calls used here:**

- `price1_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `price2_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.diff`: For SymPy differentiate algebra; for a Polars/pandas date/value expression calculate successive differences. The receiver decides which operation this name means.
- `value2_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 31–31:

```
31 return sp.factor(sp.diff(expr,theta)+sum(sp.diff(expr,k)*val for k,val in substitutions.items()))
```

**Read this block:**

- Line 31: return the result of calling `sp.factor(...)`; the arguments and preceding method chain are shown above. This ends the call.

**Syntax and calls used here:**

- `.factor`: Factor common algebraic pieces to make a symbolic derivative easier to inspect.
- `sum`: Accumulate a sequence, including generated chain-rule terms or counts.
- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

## terminal total derivative

Original function: `terminal_total_derivative`. Source begins at line 39.

This is the smaller one-branch version of the total derivative. It calculates terminal price response, computes terminal share response including repricing, and adds explicit theta, price, and share chain-rule terms. Because there is only one terminal branch, a short direct expression is clearer than building the full period-1 derivative dictionary. It is used for terminal curvature and mixed derivatives; a second plain symbolic partial derivative would hold the share fixed and give the wrong comparative static.

**Definition and stated intent:**

Source lines 39–39:

```
39 def terminal_total_derivative(expr,theta):
```

`def` creates this callable; the parentheses name its inputs (`expr`, `theta`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 40–41:

```
40 dp = price2_derivative(theta)
41 ds = sj2*s02*(sp.diff(beta,theta)+p2*sp.diff(alpha,theta)+q2*sp.diff(gamma,theta)+alpha*dp)
```

**Read this block:**

- Line 40, `dp`: Set this name to the result of calling `price2_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 41, `ds`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.

**Syntax and calls used here:**

- `price2_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.diff`: For SymPy differentiate algebra; for a Polars/pandas date/value expression calculate successive differences. The receiver decides which operation this name means.

Source lines 42–42:

```
42 return sp.factor(sp.diff(expr,theta)+sp.diff(expr,p2)*dp+sp.diff(expr,sj2)*ds)
```

**Read this block:**

- Line 42: return the result of calling `sp.factor(...)`; the arguments and preceding method chain are shown above. This ends the call.

**Syntax and calls used here:**

- `.factor`: Factor common algebraic pieces to make a symbolic derivative easier to inspect.

## profit derivative

Original function: `profit_derivative`. Source begins at line 45.

This returns the optimized two-period profit derivative with respect to theta. `partial` combines direct current demand/cost effects, direct discounting, and both branch envelope effects. `.subs(p1, markup_expression)` inserts the first-period optimal pricing equation, then `simplify` reduces the result.

The code does not explicitly multiply by dp1/dtheta because the first-period FOC makes that contribution zero at the optimum. Replacing the entire objective with current revenue before differentiating would lose continuation effects. This helper is reusable for gamma and beta, while relying on the same fixed-type, two-branch, interior optimum as the rest of the model.

**Definition and stated intent:**

Source lines 45–45:

```
45 def profit_derivative(theta):
```

`def` creates this callable; the parentheses name its inputs (`theta`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 46–49:

```
46 direct = sp.diff(beta,theta)+p1*sp.diff(alpha,theta)+q*sp.diff(gamma,theta)
47 partial = (direct*sj1*s01*(p1-c+delta*(Vj-V0))-sj1*sp.diff(c,theta)
48            +sp.diff(delta,theta)*(sj1*Vj+s01*V0)
49            +delta*(sj1*value2_derivative(theta,u,-1,p2j)+s01*value2_derivative(theta,v,1,p20)))
```

**Read this block:**

- Line 46, `direct`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 47, `partial`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

**Syntax and calls used here:**

- `.diff`: For SymPy differentiate algebra; for a Polars/pandas date/value expression calculate successive differences. The receiver decides which operation this name means.
- `value2_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 50–50:

```
50 return sp.simplify(partial.subs(p1,c-1/(alpha*s01)-delta*(Vj-V0)))
```

**Read this block:**

- Line 50: return the result of calling `sp.simplify(...)`; the arguments and preceding method chain are shown above. This ends the call.

**Syntax and calls used here:**

- `.simplify`: Try algebraic simplification; it does not numerically optimize a price or establish every economic sign.
- `.subs`: Substitute symbols/expressions in symbolic algebra; this does not by itself solve an equilibrium.

# supply_code/comparative_statics/delta_p1.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

## Module setup and direct-execution blocks

Source lines 1–3:

```
1 from model.symbols import delta
2 from comparative_statics.canonical import price1_derivative
3 dp1_ddelta = price1_derivative(delta)
```

**Read this block:**

- Line 1: from `model.symbols`, import `delta` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 2: from `comparative_statics.canonical`, import `price1_derivative` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3, `dp1_ddelta`: Set this name to the result of calling `price1_derivative(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `price1_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

# supply_code/comparative_statics/delta_p2.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

## Module setup and direct-execution blocks

Source lines 1–3:

```
1 from model.symbols import delta
2 from comparative_statics.canonical import price2_derivative
3 dp2_ddelta = price2_derivative(delta)
```

**Read this block:**

- Line 1: from `model.symbols`, import `delta` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 2: from `comparative_statics.canonical`, import `price2_derivative` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3, `dp2_ddelta`: Set this name to the result of calling `price2_derivative(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `price2_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

# supply_code/comparative_statics/gamma2_p1.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

## Module setup and direct-execution blocks

Source lines 1–1:

```
1 from comparative_statics.canonical import d2p1_dgamma2
```

**Read this block:**

- Line 1: from `comparative_statics.canonical`, import `d2p1_dgamma2` into this module's namespace. Importing can execute the imported module's top-level setup.

# supply_code/comparative_statics/gamma2_p2.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

## Module setup and direct-execution blocks

Source lines 1–1:

```
1 from comparative_statics.canonical import d2p2_dgamma2
```

**Read this block:**

- Line 1: from `comparative_statics.canonical`, import `d2p2_dgamma2` into this module's namespace. Importing can execute the imported module's top-level setup.

# supply_code/comparative_statics/gamma_p1.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

## Module setup and direct-execution blocks

Source lines 1–1:

```
1 from comparative_statics.canonical import dp1_dgamma, ds_dgamma as dsj1_dgamma_total
```

**Read this block:**

- Line 1: from `comparative_statics.canonical`, import `dp1_dgamma, ds_dgamma as dsj1_dgamma_total` into this module's namespace. Importing can execute the imported module's top-level setup.

# supply_code/comparative_statics/gamma_p1beta.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

## Module setup and direct-execution blocks

Source lines 1–4:

```
1 from model.symbols import beta, gamma
2 from comparative_statics.canonical import price1_derivative, total_derivative
3 dp1dbeta_dgamma_raw = total_derivative(price1_derivative(beta), gamma)
4 dp1dbeta_dgamma_clean = dp1dbeta_dgamma_raw
```

**Read this block:**

- Line 1: from `model.symbols`, import `beta, gamma` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 2: from `comparative_statics.canonical`, import `price1_derivative, total_derivative` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3, `dp1dbeta_dgamma_raw`: Set this name to the result of calling `total_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 4, `dp1dbeta_dgamma_clean`: Set this name to the object currently named `dp1dbeta_dgamma_raw`; plain assignment does not copy it.

**Syntax and calls used here:**

- `total_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `price1_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

# supply_code/comparative_statics/gamma_p1p2.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

## Module setup and direct-execution blocks

Source lines 1–1:

```
1 from comparative_statics.canonical import ratio_derivative, matched_j, matched_0, induce_harvest
```

**Read this block:**

- Line 1: from `comparative_statics.canonical`, import `ratio_derivative, matched_j, matched_0, induce_harvest` into this module's namespace. Importing can execute the imported module's top-level setup.

# supply_code/comparative_statics/gamma_p2.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

## Module setup and direct-execution blocks

Source lines 1–1:

```
1 from comparative_statics.canonical import dp2_dgamma, ds2_dgamma as dsj2_dgamma_total
```

**Read this block:**

- Line 1: from `comparative_statics.canonical`, import `dp2_dgamma, ds2_dgamma as dsj2_dgamma_total` into this module's namespace. Importing can execute the imported module's top-level setup.

# supply_code/comparative_statics/gamma_p2beta.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

## Module setup and direct-execution blocks

Source lines 1–1:

```
1 from comparative_statics.canonical import dp2dbeta_dgamma
```

**Read this block:**

- Line 1: from `comparative_statics.canonical`, import `dp2dbeta_dgamma` into this module's namespace. Importing can execute the imported module's top-level setup.

# supply_code/comparative_statics/gamma_profit.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

## Module setup and direct-execution blocks

Source lines 1–1:

```
1 from comparative_statics.canonical import dPi_dgamma
```

**Read this block:**

- Line 1: from `comparative_statics.canonical`, import `dPi_dgamma` into this module's namespace. Importing can execute the imported module's top-level setup.

# supply_code/comparative_statics_symbolic.py

This convenience script imports canonical expressions and, only when run directly, iterates over `(label, expression)` pairs and prints each. Tuple unpacking connects a human-readable label to its actual symbolic object. It does not compute independent derivative formulas. Reusing named expressions keeps terminal display consistent with production numeric evaluation.

## Module setup and direct-execution blocks

Source lines 1–2:

```
1 """Print verified canonical gamma derivatives."""
2 from comparative_statics.canonical import *
```

**Read this block:**

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: from `comparative_statics.canonical`, import $*$ into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 3–6:

```
3 if __name__ == "__main__":
4     for label,expr in [("dp1/dgamma",dp1_dgamma),("d2p1/dgamma2",d2p1_dgamma2),
5                        ("dprofit/dgamma",dPi_dgamma),("ratio derivative",ratio_derivative)]:
6         print(label, "=", expr)
```

**Read this block:**

- Line 3: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 4: for each entry yielded by `[('dp1/dgamma', dp1_dgamma), ('d2p1/dgamma2', d2p1_dgamma2), (...,` bind/unpack it into `(label, expr)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 6: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

# supply_code/export/___init___.py

This empty file marks an importable package directory. It defines no functions and performs no data calculations. Python can support some namespace packages without it, but an explicit marker makes the intended package structure clear.

## Module setup and direct-execution blocks

There are no statements in this file.

# supply_code/export/latex_export.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

## Module setup and direct-execution blocks

Source lines 1–2:

```
1 """Export the model's actual SymPy expressions as display equations."""
2 import sympy as sp
```

**Read this block:**

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `sympy as sp`. An as-name is a convenient local alias, not another library.

## display equation

Original function: `display_equation`. Source begins at line 5.

This returns a LaTeX display-equation string. `lhs` is already a LaTeX label; `sp.latex(expression)` serializes the actual symbolic right-hand side. `'\\[\n'` contains a backslash followed by a bracket and a newline; Python string escapes are why source contains doubled backslashes. Concatenation places the equation between display delimiters. Deriving the right side from SymPy avoids manually copying a formula that later changes in the code. The function returns text and does not itself invoke a TeX compiler.

**Definition and stated intent:**

Source lines 5–6:

```
5 def display_equation(lhs, expression):
6     """Keep the mathematical right-hand side sourced directly from SymPy."""
```

`def` creates this callable; the parentheses name its inputs (`lhs`, `expression`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 7–7:

```
7 return '\\[\n' + lhs + ' = ' + sp.latex(expression) + '\n\\]\n'
```

**Read this block:**

- Line 7: return addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

**Syntax and calls used here:**

- `.latex`: Serialize symbolic algebra as LaTeX source rather than recomputing the mathematical formula manually.

# supply_code/export/pretty_print.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

## Module setup and direct-execution blocks

Source lines 1–1:

```
1 import sympy as sp
```

**Read this block:**

- Line 1: import `sympy as sp`. An as-name is a convenient local alias, not another library.

## show

Original function: `show`. Source begins at line 4.

This is a terminal-output helper returning None. Blank `print()` creates a new line; `"=" * 70` repeats a string seventy times, whereas multiplication of numeric values has a different meaning. `sp.pprint` renders an expression in readable mathematical form in the terminal. It is useful for inspection, while `sp.latex` is the appropriate alternative for document source.

**Definition and stated intent:**

Source lines 4–4:

```
4 def show(title, expr):
```

`def` creates this callable; the parentheses name its inputs (`title`, `expr`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 5–9:

```
5 print()
6 print("=" * 70)
7 print(title)
8 print("=" * 70)
9 sp.pprint(expr)
```

**Read this block:**

- Line 5: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 6: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 7: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 8: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 9: call `sp.pprint` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.pprint`: Pretty-print symbolic mathematics to the terminal for human inspection.

**Output reminder:** reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

# supply__code/make__comparative__static__figures.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

## Module setup and direct-execution blocks

Source lines 1–6:

```
1 """Generate only the requested symmetric-switching comparative-static figures."""
2 from pathlib import Path
3 import csv
4 import numpy as np
5 import matplotlib
6 matplotlib.use('Agg')
```

**Read this block:**

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3: import `csv`. An as-name is a convenient local alias, not another library.
- Line 4: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 5: import `matplotlib`. An as-name is a convenient local alias, not another library.
- Line 6: call `matplotlib.use` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.use`: Select a Matplotlib backend; Agg renders to files without opening a GUI window.

Source lines 7–13:

```
7 import matplotlib.pyplot as plt
8 from model.numerical import evaluate, PARAMETERS
9
10 ROOT=Path(__file__).resolve().parent
11 OUT=ROOT/'writeup_math_graphs'/'figures'
12 GAMMAS=np.linspace(-3,3,601)
13 LABELS={'j':r'Initially inside ($x_0=j$)','0':r'Initially outside ($x_0=0$)'}
```

**Read this block:**

- Line 7: import `matplotlib.pyplot as plt`. An as-name is a convenient local alias, not another library.
- Line 8: from `model.numerical`, import `evaluate, PARAMETERS` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 10, `ROOT`: Set this name to the attribute `Path(__file__).resolve().parent` of the object before the dot.
- Line 11, `OUT`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 12, `GAMMAS`: Set this name to the result of calling `np.linspace(...)`; the arguments and preceding method chain are shown above.
- Line 13, `LABELS`: Set this name to a dictionary with named entries `j`, `0`.

**Syntax and calls used here:**

- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.

Source lines 64–65:

```
64 if __name__=='__main__':
65     main()
```

**Read this block:**

- Line 64: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 65: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

### save

Original function: `save`. Source begins at line 16.

This formatting/output helper takes a Matplotlib Figure and filename stem. `tight_layout` adjusts spacing; a two-element suffix loop saves both PDF and PNG; `plt.close` frees the Figure. `OUT / f"{name}.{suffix}"` constructs each Path. Shared export code prevents one figure accidentally lacking its vector PDF version. DPI matters chiefly for raster PNG content; it does not make an ordinary vector line in a PDF more mathematically precise.

**Definition and stated intent:**

Source lines 16–16:

```
16 def save(fig,name):
```

`def` creates this callable; the parentheses name its inputs (`fig`, `name`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 17–17:

```
17 fig.tight_layout()
```

**Read this block:**

- Line 17: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.tight_layout`: Adjust plot spacing to reduce overlap between labels and content.

Source lines 18–19:

```
18 for suffix in ['pdf','png']:
19     fig.savefig(OUT/f'{name}.{suffix}',dpi=160)
```

**Read this block:**

- Line 18: for each entry yielded by `['pdf', 'png']`, bind/unpack it into `suffix` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 19: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 20–20:

```
20 plt.close(fig)
```

**Read this block:**

- Line 20: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.close`: Release a Figure or file resource, according to the receiving object.

**Output reminder:** reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

## main

Original function: `main`. Source begins at line 23.

This generates all benchmark rows from the production `evaluate` function, writes a solution CSV, and plots six requested figures. The dictionary comprehension creates a list of gamma evaluations for each initial-state key. `csv.DictWriter` uses dictionary keys as field names, so output records retain branch meanings. Plot `specs` is a data list describing several similar figures; the loop executes the common plotting logic.

Separate blocks handle acquisition ratios and the two-panel price-level figure. Terminal prices can be read from either initial-state collection because their realized history determines them. A final generator computes the largest absolute FOC residual and raises if it exceeds tolerance. Plotting algebra typed anew here would risk drifting from the derivative expressions; numerical values instead come from the common symbolic-to-numerical path.

**Definition and stated intent:**

Source lines 23–23:

```
23 def main():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 24–25:

```
24 OUT.mkdir(parents=True,exist_ok=True)
25 curves={state:[evaluate(float(g),state,**PARAMETERS) for g in GAMMAS] for state in ['j','0']}
```

**Read this block:**

- Line 24: call `OUT.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 25, `curves`: Set this name to a dictionary computed one key/value pair at a time by the comprehension.

**Syntax and calls used here:**

- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- `evaluate`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.

Source lines 26–30:

```
26 with (OUT/'solutions.csv').open('w',newline='') as file:
27     writer=csv.DictWriter(file,fieldnames=list(curves['j'][0]))
28     writer.writeheader()
29     for state in ['j','0']:
30         writer.writerows(curves[state])
```

**Read this block:**

- Line 26: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 27, `writer`: Set this name to the result of calling `csv.DictWriter(...)`; the arguments and preceding method chain are shown above.
- Line 28: call `writer.writeheader` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 29: for each entry yielded by `['j', '0']`, bind/unpack it into `state` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 30: call `writer.writerows` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.open`: Open a file in the requested mode; using a with block ensures it closes on exit.
- `.DictWriter`: Create a CSV writer mapping dictionary keys to named columns in an explicit field order.
- `.writeheader`: Write the CSV's named column header before writing data rows.
- `.writerows`: Write a sequence of dictionaries/rows using the configured CSV schema.
- `list`: Materialize iterable entries as an ordered Python list.

Source lines 31–34:

```
31 specs=[('profit_by_gamma','profit',r'$\Pi^*$','Optimized two-period profit'),
32       ('gamma_profit_by_gamma','dprofit',r'$d\Pi^*/d\gamma$','Optimized profit derivative'),
33       ('gamma_p1_by_gamma','dp1',r'$dp_1^*/d\gamma$','Optimal period-1 price derivative'),
34       ('matched_price_ratio_derivative','d_matched_ratio',r'$dR_a/d\gamma$','State-matched price-ratio
   ↪ derivatives')]
```

**Read this block:**

- Line 31, `specs`: Set this name to a list containing 4 displayed entries.

Source lines 35–42:

```
35 for name,key,ylabel,title in specs:
36     fig,ax=plt.subplots(figsize=(7.2,4.5))
37     for state in ['j','0']:
38         ax.plot(GAMMAS,[r[key] for r in curves[state]],label=LABELS[state],lw=2)
39     ax.axhline(0,color='gray',lw=.7)
40     ax.set(xlabel=r'$\gamma$',ylabel=ylabel,title=title)
41     ax.legend();ax.grid(alpha=.2)
42     save(fig,name)
```

**Read this block:**

- Line 35: for each entry yielded by `specs`, bind/unpack it into `(name, key, ylabel, title)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 36, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 37: for each entry yielded by `['j', '0']`, bind/unpack it into `state` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 38: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 39: call `ax.axhline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 40: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 41: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 41: call `ax.grid` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 42: call `save` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.
- `.axhline`: Add a horizontal reference line, commonly zero for a derivative plot.
- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.legend`: Render labels previously attached to plotted series, identifying curves.
- `.grid`: Add visual grid lines without changing underlying numerical results.
- `save`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.plot`: Draw a line from supplied x/y values on the receiving plotting axes.

Source lines 43–48:

```
43 fig,ax=plt.subplots(figsize=(7.2,4.5))
44 ax.plot(GAMMAS,[r['d_induce_ratio'] for r in curves['0']],lw=2,
45         label=r'$R_{0\to j}=p_1^*(0)/p_2^*(j)$')
46 ax.axhline(0,color='gray',lw=.7)
47 ax.set(xlabel=r'$\gamma$',ylabel=r'$dR_{0\to j}/d\gamma$',title='Induce-then-harvest price-ratio derivative')
48 ax.legend();ax.grid(alpha=.2);save(fig,'induce_harvest_price_ratio_derivative')
```

**Read this block:**

- Line 43, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 44: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 46: call `ax.axhline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 47: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 48: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 48: call `ax.grid` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 48–49:

```
48 ax.legend();ax.grid(alpha=.2);save(fig,'induce_harvest_price_ratio_derivative')
49 fig,axes=plt.subplots(1,2,figsize=(10,4.2))
```

**Read this block:**

- Line 48: call `save` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 49, `(fig, axes)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.

Source lines 50–51:

```
50 for state in ['j','0']:
51     axes[0].plot(GAMMAS,[r['p1'] for r in curves[state]],label=LABELS[state],lw=2)
```

**Read this block:**

- Line 50: for each entry yielded by `['j', '0']`, bind/unpack it into `state` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 51: call `axes[0].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 52–53:

```
for state,key,label in [('j','p2j',r'Previous inside ($x_1=j$)'),('0','p20',r'Previous outside ($x_1=0$)')]:
    axes[1].plot(GAMMAS,[r[key] for r in curves['j']],label=label,lw=2)
```

**Read this block:**

- Line 52: for each entry yielded by `[('j', 'p2j', 'Previous inside ($x_1=j$)'), ('0', 'p20', 'Prev...`, bind/unpack it into `(state, key, label)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 53: call `axes[1].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 54–56:

```
for ax,title,ylabel in zip(axes,['Optimal period-1 prices','Optimal period-2 prices'],[r'$p_1^*$',r'$p_2^*$']):
    ax.axhline(0,color='gray',lw=.7);ax.set(xlabel=r'$\gamma$',ylabel=ylabel,title=title)
    ax.legend(fontsize=8);ax.grid(alpha=.2)
```

**Read this block:**

- Line 54: for each entry yielded by `zip(axes, ['Optimal period-1 prices', 'Optimal period-2 prices...`, bind/unpack it into `(ax, title, ylabel)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 55: call `ax.axhline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 55: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 56: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 56: call `ax.grid` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.

Source lines 57–58:

```
save(fig,'optimal_prices_by_gamma')
maximum=max(abs(r[k]) for state in curves for r in curves[state] for k in ['foc1','foc2j','foc20'])
```

**Read this block:**

- Line 57: call `save` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 58, `maximum`: Set this name to the result of calling `max(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `abs`: Take magnitude, removing a sign before a distance/tolerance comparison.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Source lines 59–60:

```
59 if maximum>1e-9:
60     raise AssertionError(f'FOC residual {maximum}')
```

**Read this block:**

- Line 59: test `maximum > 1e-09`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 60: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

**Syntax and calls used here:**

- `AssertionError`: Construct an exception describing why this operation cannot proceed normally.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 61–61:

```
61 print(f'Saved six requested figures and 1,202 solved observations; max FOC residual={maximum:.3g}')
```

**Read this block:**

- Line 61: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

**Output reminder:** reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

# supply_code/make_latex_document.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

## Module setup and direct-execution blocks

Source lines 1–6:

```
1 """Generate a verified symmetric-switching appendix with six selected figures."""
2 from pathlib import Path
3 import sympy as sp
4 from model.symbols import *
5 from model.utility import U1,U2
6 from model.profit import Pi
```

**Read this block:**

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3: import `sympy as sp`. An as-name is a convenient local alias, not another library.
- Line 4: from `model.symbols`, import * into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 5: from `model.utility`, import `U1, U2` into this module's namespace. Importing can execute the imported module's top-level setup.

- Line 6: from `model.profit`, import `Pi` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 7–12:

```
7 from pricing.period1_foc import foc1_collected
8 from pricing.period2_foc import foc2
9 from pricing.period1_price import p1_star
10 from pricing.period2_price import p2_star
11 from comparative_statics import canonical as C
12 from model.numerical import evaluate
```

**Read this block:**

- Line 7: from `pricing.period1_foc`, import `foc1_collected` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 8: from `pricing.period2_foc`, import `foc2` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 9: from `pricing.period1_price`, import `p1_star` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 10: from `pricing.period2_price`, import `p2_star` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 11: from `comparative_statics`, import `canonical as C` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 12: from `model.numerical`, import `evaluate` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 13–15:

```
13 from export.latex_export import display_equation
14
15 ROOT=Path(__file__).resolve().parent/'writeup_math_graphs'
```

**Read this block:**

- Line 13: from `export.latex_export`, import `display_equation` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 15, `ROOT`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

**Syntax and calls used here:**

- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 276–277:

```
276 if __name__=='__main__':
277     main()
```

**Read this block:**

- Line 276: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 277: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

## main

Original function: `main`. Source begins at line 18.

This writes a research appendix as text. The opening loop pairs LaTeX left-hand side strings with actual SymPy expressions and delegates equation formatting. The very long raw triple-quoted `document` string is document **data** inside the Python program: its LaTeX commands are interpreted later by a TeX engine, not executed as Python statements. A raw string preserves the many backslashes.

Later loops append benchmark rows and figure sections. An f-string inserts computed numbers with three decimals; adjacent raw-string fragments preserve LaTeX row-ending backslashes. The figure-existence check stops on missing assets instead of generating a broken paper. `write_text` creates `.tex`, not a PDF; compilation is a separate step. Repeated string `+=` is acceptable for this small document; accumulating a list and `"".join(parts)` would scale better for very large text assembly.

**Definition and stated intent:**

Source lines 18–18:

```
18 def main():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 19–21:

```
19 ROOT.mkdir(exist_ok=True)
20 equations='\\section{Symbolically derived equations}\n'
21 equations += r'Here $q_2=q(x_1)$, $s_2=s_2(x_1)$ and $p_2=p_2(x_1)$ describe a fixed terminal history. Price
   ↪  equations and comparative-static right-hand sides use equilibrium shares; the profit and FOCs use shares at the
   ↪  candidate prices.'+'\n'
```

**Read this block:**

- Line 19: call `ROOT.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 20, `equations`: Set this name to the literal text `'\\section{Symbolically derived equations}\n'`.
- Line 21: update `equations` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

**Syntax and calls used here:**

- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.

Source lines 22–28:

```
22 for lhs,expression in [(r'v_1',U1),(r'v_2',U2),(r'\Pi',Pi),
23                        (r'F_1',foc1_collected),(r'F_2',foc2),
24                        (r'p_1^*',p1_star),(r'p_2^*',p2_star),
25                        (r'\frac{dp_1^*}{d\gamma}',C.dp1_dgamma),
26                        (r'\frac{dp_2^*}{d\gamma}',C.dp2_dgamma),
27                        (r'\frac{d\Pi^*}{d\gamma}',C.dPi_dgamma)]:
28     equations+=display_equation(lhs,expression)
```

**Read this block:**

- Line 22: for each entry yielded by `[('v_1', U1), ('v_2', U2), ('\\Pi', Pi), ('F_1', foc1_collecte...`, bind/unpack it into `(lhs, expression)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 28: update `equations` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

**Syntax and calls used here:**

- `display_equation`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 29–29:

```
(ROOT/'sympy_equations.tex').write_text(equations)
```

**Read this block:**

- Line 29: call `(ROOT / 'sympy_equations.tex').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.write_text`: Write a string into a text file, replacing an existing file at that path.

Source lines 30–71:

```
    document=r'''\documentclass[11pt]{article}
\usepackage[margin=1in]{geometry}
\usepackage{amsmath,amssymb,amsthm,graphicx,booktabs,array}
\usepackage[hidelinks]{hyperref}
\newtheorem{proposition}{Proposition}
\newcommand{\dd}{\mathrm{d}}
\setlength{\parindent}{0pt}
\setlength{\parskip}{5pt}
\title{Two-period pricing with symmetric switching preferences}
\author{}\date{October 30, 2026}
\begin{document}\maketitle
\section{Model, histories, and normalization}
One firm sells one inside product $j$ against outside option $0$ in periods
$t=1,2$. Type $r$ is fixed and observed by the firm, which prices separately for
that type. All formulas below are conditional on $r$; its subscript and the
fixed product index are suppressed. There is no pooling of heterogeneous types.
The initial realized choice is $x_0\in\{j,0\}$. The second-period price can depend
on the realized first-period choice $x_1$. Independent Type-I extreme-value
shocks enter both alternatives in each period.

Before normalization the deterministic utilities are
\[
 V_{jt}^{\rm raw}=\beta+\alpha p_t+\gamma\mathbf1\{x_{t-1}=0\},
 \qquad V_{0t}^{\rm raw}=\gamma\mathbf1\{x_{t-1}=j\}.
\]
The payoff to switching applies in either direction. Subtracting the raw outside
utility from both alternatives gives normalized outside utility zero and
\[
 v_t(h)=\beta+\alpha p_t(h)+q(h)\gamma,\qquad
 q(j)=-1,\quad q(0)=+1.
\]
Normalization retains the outside switching payoff through the signed inside
utility difference; it does not delete that payoff. Negative $\gamma$ denotes
switching aversion and positive $\gamma$ denotes novelty preference.
\[
 s_t(h)=\frac{e^{v_t(h)}}{1+e^{v_t(h)}},\quad s_{0t}(h)=1-s_t(h),\quad
 \partial_p s_t=\alpha s_t(1-s_t),\quad
 (\partial_\gamma s_t)_p=q(h)s_t(1-s_t).
\]
A realized state $x_t$, a hypothetical switching indicator, and the probability
$s_t$ are distinct objects. Every derivative below holds the discrete history fixed.
```

Source lines 72–113:

```
\textbf{Assumptions.} $\alpha<0$, $0<\delta<1$, fixed marginal cost $c$, finite
parameters, and interior binary-logit shares. Prices are unrestricted real
```

```
numbers: introductory subsidies are allowed. Continuation values are conditional
on realized states and have no direct dependence on $p_1$. Type does not evolve.
The case $\delta=0$ is used only as a static counterfactual.

\begin{center}\begin{tabular}{ll}\toprule
Notation & Meaning\\\midrule
$q=q(x_0)$ & Signed initial-state coefficient\\
$s=s_1^*(x_0),\quad r_0=1-s$ & First-period equilibrium shares\\
$u=s_2^*(j),\quad v=s_2^*(0)$ & Inside shares in the two continuation states\\
$V_j=V_2(j),\quad V_0=V_2(0)$ & Optimized continuation profits\\
$\Delta V=V_j-V_0,\quad B=u+v$ & Continuation difference and sum of terminal shares\\
$\Pi^*$ & Expected discounted profit per initial consumer\\\bottomrule
\end{tabular}\end{center}

\section{Backward induction and optimality}
For each realized first-period choice $a\in\{j,0\}$, solve
\[
 V_2(a)=\max_{p_2(a)}s_2(a)[p_2(a)-c].
\]
Its first-order condition is
\[
 0=s_2+\alpha s_2(1-s_2)(p_2-c),\qquad
 p_2^*(a)=c-\frac1{\alpha[1-s_2^*(a)]}.
\]
This is an implicit equation: the share depends on the same price being solved.
Both $V_j$ and $V_0$ must be computed before period 1.

First-period profit integrates over both possible realized histories:
\begin{align*}
 \Pi(p_1;x_0)&=s_1(p_1-c)+\delta[s_1V_j+(1-s_1)V_0]\\
 &=\delta V_0+s_1[p_1-c+\delta\Delta V].
\end{align*}
Because the branch values do not depend directly on $p_1$,
\[
 \partial_{p_1}\{\delta[s_1V_j+(1-s_1)V_0]\}
 =\delta\alpha s_1(1-s_1)(V_j-V_0).
\]
Consequently,
\[
 0=s_1+\alpha s_1(1-s_1)[p_1-c+\delta\Delta V],\qquad
```

Source lines 114–155:

```
 p_1^*=c-\frac1{\alpha(1-s)}-\delta\Delta V.
\]
The static FOC component $c-1/[\alpha(1-s)]$ uses the share at the dynamic
optimum. It is not the counterfactual static optimum, which must be solved
separately with $\delta=0$.

\begin{proposition}[Unique optimal prices]
Under the stated assumptions each continuation price and the first-period price
has a unique finite global optimum, with no lower bound imposed on $p_1$.
\end{proposition}
\begin{proof}
Both price problems reduce, up to a constant, to maximizing
$s(p)(p-k)$ with $s(p)=\operatorname{logit}^{-1}(b+\alpha p)$.
Use $k=c$ in period 2 and $k=c-\delta\Delta V$ in period 1.
The sign of the profit derivative is the sign of
\[
 H(p)=1+e^{b+\alpha p}+\alpha(p-k),\quad
 H'(p)=\alpha(1+e^{b+\alpha p})<0.
\]
```

```
$H$ ranges from positive infinity to negative infinity, so its root is unique
and the objective increases before it and decreases afterward. The price may
lie below physical cost $c$ in period 1 when continuation incentives justify it.
\end{proof}

\section{Envelope derivatives and conditional comparative statics}
Differentiating each continuation value using the envelope theorem gives
\[
 V_2'(a)=q(a)s_2(1-s_2)(p_2^*-c)=-\frac{q(a)s_2}{\alpha}.
\]
Implicit differentiation of the terminal FOC gives the same expression for the
optimal price derivative:
\[
 p_2^{*\prime}(a)=-\frac{q(a)s_2}{\alpha},\quad
 V_j'=u/\alpha,\quad V_0'=-v/\alpha,\quad
 \Delta V'=(u+v)/\alpha.
\]
In particular, $p_2^{*\prime}(j)<0$ and $p_2^{*\prime}(0)>0$.
The total equilibrium terminal-share derivative is
$s_2'=q(a)s_2(1-s_2)^2$, including the optimal-price response.

For the first-period implicit pricing equation let
$F=p_1-c+1/[\alpha(1-s_1)]+\delta\Delta V$. At fixed history,
```

Source lines 156–197:

```
\[
 F_{p_1}=\frac1{r_0},\qquad
 F_\gamma=\frac{qs}{\alpha r_0}+\frac{\delta B}{\alpha},\qquad
 \boxed{p_1^{*\prime}=-\frac{qs+\delta r_0 B}{\alpha}}.
\]
The share therefore changes according to
\[
 s'=sr_0(q+\alpha p_1^{*\prime})=sr_0^2(q-\delta B).
\]
The first-period envelope theorem retains both continuation branches:
\begin{align*}
 \Pi^{*\prime}
 &=qsr_0[p_1^*-c+\delta\Delta V]+\delta[sV_j'+r_0V_0']\\
 &=\boxed{\frac{-qs+\delta(su-r_0v)}{\alpha}}.
\end{align*}
The term $\delta V_0'$ must not be dropped. The envelope partial derivative holds
the optimizing price fixed inside the partial derivative; it does not hold the
continuation value fixed as $\gamma$ varies.

For the analytical second price derivative define
\[
 u'=-u(1-u)^2,\quad v'=v(1-v)^2,\quad B'=u'+v'.
\]
Differentiating the first derivative and substituting $s'$ gives
\[
 \boxed{p_1^{*\prime\prime}
 =-\frac{sr_0^2(q-\delta B)^2+\delta r_0B'}{\alpha}}.
\]
This expression is used analytically in checks, rather than obtained from noisy
twice-differenced optimized prices. No second-derivative figure is included.

\begin{proposition}[Conditional signs]
For an initially outside consumer, $p_1^{*\prime}>0$ and $\Pi^{*\prime}>0$.
For an initially inside consumer,
\[
 p_1^{*\prime}>0\ \Longleftrightarrow\ \delta r_0(u+v)>s,\qquad
```

```
 \Pi^{*\prime}>0\ \Longleftrightarrow\ \delta r_0v>s(1+\delta u).
\]
Equalities give zero derivatives; reversed inequalities give negative derivatives.
\end{proposition}
\begin{proof}
For $q=1$, the price numerator $s+\delta r_0B$ is positive and $-1/\alpha>0$.
```

Source lines 198–239:

```
The profit numerator is $-s(1-\delta u)-\delta r_0v<0$ because $\delta u<1$;
dividing by $\alpha<0$ makes the derivative positive. Substituting $q=-1$
gives the two stated thresholds. These statements do not establish a common
monotonicity sign for both histories or a global sign for the price ratios.
\end{proof}

\section{Price ratios: three distinct conditioning statements}
The state-matched ratios are descriptive comparisons:
\[
 R_j=\frac{p_1^*(x_0=j)}{p_2^*(x_1=j)},\qquad
 R_0=\frac{p_1^*(x_0=0)}{p_2^*(x_1=0)}.
\]
They do not necessarily describe one realized consumer path. The acquisition
comparison is instead
\[
 R_{0\to j}=\frac{p_1^*(x_0=0)}{p_2^*(x_1=j)},
\]
conditional on acquisition in period 1. It is not an expectation over histories.
For each definition use its own numerator, denominator and state derivatives:
\[
 \boxed{R'=\frac{p_2^*p_1^{*\prime}-p_1^*p_2^{*\prime}}{(p_2^*)^2}}.
\]
Terminal prices exceed $c$; the benchmark $c=0$ therefore has strictly positive
ratio denominators. For other calibrations a ratio is only defined when its
denominator is nonzero. No expected ratio or ratio of expectations is substituted.

\section{Numerical benchmark and verification}
Every plotted point reoptimizes both terminal branches and period 1:
\[
 \beta=2,\qquad\alpha=-1,\qquad\delta=0.92,\qquad c=0,\qquad
 \gamma\in[-3,3]\ \text{on a 601-point grid}.
\]
The calibration is illustrative, not estimated. SymPy verifies raw-utility
normalization, both branch weights, the profit-derived FOCs, implicit price
derivatives and the envelope identities. Numerical checks compare all 1,202
benchmark state/grid solutions with the handoff's independent reference.
Central differences reoptimize the full model at nearby parameters; an
independent scalar profit optimizer checks optimality. Additional calibrations
check derivatives and an introductory-subsidy case. Numerical evidence is not
a proof of unqualified global comparative-static signs.

\subsection*{Selected optimized values}
```

Source lines 240–242:

```
\begin{center}\small\begin{tabular}{crrrrrr}\toprule
$x_0$ & $\gamma$ & $p_1^*$ & $p_2^*(j)$ & $p_2^*(0)$ & $\Pi^*$ & $\Pi^{*\prime}$\\\midrule
'''
```

**Read this block:**

- Line 30, `document`: Set this name to a multiline text template; its contents are data, not Python instructions.

Source lines 243–246:

```
for state in ['j','0']:
    for g in [-3,-1,0,1,3]:
        r=evaluate(g,state)
        document+=f"${state}$ & {g} & {r['p1']:.3f} & {r['p2j']:.3f} & {r['p20']:.3f} & {r['profit']:.3f} &
 ↪  {r['dprofit']:.3f}"+r'\\'+'\n'
```

**Read this block:**

- Line 243: for each entry yielded by `['j', '0']`, bind/unpack it into `state` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 244: for each entry yielded by `[-3, -1, 0, 1, 3]`, bind/unpack it into `g` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 245, `r`: Set this name to the result of calling `evaluate(...)`; the arguments and preceding method chain are shown above.
- Line 246: update `document` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

**Syntax and calls used here:**

- `evaluate`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 247–250:

```
    document+=r'''\bottomrule\end{tabular}\end{center}
\clearpage
\input{sympy_equations.tex}
'''
```

**Read this block:**

- Line 247: update `document` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 251–264:

```
specs=[
    ('matched_price_ratio_derivative','State-matched price-ratio derivatives',
     r'$R_j=p_1^*(j)/p_2^*(j)$ and $R_0=p_1^*(0)/p_2^*(0)$. Each curve holds its stated histories fixed and uses
      ↪  the analytical quotient rule.'),
    ('induce_harvest_price_ratio_derivative','Induce-then-harvest price-ratio derivative',
     r'$R_{0\to j}=p_1^*(0)/p_2^*(j)$, conditional on a first-period acquisition. This is a path-specific
      ↪  comparison, not an ex-ante expectation.'),
    ('profit_by_gamma','Optimized profit levels',
     r'Expected discounted profit includes both realized continuation histories. The two curves condition on the
      ↪  initial realized state.'),
    ('gamma_profit_by_gamma','Optimized profit derivative',
     r'Total derivatives use the envelope theorem and both continuation-value derivatives. Their signs depend on
      ↪  the initial history and the stated thresholds.'),
    ('optimal_prices_by_gamma','Optimal prices in both periods',
     r'Left: period-1 prices for both initial states. Right: period-2 prices for both realized first-period states.
      ↪  Prices are unconstrained; all shares adjust.'),
    ('gamma_p1_by_gamma','Optimal period-1 price derivative',
     r'Analytical total derivatives, with both continuation branches reoptimized. Initial-state signs are
      ↪  interpreted using the proved conditional statements.'),
]
```

**Read this block:**

- Line 251, `specs`: Set this name to a list containing 6 displayed entries.

Source lines 265–270:

```
for name,title,caption in specs:
    if not (ROOT/'figures'/f'{name}.pdf').exists():
        raise FileNotFoundError(name)
    document+='\\clearpage\n\\section*{'+title+'}\n'
    document+='\\begin{figure}[htbp]\\centering\n\\includegraphics[width=\\linewidth]{figures/'+name+'.pdf}\n'
    document+='\\caption{'+caption+r' Benchmark: $\beta=2$, $\alpha=-1$, $\delta=0.92$, $c=0$;
    $\gamma\in[-3,3]$.}'+'\n\\end{figure}\n'
```

**Read this block:**

- Line 265: for each entry yielded by `specs`, bind/unpack it into `(name, title, caption)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 266: test `not (ROOT / 'figures' / f'{name}.pdf').exists()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 267: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 268: update `document` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 269: update `document` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 270: update `document` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

**Syntax and calls used here:**

- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- `FileNotFoundError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 271–273:

```
document+='\\end{document}\n'
(ROOT/'comparative_statics_figures.tex').write_text(document)
print('Generated verified symbolic equations and appendix with six requested figures.')
```

**Read this block:**

- Line 271: update `document` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 272: call `(ROOT / 'comparative_statics_figures.tex').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 273: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

**Output reminder:** reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

## supply_code/model/___init___.py

This empty file marks an importable package directory. It defines no functions and performs no data calculations. Python can support some namespace packages without it, but an explicit marker makes the intended package structure clear.

### Module setup and direct-execution blocks

There are no statements in this file.

## supply_code/model/assumptions.py

This dictionary stores human-readable assumptions as strings. Python and SymPy do not enforce alpha<0 just because a dictionary says so. It serves documentation; the numerical evaluator performs explicit checks. Distinguish descriptive metadata from executable validation.

### Module setup and direct-execution blocks

Source lines 1–3:

```
1 assumptions = {"alpha": "<0", "delta": "0<delta<1", "prices": "unconstrained real",
2                "cost": "fixed", "shares": "strictly interior", "type": "fixed and observed",
3                "history": "realized j or 0; both continuation states integrated"}
```

**Read this block:**

- Line 1, `assumptions`: Set this name to a dictionary with named entries `alpha`, `delta`, `prices`, `cost`, `shares`, `type`, `history`.

## supply_code/model/numerical.py

Module-level tuples `_ARGS`, `_EXPRESSIONS`, and `_KEYS` establish the ordered bridge from symbolic algebra to named numerical derivative results. A leading underscore is a convention for an internal name, not access control. `sp.lambdify` executes once at import, avoiding repeated compilation of the same symbolic formulas for all 1,202 benchmark evaluations. The functions below reuse that callable. Positional order, rather than matching name spelling at runtime, connects its inputs.

### Module setup and direct-execution blocks

Source lines 1–6:

```
1 """Unconstrained backward induction; derivatives evaluated from SymPy."""
2 import numpy as np
3 import sympy as sp
4 from scipy.special import expit
5 from scipy.optimize import brentq
6 from model import symbols as S
```

**Read this block:**

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 3: import `sympy as sp`. An as-name is a convenient local alias, not another library.
- Line 4: from `scipy.special`, import `expit` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 5: from `scipy.optimize`, import `brentq` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 6: from `model`, import `symbols as S` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 7–14:

```
7 from model.utility import signed_state
8 from comparative_statics import canonical as C
```

```
9
10 PARAMETERS = dict(beta=2., alpha=-1., delta=.92, c=0.)
11 _KEYS = ("dp1", "d2p1", "dprofit", "matched_j", "matched_0", "induce_harvest", "dp2j", "dp20")
12 _ARGS = (S.alpha,S.beta,S.gamma,S.delta,S.c,S.q,S.sj1,S.u,S.v,S.p1,S.p2j,S.p20,S.Vj,S.V0)
13 _EXPRESSIONS = (C.dp1_dgamma,C.d2p1_dgamma2,C.dPi_dgamma,C.matched_j,C.matched_0,C.induce_harvest,
14                 C.price2_derivative(S.gamma,S.u,-1,S.p2j),C.price2_derivative(S.gamma,S.v,1,S.p20))
```

**Read this block:**

- Line 7: from `model.utility`, import `signed_state` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 8: from `comparative_statics`, import `canonical as C` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 10, `PARAMETERS`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 11, `_KEYS`: Set this name to a tuple containing 8 displayed entries.
- Line 12, `_ARGS`: Set this name to a tuple containing 14 displayed entries.
- Line 13, `_EXPRESSIONS`: Set this name to a tuple containing 8 displayed entries.

**Syntax and calls used here:**

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

Source lines 15–15:

```
15 _EVALUATE = sp.lambdify(_ARGS,_EXPRESSIONS,"numpy")
```

**Read this block:**

- Line 15, `_EVALUATE`: Set this name to the result of calling `sp.lambdify(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.lambdify`: Compile a symbolic expression into a numerical callable with an explicitly ordered argument list.

## inside share

Original function: `inside_share`. Source begins at line 18.

This computes one numeric binary-logit probability. `signed_state` validates the state; `expit` evaluates logistic demand stably, then `float` turns a NumPy scalar into an ordinary Python number. Direct `exp(x)/(1+exp(x))` can overflow at large x. The defaults beta=2 and alpha=-1 support the benchmark, while callers can supply other calibrations. This function alone does not validate alpha's sign; full parameter-domain checks live in `evaluate`.

**Definition and stated intent:**

Source lines 18–18:

```
18 def inside_share(price,gamma,state,beta=2.,alpha=-1.):
```

`def` creates this callable; the parentheses name its inputs (`price`, `gamma`, `state`, `beta`, `alpha`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 19–19:

```
19 return float(expit(beta+signed_state(state)*gamma+alpha*price))
```

**Read this block:**

- Line 19: return the result of calling `float(...)`; the arguments and preceding method chain are shown above. This ends the call.

**Syntax and calls used here:**

- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `expit`: Evaluate the logistic function stably, avoiding overflow from a naive exp(x)/(1+exp(x)) formula.
- `signed_state`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

## optimal price

Original function: `optimal_price`. Source begins at line 22.

This finds one optimal price for a utility intercept, negative alpha, and an effective cost. `f = lambda p: ...` defines the transformed scalar FOC. `clip` limits the exponential's argument to a representable range; it is a numerical safeguard rather than a theoretical cap on utility.

The initial lower endpoint is effective cost, where f is positive. `max` sets a reasonable initial width, and `while f(hi) > 0` doubles it until the bracket changes sign. `brentq` can then locate the zero to `xtol=1e-12`. A fixed interval is simpler but can miss a root under another calibration. Unconstrained generic minimization ignores the monotonic scalar structure and can need more evaluations. The maintained alpha<0 assumption establishes a unique root; the caller validates it.

**Definition and stated intent:**

Source lines 22–23:

```
22 def optimal_price(intercept,alpha,effective_cost):
23     # Multiplying the FOC by alpha gives a strictly decreasing function.
```

`def` creates this callable; the parentheses name its inputs (`intercept`, `alpha`, `effective_cost`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 24–27:

```
24 f = lambda p: alpha*(p-effective_cost)+1+np.exp(np.clip(intercept+alpha*p,-745,700))
25 lo = effective_cost
26 width = max(2.,2/abs(alpha))
27 hi = lo+width
```

**Read this block:**

- Line 24, `f`: Set this name to a one-expression callable, with input `p` and returned expression `alpha * (p - effective_cost) + 1 + np.exp(np.clip(in...`.
- Line 25, `lo`: Set this name to the object currently named `effective_cost`; plain assignment does not copy it.
- Line 26, `width`: Set this name to the result of calling `max(...)`; the arguments and preceding method chain are shown above.
- Line 27, `hi`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

**Syntax and calls used here:**

- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.clip`: Cap values at lower/upper limits; here it keeps starts feasible or exponentials within floating-point range.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `abs`: Take magnitude, removing a sign before a distance/tolerance comparison.

Source lines 28–30:

```
28 while f(hi) > 0:
29     width *= 2
30     hi = lo+width
```

**Read this block:**

- Line 28: repeat while `f(hi) > 0` remains true. Unlike for/range, this loop need not know the iteration count beforehand; its body must eventually change the condition.
- Line 29: update `width` using `*=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 30, `hi`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

**Syntax and calls used here:**

- `f`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 31–31:

```
31 return float(brentq(f,lo,hi,xtol=1e-12))
```

**Read this block:**

- Line 31: return the result of calling `float(...)`; the arguments and preceding method chain are shown above. This ends the call.

**Syntax and calls used here:**

- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `brentq`: Find a scalar zero within a sign-changing bracket, rather than minimize the function itself.

## evaluate

Original function: `evaluate`. Source begins at line 34.

This is backward induction expressed as ordinary assignments. It accepts gamma, initial history, and optional calibration numbers, then returns a dictionary. `np.isfinite([...]).all()` validates every numeric input at once. The terminal list comprehension solves two prices, unpacked into p2j and p20. Subsequent tuple assignments compute their shares and values; first-period effective cost then uses the value difference.

`_EVALUATE(...)` calls the numerical function built from symbolic derivatives at import time. Its argument order must match `_ARGS`; a positional swap can produce plausible but wrong derivatives. `result.update(dict(zip(_KEYS, map(float, values))))` converts and labels those outputs. State-inapplicable ratio derivatives become NaN intentionally. Final FOC residuals use original profit derivatives as a diagnostic independent of the root's transformed form. An external simultaneous optimizer would conceal the model's simpler sequential structure.

**Definition and stated intent:**

Source lines 34–34:

```
34 def evaluate(gamma,initial_state,beta=2.,alpha=-1.,delta=.92,c=0.):
```

`def` creates this callable; the parentheses name its inputs (`gamma`, `initial_state`, `beta`, `alpha`, `delta`, `c`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 35–35:

```
35 q = signed_state(initial_state)
```

**Read this block:**

- Line 35, `q`: Set this name to the result of calling `signed_state(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `signed_state`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 36–37:

```
36 if not (np.isfinite([gamma,beta,alpha,delta,c]).all() and alpha<0 and 0<=delta<1):
37     raise ValueError("Require finite parameters, alpha<0, and 0<=delta<1 (delta=0 is the static counterfactual).")
```

**Read this block:**

- Line 36: test `not (np.isfinite([gamma, beta, alpha, delta, c]).all() and alpha < 0 and (0 <...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 37: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

**Syntax and calls used here:**

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.

Source lines 38–43:

```
38 p2j,p20 = [optimal_price(beta+signed_state(state)*gamma,alpha,c) for state in ["j","0"]]
39 u,v = inside_share(p2j,gamma,"j",beta,alpha),inside_share(p20,gamma,"0",beta,alpha)
40 Vj,V0 = (p2j-c)*u,(p20-c)*v
41 DV = Vj-V0
42 p1 = optimal_price(beta+q*gamma,alpha,c-delta*DV)
43 s = inside_share(p1,gamma,initial_state,beta,alpha)
```

**Read this block:**

- Line 38, `(p2j, p20)`: Unpack a list computed by the displayed for-expression into these targets in order.
- Line 39, `(u, v)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 40, `(Vj, V0)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 41, `DV`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 42, `p1`: Set this name to the result of calling `optimal_price(...)`; the arguments and preceding method chain are shown above.
- Line 43, `s`: Set this name to the result of calling `inside_share(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `optimal_price`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `inside_share`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 44–51:

```
44 profit = s*(p1-c)+delta*(s*Vj+(1-s)*V0)
45 values = _EVALUATE(alpha,beta,gamma,delta,c,q,s,u,v,p1,p2j,p20,Vj,V0)
46 result = dict(gamma=float(gamma),initial_state=initial_state,p1=p1,p2j=p2j,p20=p20,s=s,u=u,v=v,
47              V2j=Vj,V20=V0,DeltaV=DV,profit=profit,
48              static_component=c-1/(alpha*(1-s)),continuation_adjustment=-delta*DV,
49              static_counterfactual=optimal_price(beta+q*gamma,alpha,c))
50 result.update(dict(zip(_KEYS,map(float,values))))
51 result['matched_j' if initial_state=='0' else 'matched_0'] = float('nan')
```

**Read this block:**

- Line 44, `profit`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 45, `values`: Set this name to the result of calling `_EVALUATE(...)`; the arguments and preceding method chain are shown above.
- Line 46, `result`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 50: call `result.update` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 51, `result['matched_j' if initial_state == '0' else 'matched_0']`: Store the result of calling `float(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

**Syntax and calls used here:**

- `_EVALUATE`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `.update`: Add/replace dictionary entries in place; unrelated entries remain available.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `map`: Apply a callable to each iterable entry lazily, such as converting result scalars to Python float.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 52–53:

```
52 if initial_state=='j':
53     result['induce_harvest'] = float('nan')
```

**Read this block:**

- Line 52: test `initial_state == 'j'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 53, `result['induce_harvest']`: Store the result of calling `float(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

Source lines 54–58:

```
54 result['d_matched_ratio'] = result['matched_j'] if initial_state=='j' else result['matched_0']
55 result['d_induce_ratio'] = result['induce_harvest'] if initial_state=='0' else float('nan')
56 result['foc1'] = s+alpha*s*(1-s)*(p1-c+delta*DV)
57 result['foc2j'] = u+alpha*u*(1-u)*(p2j-c)
58 result['foc20'] = v+alpha*v*(1-v)*(p20-c)
```

**Read this block:**

- Line 54, `result['d_matched_ratio']`: Store the true-branch value if `initial_state == 'j'` holds, otherwise the else-branch value in the selected entry/column.
- Line 55, `result['d_induce_ratio']`: Store the true-branch value if `initial_state == '0'` holds, otherwise the else-branch value in the selected entry/column.
- Line 56, `result['foc1']`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 57, `result['foc2j']`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 58, `result['foc20']`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.

Source lines 59–59:

```
59 return result
```

**Read this block:**

- Line 59: return the object currently named `result`; plain assignment does not copy it. This ends the call.

## solve

Original function: `solve`. Source begins at line 62.

This is a compact convenience interface. The bare `*` makes `initial_state` keyword-only; `**parameters` collects other keywords. Membership tests reject obsolete z1/z2 inputs. `parameters.pop("gamma", 0.)` retrieves and removes gamma, allowing the rest to be expanded into `evaluate` without passing gamma twice. The function returns seven values as a tuple. It is convenient for unpacking, while `evaluate` is preferable when you need named derivative or diagnostic fields.

**Definition and stated intent:**

Source lines 62–62:

```
62 def solve(*, initial_state="0", **parameters):
```

`def` creates this callable; the parentheses name its inputs (`initial_state`). A displayed `=value` supplies a default, and the colon begins its indented body.

`**parameters` collects additional keyword arguments in a dictionary.

Source lines 63–64:

```
63 if 'z1' in parameters or 'z2' in parameters:
64     raise ValueError("Old 0/1 histories are superseded: use initial_state='j' or '0'; both continuation branches
       ↪ are solved.")
```

**Read this block:**

- Line 63: test `'z1' in parameters or 'z2' in parameters`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 64: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

**Syntax and calls used here:**

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 65–66:

```
65 gamma = parameters.pop('gamma',0.)
66 r = evaluate(gamma,initial_state,**parameters)
```

**Read this block:**

- Line 65, `gamma`: Set this name to the result of calling `parameters.pop(...)`; the arguments and preceding method chain are shown above.
- Line 66, `r`: Set this name to the result of calling `evaluate(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.pop`: Retrieve and remove a dictionary entry, using the supplied default if it is absent.
- `evaluate`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 67–67:

```
67 return r['p1'],r['p2j'],r['p20'],r['s'],r['u'],r['v'],r['profit']
```

**Read this block:**

- Line 67: return a tuple of 7 results. The caller can unpack them; return ends the call, unlike print.

# supply_code/model/profit.py

These assignments construct symbolic current and continuation profit. `pi2` weights both optimized terminal values by first-period share and outside share. `Pi = pi1 + delta*pi2` creates total discounted profit. Capitalization matters: Python treats `Pi` and `pi1` as unrelated names. No optimizer runs in this file; it provides the expression that FOC modules and tests inspect.

## Module setup and direct-execution blocks

Source lines 1–6:

```
1 from model.symbols import *
2 pi1 = sj1*(p1-c)
3 profit2 = sj2*(p2-c)
4 pi2 = sj1*Vj + (1-sj1)*V0
5 Pi = pi1 + delta*pi2
6 DeltaV = Vj-V0
```

**Read this block:**

- Line 1: from `model.symbols`, import ∗ into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 2, `pi1`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 3, `profit2`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 4, `pi2`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 5, `Pi`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 6, `DeltaV`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.

# supply_code/model/shares.py

This module builds symbolic logit shares from normalized utility and defines derivative expressions. The star import makes shared symbols available directly without a module prefix, which shortens formulas but makes their origins harder for beginners to track. Expressions such as `alpha*sj1*(1-sj1)` encode the logistic price derivative using share placeholders. They are algebra, not evaluated probabilities for a dataset. The explicit-share expression and the share symbol are intentionally distinct objects.

## Module setup and direct-execution blocks

Source lines 1–6:

```
1 import sympy as sp
2 from model.symbols import *
3 from model.utility import U1, U2
4 sj1_expr = 1/(1+sp.exp(-U1))
5 sj2_expr = 1/(1+sp.exp(-U2))
6 s01_expr, s02_expr = 1-sj1_expr, 1-sj2_expr
```

**Read this block:**

- Line 1: import `sympy as sp`. An as-name is a convenient local alias, not another library.

- Line 2: from `model.symbols`, import ∗ into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3: from `model.utility`, import `U1, U2` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 4, `sj1_expr`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 5, `sj2_expr`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 6, `(s01_expr, s02_expr)`: Unpack a tuple containing 2 displayed entries into these targets in order.

**Syntax and calls used here:**

- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.

Source lines 7–11:

```
dsj1_dp1, dsj2_dp2 = alpha*sj1*s01, alpha*sj2*s02
ds01_dp1, ds02_dp2 = -dsj1_dp1, -dsj2_dp2
dsj1_dgamma = q*sj1*s01
dsj2_dgamma_direct = q2*sj2*s02
dsj2_dutility = sj2*s02
```

**Read this block:**

- Line 7, `(dsj1_dp1, dsj2_dp2)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 8, `(ds01_dp1, ds02_dp2)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 9, `dsj1_dgamma`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 10, `dsj2_dgamma_direct`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 11, `dsj2_dutility`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.

# supply_code/model/symbols.py

`sp.symbols` constructs formal mathematical objects and tuple unpacking gives them Python names. The name displayed in equations can differ from the Python identifier, e.g. `p1` represents printed `p_1`. `real=True` records a symbolic domain; `positive=True` on shares does not also encode share below one. Economic restrictions such as alpha<0 remain assumptions used in interpretation and numeric validation. `s01 = 1-sj1` defines an expression, not another independent symbol. `z1,z2 = q,q2` aliases existing signed symbols for legacy code.

## Module setup and direct-execution blocks

Source lines 1–6:

```
"""Symbols for a fixed type and signed realized histories."""
import sympy as sp
alpha, beta, gamma, delta, c = sp.symbols("alpha beta gamma delta c", real=True)
p1, p2, p2j, p20 = sp.symbols("p_1 p_2 p_{2j} p_{20}", real=True)
q, q2 = sp.symbols("q q_2", real=True)
sj1, sj2, u, v = sp.symbols("s s_2 u v", positive=True)
```

**Read this block:**

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `sympy as sp`. An as-name is a convenient local alias, not another library.
- Line 3, `(alpha, beta, gamma, delta, c)`: Unpack the result of calling `sp.symbols(...)`; the arguments and preceding method chain are shown above into these targets in order.

- Line 4, `(p1, p2, p2j, p20)`: Unpack the result of calling `sp.symbols(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 5, `(q, q2)`: Unpack the result of calling `sp.symbols(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 6, `(sj1, sj2, u, v)`: Unpack the result of calling `sp.symbols(...)`; the arguments and preceding method chain are shown above into these targets in order.

**Syntax and calls used here:**

- `.symbols`: Create SymPy symbols, which stand for algebraic variables rather than observed numbers.

Source lines 7–12:

```
s01, s02 = 1-sj1, 1-sj2
Vj, V0 = sp.symbols("V_{2j} V_{20}", real=True)
h0, h1 = sp.symbols("h_0 h_1")
p1_star_sym, p2_star_sym = sp.symbols("p_1^* p_2^*")
# Legacy names are signed coefficients now, never the former 0/1 indicators.
z1, z2 = q, q2
```

**Read this block:**

- Line 7, `(s01, s02)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 8, `(Vj, V0)`: Unpack the result of calling `sp.symbols(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 9, `(h0, h1)`: Unpack the result of calling `sp.symbols(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 10, `(p1_star_sym, p2_star_sym)`: Unpack the result of calling `sp.symbols(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 12, `(z1, z2)`: Unpack the tuple `(q, q2)` into these targets in order.

# supply_code/model/utility.py

After defining its two helpers, module-level code calls `raw_utilities` for each period, unpacks inside/outside expressions, and subtracts outside utility. `sp.expand` distributes products to obtain the normalized linear index. This module combines function definitions with actual symbolic setup work at import time. It does not read private data or solve numeric prices.

## Module setup and direct-execution blocks

Source lines 1–2:

```
import sympy as sp
from model.symbols import *
```

**Read this block:**

- Line 1: import `sympy as sp`. An as-name is a convenient local alias, not another library.
- Line 2: from `model.symbols`, import ∗ into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 17–20:

```
raw_inside1, raw_outside1 = raw_utilities(p1, q)
raw_inside2, raw_outside2 = raw_utilities(p2, q2)
U1 = sp.expand(raw_inside1 - raw_outside1)
U2 = sp.expand(raw_inside2 - raw_outside2)
```

**Read this block:**

- Line 17, `(raw_inside1, raw_outside1)`: Unpack the result of calling `raw_utilities(...)`; the arguments and preceding method chain are shown above into these targets in order.

- Line 18, `(raw_inside2, raw_outside2)`: Unpack the result of calling `raw_utilities(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 19, `U1`: Set this name to the result of calling `sp.expand(...)`; the arguments and preceding method chain are shown above.
- Line 20, `U2`: Set this name to the result of calling `sp.expand(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `raw_utilities`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.expand`: Distribute products and combine terms in a symbolic expression without solving its unknowns.

## signed state

Original function: `signed_state`. Source begins at line 5.

This converts the two accepted string state names into integer signs. Two independent `if` blocks are fine because each successful branch returns at once. Any other value reaches `raise ValueError`. `signed_state("j")` returns -1; `signed_state("0")` returns +1; numeric `0` is not string `"0"` and is rejected. A dictionary lookup could shorten this to a mapping, but these explicit branches make both accepted values and the custom error easy for a novice to see.

**Definition and stated intent:**

Source lines 5–5:

```
5 def signed_state(state):
```

`def` creates this callable; the parentheses name its inputs (`state`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 6–7:

```
6 if state == "j":
7     return -1
```

**Read this block:**

- Line 6: test `state == 'j'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 7: return the negated or sign-adjusted expression `-1`. This ends the call.

Source lines 8–9:

```
8 if state == "0":
9     return 1
```

**Read this block:**

- Line 8: test `state == '0'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 9: return the literal value `1`. This ends the call.

Source lines 10–10:

```
10 raise ValueError("state must be 'j' or '0'")
```

**Read this block:**

- Line 10: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

**Syntax and calls used here:**

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

### raw utilities

Original function: `raw_utilities`. Source begins at line 13.

Input `price` may be a SymPy symbol and `signed_history` a sign or symbolic q. The return statement contains two comma-separated expressions, so it returns a tuple `(inside_utility, outside_utility)`. The combinations `(1+q)/2` and `(1-q)/2` act as complementary indicators when q is ±1. At q=-1 the first is zero and second one; at q=+1 they reverse.

Arithmetic with SymPy objects builds algebraic expressions rather than computing a numerical utility immediately. Returning both raw utilities preserves the outside switching payoff before normalization. Writing only an inside bonus would encode a different model. Module-level unpacking later names each result and subtracts the outside utility explicitly.

**Definition and stated intent:**

Source lines 13–13:

```
13 def raw_utilities(price, signed_history):
```

`def` creates this callable; the parentheses name its inputs (`price`, `signed_history`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 14–15:

```
14 return (beta + alpha*price + gamma*(1+signed_history)/2,
15         gamma*(1-signed_history)/2)
```

**Read this block:**

- Line 14: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

## supply_code/pricing/___init___.py

This empty file marks an importable package directory. It defines no functions and performs no data calculations. Python can support some namespace packages without it, but an explicit marker makes the intended package structure clear.

### Module setup and direct-execution blocks

There are no statements in this file.

## supply_code/pricing/period1_foc.py

This module creates equivalent raw and collected forms of the dynamic first derivative. Factoring `(p1-c+delta*DeltaV)` displays the effective-margin meaning. `sp.Eq(expression, 0)` constructs a symbolic equation object; it does not solve that equation or assign zero to the expression. Keeping an equation for export beside an expression for differentiation avoids manually retyping formulas.

### Module setup and direct-execution blocks

Source lines 1–6:

```
1 import sympy as sp
2 from model.symbols import *
3 from model.profit import DeltaV
4 foc1_raw = sj1 + alpha*sj1*s01*(p1-c) + delta*alpha*sj1*s01*DeltaV
5 foc1_collected = sj1 + alpha*sj1*s01*(p1-c+delta*DeltaV)
```

```
6 foc1_eq = sp.Eq(foc1_collected, 0)
```

**Read this block:**

- Line 1: import `sympy as sp`. An as-name is a convenient local alias, not another library.
- Line 2: from `model.symbols`, import ∗ into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3: from `model.profit`, import `DeltaV` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 4, `foc1_raw`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 5, `foc1_collected`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 6, `foc1_eq`: Set this name to the result of calling `sp.Eq(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.Eq`: Create a symbolic equality object; unlike assignment it does not bind its left side to its right side.

# supply_code/pricing/period1_price.py

`p1_star` is an implicit markup expression containing an equilibrium-share symbol and both continuation values. `sp.Eq` labels it with a formal starred price for display. It is not a numerical optimal price until those endogenous objects are solved consistently. Direct substitution of an arbitrary share would not solve the model.

## Module setup and direct-execution blocks

Source lines 1–4:

```
1 import sympy as sp
2 from model.symbols import *
3 p1_star = c-1/(alpha*s01)-delta*(Vj-V0)
4 p1_star_eq = sp.Eq(p1_star_sym, p1_star)
```

**Read this block:**

- Line 1: import `sympy as sp`. An as-name is a convenient local alias, not another library.
- Line 2: from `model.symbols`, import ∗ into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3, `p1_star`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 4, `p1_star_eq`: Set this name to the result of calling `sp.Eq(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.Eq`: Create a symbolic equality object; unlike assignment it does not bind its left side to its right side.

# supply_code/pricing/period2_foc.py

The terminal FOC contains current share plus margin times its price derivative. There is no discount term because period 2 is terminal. `sp.Eq` wraps it as an equation for presentation; actual root solving happens in `model/numerical.py`. Separating the expression permits tests to compare it with a literal derivative of profit instead of trusting a displayed equation alone.

### Module setup and direct-execution blocks

Source lines 1–4:

```
1 import sympy as sp
2 from model.symbols import *
3 foc2 = sj2 + alpha*sj2*s02*(p2-c)
4 foc2_eq = sp.Eq(foc2, 0)
```

**Read this block:**

- Line 1: import `sympy as sp`. An as-name is a convenient local alias, not another library.
- Line 2: from `model.symbols`, import ∗ into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3, `foc2`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 4, `foc2_eq`: Set this name to the result of calling `sp.Eq(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.Eq`: Create a symbolic equality object; unlike assignment it does not bind its left side to its right side.

## supply_code/pricing/period2_price.py

The markup expression is cost minus `1/(alpha*s02)`, positive above cost for negative alpha and interior shares. Parentheses make the whole product the denominator: `1/alpha*s02` would mean `(1/alpha)*s02`, a different formula. This is another implicit representation because outside share depends on price.

### Module setup and direct-execution blocks

Source lines 1–4:

```
1 import sympy as sp
2 from model.symbols import *
3 p2_star = c-1/(alpha*s02)
4 p2_star_eq = sp.Eq(p2_star_sym, p2_star)
```

**Read this block:**

- Line 1: import `sympy as sp`. An as-name is a convenient local alias, not another library.
- Line 2: from `model.symbols`, import ∗ into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3, `p2_star`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 4, `p2_star_eq`: Set this name to the result of calling `sp.Eq(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.Eq`: Create a symbolic equality object; unlike assignment it does not bind its left side to its right side.

## supply_code/run_all.py

`Path(__file__).resolve().parent` locates this script's directory regardless of the shell's current directory. A list of command-argument lists describes the three reproducibility steps. `[sys.executable, *command]` uses the current Python interpreter and expands each command's arguments without a shell. `cwd=ROOT` makes relative imports and outputs predictable; `check=True` raises if a step exits unsuccessfully, preventing later artifact generation after a failed check. Running a shell string with semicolons would be less portable and would require additional quoting/exit-status care.

## Module setup and direct-execution blocks

Source lines 1–5:

```
"""Reproducibility entry point: tests, requested figures, symbolic LaTeX."""
import subprocess
import sys
from pathlib import Path
ROOT=Path(__file__).resolve().parent
```

**Read this block:**

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `subprocess`. An as-name is a convenient local alias, not another library.
- Line 3: import `sys`. An as-name is a convenient local alias, not another library.
- Line 4: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 5, `ROOT`: Set this name to the attribute `Path(__file__).resolve().parent` of the object before the dot.

**Syntax and calls used here:**

- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 6–11:

```
for command in [
    ['-m','unittest','-v','checks.test_symmetric'],
    ['make_comparative_static_figures.py'],
    ['make_latex_document.py'],
]:
    subprocess.run([sys.executable,*command],cwd=ROOT,check=True)
```

**Read this block:**

- Line 6: for each entry yielded by `[['-m', 'unittest', '-v', 'checks.test_symmetric'], ['make_com...`, bind/unpack it into `command` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 11: call `subprocess.run` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.run`: For subprocess.run, execute an argument-list command; check=True raises if its exit status is nonzero.
- A starred entry expands an iterable into separate elements, here within a list/tuple or call.

Source lines 12–12:

```
print('All checks passed; figures and LaTeX regenerated. Compile the TeX with latexmk.')
```

**Read this block:**

- Line 12: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

# supply_code/structural_switching_reference.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

## Module setup and direct-execution blocks

Source lines 1–10:

```
"""Two-period symmetric switching model: solve, differentiate, verify, plot.

Run: python structural_switching_reference.py
Requires: numpy, scipy, matplotlib.  Outputs to ./outputs.
"""
from pathlib import Path
import csv
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import brentq
```

**Read this block:**

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 6: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 7: import `csv`. An as-name is a convenient local alias, not another library.
- Line 8: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 9: import `matplotlib.pyplot as plt`. An as-name is a convenient local alias, not another library.
- Line 10: from `scipy.optimize`, import `brentq` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 12–14:

```
BETA, ALPHA, DELTA, COST = 2.0, -1.0, 0.92, 0.0
GAMMAS = np.linspace(-3.0, 3.0, 601)
OUTPUT = Path('outputs')
```

**Read this block:**

- Line 12, `(BETA, ALPHA, DELTA, COST)`: Unpack a tuple containing 4 displayed entries into these targets in order.
- Line 13, `GAMMAS`: Set this name to the result of calling `np.linspace(...)`; the arguments and preceding method chain are shown above.
- Line 14, `OUTPUT`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 130–131:

```
if __name__ == '__main__':
    main()
```

**Read this block:**

- Line 130: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.

- Line 131: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

## expit safe

Original function: `expit_safe`. Source begins at line 17.

This imports and returns SciPy's stable logistic transform. The local import occurs when the function is called; Python caches imported modules, so it is not reloading SciPy from scratch each time. A module-level import would be more conventional here. The numerical advantage over a naive exponential ratio is avoiding overflow at extreme utility indices.

**Definition and stated intent:**

Source lines 17–17:

```
17 def expit_safe(x):
```

`def` creates this callable; the parentheses name its inputs (`x`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 18–18:

```
18 from scipy.special import expit
```

**Read this block:**

- Line 18: from `scipy.special`, import `expit` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 19–19:

```
19 return expit(x)
```

**Read this block:**

- Line 19: return the result of calling `expit(...)`; the arguments and preceding method chain are shown above. This ends the call.

**Syntax and calls used here:**

- `expit`: Evaluate the logistic function stably, avoiding overflow from a naive exp(x)/(1+exp(x)) formula.

## inside share

Original function: `inside_share`. Source begins at line 22.

This independent reference maps a state to a sign with nested conditional expressions, rejects invalid states, and evaluates logistic demand. The final `else None` makes invalid input detectable before multiplying it by gamma. Defaults capture the benchmark constants at function definition time. Unlike the production state helper, the sign logic is written independently to help cross-check the implementation rather than simply calling the same helper twice.

**Definition and stated intent:**

Source lines 22–23:

```
22 def inside_share(price, gamma, state, beta=BETA, alpha=ALPHA):
23     """state='j': existing customer, state='0': outside previously."""
```

`def` creates this callable; the parentheses name its inputs (`price`, `gamma`, `state`, `beta`, `alpha`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 24–24:

```
24 q = -1 if state == 'j' else +1 if state == '0' else None
```

**Read this block:**

- Line 24, `q`: Set this name to the true-branch value if `state == 'j'` holds, otherwise the else-branch value.

**Syntax and calls used here:**

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 25–26:

```
25 if q is None:
26     raise ValueError("state must be 'j' or '0'")
```

**Read this block:**

- Line 25: test `q is None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 26: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

**Syntax and calls used here:**

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 27–27:

```
27 return expit_safe(beta + q * gamma + alpha * price)
```

**Read this block:**

- Line 27: return the result of calling `expit_safe(...)`; the arguments and preceding method chain are shown above. This ends the call.

**Syntax and calls used here:**

- `expit_safe`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

## p2 opt

Original function: `p2_opt`. Source begins at line 30.

This reference solves the terminal markup equation using a lambda and a fixed bracket [-30,40]. Its gamma/state inputs enter through the closed-over variables. The return is the `brentq` root. It is appropriate for the reference benchmark; the production solver's expanding bracket handles a wider calibration range without assuming the root remains in this interval. Separate implementations make agreement more informative than two wrappers around the same solver.

**Definition and stated intent:**

Source lines 30–31:

```
30 def p2_opt(gamma, state):
31     """Unique finite price solving p=c-1/[alpha*(1-s)] (interior)."""
```

`def` creates this callable; the parentheses name its inputs (`gamma`, `state`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 32–32:

```
32 f = lambda p: p - COST + 1 / (ALPHA * (1 - inside_share(p, gamma, state)))
```

**Read this block:**

- Line 32, `f`: Set this name to a one-expression callable, with input `p` and returned expression `p - COST + 1 / (ALPHA * (1 - inside_share(p, gamma, ....`.

**Syntax and calls used here:**

- `inside_share`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.

Source lines 33–33:

```
33 return brentq(f, -30.0, 40.0, xtol=1e-13)
```

**Read this block:**

- Line 33: return the result of calling `brentq(...)`; the arguments and preceding method chain are shown above. This ends the call.

**Syntax and calls used here:**

- `brentq`: Find a scalar zero within a sign-changing bracket, rather than minimize the function itself.

## evaluate

Original function: `evaluate`. Source begins at line 36.

This independently computes backward-induction prices, branch values, profit, and hand-written analytic gamma derivatives. Comma-separated assignments unpack small tuples; `r = 1-s` names the outside share. Conditional expressions choose the correct matched denominator, and acquisition ratios are NaN outside their applicable initial history. An `assert np.isclose(...)` checks the price decomposition internally. Production uses lambdified canonical formulas, whereas the reference writes formulas directly to provide an independent oracle. Its interface and benchmark constants are narrower than production's.

**Definition and stated intent:**

Source lines 36–36:

```
36 def evaluate(gamma, initial_state):
```

`def` creates this callable; the parentheses name its inputs (`gamma`, `initial_state`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 37–42:

```
37 p2j, p20 = p2_opt(gamma, 'j'), p2_opt(gamma, '0')
38 u, v = inside_share(p2j, gamma, 'j'), inside_share(p20, gamma, '0')
39 Vj, V0 = (p2j-COST)*u, (p20-COST)*v
40 dV = Vj - V0
41 q = -1 if initial_state == 'j' else 1
42 f = lambda p: p-COST+1/(ALPHA*(1-inside_share(p, gamma, initial_state)))+DELTA*dV
```

**Read this block:**

- Line 37, `(p2j, p20)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 38, `(u, v)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 39, `(Vj, V0)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 40, `dV`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.

- Line 41, `q`: Set this name to the true-branch value if `initial_state == 'j'` holds, otherwise the else-branch value.
- Line 42, `f`: Set this name to a one-expression callable, with input `p` and returned expression `p - COST + 1 / (ALPHA * (1 - inside_share(p, gamma, ...`.

**Syntax and calls used here:**

- `p2_opt`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `inside_share`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.

Source lines 43–48:

```
p1 = brentq(f, -30.0, 40.0, xtol=1e-13)
s = inside_share(p1, gamma, initial_state)
r = 1-s
pi = s*(p1-COST)+DELTA*(s*Vj+r*V0)
static_piece = COST - 1/(ALPHA*r)
dynamic_piece = -DELTA*dV
```

**Read this block:**

- Line 43, `p1`: Set this name to the result of calling `brentq(...)`; the arguments and preceding method chain are shown above.
- Line 44, `s`: Set this name to the result of calling `inside_share(...)`; the arguments and preceding method chain are shown above.
- Line 45, `r`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 46, `pi`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 47, `static_piece`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 48, `dynamic_piece`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.

**Syntax and calls used here:**

- `brentq`: Find a scalar zero within a sign-changing bracket, rather than minimize the function itself.

Source lines 50–56:

```
d_pi = (-q*s + DELTA*(s*u-r*v))/ALPHA
dp1 = -(q*s + DELTA*r*(u+v))/ALPHA
# ds/dgamma = s*r*(q + alpha*dp1) = s*r^2*(q - delta*(u+v)).
ds = s*r*r*(q-DELTA*(u+v))
dB = -u*(1-u)**2 + v*(1-v)**2
d2p1 = -(s*r*r*(q-DELTA*(u+v))**2 + DELTA*r*dB)/ALPHA
dp2j = -(-1)*u/ALPHA
```

**Read this block:**

- Line 50, `d_pi`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 51, `dp1`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 53, `ds`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.

- Line 54, `dB`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 55, `d2p1`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 56, `dp2j`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

**Syntax and calls used here:**

- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.

Source lines 57–61:

```
dp20 = -(+1)*v/ALPHA
matched_p2 = p2j if initial_state == 'j' else p20
matched_dp2 = dp2j if initial_state == 'j' else dp20
d_matched_ratio = (dp1*matched_p2-p1*matched_dp2)/matched_p2**2
d_induce_ratio = (dp1*p2j - p1*dp2j)/p2j**2 if initial_state == '0' else np.nan
```

**Read this block:**

- Line 57, `dp20`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 58, `matched_p2`: Set this name to the true-branch value if `initial_state == 'j'` holds, otherwise the else-branch value.
- Line 59, `matched_dp2`: Set this name to the true-branch value if `initial_state == 'j'` holds, otherwise the else-branch value.
- Line 60, `d_matched_ratio`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 61, `d_induce_ratio`: Set this name to the true-branch value if `initial_state == '0'` holds, otherwise the else-branch value.

Source lines 62–62:

```
assert np.isclose(p1, static_piece+dynamic_piece, atol=1e-9)
```

**Read this block:**

- Line 62: require `np.isclose(p1, static_piece + dynamic_piece, atol=1e-09)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

**Syntax and calls used here:**

- `.isclose`: Create approximate-equality flags rather than requiring exact floating-point representation.

Source lines 63–67:

```
return dict(gamma=gamma, initial_state=initial_state, p1=p1, p2j=p2j,
            p20=p20, profit=pi, V2j=Vj, V20=V0, DeltaV=dV,
            static_component=static_piece, continuation_adjustment=dynamic_piece,
            dp1=dp1, d2p1=d2p1, dprofit=d_pi,
            d_matched_ratio=d_matched_ratio, d_induce_ratio=d_induce_ratio)
```

**Read this block:**

- Line 63: return the result of calling `dict(...)`; the arguments and preceding method chain are shown above. This ends the call.

**Syntax and calls used here:**

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

## verify

Original function: `verify`. Source begins at line 70.

This loops over selected gamma values and both histories. For each it re-solves at gamma+h and gamma-h, then compares analytic derivatives with centered finite differences. The step h balances approximation and floating-point error; making it arbitrarily tiny does not ensure accuracy. `assert condition, details` raises AssertionError carrying a diagnostic tuple when a check fails. Ratios must be recomputed from reoptimized price levels, not differentiated holding terminal prices fixed. This is a numerical check, not a symbolic sign proof.

**Definition and stated intent:**

Source lines 70–70:

```
70 def verify():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 71–71:

```
71 h = 1e-4
```

**Read this block:**

- Line 71, `h`: Set this name to the literal value `0.0001`.

Source lines 72–86:

```
72 for g in [-2.4, -1.2, 0, 0.7, 2.6]:
73     for state in ['j', '0']:
74         result = evaluate(g, state)
75         plus, minus = evaluate(g+h, state), evaluate(g-h, state)
76         for value, key in [('dp1', 'p1'), ('dprofit', 'profit')]:
77             numerical = (plus[key]-minus[key])/(2*h)
78             assert np.isclose(result[value], numerical, atol=2e-7), (g,state,value,result[value],numerical)
79         numerical2 = (plus['dp1']-minus['dp1'])/(2*h)
80         assert np.isclose(result['d2p1'], numerical2, atol=2e-7), (g,state,'d2p1')
81         numerical_ratio = (plus['p1']/ (plus['p2j'] if state=='j' else plus['p20']) -
82                            minus['p1']/(minus['p2j'] if state=='j' else minus['p20']))/(2*h)
83         assert np.isclose(result['d_matched_ratio'], numerical_ratio, atol=2e-7)
84         if state == '0':
85             numerical_ih = (plus['p1']/plus['p2j'] - minus['p1']/minus['p2j'])/(2*h)
86             assert np.isclose(result['d_induce_ratio'], numerical_ih, atol=2e-7)
```

**Read this block:**

- Line 72: for each entry yielded by `[-2.4, -1.2, 0, 0.7, 2.6]`, bind/unpack it into `g` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 73: for each entry yielded by `['j', '0']`, bind/unpack it into `state` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 74, `result`: Set this name to the result of calling `evaluate(...)`; the arguments and preceding method chain are shown above.
- Line 75, `(plus, minus)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 76: for each entry yielded by `[('dp1', 'p1'), ('dprofit', 'profit')]`, bind/unpack it into `(value, key)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 77, `numerical`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 78: require `np.isclose(result[value], numerical, atol=2e-07)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 79, `numerical2`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

- Line 80: require `np.isclose(result['d2p1'], numerical2, atol=2e-07)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 81, `numerical_ratio`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 83: require `np.isclose(result['d_matched_ratio'], numerical_ratio, atol=2e-07)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 84: test `state == '0'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 85, `numerical_ih`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 86: require `np.isclose(result['d_induce_ratio'], numerical_ih, atol=2e-07)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

**Syntax and calls used here:**

- `evaluate`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.isclose`: Create approximate-equality flags rather than requiring exact floating-point representation.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 87–87:

```
87 print('All analytical derivative checks passed.')
```

**Read this block:**

- Line 87: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

**Output reminder:** reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

## plot series

Original function: `plot_series`. Source begins at line 90.

This plots one result field for two initial histories and saves a raster image. `[r[key] for r in rows[state]]` extracts that field from every result dictionary. Separating the data key from axis label/filename lets one helper generate price, derivative, and ratio plots. The function expects rows already aligned to GAMMAS. Its outputs belong to the retained reference program, not the six-figure production path used by `run_all.py`.

**Definition and stated intent:**

Source lines 90–90:

```
90 def plot_series(rows, key, ylabel, filename, title):
```

`def` creates this callable; the parentheses name its inputs (`rows`, `key`, `ylabel`, `filename`, `title`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 91–91:

```
91 fig, ax = plt.subplots(figsize=(8.6, 5.2))
```

**Read this block:**

- Line 91, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.

**Syntax and calls used here:**

- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.

Source lines 92–93:

```
92 for state,label in [('j',r'Initially inside ($x_0=j$)'),('0',r'Initially outside ($x_0=0$)')]:
93     ax.plot(GAMMAS, [r[key] for r in rows[state]], label=label, linewidth=2)
```

**Read this block:**

- Line 92: for each entry yielded by `[('j', 'Initially inside ($x_0=j$)'), ('0', 'Initially outside...`, bind/unpack it into `(state, label)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 93: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.plot`: Draw a line from supplied x/y values on the receiving plotting axes.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 94–99:

```
94 ax.axhline(0, color='gray', linewidth=.8)
95 ax.set(xlabel=r'$\gamma$', ylabel=ylabel, title=title)
96 ax.legend()
97 fig.tight_layout()
98 fig.savefig(OUTPUT/filename, dpi=180)
99 plt.close(fig)
```

**Read this block:**

- Line 94: call `ax.axhline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 95: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 96: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 97: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 98: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 99: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.axhline`: Add a horizontal reference line, commonly zero for a derivative plot.
- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.legend`: Render labels previously attached to plotted series, identifying curves.
- `.tight_layout`: Adjust plot spacing to reduce overlap between labels and content.
- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.
- `.close`: Release a Figure or file resource, according to the receiving object.

**Output reminder:** reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

## main

Original function: `main`. Source begins at line 102.

This first verifies the reference formulas, creates its `outputs` directory, solves benchmark rows, writes CSV, and saves several reference plots. `with ... open(..., newline="")` lets the CSV writer handle line endings correctly. Some lines contain semicolons separating multiple statements; Python executes them left-to-right just as if written on separate lines. That compact style saves space but is less friendly to beginners. Production uses another entry point and another output folder; running this does not replace `run_all.py`.

**Definition and stated intent:**

Source lines 102–102:

```
102 def main():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 103–105:

```
103 verify()
104 OUTPUT.mkdir(exist_ok=True)
105 rows = {state:[evaluate(float(g),state) for g in GAMMAS] for state in ['j','0']}
```

**Read this block:**

- Line 103: call `verify` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 104: call `OUTPUT.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 105, `rows`: Set this name to a dictionary computed one key/value pair at a time by the comprehension.

**Syntax and calls used here:**

- `verify`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- `evaluate`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.

Source lines 106–110:

```
106 with (OUTPUT/'model_results.csv').open('w', newline='') as file:
107     writer = csv.DictWriter(file, fieldnames=list(rows['j'][0]))
108     writer.writeheader()
109     for state in ['j','0']:
110         writer.writerows(rows[state])
```

**Read this block:**

- Line 106: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 107, `writer`: Set this name to the result of calling `csv.DictWriter(...)`; the arguments and preceding method chain are shown above.
- Line 108: call `writer.writeheader` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 109: for each entry yielded by `['j', '0']`, bind/unpack it into `state` and run the indented body. Changes inside an iteration can feed subsequent iterations.

- Line 110: call `writer.writerows` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.open`: Open a file in the requested mode; using a with block ensures it closes on exit.
- `.DictWriter`: Create a CSV writer mapping dictionary keys to named columns in an explicit field order.
- `.writeheader`: Write the CSV's named column header before writing data rows.
- `.writerows`: Write a sequence of dictionaries/rows using the configured CSV schema.
- `list`: Materialize iterable entries as an ordered Python list.

Source lines 111–116:

```
plot_series(rows, 'p1', r'$p_1^*$', 'price1.png', 'Optimal period-1 prices')
plot_series(rows, 'dp1', r'$dp_1^*/d\gamma$', 'dp1.png', 'First derivative of period-1 price')
plot_series(rows, 'd2p1', r'$d^2p_1^*/d\gamma^2$', 'd2p1.png', 'Second derivative of period-1 price (analytic)')
plot_series(rows, 'dprofit', r'$d\Pi^*/d\gamma$', 'dprofit.png', 'Optimized profit derivative')
plot_series(rows, 'd_matched_ratio', r'$d(p_1/p_2)/d\gamma$', 'matched_price_ratio.png', 'State-matched
 ↪  price-ratio derivatives')
fig,ax=plt.subplots(figsize=(8.6,5.2))
```

**Read this block:**

- Line 111: call `plot_series` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 112: call `plot_series` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 113: call `plot_series` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 114: call `plot_series` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 115: call `plot_series` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 116, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.

**Syntax and calls used here:**

- `plot_series`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.

Source lines 117–120:

```
ax.plot(GAMMAS, [r['d_induce_ratio'] for r in rows['0']], linewidth=2)
ax.axhline(0,color='gray',linewidth=.8)
ax.set(xlabel=r'$\gamma$', ylabel=r'$d[p_1(0)/p_2(j)]/d\gamma$', title='Induce-then-harvest price-ratio
 ↪  derivative')
fig.tight_layout();fig.savefig(OUTPUT/'induce_harvest_ratio.png',dpi=180);plt.close(fig)
```

**Read this block:**

- Line 117: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 118: call `ax.axhline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 119: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 120: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 120: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 120: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.plot`: Draw a line from supplied x/y values on the receiving plotting axes.
- `.axhline`: Add a horizontal reference line, commonly zero for a derivative plot.
- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.tight_layout`: Adjust plot spacing to reduce overlap between labels and content.
- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.
- `.close`: Release a Figure or file resource, according to the receiving object.

Source lines 121–127:

```
121 for state in ['j','0']:
122     fig,ax=plt.subplots(figsize=(8.6,5.2))
123     for key,label in [('p1',r'$p_1^*$'),('static_component','Static FOC
     ↪  component'),('continuation_adjustment',r'$-\delta\Delta V_2$')]:
124         ax.plot(GAMMAS,[r[key] for r in rows[state]],label=label,linewidth=2)
125     ax.axhline(0,color='gray',linewidth=.8)
126     ax.set(xlabel=r'$\gamma$',ylabel='Price / component',title=f'First-period decomposition: initial state
    ↪  {state}')
127     ax.legend();fig.tight_layout();fig.savefig(OUTPUT/f'decomposition_{state}.png',dpi=180);plt.close(fig)
```

**Read this block:**

- Line 121: for each entry yielded by `['j', '0']`, bind/unpack it into `state` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 122, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 123: for each entry yielded by `[('p1', '$p_1^*$'), ('static_component', 'Static FOC component...`, bind/unpack it into `(key, label)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 124: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 125: call `ax.axhline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 126: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 127: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 127: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 127: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 127: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.legend`: Render labels previously attached to plotted series, identifying curves.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 128–128:

```
128 print('Saved outputs to', OUTPUT.resolve())
```

**Read this block:**

- Line 128: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.

**Output reminder:** reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

# Tests and verification: worked calling examples

## data_code/test_summ_stats_memory.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

### Module setup and direct-execution blocks

Source lines 1–6:

```
"""Synthetic checks for sparse summaries and exact results across batch sizes."""
import contextlib
import io
from pathlib import Path
import tempfile
from types import SimpleNamespace
```

**Read this block:**

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `contextlib`. An as-name is a convenient local alias, not another library.
- Line 3: import `io`. An as-name is a convenient local alias, not another library.
- Line 4: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 5: import `tempfile`. An as-name is a convenient local alias, not another library.
- Line 6: from `types`, import `SimpleNamespace` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 7–12:

```
import unittest

import pandas as pd
import polars as pl

import summ_stats as summary
```

**Read this block:**

- Line 7: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 9: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 10: import `polars as pl`. An as-name is a convenient local alias, not another library.
- Line 12: import `summ_stats as summary`. An as-name is a convenient local alias, not another library.

Source lines 15–15:

```
15 class MemorySummaryChecks(unittest.TestCase):
```

**Read this block:**

- Line 15: define class `MemorySummaryChecks` and its methods. TestCase inheritance provides the self.assert… checking methods.

Source lines 128–129:

```
128 if __name__ == "__main__":
129     unittest.main()
```

**Read this block:**

- Line 128: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 129: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

## MemorySummaryChecks.test batches preserve global statistics

Original function: `MemorySummaryChecks.test_batches_preserve_global_statistics`. Source begins at line 16.

**What this test establishes.** Runs a small complete summary pipeline with one and three partitions, sorts outputs, and compares their actual contents. It verifies partitioning changes execution/storage rather than statistical results and preserves original metadata in the prepared sample.

**Reading its syntax.** This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

**Definition and stated intent:**

Source lines 16–16:

```
16 def test_batches_preserve_global_statistics(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 17–58:

```
17 with tempfile.TemporaryDirectory() as directory, contextlib.redirect_stdout(io.StringIO()):
18     root = Path(directory)
19     source = pl.DataFrame({
20         "household_code": [1, 1, 1, 1, 2, 2, 3], "panel_year": [2010] * 7,
21         "purchase_date": ["2010-01-02", "2010-01-09", "2010-01-09", "2010-01-23", "2010-01-02", "2010-01-09",
           ↪  "2010-01-02"],
22         "trip_code_uc": [1, 2, 2, 3, 4, 5, 6], "upc": [1, 1, 2, 3, 2, 3, 1],
23         "quantity": [2, 1, 1, 3, 1, 2, 1], "race": [1, 1, 1, 1, 2, 2, 3],
24         "hispanic_origin": [2, 2, 2, 2, 1, 1, 2], "household_income": [5, 5, 5, 5, 8, 8, 9],
25         "female_head_age": [3, 3, 3, 3, 0, 0, 4], "male_head_age": [4, 4, 4, 4, 2, 2, 5],
26         "household_size": [2] * 7, "store_code_uc": [100] * 7, "price": [1.] * 7,
27         "original_metadata": ["preserved"] * 7,
28     })
29     for name in ["merged_panel", "yogurt_estimation_sample"]:
30         source.write_parquet(root / f"{name}.parquet")
31     reference = root / "reference.csv"
32     reference.write_text("upc,flavor_descr,module\n001,vanilla,3612\n002,strawberry,3612\n003,plain,3612\n
   ↪  004,unused,3612\n")
```

```
    for batches in [1, 3]:
        out, work = root / f"out{batches}", root / f"work{batches}"
        out.mkdir()
        work.mkdir()
        args = SimpleNamespace(data_dir=root, flavors=reference, products_extra=root / "absent",
                               first_year=2006, last_year=2015, household_batches=batches, no_figures=True)
        summary.run(args, out, work)
    for name, keys in [
        ("full_panel_households", ["household_code"]), ("matched_yogurt_households", ["household_code"]),
        ("yogurt_household_flavors", ["household_code"]), ("yogurt_dynamics", ["metric"]),
        ("full_panel_overview", ["metric"]), ("matched_yogurt_overview", ["metric"]),
        ("yogurt_flavor_shares", ["flavor"]), ("full_panel_year_coverage", ["panel_year"]),
    ]:
        left = pd.read_csv(root / "out1" / f"{name}.csv").sort_values(keys).reset_index(drop=True)
        right = pd.read_csv(root / "out3" / f"{name}.csv").sort_values(keys).reset_index(drop=True)
        pd.testing.assert_frame_equal(left, right)
    prepared = pl.read_parquet(root / "out3/yogurt_estimation_sample.parquet")
    self.assertEqual(prepared.height, 6)
    self.assertTrue(set(source.columns).issubset(prepared.columns))
    self.assertEqual(set(prepared["household_code"].to_list()), {1, 2})
    self.assertEqual(prepared["original_metadata"].unique().to_list(), ["preserved"])
    self.assertEqual(set(prepared["upc"].to_list()), {"1", "2", "3"})
    self.assertEqual(prepared["flavor"].null_count(), 0)
    hh = pd.read_csv(root / "out3/yogurt_household_flavors.csv").set_index("household_code")
    self.assertEqual(set(hh.index), {1, 2})
    self.assertAlmostEqual(hh.loc[1, "flavor_HHI"], 19 / 49)
```

Source lines 59–84:

```
self.assertAlmostEqual(hh.loc[2, "flavor_HHI"], 5 / 9)
metrics = pd.read_csv(root / "out3/yogurt_dynamics.csv").set_index("metric")["value"]
self.assertEqual(metrics["Eligible adjacent single-flavor week pairs"], 1)
self.assertEqual(metrics["Switch rate in eligible pairs (%)"], 100)
self.assertEqual(metrics["Median gap between purchase weeks"], 1)
# Price filtering precedes the two-week household rule.
unpriced = source.with_columns(pl.when(
    ((pl.col("household_code") == 2) & (pl.col("purchase_date") == "2010-01-09"))
    | ((pl.col("household_code") == 1) & (pl.col("purchase_date") == "2010-01-09") & (pl.col("upc") == 1))
    ).then(None).otherwise(pl.col("price")).alias("price"))
unpriced.write_parquet(root / "yogurt_estimation_sample.parquet")
out, work = root / "filtered", root / "filtered_work"
out.mkdir()
work.mkdir()
args.household_batches = 2
summary.run(args, out, work)
filtered = pl.read_parquet(out / "yogurt_estimation_sample.parquet")
self.assertEqual(filtered.height, 3)
self.assertEqual(filtered["household_code"].unique().to_list(), [1])
self.assertTrue(filtered["price"].is_not_null().all())
self.assertEqual(pl.read_csv(out / "yogurt_price_coverage.csv")["purchase_lines_without_price_coverage"][0], 2)
# Sparse quantity output has one row per observed flavor/week, not
# one column for every flavor in the full reference.
quantities = pl.read_csv(root / "out3/yogurt_weekly_flavor_quantities.csv")
self.assertEqual(quantities.height, 6)
self.assertEqual(set(quantities.columns), {"household_code", "week", "flavor", "quantity"})
```

**Read this block:**

- Line 17: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 18, `root`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.

- Line 19, `source`: Set this name to the result of calling `pl.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 29: for each entry yielded by `['merged_panel', 'yogurt_estimation_sample']`, bind/unpack it into `name` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 30: call `source.write_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 31, `reference`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 32: call `reference.write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 33: for each entry yielded by `[1, 3]`, bind/unpack it into `batches` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 34, `(out, work)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 35: call `out.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 36: call `work.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 37, `args`: Set this name to the result of calling `SimpleNamespace(...)`; the arguments and preceding method chain are shown above.
- Line 39: call `summary.run` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 40: for each entry yielded by `[('full_panel_households', ['household_code']), ('matched_yogu...`, bind/unpack it into `(name, keys)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 46, `left`: Set this name to the result of calling `.reset_index(...)`; the arguments and preceding method chain are shown above.
- Line 47, `right`: Set this name to the result of calling `.reset_index(...)`; the arguments and preceding method chain are shown above.
- Line 48: call `pd.testing.assert_frame_equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 49, `prepared`: Set this name to the result of calling `pl.read_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 50: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 51: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 52: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 53: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 54: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 55: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 56, `hh`: Set this name to the result of calling `.set_index(...)`; the arguments and preceding method chain are shown above.
- Line 57: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 58: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 59: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 60, `metrics`: Set this name to the entry/slice selected by `pd.read_csv(root / 'out3/yogurt_dynamics.csv').set_index(...`; brackets select from the object to their left.
- Line 61: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 62: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 63: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 65, `unpriced`: Set this name to the result of calling `source.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 69: call `unpriced.write_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 70, `(out, work)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 71: call `out.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 72: call `work.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 73, `args.household_batches`: Set this name to the literal value `2`.
- Line 74: call `summary.run` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 75, `filtered`: Set this name to the result of calling `pl.read_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 76: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 77: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 78: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 79: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 82, `quantities`: Set this name to the result of calling `pl.read_csv(...)`; the arguments and preceding method chain are shown above.
- Line 83: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 84: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.
- `.redirect_stdout`: Temporarily capture printed output, keeping tests quiet without changing the tested computation.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `.read_parquet`: Read Parquet into an eager table, rather than retaining only a deferred plan.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `.assertTrue`: Fail unless the supplied condition is true; array conditions normally need any/all first.
- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.write_parquet`: Persist an eager table as Parquet, often used for small synthetic fixtures.
- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- `.run`: For subprocess.run, execute an argument-list command; check=True raises if its exit status is nonzero.
- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.
- `.StringIO`: Create an in-memory text buffer instead of writing captured output to a real file.
- `SimpleNamespace`: Construct a lightweight object with named attributes, useful for args-like synthetic test settings.
- `.reset_index`: Turn the index back into ordinary row numbering; drop=True discards the old index rather than saving it as a column.
- `.assert_frame_equal`: Compare pandas tables including values, labels, and dtype conventions rather than

only row counts.
- `set`: Build a collection of distinct values, used for column membership and missing-field checks.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.sort_values`: Order pandas rows by values; ascending can supply one direction for each listed sort key.
- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.otherwise`: Supply the fallback expression for rows not selected by the preceding condition.
- `.is_not_null`: Create a Polars nonnull mask; additional range checks can still be needed.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.then`: Supply the value/expression selected where the preceding Polars condition is true.
- `.when`: Start a Polars row-wise conditional expression, completed by then and otherwise.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.
- The | operator accepts either elementwise condition; Python's scalar or is generally unsuitable for an entire table column.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

**Output reminder:** reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

## MemorySummaryChecks.test demographics match previous fieldwise sort

Original function: `MemorySummaryChecks.test_demographics_match_previous_fieldwise_sort`. Source begins at line 86.

**What this test establishes.** Compares the optimized shared demographic-history sort to separate field-by-field selection on a synthetic example. This checks that a memory/runtime improvement preserves earliest-nonmissing semantics.

**Reading its syntax.** This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

**Definition and stated intent:**

Source lines 86–88:

```
86 def test_demographics_match_previous_fieldwise_sort(self):
87     # Unsorted histories, missing fields, conflicting same-date values and
88     # repeated purchase lines must preserve the earliest nonmissing values.
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 89–103:

```
89 source = pl.DataFrame({
90     "household_code": [1, 2, 1, 1, 1, 2],
91     "panel_year": [2011, 2010, 2010, 2010, 2010, 2011],
92     "purchase_date": ["2011-01-01", "2010-01-02", "2010-01-02", "2010-01-02", "2010-01-02", "2011-01-01"],
93     "trip_code_uc": [1, 2, 3, 3, 3, 4], "upc": [1, 2, 3, 3, 3, 4],
94     "quantity": [1.] * 6,
95 }).with_columns(pl.col("purchase_date").str.to_date())
96 fields = ["race", "income", "hispanic_origin", "household_size", "head_age",
97           "female_head_age", "male_head_age", "dma_cd"]
98 source = source.with_columns([pl.Series(c, ["9", None, None, "2", "3", "7"]) for c in fields])
99 expected = source.lazy().group_by("household_code").agg([
100     pl.col(c).sort_by(["panel_year", "purchase_date"]).drop_nulls().first().alias(c)
101     for c in fields
```

```
]).collect().sort("household_code")
actual = summary.household_summary(source.lazy()).collect().sort("household_code")
```

**Read this block:**

- Line 89, `source`: Set this name to the result of calling `.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 96, `fields`: Set this name to a list containing 8 displayed entries.
- Line 98, `source`: Set this name to the result of calling `source.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 99, `expected`: Set this name to the result of calling `.sort(...)`; the arguments and preceding method chain are shown above.
- Line 103, `actual`: Set this name to the result of calling `.sort(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.to_date`: Parse text into calendar-date values before weekday extraction and date arithmetic.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.lazy`: Wrap an eager Polars table as a lazy query so later operations can be planned together.
- `.first`: Select the first value in the current expression/group; preceding ordering and null filtering determine which one.
- `.drop_nulls`: Remove null values from the Polars expression before selecting a first observed value.
- `.sort_by`: Order values within an expression/group by another column before selecting a first value.

Source lines 104–105:

```
self.assertEqual(actual.select(["household_code"] + fields).to_dicts(), expected.to_dicts())
self.assertEqual(actual["purchase_lines"].to_list(), [4, 2])
```

**Read this block:**

- Line 104: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 105: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.

Source lines 106–113:

```
with tempfile.TemporaryDirectory() as directory:
    parts = summary.partition_purchases(source.lazy(), Path(directory), "check", 16)
    restored = pl.concat(parts).collect()
    self.assertEqual(restored.height, source.height)
    counts = restored.group_by("household_code").len().sort("household_code")
    self.assertEqual(counts["len"].to_list(), [4, 2])
    batched = pl.concat([summary.household_summary(part) for part in parts]).collect().sort("household_code")
    self.assertEqual(batched.to_dicts(), actual.to_dicts())
```

**Read this block:**

- Line 106: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 107, `parts`: Set this name to the result of calling `summary.partition_purchases(...)`; the arguments and preceding method chain are shown above.
- Line 108, `restored`: Set this name to the result of calling `.collect(...)`; the arguments and preceding method chain are shown above.
- Line 109: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 110, `counts`: Set this name to the result of calling `.sort(...)`; the arguments and preceding method chain are shown above.
- Line 111: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 112, `batched`: Set this name to the result of calling `.sort(...)`; the arguments and preceding method chain are shown above.
- Line 113: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

**Output reminder:** reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

## MemorySummaryChecks.test recovery preserves existing labels and uses earliest year

Original function: `MemorySummaryChecks.test_recovery_preserves_existing_labels_and_uses_earliest_year`. Source begins at line 115.

**What this test establishes.** Constructs reference and annual metadata cases to check that existing labels are retained and absent UPCs take earliest usable metadata. It verifies a substantive recovery precedence rule.

**Reading its syntax.** This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

**Definition and stated intent:**

Source lines 115–115:

```
115 def test_recovery_preserves_existing_labels_and_uses_earliest_year(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 116–125:

```
116 with tempfile.TemporaryDirectory() as directory, contextlib.redirect_stdout(io.StringIO()):
117     root = Path(directory)
118     ref, extra, output = root / "ref.csv", root / "extra.parquet", root / "result.csv"
119     ref.write_text("upc,flavor_descr\n001,vanilla\n002,NA\n")
120     pl.DataFrame({"upc": [1, 2, 3, 3, 4], "flavor_descr": ["plain", "berry", "cherry", "strawberry", None],
121                   "panel_year": [2006, 2006, 2007, 2006, 2006]}).write_parquet(extra)
122     summary.recover_flavor_reference(ref, extra, output)
123     result = summary.flavor_reference(output, detailed=True).sort("upc")
```

```
124     self.assertEqual(result.to_dict(as_series=False),
125                      {"upc": ["1", "2", "3"], "flavor": ["vanilla", "unknown", "strawberry"]})
```

**Read this block:**

- Line 116: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 117, `root`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 118, `(ref, extra, output)`: Unpack a tuple containing 3 displayed entries into these targets in order.
- Line 119: call `ref.write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 120: call `.write_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 122: call `summary.recover_flavor_reference` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 123, `result`: Set this name to the result of calling `.sort(...)`; the arguments and preceding method chain are shown above.
- Line 124: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.
- `.redirect_stdout`: Temporarily capture printed output, keeping tests quiet without changing the tested computation.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `.write_parquet`: Persist an eager table as Parquet, often used for small synthetic fixtures.
- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `.StringIO`: Create an in-memory text buffer instead of writing captured output to a real file.
- `.to_dict`: Convert table contents to a Python dictionary; output orientation/options determine its nested layout.

**Output reminder:** reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

# est_code/test_est.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

## Module setup and direct-execution blocks

Source lines 1–6:

```
1 """Small checks that need no Nielsen data: python3 -m unittest -v test_est.py."""
2 import unittest
3 import tempfile
4 from pathlib import Path
5 from types import SimpleNamespace
6 import polars as pl
```

**Read this block:**

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 3: import `tempfile`. An as-name is a convenient local alias, not another library.
- Line 4: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 5: from `types`, import `SimpleNamespace` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 6: import `polars as pl`. An as-name is a convenient local alias, not another library.

Source lines 7–10:

```
7 import numpy as np
8 import pandas as pd
9 from scipy.special import logsumexp
10 from est import read_flavors, prepare_data, fit_model, add_stocks, make_arrays, household_loglike, make_draws,
   ↪  parameter_names, demand_objective_gradient
```

**Read this block:**

- Line 7: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 8: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 9: from `scipy.special`, import `logsumexp` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 10: from `est`, import `read_flavors, prepare_data, fit_model, add_stocks, make_arrays, household_loglike, make_draws, parameter_names, demand_objective_gradient` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 13–13:

```
13 class EstimationChecks(unittest.TestCase):
```

**Read this block:**

- Line 13: define class `EstimationChecks` and its methods. TestCase inheritance provides the self.assert… checking methods.

Source lines 196–197:

```
196 if __name__ == "__main__":
197     unittest.main()
```

**Read this block:**

- Line 196: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 197: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

## EstimationChecks.frame

Original function: `EstimationChecks.frame`. Source begins at line 14.

This fixture method returns a tiny three-week pandas table. It includes a two-week gap, mixed counts, all three prices, and outside counts so history, menu, and likelihood tests can reuse the same explicit example. `self.frame()` returns a fresh table each time, letting one test modify it without contaminating another. A shared global mutable table would make results depend on test order.

**Definition and stated intent:**

Source lines 14–14:

```
14 def frame(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 15–20:

```
15 return pd.DataFrame({
16     "household_code": [10, 10, 10],
17     "week": pd.to_datetime(["2010-01-02", "2010-01-09", "2010-01-23"]),
18     "other": [2., 0., 1.], "berry": [0., 2., 1.], "plain": [0., 0., 0.],
19     "price_other": [1., 1.2, 1.1], "price_berry": [1.2, 1., 1.3],
20     "price_plain": [1.5, 1.4, 1.3], "outside": [5., 5., 5.]})
```

**Read this block:**

- Line 15: return the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above. This ends the call.

**Syntax and calls used here:**

- `.to_datetime`: Parse/convert dates; an explicit format avoids interpreting numeric YYYYMMDD values as timestamp offsets.

## EstimationChecks.test history excludes current counts and accumulates all occasions

Original function: `EstimationChecks.test_history_excludes_current_counts_and_accumulates_all_occasions`. Source begins at line 22.

**What this test establishes.** Checks pre-choice timing: changing a current basket must not rewrite the stock already available for that choice. It compares explicit stock vectors, not just output dimensions.

**Reading its syntax.** This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

**Definition and stated intent:**

Source lines 22–22:

```
22 def test_history_excludes_current_counts_and_accumulates_all_occasions(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 23–28:

```
23 frame = add_stocks(self.frame())
24 stock = frame[["stock_other", "stock_berry", "stock_plain"]]
25 np.testing.assert_allclose(stock.iloc[0], [0., 0., 0.])
26 np.testing.assert_allclose(stock.iloc[1], [1., 0., 0.])
27 np.testing.assert_allclose(stock.iloc[2], [1., 1., 0.])
28 changed = self.frame()
```

**Read this block:**

- Line 23, `frame`: Set this name to the result of calling `add_stocks(...)`; the arguments and preceding method chain are shown above.
- Line 24, `stock`: Set this name to the entry/slice selected by `frame[['stock_other', 'stock_berry', 'stock_plain']]`; brackets select from the object to their left.

- Line 25: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 26: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 27: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 28, `changed`: Set this name to the result of calling `self.frame(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `add_stocks`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.

Source lines 29–31:

```
29 changed.loc[2, "plain"] = 100
30 np.testing.assert_allclose(add_stocks(changed).filter(like="stock_"),
31                            frame.filter(like="stock_"))
```

**Read this block:**

- Line 29, `changed.loc[2, 'plain']`: Store the literal value `100` in the selected entry/column.
- Line 30: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.

**Output reminder:** reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

## EstimationChecks.test xi sign and unlimited history

Original function: `EstimationChecks.test_xi_sign_and_unlimited_history`. Source begins at line 33.

**What this test establishes.** Checks numerical Xi signs and persistence across long gaps. The explicit -.7/.3 expectations distinguish own-stock penalties from recent-other rewards.

**Reading its syntax.** This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

**Definition and stated intent:**

Source lines 33–33:

```
33 def test_xi_sign_and_unlimited_history(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 34–40:

```
34 frame = add_stocks(self.frame(), .7)
35 self.assertAlmostEqual(frame.loc[1, "xi_other"], -.7)
36 self.assertAlmostEqual(frame.loc[1, "xi_berry"], .3)
37 # Time gaps do not change occasion-time histories when delta=1.
```

```
old = self.frame()
old.loc[2, "week"] = pd.Timestamp("2011-04-03")
old = add_stocks(old)
```

**Read this block:**

- Line 34, `frame`: Set this name to the result of calling `add_stocks(...)`; the arguments and preceding method chain are shown above.
- Line 35: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 36: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 38, `old`: Set this name to the result of calling `self.frame(...)`; the arguments and preceding method chain are shown above.
- Line 39, `old.loc[2, 'week']`: Store the result of calling `pd.Timestamp(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 40, `old`: Set this name to the result of calling `add_stocks(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `add_stocks`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.

Source lines 41–43:

```
np.testing.assert_allclose(old.filter(like="xi_"), frame.filter(like="xi_"))
self.assertEqual(frame.loc[2, "novelty_other"], 0.)
self.assertAlmostEqual(frame.loc[2, "xi_other"], -.7)
```

**Read this block:**

- Line 41: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 42: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 43: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.

**Output reminder:** reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

## EstimationChecks.test parquet pipeline and fit

Original function: `EstimationChecks.test_parquet_pipeline_and_fit`. Source begins at line 45.

**What this test establishes.** Builds a tiny real Parquet input and runs preparation plus a deliberately short fit. It checks all-store counts, exposure-store menus, missing alternatives, exported files, and own-DMA exclusion. Its low iteration limit is a smoke check, not proof of statistical convergence.

**Reading its syntax.** This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results,

and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

**Definition and stated intent:**

Source lines 45–45:

```
def test_parquet_pipeline_and_fit(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 46–87:

```
with tempfile.TemporaryDirectory() as temp:
    root = Path(temp)
    rows = []
    def append(household, week, upc, store, dma, price, quantity=1):
        rows.append(dict(household_code=household, panel_year=2010,
                         purchase_date=str(pd.to_datetime(str(week), format="%Y%m%d").date()),
                         trip_code_uc=len(rows) + 1, upc=str(upc), store_code_uc=store,
                         quantity=quantity, price=float(price), dma_cd=dma,
                         flavor={1: "other", 2: "berry", 3: "plain"}[upc]))
    append(10, 20100102, 1, 100, 1, 3, 2)
    append(10, 20100102, 2, 200, 1, 4)
    append(10, 20100109, 2, 100, 1, 4, 2)
    for week in [20100102, 20100109]:
        for upc in [1, 2, 3]:
            append(20, week, upc, 200, 2, 2 + upc)
            append(30, week, upc, 300, 3, 6 + upc)
            append(40, week, upc, 100, 1, 2 + upc)
    # Repeat a cheap quote: it must not overweight the overall IV.
    append(20, 20100102, 1, 200, 2, 3)
    sample = root / "prepared.parquet"
    original = pl.DataFrame(rows)
    original.write_parquet(sample)
    args = SimpleNamespace(
        sample=sample, flavors=root / "not_needed.csv", rms=root / "does_not_exist.parquet",
        first_year=2010, last_year=2010, max_households=1, habit_weight=.7,
        capacity=0, no_price_control=True, household_dma_column="dma_cd", se_method="none",
        model="heterogeneous", draws=8, starts=1, maxiter=5, seed=306, output=root / "output")
    frame = prepare_data(args)
    self.assertEqual(len(frame), 2)
    self.assertEqual(frame.household_code.unique().tolist(), [10])
    self.assertEqual(frame.loc[0, "other"], 2)
    self.assertEqual(frame.loc[0, "berry"], 1)
    self.assertEqual(frame.loc[0, "store_code_uc"], 100)
    self.assertEqual(frame.loc[0, "price_other"], 3)
    np.testing.assert_allclose(frame.filter(like="stock_").iloc[0], 0)
    fit = fit_model(make_arrays(frame, 0), args)
    self.assertTrue(np.isfinite(fit.fun))
    self.assertTrue((args.output / "fit.json").exists())
    self.assertTrue((args.output / "gamma_distribution.pdf").exists())
    incomplete = original.filter(~((pl.col("purchase_date") == "2010-01-02")
                                   & (pl.col("store_code_uc") == 100) & (pl.col("upc") == "3")))
    incomplete.write_parquet(sample)
```

Source lines 88–107:

```
partial = prepare_data(args)
self.assertEqual(len(partial), 2)
self.assertFalse(partial.loc[0, "available_plain"])
```

```
self.assertTrue(np.isnan(partial.loc[0, "price_plain"]))
original.write_parquet(sample)
args.no_price_control = False
controlled = prepare_data(args)
# Pool DMA 2's quotes 3,4,5 and DMA 3's 7,8,9; mean = 6.
for name in ["iv_overall", "iv_other", "iv_berry", "iv_plain"]:
    np.testing.assert_allclose(controlled[name], 6.)
# Changing every own-DMA quote cannot change the instrument.
changed = original.with_columns(pl.when(pl.col("dma_cd") == 1)
                                .then(pl.col("price") + 100).otherwise(pl.col("price")).alias("price"))
changed.write_parquet(sample)
# Store 200 is shared across DMAs in this fixture. Update both copies
# of its quoted price to keep local store/week/UPC prices consistent.
changed = changed.with_columns(pl.when(pl.col("store_code_uc") == 200)
                               .then(pl.col("upc").cast(pl.Float64) + 2).otherwise(pl.col("price")).alias("price"))
changed.write_parquet(sample)
np.testing.assert_allclose(prepare_data(args)["iv_overall"], 6.)
```

**Read this block:**

- Line 46: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 47, `root`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 48, `rows`: Set this name to the list `[]`.
- Line 49: define nested callable `append`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.
- Line 55: call `append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 56: call `append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 57: call `append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 58: for each entry yielded by `[20100102, 20100109]`, bind/unpack it into `week` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 59: for each entry yielded by `[1, 2, 3]`, bind/unpack it into `upc` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 60: call `append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 61: call `append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 62: call `append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 64: call `append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 65, `sample`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 66, `original`: Set this name to the result of calling `pl.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 67: call `original.write_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 68, `args`: Set this name to the result of calling `SimpleNamespace(...)`; the arguments and preceding method chain are shown above.
- Line 73, `frame`: Set this name to the result of calling `prepare_data(...)`; the arguments and preceding method chain are shown above.
- Line 74: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 75: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 76: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 77: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 78: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 79: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 80: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 81, `fit`: Set this name to the result of calling `fit_model(...)`; the arguments and preceding method chain are shown above.
- Line 82: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 83: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 84: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 85, `incomplete`: Set this name to the result of calling `original.filter(...)`; the arguments and preceding method chain are shown above.
- Line 87: call `incomplete.write_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 88, `partial`: Set this name to the result of calling `prepare_data(...)`; the arguments and preceding method chain are shown above.
- Line 89: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 90: call `self.assertFalse` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 91: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 92: call `original.write_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 93, `args.no_price_control`: Set this name to the literal value `False`.
- Line 94, `controlled`: Set this name to the result of calling `prepare_data(...)`; the arguments and preceding method chain are shown above.
- Line 96: for each entry yielded by `['iv_overall', 'iv_other', 'iv_berry', 'iv_plain']`, bind/unpack it into `name` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 97: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 99, `changed`: Set this name to the result of calling `original.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 101: call `changed.write_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 104, `changed`: Set this name to the result of calling `changed.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 106: call `changed.write_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 107: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

- `append`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.write_parquet`: Persist an eager table as Parquet, often used for small synthetic fixtures.
- `SimpleNamespace`: Construct a lightweight object with named attributes, useful for args-like synthetic test settings.
- `prepare_data`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.
- `fit_model`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assertTrue`: Fail unless the supplied condition is true; array conditions normally need any/all first.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.assertFalse`: Fail unless the supplied condition is false, checking an expected absence/invalidity.
- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `len`: Count elements/rows rather than summing their numerical values.
- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.
- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- `.isnan`: Identify floating-point NaNs; NaN must not be tested with ordinary equality.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.otherwise`: Supply the fallback expression for rows not selected by the preceding condition.
- `str`: Convert to text for paths, messages, or serialization.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.then`: Supply the value/expression selected where the preceding Polars condition is true.
- `.to_datetime`: Parse/convert dates; an explicit format avoids interpreting numeric YYYYMMDD values as timestamp offsets.
- `.when`: Start a Polars row-wise conditional expression, completed by then and otherwise.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

**Output reminder:** reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

## EstimationChecks.test parquet pipeline and fit.append

Original function: `EstimationChecks.test_parquet_pipeline_and_fit.append`. Source begins at line 49.

This nested fixture builder appends one purchase dictionary to an enclosing `rows` list. Positional inputs describe household/week/UPC/store/DMA/price; quantity defaults to one. A dictionary lookup turns UPC 1/2/3 into the three flavor names. `len(rows)+1` assigns a fresh trip ID at append time. The helper reduces repetitive fixture text while keeping test prices and households explicit. It mutates the captured list and returns None; callers need not assign its result.

**Definition and stated intent:**

Source lines 49–49:

```
49 def append(household, week, upc, store, dma, price, quantity=1):
```

`def` creates this callable; the parentheses name its inputs (`household`, `week`, `upc`, `store`, `dma`, `price`, `quantity`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 50–54:

```
50 rows.append(dict(household_code=household, panel_year=2010,
51                  purchase_date=str(pd.to_datetime(str(week), format="%Y%m%d").date()),
52                  trip_code_uc=len(rows) + 1, upc=str(upc), store_code_uc=store,
53                  quantity=quantity, price=float(price), dma_cd=dma,
54                  flavor={1: "other", 2: "berry", 3: "plain"}[upc]))
```

**Read this block:**

- Line 50: call `rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `str`: Convert to text for paths, messages, or serialization.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `len`: Count elements/rows rather than summing their numerical values.
- `.to_datetime`: Parse/convert dates; an explicit format avoids interpreting numeric YYYYMMDD values as timestamp offsets.

**Output reminder:** reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

## EstimationChecks.test unavailable flavor is excluded from logit denominator

Original function: `EstimationChecks.test_unavailable_flavor_is_excluded_from_logit_denominator`. Source begins at line 109.

**What this test establishes.** Removes plain prices and independently computes the remaining two-flavor likelihood. It also requires a ValueError if an unavailable flavor is actually purchased. This catches treating a missing-price placeholder as an available free good.

**Reading its syntax.** This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

**Definition and stated intent:**

Source lines 109–109:

```
109 def test_unavailable_flavor_is_excluded_from_logit_denominator(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 110–115:

```
110 frame = self.frame()
111 frame["price_plain"] = np.nan
112 data = make_arrays(add_stocks(frame), 0)
113 theta = np.array([.1, -.2, -.4])
114 actual = household_loglike(theta, data, make_draws(8, 306), "naive")[0]
115 utility = np.array([0., .1]) - np.exp(-.4) * frame[["price_other", "price_berry"]].to_numpy()
```

**Read this block:**

- Line 110, `frame`: Set this name to the result of calling `self.frame(...)`; the arguments and preceding method chain are shown above.
- Line 111, `frame['price_plain']`: Store the attribute `np.nan` of the object before the dot in the selected entry/column.
- Line 112, `data`: Set this name to the result of calling `make_arrays(...)`; the arguments and preceding method chain are shown above.
- Line 113, `theta`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 114, `actual`: Set this name to the entry/slice selected by `household_loglike(theta, data, make_draws(8, 306), 'naive...`; brackets select from the object to their left.
- Line 115, `utility`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.

**Syntax and calls used here:**

- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `add_stocks`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `household_loglike`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `make_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.

Source lines 116–119:

```
116 expected = (frame[["other", "berry"]].to_numpy() * (utility - logsumexp(utility, axis=1, keepdims=True))).sum()
117 self.assertAlmostEqual(actual, expected)
118 self.assertFalse(data["available"][:, 2].any())
119 frame.loc[0, "plain"] = 1
```

**Read this block:**

- Line 116, `expected`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.
- Line 117: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 118: call `self.assertFalse` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 119, `frame.loc[0, 'plain']`: Store the literal value `1` in the selected entry/column.

**Syntax and calls used here:**

- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `logsumexp`: Compute log(sum(exp(values))) stably; axis selects alternatives, draws, or another reduction dimension.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `.assertFalse`: Fail unless the supplied condition is false, checking an expected absence/invalidity.
- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.

Source lines 120–121:

```
120 with self.assertRaisesRegex(ValueError, "Purchased flavors"):
121     make_arrays(add_stocks(frame), 0)
```

**Read this block:**

- Line 120: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 121: call `make_arrays` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.assertRaisesRegex`: Expect a particular exception and matching error text inside the with block; no exception is a test failure.

**Output reminder:** reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

## EstimationChecks.test persistent household integration

Original function: `EstimationChecks.test_persistent_household_integration`. Source begins at line 123.

**What this test establishes.** Independently computes a draw's full household history before averaging draws. This catches a subtle wrong model that integrates each week separately.

**Reading its syntax.** This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

**Definition and stated intent:**

Source lines 123–123:

```
123 def test_persistent_household_integration(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 124–130:

```
124 data = make_arrays(add_stocks(self.frame()), 7)
125 draws = make_draws(8, 306)
126 names = parameter_names("heterogeneous", True)
127 theta = np.array([.4 if n.startswith("sd_") else .2 for n in names])
128 actual = household_loglike(theta, data, draws, "heterogeneous")[0]
129 # Independently calculate each fixed type's full household history.
130 by_draw = []
```

**Read this block:**

- Line 124, `data`: Set this name to the result of calling `make_arrays(...)`; the arguments and preceding method chain are shown above.
- Line 125, `draws`: Set this name to the result of calling `make_draws(...)`; the arguments and preceding method chain are shown above.
- Line 126, `names`: Set this name to the result of calling `parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 127, `theta`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 128, `actual`: Set this name to the entry/slice selected by `household_loglike(theta, data, draws, 'heterogeneous')[0]`; brackets select from the object to their left.

- Line 130, `by_draw`: Set this name to the list `[]`.

**Syntax and calls used here:**

- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `add_stocks`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `make_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- `household_loglike`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 131–137:

```
131 for z in draws:
132     alpha = -np.exp(.2 + .4*z[0])
133     gamma = .2 + .4*z[1]
134     utility = np.array([.2, .4, .4]) + alpha*data["prices"] + gamma*data["xi"]
135     utility = np.column_stack([utility, np.zeros(3)])
136     logp = utility - logsumexp(utility, axis=1, keepdims=True)
137     by_draw.append(np.sum(data["counts"]*logp))
```

**Read this block:**

- Line 131: for each entry yielded by `draws`, bind/unpack it into `z` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 132, `alpha`: Set this name to the negated or sign-adjusted expression `-np.exp(0.2 + 0.4 * z[0])`.
- Line 133, `gamma`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 134, `utility`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 135, `utility`: Set this name to the result of calling `np.column_stack(...)`; the arguments and preceding method chain are shown above.
- Line 136, `logp`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 137: call `by_draw.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.column_stack`: Place one-dimensional vectors beside each other as columns of a matrix.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `logsumexp`: Compute log(sum(exp(values))) stably; axis selects alternatives, draws, or another reduction dimension.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.

Source lines 138–138:

```
138 self.assertAlmostEqual(actual, logsumexp(by_draw)-np.log(8))
```

**Read this block:**

- Line 138: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.

**Output reminder:** reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

## EstimationChecks.test analytic gradient

Original function: `EstimationChecks.test_analytic_gradient`. Source begins at line 140.

**What this test establishes.** Perturbs each coefficient and compares centered finite-difference objective slopes with the supplied analytic gradient. This checks derivative values rather than just whether optimization returns a result.

**Reading its syntax.** This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

**Definition and stated intent:**

Source lines 140–140:

```
140 def test_analytic_gradient(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 141–141:

```
141 from scipy.optimize._numdiff import approx_derivative
```

**Read this block:**

- Line 141: from `scipy.optimize._numdiff`, import `approx_derivative` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 142–152:

```
142 for capacity in [0, 7]:
143     frame = add_stocks(self.frame())
144     frame["cf_other"], frame["cf_berry"], frame["cf_plain"] = .1, -.2, .3
145     data = make_arrays(frame, capacity)
146     for model in ["naive", "homogeneous", "mixed_naive", "heterogeneous"]:
147         theta = np.full(len(parameter_names(model, bool(capacity), True)), .2)
148         draws = make_draws(8, 306)
149         value, gradient = demand_objective_gradient(theta, data, draws, model)
150         objective = lambda t: -household_loglike(t, data, draws, model).mean()
151         self.assertAlmostEqual(value, objective(theta))
152         np.testing.assert_allclose(gradient, approx_derivative(objective, theta).ravel(), rtol=1e-5, atol=1e-7)
```

**Read this block:**

- Line 142: for each entry yielded by `[0, 7]`, bind/unpack it into `capacity` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 143, `frame`: Set this name to the result of calling `add_stocks(...)`; the arguments and preceding method chain are shown above.
- Line 144, `(frame['cf_other'], frame['cf_berry'], frame['cf_plain'])`: Unpack a tuple containing 3 displayed entries into these targets in order.
- Line 145, `data`: Set this name to the result of calling `make_arrays(...)`; the arguments and preceding method chain are shown above.
- Line 146: for each entry yielded by `['naive', 'homogeneous', 'mixed_naive', 'heterogeneous']`, bind/unpack it into `model` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 147, `theta`: Set this name to the result of calling `np.full(...)`; the arguments and preceding method chain are shown above.
- Line 148, `draws`: Set this name to the result of calling `make_draws(...)`; the arguments and preceding method chain are shown above.
- Line 149, `(value, gradient)`: Unpack the result of calling `demand_objective_gradient(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 150, `objective`: Set this name to a one-expression callable, with input `t` and returned expression `-household_loglike(t, data, draws, model).mean()`.
- Line 151: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 152: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `add_stocks`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.full`: Allocate an array filled with one value, such as equal histogram weights.
- `make_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `demand_objective_gradient`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.
- `len`: Count elements/rows rather than summing their numerical values.
- `objective`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.
- `bool`: Convert scalar truthiness to an explicit Boolean flag.
- `approx_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `household_loglike`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.

**Output reminder:** reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

## EstimationChecks.test four model specifications

Original function: `EstimationChecks.test_four_model_specifications`. Source begins at line 154.

**What this test establishes.** Checks parameter layouts/likelihood behavior for the four supported specifications. The labels define vector meaning; silently changed ordering would corrupt estimates even when vector lengths still match.

**Reading its syntax.** This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

**Definition and stated intent:**

Source lines 154–154:

```
154 def test_four_model_specifications(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 155–166:

```
155 for model, gamma, alpha_sd, gamma_sd in [
156     ("naive", False, False, False),
157     ("homogeneous", True, False, False),
158     ("mixed_naive", False, True, False),
159     ("heterogeneous", True, True, True),
160 ]:
161     with self.subTest(model=model):
162         names = parameter_names(model, False)
163         self.assertEqual("mean_gamma" in names, gamma)
164         self.assertEqual("sd_log_minus_alpha" in names, alpha_sd)
165         self.assertEqual("sd_gamma" in names, gamma_sd)
166         self.assertFalse(any("sd_beta" in n for n in names))
```

**Read this block:**

- Line 155: for each entry yielded by `[('naive', False, False, False), ('homogeneous', True, False, ...`, bind/unpack it into `(model, gamma, alpha_sd, gamma_sd)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 161: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 162, `names`: Set this name to the result of calling `parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 163: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 164: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 165: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 166: call `self.assertFalse` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `.assertFalse`: Fail unless the supplied condition is false, checking an expected absence/invalidity.

- `any`: Require at least one condition to be true; this reduces a sequence to one Boolean.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

**Output reminder:** reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

## EstimationChecks.test mixed naive integrates only alpha

Original function: `EstimationChecks.test_mixed_naive_integrates_only_alpha`. Source begins at line 168.

**What this test establishes.** Checks that mixed-naive choice probabilities vary with alpha draws while excluding a gamma effect. It independently constructs the corresponding integrated likelihood.

**Reading its syntax.** This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

**Definition and stated intent:**

Source lines 168–168:

```
168 def test_mixed_naive_integrates_only_alpha(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 169–173:

```
169 data = make_arrays(add_stocks(self.frame()), 0)
170 draws = make_draws(8, 306)
171 theta = np.array([.1, -.2, -.4, .6])
172 actual = household_loglike(theta, data, draws, "mixed_naive")
173 independent = []
```

**Read this block:**

- Line 169, `data`: Set this name to the result of calling `make_arrays(...)`; the arguments and preceding method chain are shown above.
- Line 170, `draws`: Set this name to the result of calling `make_draws(...)`; the arguments and preceding method chain are shown above.
- Line 171, `theta`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 172, `actual`: Set this name to the result of calling `household_loglike(...)`; the arguments and preceding method chain are shown above.
- Line 173, `independent`: Set this name to the list `[]`.

**Syntax and calls used here:**

- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `add_stocks`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `make_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `household_loglike`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 174–178:

```
174 for z in draws:
175     alpha = -np.exp(-.4 + .6*z[0])
176     utility = np.array([0., .1, -.2]) + alpha*data["prices"]
177     logp = utility - logsumexp(utility, axis=1, keepdims=True)
178     independent.append(np.sum(data["counts"]*logp))
```

**Read this block:**

- Line 174: for each entry yielded by `draws`, bind/unpack it into `z` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 175, `alpha`: Set this name to the negated or sign-adjusted expression `-np.exp(-0.4 + 0.6 * z[0])`.
- Line 176, `utility`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 177, `logp`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 178: call `independent.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `logsumexp`: Compute log(sum(exp(values))) stably; axis selects alternatives, draws, or another reduction dimension.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.

Source lines 179–184:

```
179 np.testing.assert_allclose(actual, [logsumexp(independent)-np.log(len(draws))])
180 changed = dict(data, xi=data["xi"] + 100.)
181 np.testing.assert_allclose(actual, household_loglike(theta, changed, draws, "mixed_naive"))
182 np.testing.assert_allclose(
183     household_loglike(theta[:3], data, draws, "naive"),
184     household_loglike(np.r_[theta[:3], 0.], data, draws, "mixed_naive"))
```

**Read this block:**

- Line 179: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 180, `changed`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 181: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 182: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.
- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.
- `len`: Count elements/rows rather than summing their numerical values.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

**Output reminder:** reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

## EstimationChecks.test zero heterogeneity nests homogeneous model

Original function: `EstimationChecks.test_zero_heterogeneity_nests_homogeneous_model`. Source begins at line 186.

**What this test establishes.** Sets random-coefficient SDs to zero and compares against the common-coefficient model. A nesting identity is a strong check on parameter transforms and integration.

**Reading its syntax.** This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

**Definition and stated intent:**

Source lines 186–186:

```
186 def test_zero_heterogeneity_nests_homogeneous_model(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 187–193:

```
187 data = make_arrays(add_stocks(self.frame()), 0)
188 fixed = np.array([.1, -.2, -.4, .3])
189 random = np.r_[fixed, np.zeros(2)]
190 draws = make_draws(8, 306)
191 np.testing.assert_allclose(
192     household_loglike(fixed, data, draws, "homogeneous"),
193     household_loglike(random, data, draws, "heterogeneous"))
```

**Read this block:**

- Line 187, `data`: Set this name to the result of calling `make_arrays(...)`; the arguments and preceding method chain are shown above.
- Line 188, `fixed`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 189, `random`: Set this name to the entry/slice selected by `np.r_[fixed, np.zeros(2)]`; brackets select from the object to their left.
- Line 190, `draws`: Set this name to the result of calling `make_draws(...)`; the arguments and preceding method chain are shown above.
- Line 191: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `add_stocks`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `make_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.
- `household_loglike`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

**Output reminder:** reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

# est_code/test_price_control.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

## Module setup and direct-execution blocks

Source lines 1–6:

```
1 """Checks for DMA exclusion, first-stage OLS, and two-stage inference."""
2 import contextlib
3 import io
4 import tempfile
5 import unittest
6 from pathlib import Path
```

**Read this block:**

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `contextlib`. An as-name is a convenient local alias, not another library.
- Line 3: import `io`. An as-name is a convenient local alias, not another library.
- Line 4: import `tempfile`. An as-name is a convenient local alias, not another library.
- Line 5: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 6: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 7–14:

```
7 from types import SimpleNamespace
8
9 import numpy as np
10 import pandas as pd
11 from scipy.optimize import OptimizeResult
12 from unittest.mock import patch
13
14 from price_control import other_dma_instruments, absorb_effects, add_price_controls
```

**Read this block:**

- Line 7: from `types`, import `SimpleNamespace` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 9: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 10: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 11: from `scipy.optimize`, import `OptimizeResult` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 12: from `unittest.mock`, import `patch` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 14: from `price_control`, import `other_dma_instruments, absorb_effects, add_price_controls` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 15–16:

```
15 from bootstrap_se import resample_households, bootstrap_standard_errors
16 from est import FLAVORS, make_arrays, optimize_demand
```

**Read this block:**

- Line 15: from `bootstrap_se`, import `resample_households, bootstrap_standard_errors` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 16: from `est`, import `FLAVORS, make_arrays, optimize_demand` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 43–43:

```
43 class PriceControlChecks(unittest.TestCase):
```

**Read this block:**

- Line 43: define class `PriceControlChecks` and its methods. TestCase inheritance provides the self.assert… checking methods.

Source lines 136–137:

```
136 if __name__ == "__main__":
137     unittest.main()
```

**Read this block:**

- Line 136: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 137: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

## synthetic panel

Original function: `synthetic_panel`. Source begins at line 19.

This constructs a reproducible simulated panel with 60 households and eight periods, first-stage prices, residuals, utilities, and multinomial package counts. `repeat` holds each household ID for eight rows; `tile` repeats the eight-date sequence for all households. Stable softmax subtracts each row's largest utility before exponentiating. `rng.multinomial(8,p)` samples an eight-package vector whose entries sum to eight. This gives the tests known structure without private data; it is a test fixture, not estimated reality.

**Definition and stated intent:**

Source lines 19–19:

```
19 def synthetic_panel():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 20–25:

```
20 rng = np.random.default_rng(120)
21 n_households, periods = 60, 8
22 size = n_households * periods
23 frame = pd.DataFrame({"household_code": np.repeat(np.arange(n_households), periods),
24                       "week": np.tile(pd.date_range("2010-01-02", periods=periods, freq="7D"), n_households),
25                       "dma_cd": np.repeat(np.arange(n_households) % 4 + 1, periods)})
```

**Read this block:**

- Line 20, `rng`: Set this name to the result of calling `np.random.default_rng(...)`; the arguments and preceding method chain are shown above.
- Line 21, `(n_households, periods)`: Unpack the tuple `(60, 8)` into these targets in order.
- Line 22, `size`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 23, `frame`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.default_rng`: Create a local seeded random Generator, avoiding dependence on a shared global random stream.
- `.repeat`: Repeat each element consecutively, e.g. each household ID across all its periods.
- `.tile`: Repeat a complete sequence, e.g. the period-date sequence for every household.
- `.arange`: Create evenly stepped values with an excluded stop, commonly row positions 0 through n-1.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.

Source lines 26–30:

```
26 for flavor in FLAVORS:
27     frame["xi_" + flavor] = rng.normal(size=size)
28     frame["iv_" + flavor] = rng.uniform(.5, 2.5, size=size)
29     frame["price_" + flavor] = (1 + .8*frame["iv_" + flavor]
30                                 + .1*frame["xi_" + flavor] + rng.normal(0, .2, size))
```

**Read this block:**

- Line 26: for each entry yielded by `FLAVORS`, bind/unpack it into `flavor` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 27, `frame['xi_' + flavor]`: Store the result of calling `rng.normal(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 28, `frame['iv_' + flavor]`: Store the result of calling `rng.uniform(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 29, `frame['price_' + flavor]`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.

**Syntax and calls used here:**

- `.normal`: Draw normal random values with the supplied location/scale, used for starts or synthetic data.
- `.uniform`: Draw values from a continuous uniform interval for controlled synthetic inputs.

Source lines 31–37:

```
31 controlled, _ = add_price_controls(frame)
32 utility = (np.array([0., .4, -.3])
33            - controlled[["price_"+f for f in FLAVORS]].to_numpy()
34            + .8*controlled[["cf_"+f for f in FLAVORS]].to_numpy())
35 probs = np.exp(utility - utility.max(axis=1, keepdims=True))
36 probs /= probs.sum(axis=1, keepdims=True)
37 counts = np.array([rng.multinomial(8, p) for p in probs])
```

**Read this block:**

- Line 31, `(controlled, _)`: Unpack the result of calling `add_price_controls(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 32, `utility`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 35, `probs`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 36: update `probs` using `/=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 37, `counts`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `add_price_controls`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.

- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.multinomial`: Draw a vector of category counts summing to the specified total, given category probabilities.

Source lines 38–39:

```
38 for j, flavor in enumerate(FLAVORS):
39     frame[flavor] = counts[:, j]
```

**Read this block:**

- Line 38: for each entry yielded by `enumerate(FLAVORS)`, bind/unpack it into `(j, flavor)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 39, `frame[flavor]`: Store the entry/slice selected by `counts[:, j]`; brackets select from the object to their left in the selected entry/column.

**Syntax and calls used here:**

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.

Source lines 40–40:

```
40 return frame
```

**Read this block:**

- Line 40: return the object currently named `frame`; plain assignment does not copy it. This ends the call.

## PriceControlChecks.test excludes consumer dma even when exposure store elsewhere

Original function: `PriceControlChecks.test_excludes_consumer_dma_even_when_exposure_store_elsewhere`. Source begins at line 44.

**What this test establishes.** Exercises the retained UPC-matched IV helper with explicit tiny prices. It excludes the consumer DMA rather than assuming exposure store and consumer are in the same market, and checks incomplete baskets become missing.

**Reading its syntax.** This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

**Definition and stated intent:**

Source lines 44–44:

```
44 def test_excludes_consumer_dma_even_when_exposure_store_elsewhere(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 45–53:

```
45 week = pd.Timestamp("2010-01-02")
46 weekly = pd.DataFrame({"store_code_uc": [99, 99, 99], "week": [week]*3,
47                        "dma_cd": [1, 2, 4]})
```

```
48 exposure = pd.DataFrame({"store_code_uc": [99, 99], "week": [week]*2,
49                          "upc": ["a", "b"], "flavor": ["berry", "berry"]})
50 prices = pd.DataFrame({"upc": ["a"]*3+["b"]*3, "week": [week]*6,
51                       "dma_code": [1, 2, 3]*2, "dma_price": [1., 3., 5., 2., 4., 6.]})
52 result = other_dma_instruments(weekly, exposure, prices)
53 np.testing.assert_allclose(result["iv_berry"], [4.5, 3.5, 3.5])
```

**Read this block:**

- Line 45, `week`: Set this name to the result of calling `pd.Timestamp(...)`; the arguments and preceding method chain are shown above.
- Line 46, `weekly`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 48, `exposure`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 50, `prices`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 52, `result`: Set this name to the result of calling `other_dma_instruments(...)`; the arguments and preceding method chain are shown above.
- Line 53: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `other_dma_instruments`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.

Source lines 55–57:

```
55 missing = prices.loc[~((prices["upc"] == "b") & (prices["dma_code"] != 1))]
56 result = other_dma_instruments(weekly, exposure, missing)
57 self.assertTrue(np.isnan(result.loc[0, "iv_berry"]))
```

**Read this block:**

- Line 55, `missing`: Set this name to the entry/slice selected by `prices.loc[~((prices['upc'] == 'b') & (prices['dma_code']...`; brackets select from the object to their left.
- Line 56, `result`: Set this name to the result of calling `other_dma_instruments(...)`; the arguments and preceding method chain are shown above.
- Line 57: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.
- `.assertTrue`: Fail unless the supplied condition is true; array conditions normally need any/all first.
- `.isnan`: Identify floating-point NaNs; NaN must not be tested with ordinary equality.

**Output reminder:** reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

## PriceControlChecks.test absorption matches explicit dummy ols

Original function: `PriceControlChecks.test_absorption_matches_explicit_dummy_ols`. Source begins at line 59.

**What this test establishes.** Compares repeated demeaning to a small regression on explicit week/DMA dummy columns. The dummy matrix is feasible in a fixture and supplies a transparent independent alternative to the production memory-saving algorithm.

**Reading its syntax.** This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

**Definition and stated intent:**

Source lines 59–59:

```
59 def test_absorption_matches_explicit_dummy_ols(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 60–66:

```
60 rng = np.random.default_rng(20)
61 weeks = pd.Series([1, 1, 2, 2, 2, 3, 3, 4, 4, 4])
62 dma = pd.Series([1, 2, 1, 2, 3, 1, 3, 1, 2, 3])
63 x = rng.normal(size=(len(weeks), 3))
64 dummies = pd.concat([pd.get_dummies(weeks, dtype=float),
65                      pd.get_dummies(dma, dtype=float)], axis=1).to_numpy()
66 expected = x - dummies @ np.linalg.lstsq(dummies, x, rcond=None)[0]
```

**Read this block:**

- Line 60, `rng`: Set this name to the result of calling `np.random.default_rng(...)`; the arguments and preceding method chain are shown above.
- Line 61, `weeks`: Set this name to the result of calling `pd.Series(...)`; the arguments and preceding method chain are shown above.
- Line 62, `dma`: Set this name to the result of calling `pd.Series(...)`; the arguments and preceding method chain are shown above.
- Line 63, `x`: Set this name to the result of calling `rng.normal(...)`; the arguments and preceding method chain are shown above.
- Line 64, `dummies`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 66, `expected`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.

**Syntax and calls used here:**

- `.default_rng`: Create a local seeded random Generator, avoiding dependence on a shared global random stream.
- `.normal`: Draw normal random values with the supplied location/scale, used for starts or synthetic data.
- `len`: Count elements/rows rather than summing their numerical values.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- `.get_dummies`: Construct indicator columns for categories, used as a small explicit fixed-effect benchmark in tests.
- `.lstsq`: Solve least squares without explicitly inverting a cross-product matrix; [0] keeps its coefficient vector.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.

Source lines 67–67:

```
67 np.testing.assert_allclose(absorb_effects(x, [weeks, dma]), expected, atol=1e-8)
```

**Read this block:**

- Line 67: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.
- `absorb_effects`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

**Output reminder:** reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

## PriceControlChecks.test first stage residuals orthogonal and in utility

Original function: `PriceControlChecks.test_first_stage_residuals_orthogonal_and_in_utility`. Source begins at line 69.

**What this test establishes.** Checks positive first-stage relevance in controlled data, residual orthogonality, zero group means, and that the demand arrays actually contain the computed controls.

**Reading its syntax.** This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

**Definition and stated intent:**

Source lines 69–69:

```
69 def test_first_stage_residuals_orthogonal_and_in_utility(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 70–73:

```
70 frame = synthetic_panel()
71 controlled, report = add_price_controls(frame)
72 self.assertTrue((report["iv_coefficient"] > .7).all())
73 self.assertTrue((report["iv_cluster_F"] > 10).all())
```

**Read this block:**

- Line 70, `frame`: Set this name to the result of calling `synthetic_panel(...)`; the arguments and preceding method chain are shown above.
- Line 71, `(controlled, report)`: Unpack the result of calling `add_price_controls(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 72: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 73: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `synthetic_panel`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `add_price_controls`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assertTrue`: Fail unless the supplied condition is true; array conditions normally need any/all first.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.

Source lines 74–77:

```
74 for flavor in FLAVORS:
75     residual = controlled["cf_" + flavor]
76     self.assertAlmostEqual(float(residual @ frame["iv_" + flavor]), 0., places=7)
77     np.testing.assert_allclose(residual.groupby(frame["week"]).mean(), 0., atol=1e-8)
```

**Read this block:**

- Line 74: for each entry yielded by `FLAVORS`, bind/unpack it into `flavor` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 75, `residual`: Set this name to the entry/slice selected by `controlled['cf_' + flavor]`; brackets select from the object to their left.
- Line 76: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 77: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.
- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.

Source lines 78–80:

```
78 data = make_arrays(controlled, 0)
79 self.assertTrue(data["control"])
80 np.testing.assert_allclose(data["cf"], controlled[["cf_"+f for f in FLAVORS]])
```

**Read this block:**

- Line 78, `data`: Set this name to the result of calling `make_arrays(...)`; the arguments and preceding method chain are shown above.
- Line 79: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 80: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

**Output reminder:** reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

## PriceControlChecks.test first stage uses only covered flavor prices

Original function: `PriceControlChecks.test_first_stage_uses_only_covered_flavor_prices`. Source begins at line 82.

**What this test establishes.** Moves purchases away from unavailable alternatives, removes their prices, and checks each first stage uses only its covered rows. It also verifies zero residual placeholders remain finite and masked.

**Reading its syntax.** This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

**Definition and stated intent:**

Source lines 82–82:

```
82 def test_first_stage_uses_only_covered_flavor_prices(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 83–83:

```
83 frame = synthetic_panel()
```

**Read this block:**

- Line 83, `frame`: Set this name to the result of calling `synthetic_panel(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `synthetic_panel`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 84–89:

```
84 for j, flavor in enumerate(FLAVORS):
85     missing = np.arange(len(frame)) % 4 == j
86     next_flavor = FLAVORS[(j + 1) % 3]
87     frame.loc[missing, next_flavor] += frame.loc[missing, flavor]
88     frame.loc[missing, flavor] = 0
89     frame.loc[missing, "price_" + flavor] = np.nan
```

**Read this block:**

- Line 84: for each entry yielded by `enumerate(FLAVORS)`, bind/unpack it into `(j, flavor)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 85, `missing`: Set this name to the Boolean condition `np.arange(len(frame)) % 4 == j`; it can be a scalar or a row/array mask depending on its inputs.
- Line 86, `next_flavor`: Set this name to the entry/slice selected by `FLAVORS[(j + 1) % 3]`; brackets select from the object to their left.
- Line 87: update `frame.loc[missing, next_flavor]` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 88, `frame.loc[missing, flavor]`: Store the literal value `0` in the selected entry/column.
- Line 89, `frame.loc[missing, 'price_' + flavor]`: Store the attribute `np.nan` of the object before the dot in the selected entry/column.

**Syntax and calls used here:**

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.arange`: Create evenly stepped values with an excluded stop, commonly row positions 0 through n-1.
- `len`: Count elements/rows rather than summing their numerical values.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.

Source lines 90–90:

```
90 controlled, report = add_price_controls(frame)
```

**Read this block:**

- Line 90, `(controlled, report)`: Unpack the result of calling `add_price_controls(...)`; the arguments and preceding method chain are shown above into these targets in order.

**Syntax and calls used here:**

- `add_price_controls`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 91–96:

```
91 for flavor in FLAVORS:
92     available = frame["price_" + flavor].notna()
93     self.assertEqual(report.set_index("flavor").loc[flavor, "observations"], available.sum())
94     self.assertTrue((controlled.loc[~available, "cf_" + flavor] == 0).all())
95     self.assertAlmostEqual(float(controlled.loc[available, "cf_" + flavor]
96                                  @ frame.loc[available, "iv_" + flavor]), 0., places=7)
```

**Read this block:**

- Line 91: for each entry yielded by `FLAVORS`, bind/unpack it into `flavor` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 92, `available`: Set this name to the result of calling `frame['price_' + flavor].notna(...)`; the arguments and preceding method chain are shown above.
- Line 93: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 94: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 95: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.notna`: Create a pandas mask marking nonmissing values; it does not establish numeric positivity or finiteness.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `.assertTrue`: Fail unless the supplied condition is true; array conditions normally need any/all first.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

Source lines 97–99:

```
97 arrays = make_arrays(controlled, 0)
98 self.assertFalse(arrays["available"].all())
99 self.assertTrue(np.isfinite(arrays["cf"]).all())
```

**Read this block:**

- Line 97, `arrays`: Set this name to the result of calling `make_arrays(...)`; the arguments and preceding method chain are shown above.
- Line 98: call `self.assertFalse` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 99: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assertFalse`: Fail unless the supplied condition is false, checking an expected absence/invalidity.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.

**Output reminder:** reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

## PriceControlChecks.test resampling preserves histories and relabels duplicates

Original function: `PriceControlChecks.test_resampling_preserves_histories_and_relabels_duplicates`. Source begins at line 101.

**What this test establishes.** Resamples a 60-household/eight-period fixture and verifies there are 60 newly labeled histories, each with eight rows and no repeated new-household/week key. Sampling whole groups is compared with the behavior a row bootstrap would break.

**Reading its syntax.** This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

**Definition and stated intent:**

Source lines 101–101:

```
101 def test_resampling_preserves_histories_and_relabels_duplicates(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 102–106:

```
102 frame = synthetic_panel()
103 sampled = resample_households(frame, np.random.default_rng(1))
104 self.assertEqual(sampled["household_code"].nunique(), 60)
105 np.testing.assert_array_equal(sampled.groupby("household_code").size(), 8)
106 self.assertFalse(sampled.duplicated(["household_code", "week"]).any())
```

**Read this block:**

- Line 102, `frame`: Set this name to the result of calling `synthetic_panel(...)`; the arguments and preceding method chain are shown above.
- Line 103, `sampled`: Set this name to the result of calling `resample_households(...)`; the arguments and preceding method chain are shown above.
- Line 104: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 105: call `np.testing.assert_array_equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 106: call `self.assertFalse` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `synthetic_panel`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `resample_households`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

- `.default_rng`: Create a local seeded random Generator, avoiding dependence on a shared global random stream.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `.nunique`: Count distinct pandas values, rather than counting repeated purchase rows as separate identities.
- `.assert_array_equal`: Check that corresponding array entries agree exactly, appropriate for integer identities/counts.
- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `.assertFalse`: Fail unless the supplied condition is false, checking an expected absence/invalidity.
- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `.duplicated`: Flag repeated keys/rows so validation can detect ambiguity instead of resolving it silently.

**Output reminder:** reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

## PriceControlChecks.test bootstrap refits first stage and reports empirical sd

Original function: `PriceControlChecks.test_bootstrap_refits_first_stage_and_reports_empirical_sd`. Source begins at line 108.

**What this test establishes.** Wraps the first-stage callable with a mock to count 20 refits, supplies a cheap residual-dependent fake estimator, and compares output SEs with SDs calculated from saved bootstrap draws. This isolates two-stage orchestration from optimization.

**Reading its syntax.** This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

**Definition and stated intent:**

Source lines 108–108:

```
108 def test_bootstrap_refits_first_stage_and_reports_empirical_sd(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 109–109:

```
109 frame = synthetic_panel()
```

**Read this block:**

- Line 109, `frame`: Set this name to the result of calling `synthetic_panel(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `synthetic_panel`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 110–125:

```
110 with tempfile.TemporaryDirectory() as directory:
111     args = SimpleNamespace(seed=306, bootstrap_reps=20, no_price_control=False,
112                            output=Path(directory))
113     def estimate(sample, initial):
114         # Explicitly depend on the recomputed residual, so frozen residuals
115         # would produce a different bootstrap distribution.
116         theta = np.array([sample["cf_other"].std(), sample["cf_berry"].std()])
```

```
117         return OptimizeResult(x=theta, fun=1., success=True, message="test fit")
118     with patch("bootstrap_se.add_price_controls", wraps=add_price_controls) as first_stage:
119         with contextlib.redirect_stdout(io.StringIO()):
120             table, report = bootstrap_standard_errors(frame, args, estimate, np.zeros(2), ["a", "b"])
121     self.assertEqual(first_stage.call_count, 20)
122     self.assertEqual(report["usable"], 20)
123     draws = pd.read_csv(Path(directory) / "bootstrap_draws.csv")
124     np.testing.assert_allclose(table["std_error"], draws[["a", "b"]].std(ddof=1))
125     self.assertTrue((table["std_error"] > 0).all())
```

**Read this block:**

- Line 110: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 111, `args`: Set this name to the result of calling `SimpleNamespace(...)`; the arguments and preceding method chain are shown above.
- Line 113: define nested callable `estimate`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.
- Line 118: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 119: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 120, `(table, report)`: Unpack the result of calling `bootstrap_standard_errors(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 121: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 122: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 123, `draws`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.
- Line 124: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 125: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.
- `SimpleNamespace`: Construct a lightweight object with named attributes, useful for args-like synthetic test settings.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.
- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.
- `.assertTrue`: Fail unless the supplied condition is true; array conditions normally need any/all first.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `OptimizeResult`: Construct a SciPy-style result object; test fixtures can provide one without a genuine optimizer run.
- `patch`: Temporarily replace/wrap a callable in a test so its invocations can be observed.
- `.std`: Compute standard deviation; ddof=1 requests sample SD rather than dividing squared deviations by the full count.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.redirect_stdout`: Temporarily capture printed output, keeping tests quiet without changing the tested computation.
- `bootstrap_standard_errors`: Call this project helper/callable with the displayed arguments; its own walk-through gives the input/output contract.

- `.StringIO`: Create an in-memory text buffer instead of writing captured output to a real file.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.

## PriceControlChecks.test bootstrap refits first stage and reports empirical sd.estimate

Original function: `PriceControlChecks.test_bootstrap_refits_first_stage_and_reports_empirical_sd.estimate`. Source begins at line 113.

This nested fake estimator returns an `OptimizeResult` with coefficients equal to resampled residual SDs. It deliberately depends on the recomputed controls, so frozen controls would change the bootstrap distribution. It declares success and a harmless objective because the test targets bootstrap orchestration, not numerical optimization. Mocking the demand fit makes this check faster and isolates whether first stages are rerun and empirical SEs calculated correctly.

**Definition and stated intent:**

Source lines 113–115:

```
113 def estimate(sample, initial):
114     # Explicitly depend on the recomputed residual, so frozen residuals
115     # would produce a different bootstrap distribution.
```

`def` creates this callable; the parentheses name its inputs (`sample`, `initial`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 116–116:

```
116 theta = np.array([sample["cf_other"].std(), sample["cf_berry"].std()])
```

**Read this block:**

- Line 116, `theta`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.std`: Compute standard deviation; ddof=1 requests sample SD rather than dividing squared deviations by the full count.

Source lines 117–117:

```
117 return OptimizeResult(x=theta, fun=1., success=True, message="test fit")
```

**Read this block:**

- Line 117: return the result of calling `OptimizeResult(...)`; the arguments and preceding method chain are shown above. This ends the call.

**Syntax and calls used here:**

- `OptimizeResult`: Construct a SciPy-style result object; test fixtures can provide one without a genuine optimizer run.

## PriceControlChecks.test controlled demand optimizer runs

Original function: `PriceControlChecks.test_controlled_demand_optimizer_runs`. Source begins at line 127.

**What this test establishes.** Fits a small controlled naive demand model and checks convergence, a four-coefficient vector, and finite objective. This is a controlled optimizer smoke test, not an assertion about private-data fit quality.

**Reading its syntax.** This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results,

and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

**Definition and stated intent:**

Source lines 127–127:

```
127 def test_controlled_demand_optimizer_runs(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 128–133:

```
128 controlled, _ = add_price_controls(synthetic_panel())
129 args = SimpleNamespace(model="naive", draws=8, seed=306, starts=2, maxiter=150)
130 fit, _, _ = optimize_demand(make_arrays(controlled, 0), args, quiet=True)
131 self.assertTrue(fit.success)
132 self.assertEqual(len(fit.x), 4)  # Two betas, mean log(-alpha), price control.
133 self.assertTrue(np.isfinite(fit.fun))
```

**Read this block:**

- Line 128, `(controlled, _)`: Unpack the result of calling `add_price_controls(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 129, `args`: Set this name to the result of calling `SimpleNamespace(...)`; the arguments and preceding method chain are shown above.
- Line 130, `(fit, _, _)`: Unpack the result of calling `optimize_demand(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 131: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 132: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 133: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `add_price_controls`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `synthetic_panel`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `SimpleNamespace`: Construct a lightweight object with named attributes, useful for args-like synthetic test settings.
- `optimize_demand`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assertTrue`: Fail unless the supplied condition is true; array conditions normally need any/all first.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `len`: Count elements/rows rather than summing their numerical values.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.

**Output reminder:** reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

# est__code/test__summary__stats.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

## Module setup and direct-execution blocks

Source lines 1–6:

```
1 """Checks for the sibling summary script, using synthetic purchases only."""
2 import importlib.util
3 import os
4 from pathlib import Path
5 import tempfile
6 import unittest
```

**Read this block:**

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `importlib.util`. An as-name is a convenient local alias, not another library.
- Line 3: import `os`. An as-name is a convenient local alias, not another library.
- Line 4: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 5: import `tempfile`. An as-name is a convenient local alias, not another library.
- Line 6: import `unittest`. An as-name is a convenient local alias, not another library.

Source lines 8–10:

```
8 import numpy as np
9 import pandas as pd
10 import polars as pl
```

**Read this block:**

- Line 8: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 9: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 10: import `polars as pl`. An as-name is a convenient local alias, not another library.

Source lines 13–13:

```
13 class SummaryChecks(unittest.TestCase):
```

**Read this block:**

- Line 13: define class `SummaryChecks` and its methods. TestCase inheritance provides the self.assert… checking methods.

Source lines 61–62:

```
61 if __name__ == "__main__":
62     unittest.main()
```

**Read this block:**

- Line 61: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 62: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

## SummaryChecks.test demographics and mixed baskets

Original function: `SummaryChecks.test_demographics_and_mixed_baskets`. Source begins at line 14.

**What this test establishes.** Uses tiny tables to check household demographic selection, missing categories, and mixed-basket summary behavior. Explicit source values make it possible to verify denominators by hand.

**Reading its syntax.** This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

**Definition and stated intent:**

Source lines 14–15:

```
14 def test_demographics_and_mixed_baskets(self):
15     # The override lets a prepared replacement be tested before installation.
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 16–19:

```
16 script = Path(os.environ.get("SUMMARY_SCRIPT", Path(__file__).parent / "../data_code/summ_stats.py"))
17 spec = importlib.util.spec_from_file_location("summary", script)
18 summary = importlib.util.module_from_spec(spec)
19 spec.loader.exec_module(summary)
```

**Read this block:**

- Line 16, `script`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 17, `spec`: Set this name to the result of calling `importlib.util.spec_from_file_location(...)`; the arguments and preceding method chain are shown above.
- Line 18, `summary`: Set this name to the result of calling `importlib.util.module_from_spec(...)`; the arguments and preceding method chain are shown above.
- Line 19: call `spec.loader.exec_module` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.

Source lines 20–58:

```
20 with tempfile.TemporaryDirectory() as directory:
21     root = Path(directory)
22     source = pl.DataFrame({
23         "household_code": [1, 1, 1, 1, 2], "panel_year": [2010]*5,
24         "purchase_date": ["2010-01-02", "2010-01-02", "2010-01-09", "2010-01-23", "2010-01-03"],
25         "trip_code_uc": [1, 1, 2, 3, 4], "upc": [1, 2, 2, 3, 1], "quantity": [2, 1, 1, 3, 1],
26         "race": [1, 1, 1, 1, 2], "hispanic_origin": [2, 2, 2, 2, 1],
27         "household_income": [5, 5, 5, 5, 8], "female_head_age": [3, 3, 3, 3, 0],
28         "male_head_age": [4, 4, 4, 4, 2], "household_size": [2]*5})
29     source.write_parquet(root / "sample.parquet")
30     scan = summary.scan_purchases(root / "sample.parquet", 2006, 2015)
31     households =
    ↪ summary.household_summary(scan).collect(engine="streaming").to_pandas().set_index("household_code")
32     self.assertEqual(len(households), 2)
33     self.assertEqual(households.loc[1, "trips"], 3)
```

```
    self.assertEqual(households.loc[2, "race"], "Hispanic")
    self.assertEqual(households.loc[2, "race_original"], "2")
    summary.demographic_tables(households.reset_index(), root, "test")
    race = pd.read_csv(root / "test_race.csv")
    np.testing.assert_allclose(race["percent_all_households"], 50.)
    reference = pl.DataFrame({"upc": ["1", "2", "3"], "flavor": ["other", "berry", "plain"]})
    weekly, _ = summary.yogurt_weeks(scan, reference)
    self.assertEqual(pd.Timestamp(weekly.filter(pl.col("household_code") ==
     ↪  2).select("week").collect(engine="streaming").item()), pd.Timestamp("2010-01-09"))
    repeated = summary.repeated_yogurt_households(scan)
    self.assertEqual(summary.household_summary(repeated).collect(engine="streaming")["household_code"].to_list(),
     ↪  [1])
    # Several purchases in a single week still do not qualify.
    one_week = scan.filter(pl.col("purchase_date") <= pl.date(2010, 1, 3))
    self.assertEqual(summary.repeated_yogurt_households(one_week).collect().height, 0)
    shares, transitions = summary.flavor_summaries(weekly, root)
    summary.make_hhi_figures(root)
    metrics = pd.read_csv(root / "yogurt_household_flavors.csv").set_index("household_code")
    self.assertAlmostEqual(metrics.loc[1, "flavor_HHI"], 17 / 49)
    self.assertEqual(metrics.loc[2, "flavor_HHI"], 1.)
    for kind in ["histogram", "cdf"]:
        self.assertTrue((root / f"flavor_hhi_{kind}.pdf").exists())
    baskets = pd.read_csv(root / "yogurt_weekly_baskets.csv")
    self.assertEqual(baskets.loc[(baskets.household_code == 1) & (baskets.week == "2010-01-02"),
     ↪  "flavors_in_basket"].iloc[0], 2)
    self.assertTrue(transitions.isna().all().all())
    self.assertAlmostEqual(shares.percent_packages.sum(), 100.)
    self.assertTrue((root / "test_income.tex").exists())
```

**Read this block:**

- Line 20: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 21, `root`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 22, `source`: Set this name to the result of calling `pl.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 29: call `source.write_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 30, `scan`: Set this name to the result of calling `summary.scan_purchases(...)`; the arguments and preceding method chain are shown above.
- Line 31, `households`: Set this name to the result of calling `.set_index(...)`; the arguments and preceding method chain are shown above.
- Line 32: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 33: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 34: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 35: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 36: call `summary.demographic_tables` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 37, `race`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.
- Line 38: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 39, `reference`: Set this name to the result of calling `pl.DataFrame(...)`; the arguments and preceding method chain are shown above.

- Line 40, `(weekly, _)`: Unpack the result of calling `summary.yogurt_weeks(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 41: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 42, `repeated`: Set this name to the result of calling `summary.repeated_yogurt_households(...)`; the arguments and preceding method chain are shown above.
- Line 43: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 45, `one_week`: Set this name to the result of calling `scan.filter(...)`; the arguments and preceding method chain are shown above.
- Line 46: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 47, `(shares, transitions)`: Unpack the result of calling `summary.flavor_summaries(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 48: call `summary.make_hhi_figures` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 49, `metrics`: Set this name to the result of calling `.set_index(...)`; the arguments and preceding method chain are shown above.
- Line 50: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 51: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 52: for each entry yielded by `['histogram', 'cdf']`, bind/unpack it into `kind` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 53: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 54, `baskets`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.
- Line 55: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 56: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 57: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 58: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.
- `.write_parquet`: Persist an eager table as Parquet, often used for small synthetic fixtures.
- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.
- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `.assertTrue`: Fail unless the supplied condition is true; array conditions normally need any/all first.
- `len`: Count elements/rows rather than summing their numerical values.
- `.reset_index`: Turn the index back into ordinary row numbering; drop=True discards the old index rather than saving it as a column.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.

- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- `.to_pandas`: Convert a Polars table to pandas, crossing from query/table preparation to eager labeled analysis.
- `.item`: Extract a scalar from a one-element array/one-cell result rather than passing a table wrapper onward.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `.isna`: Create a pandas mask marking missing values; compare masks rather than using equality to NaN.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

**Output reminder:** reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

# supply_code/checks/___init___.py

This empty file marks an importable package directory. It defines no functions and performs no data calculations. Python can support some namespace packages without it, but an explicit marker makes the intended package structure clear.

## Module setup and direct-execution blocks

There are no statements in this file.

# supply_code/checks/check_alpha.py

This is a compatibility entry point for the canonical unittest checks. Importing a TestCase class makes its methods discoverable to unittest; the guarded unittest.main() runs the selected checks when invoked directly. It does not define another derivative model.

## Module setup and direct-execution blocks

Source lines 1–3:

```
1 """Compatibility entry point for the canonical symmetric-switching checks."""
2 import unittest
3 from checks.test_symmetric import SymbolicChecks
```

**Read this block:**

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 3: from `checks.test_symmetric`, import `SymbolicChecks` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 4–5:

```
4 if __name__ == "__main__":
5     unittest.main()
```

**Read this block:**

- Line 4: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 5: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

# supply_code/checks/check_beta.py

This is a compatibility entry point for the canonical unittest checks. Importing a TestCase class makes its methods discoverable to unittest; the guarded unittest.main() runs the selected checks when invoked directly. It does not define another derivative model.

## Module setup and direct-execution blocks

Source lines 1–3:

```
"""Compatibility entry point for the canonical symmetric-switching checks."""
import unittest
from checks.test_symmetric import SymbolicChecks
```

**Read this block:**

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 3: from `checks.test_symmetric`, import `SymbolicChecks` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 4–5:

```
if __name__ == "__main__":
    unittest.main()
```

**Read this block:**

- Line 4: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 5: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

# supply_code/checks/check_delta.py

This is a compatibility entry point for the canonical unittest checks. Importing a TestCase class makes its methods discoverable to unittest; the guarded unittest.main() runs the selected checks when invoked directly. It does not define another derivative model.

## Module setup and direct-execution blocks

Source lines 1–3:

```
"""Compatibility entry point for the canonical symmetric-switching checks."""
import unittest
from checks.test_symmetric import SymbolicChecks
```

**Read this block:**

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 3: from `checks.test_symmetric`, import `SymbolicChecks` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 4–5:

```
if __name__ == "__main__":
    unittest.main()
```

**Read this block:**

- Line 4: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 5: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

# supply__code/checks/check__focs.py

This is a compatibility entry point for the canonical unittest checks. Importing a TestCase class makes its methods discoverable to unittest; the guarded unittest.main() runs the selected checks when invoked directly. It does not define another derivative model.

## Module setup and direct-execution blocks

Source lines 1–3:

```
"""Compatibility entry point for the canonical symmetric-switching checks."""
import unittest
from checks.test_symmetric import SymbolicChecks
```

**Read this block:**

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 3: from `checks.test_symmetric`, import `SymbolicChecks` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 4–5:

```
if __name__ == "__main__":
    unittest.main()
```

**Read this block:**

- Line 4: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 5: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

# supply__code/checks/check__gamma.py

This is a compatibility entry point for the canonical unittest checks. Importing a TestCase class makes its methods discoverable to unittest; the guarded unittest.main() runs the selected checks when invoked directly. It does not define another derivative model.

## Module setup and direct-execution blocks

Source lines 1–3:

```
"""Compatibility entry point for the canonical symmetric-switching checks."""
import unittest
from checks.test_symmetric import SymbolicChecks
```

**Read this block:**

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `unittest`. An as-name is a convenient local alias, not another library.

- Line 3: from `checks.test_symmetric`, import `SymbolicChecks` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 4–5:

```
4 if __name__ == "__main__":
5     unittest.main()
```

**Read this block:**

- Line 4: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 5: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

# supply_code/checks/check_numerical.py

This is a compatibility entry point for the canonical unittest checks. Importing a TestCase class makes its methods discoverable to unittest; the guarded unittest.main() runs the selected checks when invoked directly. It does not define another derivative model.

## Module setup and direct-execution blocks

Source lines 1–3:

```
1 """Compatibility entry point for the canonical symmetric-switching checks."""
2 import unittest
3 from checks.test_symmetric import NumericalChecks
```

**Read this block:**

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 3: from `checks.test_symmetric`, import `NumericalChecks` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 4–5:

```
4 if __name__ == "__main__":
5     unittest.main()
```

**Read this block:**

- Line 4: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 5: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

# supply_code/checks/check_pricing_derivatives.py

This is a compatibility entry point for the canonical unittest checks. Importing a TestCase class makes its methods discoverable to unittest; the guarded unittest.main() runs the selected checks when invoked directly. It does not define another derivative model.

## Module setup and direct-execution blocks

Source lines 1–3:

```
"""Compatibility entry point for the canonical symmetric-switching checks."""
import unittest
from checks.test_symmetric import SymbolicChecks
```

**Read this block:**

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 3: from `checks.test_symmetric`, import `SymbolicChecks` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 4–5:

```
if __name__ == "__main__":
    unittest.main()
```

**Read this block:**

- Line 4: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 5: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

# supply_code/checks/test_symmetric.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

## Module setup and direct-execution blocks

Source lines 1–6:

```
"""Symbolic identities and independent reoptimized finite differences."""
import unittest
import numpy as np
import sympy as sp
from scipy.optimize import minimize_scalar
from model.symbols import *
```

**Read this block:**

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 3: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 4: import `sympy as sp`. An as-name is a convenient local alias, not another library.
- Line 5: from `scipy.optimize`, import `minimize_scalar` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 6: from `model.symbols`, import $*$ into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 7–12:

```
from model.utility import raw_utilities, U1, U2
from model.shares import sj1_expr, sj2_expr
from model.profit import Pi
from pricing.period1_foc import foc1_collected
from pricing.period2_foc import foc2
```

```
12 from comparative_statics import canonical as C
```

**Read this block:**

- Line 7: from `model.utility`, import `raw_utilities, U1, U2` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 8: from `model.shares`, import `sj1_expr, sj2_expr` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 9: from `model.profit`, import `Pi` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 10: from `pricing.period1_foc`, import `foc1_collected` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 11: from `pricing.period2_foc`, import `foc2` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 12: from `comparative_statics`, import `canonical as C` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 13–14:

```
13 from model.numerical import evaluate, inside_share
14 import structural_switching_reference as reference
```

**Read this block:**

- Line 13: from `model.numerical`, import `evaluate, inside_share` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 14: import `structural_switching_reference as reference`. An as-name is a convenient local alias, not another library.

Source lines 17–17:

```
17 class SymbolicChecks(unittest.TestCase):
```

**Read this block:**

- Line 17: define class `SymbolicChecks` and its methods. TestCase inheritance provides the self.assert… checking methods.

Source lines 57–57:

```
57 class NumericalChecks(unittest.TestCase):
```

**Read this block:**

- Line 57: define class `NumericalChecks` and its methods. TestCase inheritance provides the self.assert… checking methods.

Source lines 134–135:

```
134 if __name__=='__main__':
135     unittest.main()
```

**Read this block:**

- Line 134: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 135: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

## SymbolicChecks.equal

Original function: `SymbolicChecks.equal`. Source begins at line 18.

This helper asserts symbolic equality by simplifying the difference to zero. `self` supplies the current unittest instance, so `self.assertEqual` records a normal test failure instead of only printing a warning. Structural `left == right` may fail for algebraically equivalent expressions written in different forms. Simplifying their difference is a stronger algebraic comparison for these expressions, though symbolic simplification is not a universal theorem prover.

**Definition and stated intent:**

Source lines 18–18:

```
18 def equal(self,left,right):
```

`def` creates this callable; the parentheses name its inputs (`self`, `left`, `right`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 19–19:

```
19 self.assertEqual(sp.simplify(left-right),0)
```

**Read this block:**

- Line 19: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `.simplify`: Try algebraic simplification; it does not numerically optimize a price or establish every economic sign.

**Output reminder:** reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

## SymbolicChecks.test raw normalization and both states

Original function: `SymbolicChecks.test_raw_normalization_and_both_states`. Source begins at line 21.

**What this test establishes.** Evaluates both signed histories and symbolically verifies subtracting raw outside utility yields beta+alpha*price*+*q*gamma. This catches accidentally deleting outside switching utility.

**Reading its syntax.** This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

**Definition and stated intent:**

Source lines 21–21:

```
21 def test_raw_normalization_and_both_states(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 22–24:

```
22 for history in [-1,1]:
23     a,b = raw_utilities(p1,history)
24     self.equal(a-b,beta+alpha*p1+history*gamma)
```

**Read this block:**

- Line 22: for each entry yielded by `[-1, 1]`, bind/unpack it into `history` and run the indented body. Changes inside an iteration can feed subsequent iterations.

- Line 23, `(a, b)`: Unpack the result of calling `raw_utilities(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 24: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `raw_utilities`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 25–25:

```
25 self.equal(U1,beta+alpha*p1+q*gamma)
```

**Read this block:**

- Line 25: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Output reminder:** reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

## SymbolicChecks.test focs from actual profit

Original function: `SymbolicChecks.test_focs_from_actual_profit`. Source begins at line 27.

**What this test establishes.** Differentiates literal symbolic profits and compares them to stored FOCs, including both continuation weights. It checks the objective behind the equations rather than merely rearranging a copied FOC.

**Reading its syntax.** This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

**Definition and stated intent:**

Source lines 27–27:

```
27 def test_focs_from_actual_profit(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 28–33:

```
28 literal = Pi.subs(sj1,sj1_expr)
29 expected = foc1_collected.subs(sj1,sj1_expr)
30 self.equal(sp.diff(literal,p1),expected)
31 self.equal(sp.diff((p2-c)*sj2_expr,p2),foc2.subs(sj2,sj2_expr))
32 self.equal(sp.diff(literal,Vj),delta*sj1_expr)
33 self.equal(sp.diff(literal,V0),delta*(1-sj1_expr))
```

**Read this block:**

- Line 28, `literal`: Set this name to the result of calling `Pi.subs(...)`; the arguments and preceding method chain are shown above.
- Line 29, `expected`: Set this name to the result of calling `foc1_collected.subs(...)`; the arguments and preceding method chain are shown above.
- Line 30: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 31: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 32: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 33: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.subs`: Substitute symbols/expressions in symbolic algebra; this does not by itself solve an equilibrium.
- `.diff`: For SymPy differentiate algebra; for a Polars/pandas date/value expression calculate successive differences. The receiver decides which operation this name means.

Source lines 34–34:

```
34 self.equal(sp.diff(Vj,p1),0)
```

**Read this block:**

- Line 34: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Output reminder:** reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

## SymbolicChecks.test envelope and implicit gamma derivatives

Original function: `SymbolicChecks.test_envelope_and_implicit_gamma_derivatives`. Source begins at line 36.

**What this test establishes.** Verifies explicit gamma price/profit/curvature identities and total derivatives of the implicit FOC. Algebraic comparison catches missing chain-rule and continuation terms.

**Reading its syntax.** This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

**Definition and stated intent:**

Source lines 36–36:

```
36 def test_envelope_and_implicit_gamma_derivatives(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 37–43:

```
37 self.equal(C.dp1_dgamma,-(q*sj1+delta*(1-sj1)*(u+v))/alpha)
38 self.equal(C.dPi_dgamma,(-q*sj1+delta*(sj1*u-(1-sj1)*v))/alpha)
39 expected2 = -(sj1*(1-sj1)**2*(q-delta*(u+v))**2
40              +delta*(1-sj1)*(-u*(1-u)**2+v*(1-v)**2))/alpha
41 self.equal(C.d2p1_dgamma2,expected2)
42 self.equal(C.ds_dgamma,sj1*(1-sj1)**2*(q-delta*(u+v)))
43 self.equal(C.dVj_dgamma,u/alpha)
```

**Read this block:**

- Line 37: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 38: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 39, `expected2`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

- Line 41: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 42: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 43: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.

Source lines 44–49:

```
44 self.equal(C.dV0_dgamma,-v/alpha)
45 # Differentiate implicit pricing equations with endogenous shares.
46 F = p1-c+1/(alpha*(1-sj1))+delta*(Vj-V0)
47 self.equal(C.total_derivative(F,gamma),0)
48 self.equal(sp.diff(p2-c+1/(alpha*(1-sj2)),p2)
49           +sp.diff(p2-c+1/(alpha*(1-sj2)),sj2)*alpha*sj2*(1-sj2),1/(1-sj2))
```

**Read this block:**

- Line 44: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 46, `F`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 47: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 48: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.diff`: For SymPy differentiate algebra; for a Polars/pandas date/value expression calculate successive differences. The receiver decides which operation this name means.

**Output reminder:** reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

## SymbolicChecks.test all parameter implicit derivatives

Original function: `SymbolicChecks.test_all_parameter_implicit_derivatives`. Source begins at line 51.

**What this test establishes.** Loops through alpha, beta, delta, and cost and verifies that differentiating the equilibrium pricing identity along each solution gives zero. This extends checks beyond the headline gamma plots.

**Reading its syntax.** This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

**Definition and stated intent:**

Source lines 51–51:

```
51 def test_all_parameter_implicit_derivatives(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 52–52:

```
52 F = p1-c+1/(alpha*(1-sj1))+delta*(Vj-V0)
```

**Read this block:**

- Line 52, `F`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Source lines 53–54:

```
53 for theta in [alpha,beta,delta,c]:
54     self.equal(C.total_derivative(F,theta),0)
```

**Read this block:**

- Line 53: for each entry yielded by `[alpha, beta, delta, c]`, bind/unpack it into `theta` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 54: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Output reminder:** reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

## NumericalChecks.test reference on entire benchmark grid

Original function: `NumericalChecks.test_reference_on_entire_benchmark_grid`. Source begins at line 58.

**What this test establishes.** Solves both histories at all 601 gamma grid points and compares named numerical outputs against the independently coded reference. It also checks original FOC residual magnitudes.

**Reading its syntax.** This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

**Definition and stated intent:**

Source lines 58–58:

```
58 def test_reference_on_entire_benchmark_grid(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 59–64:

```
59 for g in np.linspace(-3,3,601):
60     for state in ['j','0']:
61         ours, original = evaluate(float(g),state),reference.evaluate(float(g),state)
62         for key in ['p1','p2j','p20','profit','dp1','d2p1','dprofit','d_matched_ratio']:
63             self.assertAlmostEqual(ours[key],original[key],places=9)
64         self.assertLess(max(abs(ours[k]) for k in ['foc1','foc2j','foc20']),1e-10)
```

**Read this block:**

- Line 59: for each entry yielded by `np.linspace(-3, 3, 601)`, bind/unpack it into `g` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 60: for each entry yielded by `['j', '0']`, bind/unpack it into `state` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 61, `(ours, original)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 62: for each entry yielded by `['p1', 'p2j', 'p20', 'profit', 'dp1', 'd2p1', 'dprofit', 'd_ma...`, bind/unpack it into `key` and run the indented body. Changes inside an iteration can feed subsequent iterations.

- Line 63: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 64: call `self.assertLess` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.
- `.assertLess`: Fail unless the first value is strictly smaller than the second.
- `evaluate`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `abs`: Take magnitude, removing a sign before a distance/tolerance comparison.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

**Output reminder:** reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

## NumericalChecks.test independent reoptimization and derivatives

Original function: `NumericalChecks.test_independent_reoptimization_and_derivatives`. Source begins at line 66.

**What this test establishes.** At several calibrations compares analytic slopes and curvature with finite differences of fully reoptimized solutions, ratio derivatives with recomputed ratios, and p1 with a separate direct profit optimizer. Different computational routes can catch shared-expression mistakes.

**Reading its syntax.** This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

**Definition and stated intent:**

Source lines 66–66:

```
66 def test_independent_reoptimization_and_derivatives(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 67–67:

```
67 h=1e-4
```

**Read this block:**

- Line 67, `h`: Set this name to the literal value `0.0001`.

Source lines 68–91:

```
68 for parameters in [dict(beta=2,alpha=-1,delta=.92,c=0),
69                    dict(beta=-1,alpha=-2,delta=.7,c=1),
70                    dict(beta=1.5,alpha=-.7,delta=.4,c=.3)]:
71     for g in [-2.4,-1.2,0,.7,2.6]:
72         for state in ['j','0']:
73             r=evaluate(g,state,**parameters)
74             plus,minus=evaluate(g+h,state,**parameters),evaluate(g-h,state,**parameters)
75             for derivative,level in [('dp1','p1'),('dprofit','profit'),('dp2j','p2j'),('dp20','p20')]:
76                 self.assertAlmostEqual(r[derivative],(plus[level]-minus[level])/(2*h),delta=3e-7)
```

```
77            self.assertAlmostEqual(r['d2p1'],(plus['dp1']-minus['dp1'])/(2*h),delta=3e-7)
78            self.assertAlmostEqual(r['d2p1'],(plus['p1']-2*r['p1']+minus['p1'])/h**2,delta=3e-5)
79            denominator='p2j' if state=='j' else 'p20'
80            dr=(plus['p1']/plus[denominator]-minus['p1']/minus[denominator])/(2*h)
81            self.assertAlmostEqual(r['d_matched_ratio'],dr,delta=3e-7)
82            if state=='0':
83                self.assertAlmostEqual(r['d_induce_ratio'],(plus['p1']/plus['p2j']-minus['p1']/minus['p2j'])/(2*
                  ↪ h),delta=3e-7)
84            self.assertAlmostEqual(r['p1'],r['static_component']+r['continuation_adjustment'],places=10)
85            # Independent optimizer of the complete two-branch profit.
86            def negative_profit(price):
87                share=inside_share(price,g,state,parameters['beta'],parameters['alpha'])
88                return -(share*(price-parameters['c'])+parameters['delta']*(share*r['V2j']+(1-share)*r['V20']))
89            optimum=minimize_scalar(negative_profit,bounds=(r['p1']-10,r['p1']+10),method='bounded')
90            self.assertAlmostEqual(optimum.x,r['p1'],delta=1e-5)
91            self.assertLess(negative_profit(r['p1']),(negative_profit(r['p1']+h)+negative_profit(r['p1']-h))/2)
```

**Read this block:**

- Line 68: for each entry yielded by `[dict(beta=2, alpha=-1, delta=0.92, c=0), dict(beta=-1, alpha=...`, bind/unpack it into `parameters` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 71: for each entry yielded by `[-2.4, -1.2, 0, 0.7, 2.6]`, bind/unpack it into `g` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 72: for each entry yielded by `['j', '0']`, bind/unpack it into `state` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 73, `r`: Set this name to the result of calling `evaluate(...)`; the arguments and preceding method chain are shown above.
- Line 74, `(plus, minus)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 75: for each entry yielded by `[('dp1', 'p1'), ('dprofit', 'profit'), ('dp2j', 'p2j'), ('dp20...`, bind/unpack it into `(derivative, level)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 76: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 77: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 78: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 79, `denominator`: Set this name to the true-branch value if `state == 'j'` holds, otherwise the else-branch value.
- Line 80, `dr`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 81: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 82: test `state == '0'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 83: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 84: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 86: define nested callable `negative_profit`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.
- Line 89, `optimum`: Set this name to the result of calling `minimize_scalar(...)`; the arguments and preceding method chain are shown above.
- Line 90: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 91: call `self.assertLess` with the displayed arguments. No assignment stores its return here; its

documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `evaluate`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `minimize_scalar`: Numerically minimize a one-variable objective, used as an independent check of price optimality.
- `.assertLess`: Fail unless the first value is strictly smaller than the second.
- `inside_share`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `negative_profit`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

## NumericalChecks.test independent reoptimization and derivatives.negative profit

Original function: `NumericalChecks.test_independent_reoptimization_and_derivatives.negative_profit`. Source begins at line 86.

This nested objective accepts a candidate first-period price, computes its logit share, and returns the negative of complete two-branch discounted profit. It captures current gamma, state, calibration, and solved continuation values from the enclosing test loop. `minimize_scalar` minimizes, so negating profit turns maximization into minimization. Using a fresh direct objective here checks the production root against another computational route.

**Definition and stated intent:**

Source lines 86–86:

```
86 def negative_profit(price):
```

`def` creates this callable; the parentheses name its inputs (`price`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 87–87:

```
87 share=inside_share(price,g,state,parameters['beta'],parameters['alpha'])
```

**Read this block:**

- Line 87, `share`: Set this name to the result of calling `inside_share(...)`; the arguments and preceding method chain are shown above.

**Syntax and calls used here:**

- `inside_share`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 88–88:

```
88 return -(share*(price-parameters['c'])+parameters['delta']*(share*r['V2j']+(1-share)*r['V20']))
```

**Read this block:**

- Line 88: return the negated or sign-adjusted expression `-(share * (price - parameters['c']) + parameters['delta']...`. This ends the call.

## NumericalChecks.test all parameter and mixed derivatives numerically

Original function: `NumericalChecks.test_all_parameter_and_mixed_derivatives_numerically`. Source begins at line 93.

**What this test establishes.** Lambdifies general parameter derivatives, perturbs alpha/beta/delta/c separately, and compares finite differences. It also checks a beta-gamma mixed derivative using reoptimized results.

**Reading its syntax.** This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

**Definition and stated intent:**

Source lines 93–93:

```
93 def test_all_parameter_and_mixed_derivatives_numerically(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 94–95:

```
94 parameters=dict(beta=2.,alpha=-1.,delta=.92,c=0.)
95 arguments=(alpha,beta,gamma,delta,c,q,sj1,u,v,p1,p2j,p20,Vj,V0)
```

**Read this block:**

- Line 94, `parameters`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 95, `arguments`: Set this name to a tuple containing 14 displayed entries.

**Syntax and calls used here:**

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

Source lines 96–98:

```
96 def arguments_at(result):
97     return (-1.,2.,result['gamma'],.92,0.,-1 if result['initial_state']=='j' else 1,
98
 ↪  result['s'],result['u'],result['v'],result['p1'],result['p2j'],result['p20'],result['V2j'],result['V20'])
```

**Read this block:**

- Line 96: define nested callable `arguments_at`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 99–112:

```
99 for theta,name in [(alpha,'alpha'),(beta,'beta'),(delta,'delta'),(c,'c')]:
100     expression=C.price1_derivative(theta)
101     terminal=C.price2_derivative(theta,u,-1,p2j)
102     evaluator=sp.lambdify(arguments,[expression,terminal],'numpy')
103     for state in ['j','0']:
104         for g in [-1.,1.]:
105             result=evaluate(g,state,**parameters)
106             h=1e-5
107             plus_parameters=dict(parameters);plus_parameters[name]+=h
108             minus_parameters=dict(parameters);minus_parameters[name]-=h
109             plus=evaluate(g,state,**plus_parameters);minus=evaluate(g,state,**minus_parameters)
110             analytical=evaluator(*arguments_at(result))
111             self.assertAlmostEqual(float(analytical[0]),(plus['p1']-minus['p1'])/(2*h),delta=2e-6)
112             self.assertAlmostEqual(float(analytical[1]),(plus['p2j']-minus['p2j'])/(2*h),delta=2e-6)
```

**Read this block:**

- Line 99: for each entry yielded by `[(alpha, 'alpha'), (beta, 'beta'), (delta, 'delta'), (c, 'c')]`, bind/unpack it into `(theta, name)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 100, `expression`: Set this name to the result of calling `C.price1_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 101, `terminal`: Set this name to the result of calling `C.price2_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 102, `evaluator`: Set this name to the result of calling `sp.lambdify(...)`; the arguments and preceding method chain are shown above.
- Line 103: for each entry yielded by `['j', '0']`, bind/unpack it into `state` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 104: for each entry yielded by `[-1.0, 1.0]`, bind/unpack it into `g` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 105, `result`: Set this name to the result of calling `evaluate(...)`; the arguments and preceding method chain are shown above.
- Line 106, `h`: Set this name to the literal value `1e-05`.
- Line 107, `plus_parameters`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 107: update `plus_parameters[name]` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 108, `minus_parameters`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 108: update `minus_parameters[name]` using `-=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 109, `plus`: Set this name to the result of calling `evaluate(...)`; the arguments and preceding method chain are shown above.
- Line 109, `minus`: Set this name to the result of calling `evaluate(...)`; the arguments and preceding method chain are shown above.
- Line 110, `analytical`: Set this name to the result of calling `evaluator(...)`; the arguments and preceding method chain are shown above.
- Line 111: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 112: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.lambdify`: Compile a symbolic expression into a numerical callable with an explicitly ordered argument list.
- `evaluate`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `evaluator`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `arguments_at`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- A starred entry expands an iterable into separate elements, here within a list/tuple or call.

Source lines 113–114:

```
113 mixed=sp.lambdify(arguments,C.total_derivative(C.price1_derivative(beta),gamma),'numpy')
114 slope=sp.lambdify(arguments,C.price1_derivative(beta),'numpy')
```

**Read this block:**

- Line 113, `mixed`: Set this name to the result of calling `sp.lambdify(...)`; the arguments and preceding method chain are shown above.
- Line 114, `slope`: Set this name to the result of calling `sp.lambdify(...)`; the arguments and preceding method chain are shown above.

Source lines 115–119:

```
115 for state in ['j','0']:
116     g=.7;h=1e-4
117     actual=mixed(*arguments_at(evaluate(g,state)))
118     expected=(slope(*arguments_at(evaluate(g+h,state)))-slope(*arguments_at(evaluate(g-h,state))))/(2*h)
119     self.assertAlmostEqual(float(actual),float(expected),delta=2e-6)
```

**Read this block:**

- Line 115: for each entry yielded by `['j', '0']`, bind/unpack it into `state` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 116, `g`: Set this name to the literal value `0.7`.
- Line 116, `h`: Set this name to the literal value `0.0001`.
- Line 117, `actual`: Set this name to the result of calling `mixed(...)`; the arguments and preceding method chain are shown above.
- Line 118, `expected`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 119: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `mixed`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `slope`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

## NumericalChecks.test all parameter and mixed derivatives numerically.arguments at

Original function: `NumericalChecks.test_all_parameter_and_mixed_derivatives_numerically.arguments_at`. Source begins at line 96.

This nested adapter turns a result dictionary into the exact tuple expected by lambdified symbolic derivatives. The q sign uses initial history, and branch prices/shares/values retain their order. Its benchmark constants deliberately match the enclosing test calibration. A tuple is appropriate because the numerical evaluator uses positional arguments; a dictionary would require explicit ordered extraction anyway. Swapping u/v could make a wrong formula look correct at the symmetric gamma=0 point, so tests also use nonzero gamma.

**Definition and stated intent:**

Source lines 96–96:

```
96 def arguments_at(result):
```

`def` creates this callable; the parentheses name its inputs (`result`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 97–98:

```
97 return (-1.,2.,result['gamma'],.92,0.,-1 if result['initial_state']=='j' else 1,
98         result['s'],result['u'],result['v'],result['p1'],result['p2j'],result['p20'],result['V2j'],result['V20'])
```

**Read this block:**

- Line 97: return a tuple of 14 results. The caller can unpack them; return ends the call, unlike print.

**Syntax and calls used here:**

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

## NumericalChecks.test zero switching static case and symmetry

Original function: `NumericalChecks.test_zero_switching_static_case_and_symmetry`. Source begins at line 121.

**What this test establishes.** Checks gamma=0 branch equality, delta=0 equality with the true static counterfactual, terminal history/sign symmetry, and an admissible negative introductory price. These limit cases catch state and price-floor regressions.

**Reading its syntax.** This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

**Definition and stated intent:**

Source lines 121–121:

```
121 def test_zero_switching_static_case_and_symmetry(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 122–127:

```
122 for state in ['j','0']:
123     r=evaluate(0,state)
124     self.assertAlmostEqual(r['p1'],r['p2j'])
125     self.assertAlmostEqual(r['p2j'],r['p20'])
126     static=evaluate(2,state,delta=0)
127     self.assertAlmostEqual(static['p1'],static['static_counterfactual'])
```

**Read this block:**

- Line 122: for each entry yielded by `['j', '0']`, bind/unpack it into `state` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 123, `r`: Set this name to the result of calling `evaluate(...)`; the arguments and preceding method chain are shown above.
- Line 124: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 125: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 126, `static`: Set this name to the result of calling `evaluate(...)`; the arguments and preceding method chain are shown above.
- Line 127: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `evaluate`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.

Source lines 128–129:

```
128 for g in [-3,-1,1,3]:
129     self.assertAlmostEqual(evaluate(g,'j')['p2j'],evaluate(-g,'0')['p20'])
```

**Read this block:**

- Line 128: for each entry yielded by `[-3, -1, 1, 3]`, bind/unpack it into `g` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 129: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 131–131:

```
131 self.assertLess(evaluate(-8,'0')['p1'],0)
```

**Read this block:**

- Line 131: call `self.assertLess` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

**Syntax and calls used here:**

- `.assertLess`: Fail unless the first value is strictly smaller than the second.

**Output reminder:** reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

# Coverage, rebuilding, and next steps

This edition includes 71 Python files, 104 named functions/methods (including nested helpers), and 3,827 source lines. Every AST statement start is accounted for in the annotated listings. Empty files and top-level scripts are included. Lambda expressions are explained at their use sites rather than counted as named functions. Large raw LaTeX strings are reproduced as data; their mathematical content is discussed in the separate economic guide.

The build reads Python files as text and parses their syntax; it never imports or executes the private-data processing scripts. The companion JSON manifest records source hashes, function names/lines, and statement coverage. If source changes, rebuild the book so printed line references match the new snapshot. A newly added function requires a teaching note before the build succeeds.

Rebuild from the repository's Code directory:

```
python3 final_draft/build_python101.py
```

Use `--markdown-only` to regenerate the readable source/coverage manifest without invoking Pandoc/XeLaTeX. The source lessons are in `final_draft/python101/`. The generated Markdown is `final_draft/PYTHON101_WALKTHROUGH.md`.

Keep this book beside the separate `complete_code_guides.pdf`: use that guide for model derivations and this one for what the corresponding Python actually says. Try changing tiny test fixtures or tracing values on paper before editing the large server pipelines.